Using Genetic Programming for Feature Creation with a Genetic Algorithm Feature Selector

Conference Paper *in* Lecture Notes in Computer Science · January 2010

DOI: 10.1007/978-3-540-30217-9_117

1 author:

Matthew G. Smith
University of the West of England, Bristol
6 PUBLICATIONS **235** CITATIONS

Using Genetic Programming for Feature Creation with a Genetic Algorithm Feature Selector

Matthew Goble Smith
. .

Bristol Institute of Technology
University of the West of England

Thesis submitted for the Degree of Doctor of Philosophy
at the
University of the West Of England

· June 2010 ·

Abstract

This thesis examines the use of Genetic Programming and a Genetic Algorithm in a wrapper approach to pre-process data before it is classified by a (relatively simple) classifier such as C4.5. Genetic Programming is combined with a Genetic Algorithm to construct and select new features from those available in the data, a potentially significant process for data mining since it gives consideration to hidden relationships between features. Pre-processing data in this fashion for a simple classifier makes explicit the discovery of hidden relationships that is made implicitly by a more complex classifier such as an Artificial Neural Network or a Support Vector Machine. The algorithm is applied initially to C4.5, then to IBk and Naïve Bayes, and allowed to select the most appropriate classifier type. Techniques are employed to improve the human readability of these new features and extract more information about the domain. Finally ensembles of heterogenous classifiers are constructed, both from the final population of a single run and by assembling the fittest individuals from many runs.

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1

Introduction

The seed that eventually grew into the thesis presented here was planted when, during a Masters course in Machine Learning, I was presented with 'The Great Energy Predictor Shootout'[1]. Key to being able to make predictions about energy usage was the individual's ability to transform the data into a format intelligible to the prediction algorithm (primarily time-related attributes such as time of day/week). I wasn't very happy with this - I wanted an algorithm that would do this for me, so that I didn't have to have such domain-specific knowledge or have to perform any data transformation at all (I'm just lazy at heart). Thus was born the algorithm that is the focus of this thesis - a pre-processor that would perform feature construction and selection so as to improve the accuracy of a base classifier, and ultimately be able to present the user with the domain knowledge gained during the process.

The research presented in this thesis evolved to investigate the hypothesis that this algorithm could be used to improve the performance of a simple classifier (C4.5, Nearest Neighbour or Naïve Bayes) so as to make it competitive with the accuracy of a more complex classifier such as a Support Vector Machine (SVM) or Artificial Neural Network (ANN) (both of which are capable of performing feature construction implicitly), with the added advantage of making explicit the extraction of relationships between attributes in the dataset so as to make them visible and easily understood by a human (something that is often difficult with SVMs or ANNs). This has been investigated with experiments on 10 datasets from the UCI Machine Learning Repository [23], using two sets of 10-fold cross-validation on each dataset. The signif-

[1] `http://repository.tamu.edu/handle/1969.1/2137`

icance (or otherwise) of the experimental results has been established using a paired Students t-test on the results for each dataset, and on the results of the 10 datasets combined.

The content of this thesis is laid out as follows: Chapter 2 provides background on machine learning techniques (including evolutionary algorithms) and the types of classifiers employed here. Chapter 3 introduces the algorithm presented in this thesis (the GAP algorithm), while chapter 4 presents some expansions and modifications to it. Chapter 5 focuses on methods to improve the human readability (visibility) of features derived by the algorithm. Chapter 6 explores the use of ensembles where each individual has a different set of features as constructed by the GAP algorithm. Chapter 7 presents a case study using a real world dataset to confirm the conclusions drawn in previous chapters.

There are a number of versions of the GAP algorithm presented in this thesis, they are numbered according to the chapter within which they are presented, as listed on page x. The algorithms are presented in the order in which they were developed - the initial two-stage version presented in chapter 3 is algorithm 3.1. The primary version of the algorithm (the result of a number of modifications) is presented toward the end of chapter 4, as algorithm 4.3. Algorithms 4.1 and 4.2 represent intermediary stages, and are not referred to outside of the context of that chapter. A further modification to include the use of multi-objective techniques is presented in chapter 5 as algorithm 5.1. Both algorithms 4.3 and 5.1 are used and referred to in subsequent chapters, including the case study.

Chapter 2

Background

2.1 Data Mining and Classification

Data Mining (aka knowledge extraction or information discovery) is the extraction of useful, previously unknown information from a large body of data. Manual techniques for identifying patterns have been available for centuries, for example the probability-based Bayes theorem developed in the eighteenth century.[1] However as the amount of data collected has grown ever more quickly (for instance the recording of every transaction at supermarket tills, the terabytes of data returned by satellites in Earth orbit, the explosion in the amount of content on the internet, or the huge amount of data on the human genome), the utility of manual approaches has diminished as rapidly as the need for automated tools has grown.

There are two primary aims in data mining: to predict the unknown (the outcome of an event or the future value of a variable of interest); and to describe the patterns in the data that enabled the prediction, in a way that is intelligible to humans. Prediction in this case takes one of two forms: classification and regression. *Classification* involves assigning examples to one of a given set of categories, for instance diagnosing whether or not someone has a particular medical condition, or determining a letter in optical character recognition. *Regression* involves assigning a real-valued number to the prediction, often in a time series such as predicting tomorrow's share price for a company.

The computer science discipline concerned with developing algo-

[1] Bayes theorem relates the *conditional* probability of an event A given that event B has occurred, to the *prior* probability of event A (the probability of event A before B occurred).

rithms for automated data mining is known as *machine learning*, the development of 'techniques and tools with the ability to intelligently and automatically assist humans in analysing the mountains of data for nuggets of useful information' [63]. Machine learning is increasingly becoming ubiquitous, from the categorisation of news stories by aggregators like Google News and the suggestion of songs by online services such as Pandora, to the analysis of shopping habits by supermarkets (the reason for that loyalty card in your wallet). Machine learning tools monitor all credit card transactions looking for patterns of fraud, and are used to manage financial portfolios (one of the few companies to publicly describe their investment systems are LBS Capital Management, who have used a combination of expert systems,[2] neural networks and genetic algorithms since 1993 [62]). In science the uses of machine learning range from image analysis in astronomy (for instance the recently announced Palomar Transient Factory will sift terabytes of data each night 'to discover relatively rare and fleeting cosmic events like supernovae and gamma ray bursts')[3] to medical diagnosis (e.g. the identification of discriminatory genes in gene expression profiles for the diagnosis of cancer [112]). Machine learning is used to optimise processes in manufacturing (for instance in the etching of semiconductor chips [85]), even providing patentable designs (such as an improved design evolved by Genetic Programming for the PID controller widely used in industrial control systems [99]). Machine learning is not limited solely to data mining - for instance all the entrants in the DARPA grand challenge for driverless cars use machine learning techniques of one form or another (pattern recognition being extremely important in allowing the machine to 'see' upcoming obstacles).

Machine learning algorithms can be grouped according to the input provided during training:

Unsupervised learning. This often involves clustering (using algorithms such as k-means clustering) - finding similarities in the training data and assigning examples to naturally occurring

[2]Expert systems occupy a different field of artificial intelligence to machine learning, attempting to codify the domain-specific knowledge of a subject matter expert using a series of *if-then* rules, rather than the algorithm learning for itself.

[3]The Palomar Transient Factory will take 100-megapixel images to be examined in real time to 'identify "transient" sources, cosmic objects that change in brightness or position, by comparing the new observations with all of the data collected from previous nights', allowing other telescopes to be trained on the event. More details can be found online: `http://newscenter.lbl.gov/press-releases/2009/06/15/nersc-helps-expose-cosmic-transients/`.

groups. In this case the training data is not labelled, the algorithm must decide for itself how many categories are appropriate and how the examples are organised. An example is the automated grouping of search results into related areas (as used by the Clusty search engine[4]). Another form of unsupervised learning is **Reinforcement learning** in which the algorithm (or agent) learns the most appropriate sequence of actions based on its interaction with the world, for instance learning a game or exploring a maze - the agent must learn by trial and error which actions lead to a long-term reward (winning the game or exiting the maze) without being told whether the intermediary steps are good or bad. A practical example of reinforcement learning would be the adaptive routing of packets within a network.

Supervised learning. In supervised learning the algorithm is provided with a set of labelled examples (training data) and must create a function that maps the input vector to the desired output. Most classification algorithms use supervised learning and it is the form referred to throughout this thesis. For example in medical diagnosis the algorithm might be provided with training data consisting of patient attributes such as age, weight, height etc., labelled according to whether they have liver disorder; for letter recognition the attributes might consist of a greyscale rating from 0-255 for each cell in an 8x8 grid, with labels A-Z. In both cases the algorithm must learn a function that maps the input attributes to the desired class label.

The major machine learning algorithms used in supervised learning are as follows:

Decision trees such as C4.5 look for a feature and value that best split the training data, so that ideally all instances in one group are of the same class (e.g. all patients whose weight is greater than 115kg have liver disorder). This is repeated iteratively, splitting the data at each node until all instances are in 'leaf' nodes with a uniform class label. See section 2.1.2 for details of the C4.5 decision tree algorithm.

[4] `http://www.clusty.com`

Instance based classifiers such as k-nearest-neighbour keep the training data in memory and when asked to classify a new instance look for the training instance most similar to it, assigning the new instance to the same class. See section 2.1.3 for details of IBk, a k-nearest-neighbour algorithm.

Probabilistic models are often based on Bayes' theorem. For instance the Naïve Bayes classifier estimates the probability that a piece of fruit is a grape based on the probabilities that it is green, ovoid in shape and a centimetre across (it is *naïve* because it assumes that the probability of being green is independent of the probability of being ovoid). See section 2.1.4 for more on Naïve Bayes. Probabilistic graphical models such as Bayesian networks (including Hidden Markov Models[5]) are capable of representing causal relations between variables (the notion that rain means things get wet) as well as their conditional independence via directed acyclic graphs.

Artificial neural networks (commonly just "neural networks") attempt to simulate the structure of biological neural networks, in practice functioning as non-linear statistical models. They consist of a group of interconnected artificial neurons (mathematical models) that adapt throughout the learning process by adjusting the relative weights of the connections. Types of network such as Multi-Layer Perceptron (MLP) and Radial Basis Function (RBF) are used in supervised learning (though other forms such as Self Organizing Maps can be used for unsupervised learning).

Support vector machines (SVMs) employ a linear classifier. Linear classifiers attempt to find a straight line that divides instances of one class from another (e.g. if the data has two classes A and B and attributes x and y these can be plotted on a graph - a linear classifier will attempt to draw a straight line that leaves all instances of class A on one side and all of class B on the other, as illustrated in figure 2.1. In three dimensions the line becomes a plane, in higher dimensions a hyperplane)[6]. SVMs use a "ker-

[5] A Hidden Markov Model assumes that the system being modelled is a Markov process (a memoryless system in which the future state of the system depends entirely on its current state, and previous states are irrelevant) with an unknown state.

[6] The dimensions in this case are not physical, but a purely mathematical construct. There is one dimension for each attribute in the dataset.

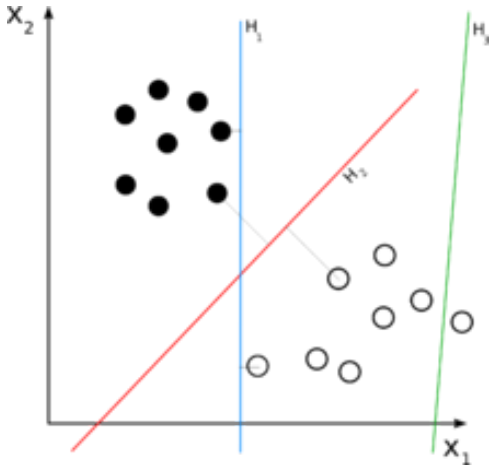


Figure 2.1: Hyperplanes for a 2-class linear classifier. H3 doesn't separate the 2 classes. H1 does, with a small margin and H2 with the maximum margin. Figure reproduced from Wikipedia `http://en.wikipedia.org/wiki/Linear_classifier`.

nel function" to project the data into a higher dimensional space where it may be more easily separated into different classes by a hyperplane (the kernel function may be non-linear, so that even if the data is not linearly separable in the original input space it may be in the higher-dimensional space into which it has been transformed).

The more complicated classifiers such as SVM and ANNs operate a form of implicit feature construction within the algorithm (the transformation of data into a higher dimensional space by kernel functions in SVM, and the combination of multiple inputs in hidden neurons in an ANN). This contributes to the high level of success of such classifiers, but it is often difficult to extract information about the relationships between attributes used by the algorithm. By pre-processing data the GAP algorithm presented in this thesis aims to provide simpler classifiers such as decision trees, instance based classifiers and probabilistic models with the ability to construct new features; with the added advantage of making explicit information about the relationships between attributes in the data.

2.1.1 Inductive bias

All Machine Learning algorithms have an inherent bias toward a particular style of learning, and indeed need an appropriate bias to "make

an inductive leap". Mitchell defines bias as

> Any basis for choosing one generalization over another, other than strict consistency with the observed training examples. [125]

Mitchell has characterised supervised learning as searching through a very large space of all possible hypotheses (the Hypothesis Space) "to locate the hypothesis that is most consistent with the available training examples" [124]. The search through this hypothesis space is guided by the inductive bias of the algorithm that determines the direction of the search within the hypothesis space (for instance ID3 attempts to maximise information gain at each node of the decision tree), while the subset of the hypothesis space accessible to the algorithm is also limited by inductive bias (for instance the choice of the set of features that have been made available to it). Without an inductive bias of some form machine learning algorithms would be unable to generalise to unseen training examples (that a small green ovoid fruit is a grape tells you nothing, on its own, about small red ovoid fruit) - this is the No Free Lunch theorem [200]. There are a number of different types of inductive bias, all machine learning algorithms will possess at least one. A few examples follow.

Conditional independence. A bias of the Naïve Bayes classifier, the assumption that all features are conditionally independent of one another.

Maximum margin. A bias of Support Vector Machines. Linear classifiers try to draw a boundary line with the maximum width (the greatest distance between the line and the nearest instance). The assumption is that the best division between classes is the one that separates instances by the greatest distance.

Nearest neighbours. A bias of k-nearest-neighbour. The assumption that all instances within a small neighbourhood of the feature space share the same class.

Minimum description length. This is the principle of *Occam's Razor*, that simpler explanations are to be preferred over complicated

ones.[7] Decision tree algorithms use this bias by preferring shorter trees to longer ones.

Minimum features. Features should be ignored unless there is positive evidence that they are beneficial to classification. This is the assumption underlying feature selection algorithms.

2.1.2 Decision Trees using C4.5

The decision tree learning algorithm ID3 was introduced by Quinlan in the late 1970s [156], and was followed by its successor C4.5 [158]. The basic algorithm for ID3 is presented below and mention is made of some of the improvements incorporated into C4.5. It should be noted that the algorithm presented in this thesis treats C4.5 and other classifiers as a 'black box', no attempt is made to modify its operation and no understanding of it is required to follow the rest of this thesis.

ID3 works by constructing a decision tree from the top down using a greedy search (the 'best' attribute is always selected to split the data) and never backtracking (once the decision is made to test a particular attribute this decision is never reconsidered and the attribute/value never tested further down the tree). The primary question for ID3 is how to select an attribute to test at each node, beginning with the root node.

ID3 makes this decision by considering the *information gain* of each attribute. Information gain measures the reduction in *entropy* associated with splitting the data according to a particular attribute (in its basic form ID3 can only consider discrete value attributes, though C4.5 is extended to be able to consider continuous numbers). Entropy is a measure used in information theory to measure the level of 'confusion' present in a set of data. Quinlan states that "when applied to the set of training cases, $entropy(S)$ measures the average amount of information needed to identify the class of a case in S" [158]. E.g., for a binary class if all instances in a dataset have the same value (either all positive or all negative) then the entropy for that dataset is 0. If there are exactly the same number of instances with positive values as those with negative then the entropy is 1.

[7] "...other things being equal – the simplest hypothesis proposed as an explanation of phenomena is more likely to be the true one than is any other available hypothesis, that its predictions are more likely to be true than those of any other available hypothesis, and that it is an ultimate a priori epistemic principle that simplicity is evidence for truth." [180]

In a decision tree, the attribute with the highest information gain at a decision node can be thought of as the one that best splits the dataset into subsets with the greatest 'purity', so that child nodes are most likely to contain instances that all have the same class. At each node in the decision tree the information gain is calculated for every remaining attribute and the attribute with the highest gain is selected to split on at that node. The new node has one branch for every value of the chosen attribute and the dataset is partitioned according to the values for that attribute. The chosen attribute will never again be considered for testing within this subtree and each branch will only look at the relevant partition of the dataset when considering what attribute to test next. The process stops when every instance in the dataset at a particular node has the same class value (i.e. the entropy for the node is 0). These are the 'leaf' nodes of the tree and represent a classification.

C4.5 introduces a number of extensions and improvements to ID3, the first of which is to replace information gain as the selection criteria with the *Information Gain Ratio* criterion. Recall that the attribute with the highest information gain is the one whose child nodes are most likely to contain instances all having the same class. This leads the information gain criterion to prefer attributes with many values (many values = many child nodes = many subsets, and so more chance of the instances all having the same class). "The bias inherent in the gain criterion can be rectified by a kind of normalisation in which the apparent gain attributable to tests with many outcomes is adjusted" ([158], p. 23). Gain Ratio achieves this by dividing the gain term by a *split info* term that represents the potential information generated by splitting the dataset into subsets (very similar to the gain term, but not limited to information relevant to classification). The choice of attribute suggested by the gain ratio measure is subject to the limitation that each child node must have at least 2 training cases (there is a parameter setting to change the default minimum of 2). This avoids near-trivial splits of the training data and is useful when the amount of training data is small.

Continuous attributes are handled by choosing a threshold value and using binary tests with outcomes of the form $a <= b$ and $a > b$. The training cases are sorted on the values of continuous attribute A

$\{v_1, v_2, \ldots, v_m\}$ and any threshold between v_i and v_{i+1} is considered as a possible binary attribute. The restriction that any attribute can only be considered once for a test is applied to the binary split so that other splits of that attribute can be considered in the subtree. For example if the split at a node is on an attribute *age* ($<= 75$, > 75), splits on other values of age (e.g. $<= 60$, > 60) are still allowed in the subtrees of that node.

Missing values are handled by assuming that unknown values are distributed probabilistically according to the relative frequency of known values. An instance with an unknown attribute is divided into fragments with weights proportional to these relative frequencies (so a single case with unknown values may follow many paths in the tree).

As it is possible to construct overly complicated trees that perform perfectly on the training data but poorly on test data (trees that 'overfit' the training data), C4.5 includes methods to 'prune' decision trees using *reduced-error pruning*. It does this by replacing a sub tree with a single node (or with one of its branches) - resulting in smaller trees that are more accurate on test data. The rationale for pruning is: "Start from the bottom of the tree and examine each nonleaf subtree. If replacement of this subtree with a leaf, or with its most frequently used branch, would lead to a lower predicted error rate, then prune the tree accordingly" ([158], p. 39). C4.5 uses a very pessimistic, but statistically unsound[8] method for estimating the error rate by treating the number of cases n at a node as a sample of which e are incorrectly classified, from which the true error rate of the whole population can be estimated. The probability of error has a distribution summarised by a pair of confidence limits - the upper limit can be found from the binomial distribution for a given confidence level. C4.5 uses this upper limit as the predicted error rate. The default confidence level of 0.25 can be changed using C4.5's Confidence Factor parameter.

Multivariate decision trees

A limitation of C4.5 and other *Univariate* decision trees is that they can only consider a single attribute at any given node. As well as lim-

[8] In Quinlan's own words the reasoning behind his choice of error rate estimate 'does violence to statistical notions of sampling and confidence limits ... Like many heuristics with questionable underpinnings, however, the estimates that it produces seem frequently to produce acceptable results' [158] p. 41.

iting the solutions that can be discovered (a trivial example being that C4.5 cannot generalise to distinguish between squares and rectangles given only length and width) this can lead to a number of problems (as described by Liu and Setiono [116]): (1) subtrees being replicated in different branches of the tree, (2) features that are tested more than once along a particular branch (note that this can only happen with continuous attributes), and (3) the dataset becoming increasingly fragmented as a branch is traversed. This can lead to difficulty constructing succinct trees and being over-sensitive to noise in the dataset.

Multivariate decision trees [39] are designed to overcome these problems by considering multiple attributes at a single decision node. "Testing on a single feature is equivalent to finding an axis-parallel hyperplane to partition the data, while testing on multiple features we try to find an oblique hyperplane" [116]. The challenge is to find the right combination of features at each node, and the right coefficients with which to combine them (i.e. to determine the dimensions and orientation of the hyperplane). An exhaustive search is often impossible in practice so a number of approaches have been suggested (including forward selection and backward elimination to find the features; Recursive Least Squares or simulated annealing to determine the coefficients), none conclusively better than the others. Multivariate decision trees tend to be much smaller than their univariate counterparts, though the individual nodes are considerably more complex.

Where univariate trees naturally work on symbolic attributes and continuous attributes must be converted into sets of binary tests ($a <= b$), multivariate trees naturally work on numeric attributes and it is symbolic attributes that must be converted into a set of numeric (boolean) attributes, one for each symbolic value.

2.1.3 Nearest Neighbour algorithm: IBk

IBk [3] is an *instance-based learning* algorithm developed by Aha, Kibler and Albert, which extends the nearest neighbour classifier. Instance-based learning assumes that similar instances have similar classifications, and thus an instance is assigned the same class as its closest neighbour. The simplest way to visualise this is to assume a dataset with two real-valued attributes x and y. All known instances

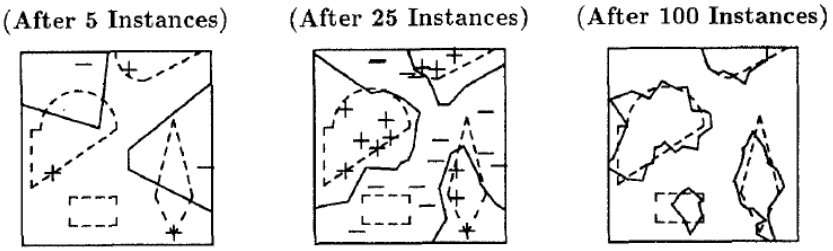


Figure 2.2: "The extension of IB1's concept description, denoted by the solid lines, improves with training. Dashed lines delineate the four disjuncts of the target concept. Positive (+) and negative (-) instances are shown where possible."' Figure reproduced from [3].

are plotted on a graph. When a new instance is to be classified it is plotted on the graph and assigned the same class as the nearest existing instance (identified as the one from which the shortest straight line to the new instance can be drawn). (This assumes that $k = 1$; if $k = 3$ [for example] then the new instance would be classified according to the majority class of the nearest 3 existing instances). This graph represents a *concept description*, a function mapping an instance to a classification. The *concept boundary* can be thought of as a line plotted on the graph indicating where the prediction changes from one class to another. If a new instance is plotted on one side of the line its predicted class will be class a, on the other side class b. This is illustrated in figure 2.2.

IBk consists of 3 components:

1. *Similarity Function.* This computes the similarity between the instance to be classified and the existing instances in the concept description.

2. *Classification Function.* Takes the output of the similarity function and the classifications of the instances in the concept description, producing a classification for the instance (the classification of the single most similar instance when $k = 1$).

3. *Concept Description Updater.* Holds the classifications of the instances and decides which instances should be added to the concept description (all instances in the simplest case).

The similarity metric used by IBk is shown in equation 2.1, where n is the number of attributes. For numeric attributes, $f(x_i, y_i) = (x_i -$

$y_i)^2$; while for boolean and discrete attributes $f(x_i, y_i) = (x_i \neq y_i)$. Numeric attributes are normalised. Missing attributes are assumed to be maximally different from present values, unless both are missing in which case $f(x_i, y_i) = 1$.

$$\text{Similarity}(x, y) = -\sqrt{\sum_{i=1}^{n} f(x_i, y_i)}, \tag{2.1}$$

The simplest form of IBk is IB1, which stores and uses all instances in its concept description (the boundary of which is identified by the dashed lines in figure 2.2). In describing how IBk approximates the target concept, Aha et al. use the following definitions, illustrated in figure 2.3:[9]

Definition: The ϵ-ball about a point x in $\Re^n$ is the set of points within distance ϵ of x: $\{y \in \Re^n | distance(x, y) < \epsilon\}$. (The set of real numbers within distance ϵ of x) [10].

Definition: Let X be a subset of $\Re^n$ with a fixed probability distribution. A subset S of X is an $< \epsilon, \gamma >$-net for X if, for all x in X, except for a set with probability less than γ, there exists an s in S such that $|s - x| < \epsilon$. (An $< \epsilon, \gamma >$-net S can be thought of as an approximation of the set X).

Definition: For any $\epsilon > 0$, let the ϵ-core of a set C (the set of all points within a concept boundary) be all those points of C such that the ϵ-ball about them is contained in C.

Definition: The ϵ-neighbourhood of C is defined as the set of points that are within ϵ of some point of C.

Definition: The set of points C' is an $< \epsilon, \gamma >$-approximation of C if, ignoring some set with probability less than ϵ, it contains the ϵ-core of C and is contained in the ϵ-neighbourhood of C.

Aha et al. demonstrate how IB1's concept description nearly always converges to a set "which, except for a set of probability less than γ, lies between the ϵ-core and the ϵ-neighbourhood of the concept." [3] IB1 is the version of IBk used with the GAP algorithm[11] (with $k = 1$). There are two other forms: IB2 and IB3.

[9] For assistance in understanding any of the mathematical symbols found in this thesis, the reader is directed to `http://en.wikipedia.org/wiki/Table_of_mathematical_symbols`.

[10] Where x is a vector of n real numbers. However for the purposes of these definitions it may be simpler to consider x to be a single real number.

[11] IB1 is used simply because it is the implementation you get if you call the IBk constructor with no arguments in Weka.

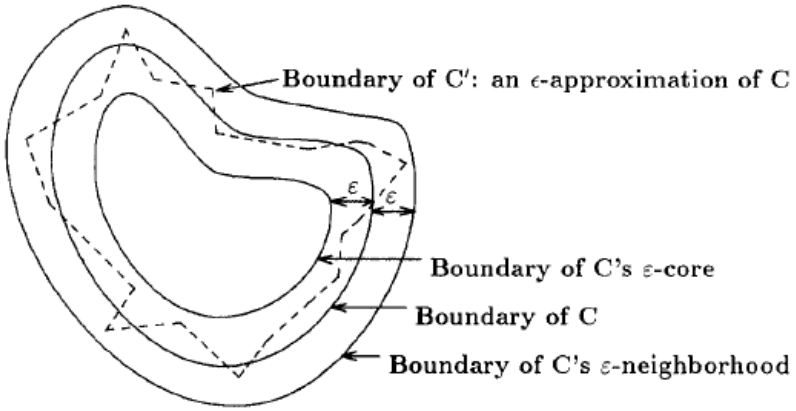


Figure 2.3: Illustration of IBk concepts and definitions. Figure reproduced from [3].

IB2 attempts to reduce the storage demands of IB1 (which must remember every training instance), relying on the fact that only a subset of the instances are required to identify where the concept boundary lies and so only these instances need to be saved. (These are the instances between the ϵ-core and the ϵ-neighbourhood of C). As it is impossible to identify this set of instances with certainty without full knowledge of the concept boundary, Aha et al. approximate it using the set of instances mis-classified by IB1 (relying on the intuition that mis-classifications occur because the instance lies near the concept boundary, within the ϵ-neighbourhood but outside the ϵ-core). While this does reduce the storage requirements it also makes IB2 more susceptible to noise in the dataset (already a weakness of IB1) - because noisy instances are precisely those most likely to be mis-classified by IB1.

IB3 extends IB2 to address this weakness by maintaining a classification record for each instance (a count of subsequent instances classified/misclassified by reference to this instance) and using significance tests to identify and discard those instances believed to be noisy. During training the classification records of instances already seen are updated with the results of classifying the latest instance. IB3 then accepts an instance if its accuracy is significantly greater than observed frequency of its class; discarding the instance if its accuracy is significantly less. The confidence level required for accepting an instance (90%) is higher than that required to reject it (75%) on the grounds that IBk should be

quicker to reject poorly performing instances than to accept that one is truly accurate.

2.1.4 A Bayesian Probability algorithm: Naïve Bayes

A Naïve Bayes algorithm [88] classifies instances according to Bayesian probability (which allows the calculation of $p(h|d)$, the probability that a hypothesis h is true given data d), in a very efficient manner (it is far quicker than either IBk or C4.5). It is *naïve* because it makes two assumptions: that predictive attributes are conditionally independent of one another given the class;[12] and that there are no hidden attributes that influence the prediction. The probability of an instance with observed variables x having a class label c is then defined by equation 2.2. This defines the probability that the class C of the instance will have the label c given that the attribute vector X has particular values x ($X = x$ such that $X_1 = x_1 \wedge X_2 = x_2 \wedge \ldots X_k = x_k$).

$$p(C = c|X = x) = \frac{p(C = c)p(X = x|C = c)}{p(X = x)} \tag{2.2}$$

Because the attributes are assumed to be conditionally independent, $p(X = x|C = c)$ becomes:

$$p(X = x|C = c) = \prod_i p(X_i = x_i|C = c)$$

For nominal attributes $p(X = x|C = c)$ takes the form of a single real-valued number between 0 and 1, representing the probability that attribute X will have the value x when the class is c. Estimating the probability that the attribute will take on each value given a particular class c is performed by dividing the count of instances with that value and class c by the total number of instances with class c in the sample.

Numeric attributes are modeled by a continuous probability distribution, frequently (and as used with the GAP algorithm) using the normal (or Gaussian) distribution, which can be represented in terms of the mean (μ) and standard deviation (σ) of the attribute. Thus $p(X = x|C = c) = g(x; \mu, \sigma)$, the probability density function for a

[12] For example if a dataset consists of attributes A, B and class C, Naïve Bayes assumes that the conditional probability distribution of A is independent of the conditional probability distribution of B, given class c. The conditional probability distribution of A is simply the probability distribution of A under the condition that $C = c$ (Class C has the particular value c).

Gaussian distribution shown in equation 2.3. The estimates for the mean and standard deviation of a normal distribution are the mean and standard deviation of the sample (i.e. for a particular class c, the mean and standard deviation of the attribute among those instances having class c).

$$g(x;\mu,\sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \tag{2.3}$$

Bayesian Networks

The assumption of conditional independence by Naïve Bayes is what makes it possible to compute the posterior probability of a given class for a set of attributes, but it is not often a realistic assumption (though it continues to be successful on a number of datasets despite this). The success of Naïve Bayes despite the unrealistic assumption of independence motivated researchers such as Friedman et al. [69] to investigate the use of Bayesian Network Classifiers to see if the performance could be improved by avoiding unwarranted assumptions of conditional independence of variables. Bayesian networks (of which Naïve Bayes is a special case) are Directed Acyclic Graphs in which the vertices or nodes are the attributes of the dataset (including the class) and the edges are used to represent conditional dependencies - nodes that are not connected by an edge are conditionally independent of one another. Bayesian networks have been shown to improve on the performance of Naïve Bayes when an appropriate network is used, the problem becomes one of learning the most appropriate network for the dataset at hand, for which a number of techniques exist (including statistical techniques such as hill climbing, simulated annealing and tabu search, and also other machine learning algorithms such as tree augmented naïve bayes [69] or Full bayesian networks [178].

2.2 Evolutionary Algorithms

Evolutionary algorithms have been an area of active research since Nils Aall Barricelli's often overlooked work on simulating evolution in the early 1950s [17]. Practical applications of evolutionary algorithms started appearing in the early 1970s with Rechenberg's (independently

developed) Evolution Strategies [160] which dealt with parameter optimisation for hydrodynamic problems like shape optimisation of a bent pipe and of a flashing nozzle.

Evolutionary algorithms attempt to simulate the processes of natural evolution: evaluation, selection, recombination and mutation. These processes act on a population of individuals, each of which represents a search point in the space of potential solutions to a given problem, and are guided by a notion of an individual's 'fitness' - its performance on the problem. The population is usually initialised at random and undergoes a cycle of selection, recombination and mutation. Recombination allows the exchange of information between individuals and mutation introduces new (or re-introduces old) information. Fitter individuals are selected to reproduce more often and over time the quality of the population increases. The outline of a typical evolutionary algorithm is shown below.

Algorithm 2.1 A Typical Evolutionary Algorithm

1. Let t = 0;
2. initialise population P(t);
3. evaluate P(t);
4. until (done):
   (a) t = t + 1;
   (b) P(t) = select P(t-1);
   (c) replicate P(t);
   (d) mutate P(t);
   (e) evaluate P(t);

Here t is the generation number and $P(t)$ is the population at generation t. The termination criterion depends on the implementation and may use the number of generations elapsed, CPU time elapsed, absolute or relative progress, etc.

2.2.1 Genetic Algorithms

Genetic Algorithms were mainly developed in Michigan by Holland [83] and were used on parameter optimisation problems. They traditionally operate on populations of bit-strings - binary strings of a fixed length that represent parameters to be optimised (switched on or off). The

bits in the string are often referred to as *alleles* with possible values 0 and 1. Genes are usually single bits with a particular value. The term *chromosome* usually refers to a particular individual in the population (which generally consists of 50 to 1,000 individuals). Two individuals in a population of bit-strings of length 5 might look like this:

```
10001
01101
```

A standard GA works according to the algorithm outlined for Evolutionary Algorithms above.

Initialisation: An initial large, random population of bit-strings is created. A large population is needed to scatter search points widely throughout the search space. Initialisation is followed by repeated cycles of the next three stages.

Evaluation and selection: The fitness of each individual is calculated and individuals are selected to reproduce according to their fitness. The simplest method of selection is 'roulette-wheel sampling' (fitness-proportional selection) "which is conceptually equivalent to giving each individual a slice of a circular roulette wheel equal in area to the individual's fitness" [124]. Another method is tournament selection in which two or more individuals are chosen at random from the population. With some probability p the fittest of the individuals is chosen, otherwise one of the individuals is chosen at random. Tournament selection can overcome a problem with roulette wheel selection in which the most fit individuals come to dominate the population in early generations, leading to a stagnant population in later generations.

Replication (crossover): Pairs of parents are selected, and two offspring are created by crossover at a random point in the bit-string with probability Pc (the crossover rate, generally in the region of 0.6). If crossover doesn't take place, each offspring is an exact copy of its parent. The pair of individuals shown above might be crossed after the second bit to produce these offspring: (the "|" symbol indicating the crossover point)

Parent	Offspring
10|001	10101
01|101	01001

This form of replication is known as *single-point* crossover. Another form is *two-point* crossover in which two random points in the bit string are chosen and the middle sections are swapped between the offspring:

Parent	Offspring
1|00|01	11101
0|11|01	00001

A third form of crossover is *uniform* crossover, in which a probability (known as the mixing ratio) is used to decide which parent will contribute to which offspring *separately for each gene*. If the mixing ratio is 0.5 then approximately half the genes will come from the first parent, half from the second: (the subscripts indicate which parent each gene is from)

Parent	Offspring
10001	$1_1 1_2 0_1 0_1 1_2$
01101	$0_2 0_1 1_2 0_2 1_1$

Mutation: Each bit in the offspring is flipped with a small probability (generally around 0.01 - 0.001). Mutation has a much smaller role than replication and is often considered a background operator (this is in contrast to evolution strategies in which mutation is the primary operator). Its main purpose is to prevent the permanent loss of an allele at a particular position in the string, which would otherwise reduce the search space, for instance if the parents shown above are the only members of the population there is no other way to get a 1 in the fourth bit or a 0 in the fifth.

A single offspring	11101
mutated in the fifth bit	11100

The process continues through cycles of evaluation, selection, crossover and mutation until the termination criteria are met - generally that a fixed number of generations has been run.

One variation on the process of selecting the next generation is frequently employed: *elitist selection*, or *elitism*, which involves ensuring that the fittest member(s) of the population survive into the next generation so that the fittest individuals are not lost by random chance. This ranges from copying the single fittest member of the current population unchanged into the next generation (partial elitism); to creating the child generation as described above, then keeping the fittest half of the combined parent and child generations (full elitism).

Schemata

The essentials of GA theory are based on the notion of *schemata*. A *schema* has an alphabet of three symbols (0, 1, *) and the same length as the bit-strings in the population.

Instances of the schema are all strings that match it on every position other than * (e.g. all the bit-strings shown so far are instances of the schema $***0*$).

The **defining length** of the schema $\delta(S)$ is the distance between the first and last fixed positions (e.g. $\delta(*0**1) = 3$).

The **order** of the schema $\sigma(S)$ is the number of fixed positions (e.g. $\sigma(*0**1) = 2$).

The **fitness** of a schema is the average fitness of all instances of it in the population.

Holland [83] uses the *schema growth equation* to describe the expected number of instances of a schema in the next generation, and states the result as the

Schema Theorem: Short, low-order, above-average schemata receive exponentially increasing trials in subsequent generations of a genetic algorithm.

This means that GAs explore the search space by short, low-order schemata. In a population of n strings there are many more than n schemata represented (up to 2^n), and Holland has shown that at least n^3 of them are processed usefully. These are selected at an exponentially increasing rate and are not easily disrupted by crossover and mutation (unlike schemata with high order and long defining length).

Building Block Hypothesis: "A genetic algorithm seeks near optimal performance through the juxtaposition of short, low-order,

> high-performance schemata, called the building blocks" [123]. Building blocks and their combination by crossover to form longer and longer useful substrings are the primary mechanism in a GA.

Although the schema theorem has been used for many years to explain why GAs work, it came under widespread criticism in the 1990s (e.g. by Altenberg [7], who states that the schema theory fails to demonstrate that recombining schemata with above-average fitness produces offspring with above-average fitness). Criticism often focused on the limitation of the schema theorem to predicting the number of instances of a schema in the next generation only - it cannot easily predict behaviour over multiple generations. In particular,

> "schema theorems give only a lower bound for the expected value of the number of instances of schema H at the next generation ... In addition, since the schema theory provides only a lower bound, some people argue that the predictions of schema theory are not very useful even for a single generation ahead." [147]

Such criticism led theorists to investigate alternatives to the schema theorem such as Markov chains [24], as well as further development of schema theory by Stephens and Walbroek [177] which "provides exact information about the distribution of the population at the next generation in terms of quantities measured at the current generation, without having to actually run the algorithm". [151].

Integer and real-value encodings

When genetic algorithms are used to optimise integer-valued parameters, the integers are usually encoded in binary first (e.g. four bits can be used to represent numbers between 0 [0000] and 15 [1111]). However this can result in premature convergence of the population at 'hamming walls',[13] where a small change in the integer value requires many changes in the binary encoding (for instance the binary representation of 7 [0111] is very different to the binary representation of 8 [1000], requiring every bit to change). For this reason integers are often encoded

[13]The *hamming distance* between two strings of equal length is the number of substitutions required to change one string into the other. So the hamming distance between 001$\underline{1}$ and 001$\underline{0}$ is 1, while the hamming distance between $\underline{011}$1 and $\underline{100}$1 is 3.

using *Gray coding* (named after its inventor, Frank Gray), in which the binary representation of adjacent integers differs only in a single bit, so that small changes in integer values are readily accomplished by small mutations.

A form of genetic algorithm for optimising real-valued parameters uses arrays of real numbers instead of bit strings. According to the schema theorem shorter alphabets should produce better results, but despite this good results have been achieved using real-valued GAs. This requires a different form of crossover operator, such as arithmetic crossover:

$$\begin{aligned} Offspring1 &= a * Parent1 + (1 - a) * Parent2 \\ Offspring2 &= (1 - a) * Parent1 + a * Parent2 \end{aligned}$$

Where a is a random weight in the range 0-1, determined for each crossover operation. In the following example a is set at 0.7:

Parents	Offspring
`[0.00] [0.30] [1.40] [0.20] [7.40]`	`[0.30] [0.36] [2.33] [0.17] [6.86]`
`[1.00] [0.50] [4.50] [0.10] [5.60]`	`[0.70] [0.40] [2.98] [0.15] [6.84]`

2.2.2 Genetic Programming

Genetic Programming (GP), primarily developed by John Koza in the late 1980s and 1990s, extends the concept of genetic algorithms by evolving computer programs (in the form of program trees) instead of bit strings. In order to apply GP to a particular domain, appropriate functions and terminals (the nodes and leaf nodes of the tree) must be specified. For instance the functions could be arithmetic functions {*, /, +, -, %} (or trigonometric or logarithmic), boolean {and, not, or, xor}, or program control statements such as `if`, `goto` etc.; while the terminals could be primitive attributes or numeric constants {2, 3, π, etc.}. The chosen function set should be *sufficient*, in that it should be possible to express a solution to the problem using only those functions. A limited set of simple functions such as the arithmetic and boolean functions listed above is often sufficient to solve many problems (or at least to provide a good approximation to the solution), while too many functions can make the search space too large for the easy discovery of

solutions.

> "Another important property of the function set is that each function should be able to handle gracefully all values it might receive as input. This is called the *closure* property. The most common example of a function that does not satisfy the closure property is the division operator [which] cannot accept zero as input. Instead ... one may define a new function called protected division [which] is just like normal division except for zero denominator inputs [in which case it returns] a very big number or zero. All functions ... must be able to accept all possible inputs because if there is any way to crash the system, the boiling genetic soup will certainly hit upon it." [16]

Figure 2.4 shows an example of a GP tree, equivalent to the expression $\frac{(a/w)+l}{a/w}$, in which the attributes are a (Area), l (Length) and w (Width) (these are the attributes of an example 'shapes' dataset further discussed in section 3.2); and the function set is {*, /, +, -, %}.

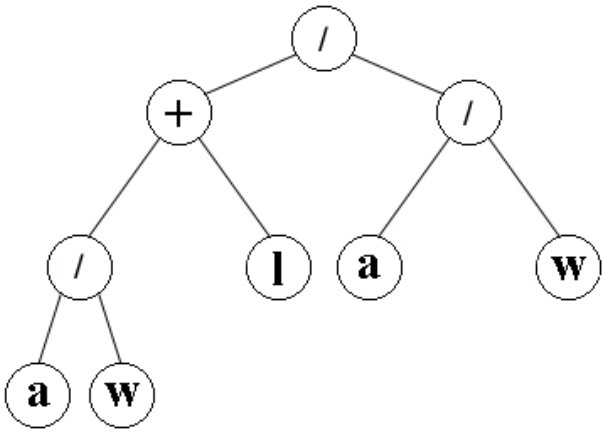


Figure 2.4: Example GP tree, equivalent to the expression $\frac{(a/w)+l}{a/w}$

Like GAs, a GP algorithm maintains a population of individuals (program trees) which are operated on at each generation by the processes of evaluation, selection, replication, and mutation.

Initialisation: An initial large, random population of program trees is created. One of the main parameters in a GP run specifies the

maximum size of a tree, usually specified as the maximum depth but sometimes as the maximum number of nodes. The depth of a node is defined as "the minimal number of nodes that must be traversed to get from the root node of the tree to the selected node" [16]. The tree shown in figure 2.4 has a depth of 4, and a count of 9 nodes. The size of the program tree required to find a solution is problem dependent and important to get right. If the maximum size of trees is too low a solution will never be found; too high and, as with too large a function set, the search space will be too big to easily find a solution (this will be compounded by the *bloat* of program trees - see section 2.2.2).[14] There are two methods in general use to initialise a tree structure: *full* and *grow* [97]. In the grow method, nodes are selected at random from the combined set of functions and terminals (except the root node, which uses only functions). This causes some branches to end before the maximum depth has been reached (a branch stops when a terminal has been selected, becoming a leaf node); causing the tree to be irregular in shape. Figure 2.4 is an example of a tree initialised with the grow method. Using the full method nodes can only use functions until the branch has reached the maximum depth, at which point the node can only choose a terminal. This means that every branch reaches the full depth.

The most popular way to ensure diversity in the initial population is to combine the full and grow methods using the **ramped half-and-half** technique. Using this technique the population is divided equally into groups of trees to be initialised with depths between 2 and the maximum depth (e.g. if the maximum depth is 6, there will be 5 groups initialised with depths of 2, 3, 4, 5, and 6). Within each depth group, half the individuals are initialised with the full method, half with grow. This means that the initial population tends to have a high proportion of symmetric ('bushy') trees, more likely to produce good solutions for problems where solutions should treat all inputs with equal weight than for problems exhibiting a degree of asymmetry (e.g. where solutions are deep but narrow or some inputs should have more weight (be closer to

[14] There are also techniques for adapting the maximum allowed size or depth dynamically during the run, useful when the appropriate program size is not known. See for example [169, 170].

the root node) than others).

A number of alternative initialisation techniques have been proposed, such as Langdon's *ramped uniform initialisation* [106] which accepts a range of tree *sizes* (rather than maximum *depth*) and creates equal numbers of trees for each size in the range (this also allows the user to choose to avoid including trees that are too small to be useful in the initial population). A new initialisation technique that seems particularly suited to producing trees that construct features rather than programs is presented in section 3.3.1.

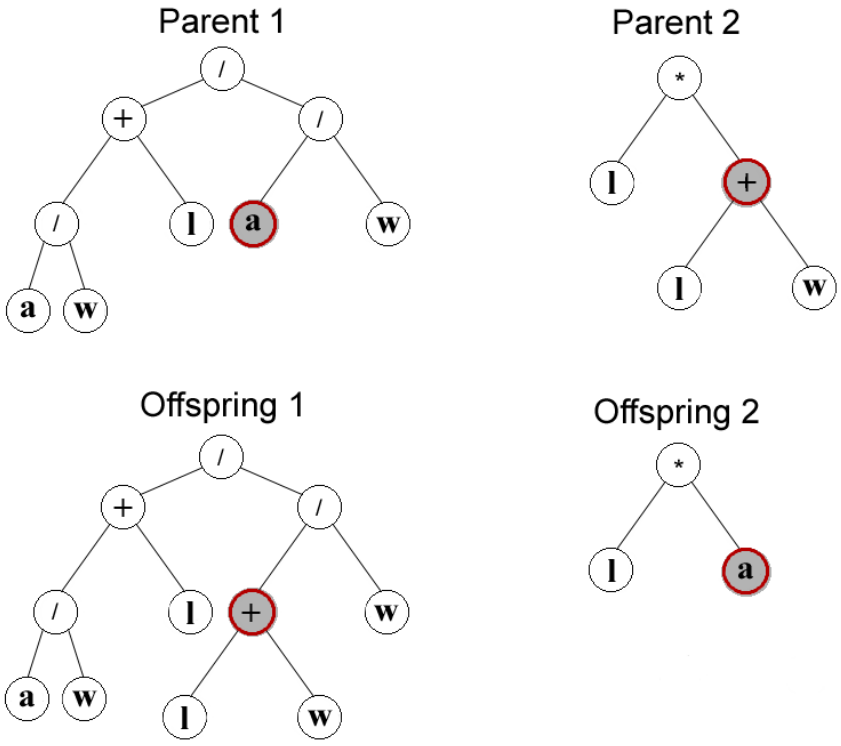


Figure 2.5: GP crossover example

Evaluation and selection: The fitness of individuals is generally determined by executing the program on a set of training data (e.g. for a classification task using the percentage correctly classified, or using the absolute value of the difference between the output of the program and the correct value for regression task). The fitness is then used to probabilistically select individuals for replication by crossover (using the same range of selection techniques as for

genetic algorithms). In Koza's original experiments [97] "10% of the current population is retained unchanged in the next generation. The remainder of the new generation is created by applying crossover to pairs of programs from the current generation, again selected probabilistically according to their fitness" [124].

Crossover: Crossover in GP is achieved by replacing a randomly chosen subtree in one individual with a randomly chosen subtree from another individual, as illustrated in figure 2.5. A number of different crossover operators have been proposed for GP, which generally aim to cross subtrees in the same position in both parents (*homologous* crossover), such as one-point crossover [149] where the parent trees are analysed to identify areas with the same structure (topology), from which a crossover point common to both parents is selected and the corresponding subtrees are swapped as normal (illustrated in figure 2.6[15]).

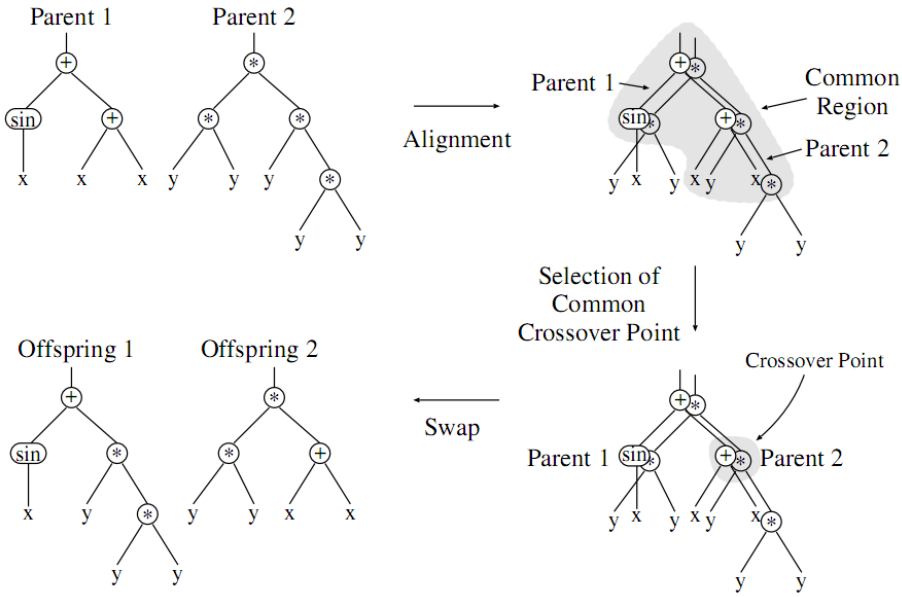


Figure 2.6: One-point crossover example. Image reproduced from [151].

Mutation: Although Koza did not use the mutation operator in his original experiments, it can be applied by replacing a randomly chosen subtree with a new randomly generated program tree, as

[15]Figure 2.6 is reproduced from 'A field guide to genetic programming' [151] which is licensed under the Creative Commons Attribution- Noncommercial-No Derivative Works 2.0 UK: England & Wales License (see `http://creativecommons.org/licenses/by-nc-nd/2.0/uk/`).

illustrated in figure 2.7. It is also possible to employ point mutation, in which the function of a randomly chosen node is replaced by a random function (or if it is a terminal node it is replaced by a randomly chosen terminal), so that the structure of the tree is not mutated. Due to the large number of functions and terminals in a GP population it is rare to lose all instances of a particular function or terminal, and so mutation is thought to play a less important role than in a conventional genetic algorithm.

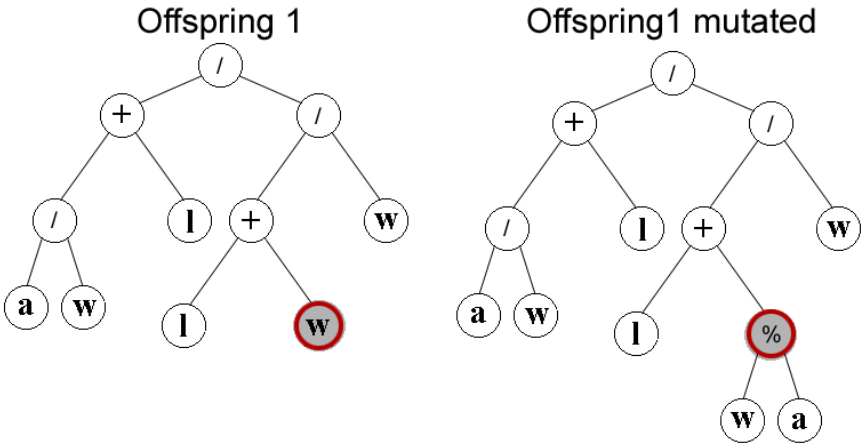


Figure 2.7: GP subtree mutation example

Automatically Defined Functions

Automatically defined functions (ADFs) [96] serve the same purpose in genetic programming as subroutines and functions do in human programming. To use ADFs, Koza designed a tree structure that consisted of one (or more) Result Producing Branch and one or more ADF branches. The ADF branches are made available as part of the function set for the result producing branch, which can reuse an ADF branch by including it as a node at many different points in the branch. ADFs can sometimes also refer to each other, but in order to prevent infinite recursion ADF branches are often ordered and prevented from referring to any ADF that comes after (so that ADF_2 can refer to ADF_0 and ADF_1, but ADF_1 can only refer to ADF_0).

Usually the number of ADFs and the number of their arguments must be specified as parameters at the start of a run, but Koza also proposed *architecture altering operations* [98] to add new ADF branches,

clone them and alter the number of parameters. ADFs impose restrictions on the crossover operation, so that during crossover a subtree selected from an ADF can only be swapped with a subtree from another ADF in the same position, and cannot cross the boundary between ADFs (or between an ADF and the RPB - so that a subtree from ADF_0 can only be swapped with a subtree from ADF_0, and not with one from ADF_1 or the RBF).

Koza's ADFs belong to a specific individual but Angeline and Pollack [11] proposed an alternative form of ADF called *module acquisition* in which subtrees of fit individuals are defined as modules and placed in a module library available to the whole population. The modules are protected from further evolution, but the leaf nodes of the module become parameters so that individuals can use different parameters with the module.

GP Schema Theory

Koza's original work [97] included an explanation of GP schema theory, based on Holland's schema theorem but replacing the alphabet of three symbols {0, 1, #} with LISP S-expressions (e.g. schema 'H = {(∗ Width Length), (+ Area Length)}' has as instances any program trees containing both (Width ∗ Length) and (Area + Length) at least once). O'Reilly and Oppacher [136] extended Koza's GP schema theory to include concepts of crossover, schema order and defining length (using '#' as a wildcard symbol representing any subtree). Poli and Langdon [149] introduced a simpler definition of a GP schema theorem composed of functions and terminals with '=' as a wildcard symbol representing any single terminal or function. Using the new schema definition and one-point crossover they were able to derive a new schema theorem and show that it "is the natural counterpart for GP of the GA schema theorem and that the GP schema theorem asymptotically tends to it". Poli went on [146–148] to further extend the schema theory to define formulae for the expected number of instances of a schema in the next generation (previous theories had, like GA schema theories before Stephens and Walbroek, only been able to give the lower bound), initially only for one-point crossover but later [152–154] for all forms of GP crossover.

The theory of Markov chains has also been applied to GP [154] but

is not yet as descriptive of GP as schema theory, and neither theory has been able to fully describe the behaviour of GP:

> "... There is no representation of the fact that in GP, unlike in other evolutionary techniques, the fitness function involves the execution of computer programs on a variety of inputs. In other words, schema theories and Markov chains do not tell us how fitness is distributed in the search space. Yet, without this information, we have no way of closing the loop and fully characterising the behaviour of a GP system which is always the result of the interaction between the fitness function and the search biases of the representation and genetic operations used in the system." [151]

Bloat

In 1994 Tackett [181] first observed that after a while individuals in a GP population tend grow uncontrollably (without materially improving their fitness) until they reach the maximum allowed size (that is, the population tends to "bloat"). The extra portions of code were dubbed "introns" in contemporary work by Angeline [9] (referring to code segments (subtrees) that could be removed without altering the result of the program - effectively inactive code). Bloat is a persistent problem in GP, with studies [131, 132] showing that late in a GP run introns tend to grow exponentially and come to constitute almost all the code in an entire population, causing the population to stagnate. Thus not only does code bloat require extra work to execute a GP program and make it more difficult to see what the program is doing, it prevents further evolution of the population.

There are a number of theories to explain bloat, none wholly accepted as fully explaining the phenomenon. Poli, Langdon and McPhee [151] have outlined the four major explanations:

The replication accuracy theory [121] concentrates on the fact that crossover is on average a *destructive* operator.[16] Because bloated individuals have large sections of code that do not affect

[16] Studies have shown that approximately 75% of all crossover events result in offspring that have less than half the fitness of their parents (i.e. crossover removes code that is useful at that location and replaces it with code that is not) [132]. Of course, it is not *always* a destructive event, and those few successful crossover events are what (together with mutation) drives the population forward.

their output, crossover is less likely to destroy their fitness than that of an individual with only functional code (they are more likely to be accurately replicated). Bloated individuals therefore spread throughout the population.

The removal bias theory [176] observes that introns tend to occur low in the tree and therefore occupy smaller than average subtrees. Although these inactive subtrees are less likely to be selected for crossover (as they are smaller than average), because they are inactive offspring tend to survive events in which they are selected for crossover (having the same fitness as the parent). Because the new subtree is on average larger than the excised one this results in offspring that are larger than the parent but with the same fitness - so individuals grow over time.

The nature of program search spaces theory [108, 109] states that, because there are more long programs than short ones in the search space, there will be more long programs than short ones with a given level of fitness. As the GP run samples more of the search space over time, it will inevitably discover longer programs because more of them exist.

The crossover bias theory [59, 150] states that although *on average* crossover events do not change the size of the individuals involved, individual crossover events will result in offspring smaller or larger than their parents. "In particular, crossover pushes the population towards a particular distribution of program sizes, known as a *Lagrange distribution of the second kind*, where small programs have a much higher frequency than longer ones." [151] Because these small programs have less chance of solving the problem they have less chance of surviving to produce offspring than large programs, the large programs consequently produce more offspring and thus spread throughout the population, increasing in size at each generation.

A number of methods exist to tackle bloat, the simplest of which is to limit the size (or depth) that individuals are allowed to reach. This is often fixed for the duration of the run and enforced by replacing offspring that exceed the limit with the parent [97]. However this leads

to situations where parents at or near the limit are more likely to be replicated unchanged and so come to dominate the population. To overcome this, where offspring are too large they are deleted and the crossover event is repeated, or alternatively the offspring are retained but assigned a fitness of 0.

It is possible to modify the crossover operator so that it is less likely to cause bloat. For instance introducing a 'Deleting Crossover' operator [27] to remove introns[17]; or reducing the destructiveness of the crossover operator using Explicitly Defined Introns [132] to modify the probability of crossover at different points in the tree. More recently the 'size fair crossover' operator was introduced [106], which restricts the size of the subtree selected in the second parent to ensure that it is not "unfairly" large in comparison to the subtree from the first parent (which is selected at random as normal). Modifications can also be made to the mutation operator to prevent mutation from creating offspring too much larger than the parent ([92, 106], or ensuring that the offspring is smaller ([10, 93]).

The most popular method for tackling bloat, however, is to change the fitness function to take the size of the program into account - applying *parsimony* pressure. This is accomplished either by modifying the fitness function directly so that a proportion of fitness is derived from the program's size, or by splitting the fitness function into multiple objectives to be optimised simultaneously (one for the performance of the program, another for its size) using *multi-objective optimisation*. Both approaches are discussed further in chapter 5.

Linear and graph-based GP

Instead of a tree structure linear GP [14, 33] uses a straightforward sequence of instructions, which can be fixed or variable in length. The instructions in linear GP are imperative instructions in either an imperative programming language (interpreted by a virtual machine) or directly in machine code. Imperative instructions read from source registers (memory), perform an operation and write to a destination register in either a 2-register (in which case, one of the source registers may also be the destination register, e.g. $r_i = r_i + r_j$) or 3-register for-

[17]The Deleting Crossover operator works by marking all the edge nodes that are traversed during evaluation of the fitness of a tree. All subtrees at the edges that have not been marked are redundant and are each replaced by a single node (randomly generated).

mat (e.g. $r_i = r_j + r_k$). In this way linear GP instructions communicate with each other only via memory registers and there is no distinction between function and terminal nodes as in tree-based GP. In addition to the *input registers*, which are populated with the program inputs at the start of execution, there are a user-defined number of *calculation registers* that may be initialised with a constant (e.g. 1). All registers may be written to by the program (unless constants are in use, in which case those registers populated with constants at the start of the run may be write-protected and used only as input registers) and one or more of the input or calculation registers will be designated as *output registers*, from which the output of the program will be read when it has finished executing. Figure 2.8 shows an example linear GP program, in which instructions of the form $r_i = r_j + r_k$ are represented as $< +, r_i, r_j, r_k >$.

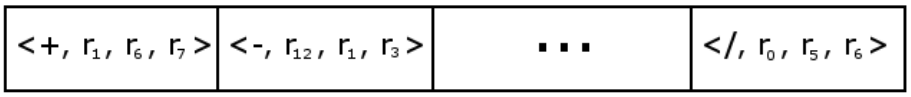
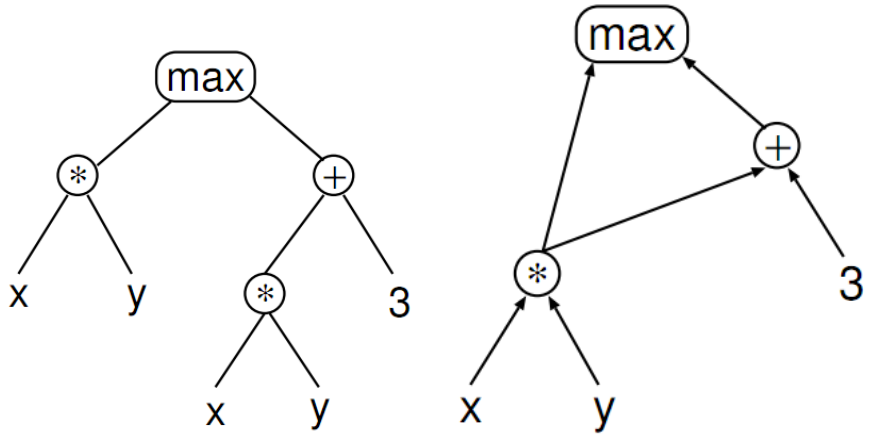


Figure 2.8: Linear GP example

Variations of GP using graph-based structures have also been proposed using directed graphs, of which the tree structure is just one form. Teller proposed *parallel algorithm discovery and orchestration* (PADO) [184] in which individuals consist of a main program and several sub-programs (ADFs), with each program having a stack and an indexed memory used for storing intermediate values and for communication between programs within the individual. Programs are comprised of a number of nodes in a directed graph (each node having one or more incoming arcs and zero or more outgoing arcs) with sub-programs being represented by a single node within the main program. If a program stops (enters a node with no outgoing arcs) before a set minimum execution time it is restarted with the stack and indexed memory left unchanged, and is stopped by an external agent if it fails to stop before a set maximum execution time.

Poli proposed *parallel distributed GP* (PDGP) [145] in which individuals are represented as directed graphs with functions and terminals as nodes and edges (arcs) indicating the flow of control and data. Figure 2.9 shows an example (standard) gp tree and the equivalent (but

more compact) PDGP tree.[18] This can be extended by assigning labels to the edges, with the labels becoming another form of operator (the *label set*) in addition to the function and terminal sets - for example in using PDGP to evolve a neural network the edges represent connections between nodes (neurons) with labels specifying the weight of the connection. Poli also suggested using *automatically defined edges*, akin to ADFs in which the edge invokes another graph.



(a) GP tree (b) PDGP graph

Figure 2.9: "A sample tree where the same subtree is used twice (a) and the corresponding graph-based representation of the same program (b). The graph representation may be more efficient since it makes it possible to avoid the repeated evaluation of the same subtree." Figure reproduced from [151].

2.3 Multi-Objective techniques

> The concept of Pareto optimum was formulated by Vilfredo Pareto in the XIX century, and constitutes by itself the origin of research in multi-objective optimization.[50].

Many real-world problems involve the simultaneous optimisation of several, often competing, objectives (for instance finding a material that is both strong and light). There is usually no way to identify a single optimal solution to such competing objectives, instead a human

[18] Figure 2.9 is reproduced from 'A field guide to genetic programming' [151] which is licensed under the Creative Commons Attribution- Noncommercial-No Derivative Works 2.0 UK: England & Wales License (see `http://creativecommons.org/licenses/by-nc-nd/2.0/uk/`).

decision maker must choose between a number of alternative solutions. These solutions can all be considered to be optimal in the sense that no one solution is superior to another when all objectives are taken into account (taking our earlier example, one solution might be stronger than another but it isn't both stronger and lighter). These solutions are called the set of *Pareto-optimal* solutions or *non-dominated* solutions. The Pareto-optimal solutions cannot be improved in regard to one objective without degrading performance in regard one or more other objectives - thus they are globally optimal solutions.[19]

Evolutionary Algorithms were first applied to multi-objective optimisation in the mid-1980s (probably by Schaffer [166], though his Vector Evaluated Genetic Algorithm did not make use of the concept of Pareto optimum) and have since grown in popularity. Evolutionary algorithms are particularly suited to multi-objective optimisation as, 'due to their inherent parallelism and their capability to exploit similarities of solutions by recombination, they are able to approximate the Pareto-optimal front in a single optimization run' [208]. It has been suggested that they are more effective at multi-objective optimisation than other blind search strategies ([67, 209]). For an overview of multi-objective optimisation using evolutionary algorithms, see [50, 55, 67, 209].

2.3.1 Enforcing parsimony using Multi-Objective optimisation: POPE-GP

Multi-objective optimisation has become a popular approach to enforcing parsimony since initial studies by Bleuler et al. [26] and DeJong et al. [54]. Instead of applying parsimony pressure by modifying the fitness function in order to find a single optimal solution; multi-objective optimisation techniques such as SPEA2 [26] and NSGA-II [56] can be used to treat fitness and size as independent objectives and 'try to find or approximate the so-called Pareto-optimal set consisting of solutions which cannot be improved in one objective without degradation in another' [26].

[19] For instance assume that two objectives x and y are measured as real-valued numbers between 0 and 1: {x, y} and the Pareto set consists of four individuals: {1, 0}; {0, 1}; {0.5, 0.5} and {0.51, 0.25}. {1, 0}; {0, 1} belong to the Pareto set because they maximise objectives x and y respectively, while {0.5, 0.5} and {0.51, 0.25} are in the Pareto set because there is no other individual that is better than (dominates) them on both objectives, despite the fact that they can be beaten on either one objective or the other. An individual {0.5, 0.49} would not be in the Pareto set because it is dominated by {0.5, 0.5} (which is equal with respect to x and better than with respect to y).

Bernstein et al. [21] developed Pseudo-Objective Parsimony Enforcement GP (POPE-GP), using the NSGA-II algorithm to look for solutions that are both parsimonious and offer improved ability to generalise to unseen problems. Parrott et al. [142] later extended POPE-GP to reduce the chance of early convergence on poor but easy to find solutions.

There follows a description of the way in which the NSGA-II (and therefore the POPE-GP) algorithm works. The definitions used here are taken from [21]. The function of NSGA-II rests on the notions of Dominance and Crowding Distance:

Definition 1: The dominance relation $>_d$ between two solutions i and j: $j >_d i$ holds over the set of objectives Θ if:

$$\forall\theta \in \Theta(f_\theta(j) \geq f_\theta(i)) \wedge \exists\theta \in \Theta(f_\theta(j) > f_\theta(i))$$

That is, solution j dominates solution i if it achieves equal or greater fitness than solution i on all objectives, and greater fitness on at least one objective, where f_θ is the fitness of a solution under objective θ and all objectives are maximisation objectives.

Definition 2: A solution i is said to be non-dominated in a solution set P if

$$\neg\exists j \in P(j >_d i)$$

That is, a solution i is said to be non-dominated within a set if there are no other solutions in that set that dominate it.

Definition 3: A solution i is said to be a member of the Pareto Front of a problem if it is non-dominated in the set of all possible solutions S.

Definition 4: The first non-dominated front N_1 of a population P is the set of individuals that are non-dominated in that population.
Note that this approximates, but is not the same as, the Pareto Front because the individuals are non-dominated within the population, not the set of all possible solutions.

Definition 5: The n^{th} non-dominated front N_n of a population P is

the first non-dominated front of the population P'_n where

$$P'_n = P - \bigcup_{i=1}^{n..1} N_i$$

That is, the n$^{\text{th}}$ non-dominated front is the first non-dominated front of the remaining population when all the preceding $n-1$ non-dominated fronts have been removed.

Definition 6: The function $N(i)$ returns the number of the non-dominated front of which an individual i is a member:

$$N(i) = q \quad \text{iff} \quad i \in N_q$$

Definition 7: The crowding distance $C(i)$ of a particular individual i is equal to the sum of the distance between i's nearest neighbours to either side on its non-dominated front for all objectives. If i does not have a neighbour on one side (i.e. it is on the edge of the front) then its crowding distance is deemed to be infinite.
Thus the crowding distance rewards (is higher for) individuals in areas of the search space where there are few other individuals.

Definition 8: The crowded-comparison operator $\prec_n$. For two individuals i and j we say that $i \prec_n j$ iff:

$$N(i) < N(j), \text{or}$$
$$N(i) = N(j) \text{ and } C(i) > C(j)$$

At each generation the current population P_n is grouped into non-dominated fronts, and the crowding distance is calculated for individuals within each front. Tournament selection (based on the crowded-comparison operator $\prec_n$) and 'the user's choice of genetic operators' [21] are used to create a child population (of the same size as the parent population P). The use of tournament selection based on the crowded-comparison operator ensures that individuals from higher non-dominated fronts are selected over those from lower fronts, while less crowded individuals are preferentially selected within a front. The child population is then evaluated and the combined population (parents P_n

plus the child generation) is grouped into non-dominated fronts and crowding distance calculated. The next generation P_{n+1} is formed from the top half of the combined population. POPE-GP is therefore fully elitist. The population has a fixed size (50-500 in [21]) and is run for a fixed number of generations (150 in [21]), at which point the entire first non-dominated front is evaluated against the test data.

2.4 Feature Transformation: Extraction, Selection and Construction

The term *feature extraction* describes methods for reducing the number (dimensionality) of features from potentially very many features to a more manageable number, while still describing the data with sufficient accuracy to allow classification (it is often associated with filtering for image processing in the wider literature). While generally held to be a broad category that encompasses both selection and construction, also covering approaches that do not fall neatly into either category (such as Principal Component Analysis and Wavelet Transformations), feature extraction is referred to in machine learning literature as an alternative to *feature subset selection* and *feature construction* (or occasionally as being synonymous with feature construction). Liu and Motoda [113] provide the following definitions:

> '**Feature Construction** is a process that discovers missing information about the relationships between features and augments the space of features by inferring or creating additional features...
>
> **Subset Selection** is different from [feature construction] in that no new features will be generated, but only a subset of original features is selected and the feature space is reduced...It is common that some constructed features are not useful at all. Subset selection can then remove these useless features...
>
> **Feature Extraction** [operates to *replace* original features] through some functional mapping [e.g. by altering the scale of a feature] ... feature construction often expands the feature space, whereas feature extraction usually reduces the

feature space.'

Techniques for transforming features are frequently used in statistics to make data fit a normal distribution more closely, stabilise the variance or improve correlation between input and predicted variables. These are Power Transformations such as Box-Cox which have a power parameter λ. Using statistical techniques to estimate the value of λ makes it possible to identify the most appropriate transformation for the data, including logarithm, inverse, square root and the identity transform (i.e. no transformation).

Feature transformation methods can otherwise be divided into three groups, according to how the technique interacts with the classifier:

- Filter techniques are completely independent of the classifier to be used. They evaluate the utility of features directly from the data itself (e.g. using statistical methods), without any feedback from the classifier. In most cases features are scored according to how closely related they are to the classification variable, and low scoring variables discarded.

- Wrapper techniques rely on feedback from the classifier to determine the utility of features. The extracted features are thus optimised specifically for the classifier to be used. Wrapper approaches treat the classifier as a 'black box' and tend to involve the use of cross validation[20] to evaluate a classifier, though separate train and test sets may also be used.

- Embedded techniques are specific to a classifier and form part of the classifier itself, and so do not perform any pre-processing per se. These techniques often form an intrinsic part of the classifier (e.g. the use of Information Gain Ratio by C4.5 to select a feature for use at each node in the tree); though it also refers to the introduction of additional techniques to enhance an existing classifier. For instance Ekárt and Márkus [60] use GP to evolve new features that are useful at specific points in the decision tree by working interactively with C4.5. They do this by invoking a

[20] Cross-validation is a technique for estimating the ability of a classifier to generalise to unseen data. The most common form, N-fold cross validation, involves partitioning the data into equal subsets or 'folds' and using one subset as test data with the rest used to train the classifier, repeated once for each fold. So in 10-fold cross validation the classifier will be trained 10 times, each time using a different 10% of the data for testing and the remaining 90% for training. Accuracy on unseen data is estimated by taking an average over the 10 folds.

GP algorithm when constructing a new node in the decision tree, for instance when a leaf node incorrectly classifies some instances. Information gain is used as the fitness criterion while the GP is trained only on those instances relevant at that node of the tree.

For an overview of the current state of feature extraction techniques (excluding evolutionary approaches), see [79].

2.4.1 Feature Selection

Feature subset selection is increasingly important for classification and data mining, particularly with recent developments in genetics (e.g. in the diagnosis of cancer using gene expression profiles [112], where there can be thousands of genes expressed). Many different methods exist for selecting a feature subset, almost all of which are filter approaches and which can be roughly grouped as follows (though other writers have used different classifications, for instance into exhaustive, heuristic and randomised search [168]).

Statistical methods. These include the well known Chi-squared (X^2) test which measures divergence from the distribution expected assuming that feature values are independent of the class value. Other examples include Spearman's rank correlation coefficient (ρ), a non-parametric measure of whether two variables are correlated; and Pearson product-moment correlation coefficient (r), which measures the degree of linear relationship between two variables. Generally these techniques are used to select those features that possess the strongest correlation with the classification variable (the 'maximum-relevance selection'), e.g. by forward selection which starts with an empty set and at each stage tests all remaining features, selecting the single most strongly correlated feature to add to the set (as long as the correlation achieves a specified level of significance). Another powerful approach is to select features that are both strongly correlated to the class and mutually far away from each other ('minimum-Redundancy-Maximum-Relevance selection': mRMR [143]).

Distance measures, such as the popular filter method relief-F [95] which produces higher weightings for attributes that have instances close to neighbours of the same class and farther from instances of a different class. Unlike most filter techniques, relief-F is multivariate,

i.e. it is able to take account of dependencies between variables. It can also deal with noisy and missing data. Other distance measures include Euclidean Distance and Cosine similarity.

Clustering methods divide the feature space into clusters of similar features, selecting a representative feature from each cluster. Recent examples include [126] and [52].

Information theory. Information gain measures the decrease in entropy when the feature is present compared with when it is absent. Mutual information measures the mutual dependence of two variables, and can be used to characterize both the relevance and redundancy of variables (e.g. in minimum-Redundancy-Maximum-Relevance, above). The 'signal-to-noise-ratio' measures the ratio of useful information to false or irrelevant data, and can be used to identify features that contribute highly to classification [105].

Evolutionary approaches. Genetic algorithms are well suited to performing feature selection, and there have been many examples in the literature since they were first used for feature selection in the design of automatic pattern classifiers by Siedlecki and Sklansky in 1989 [168]. GAs have been used to select features for a wide range of classifiers, including: k-nearest-neighbour (knn) [155, 168]; decision trees [163]; rule induction systems [190]; Bayesian classifiers [185]; and neural networks [71, 201] (Brill et al. [71], in selecting features for a neural network, reduced the computational effort required by an order of magnitude by evaluating the features using knn instead of a neural network, and training the knn on only a subset of the data on any given evaluation).

Wrapper approaches can also be used, such as **forward selection**, a stepwise regression approach common in statistics. This approach can be used with statistical methods such as the F-test, but it has also been applied by Kohavi and John [94] to other learning algorithms such as decision trees and Naïve Bayes. Starting with an empty subset of features, each of the N available features is tested in turn and the most predictive is added to the subset. Each remaining feature is tested in combination with the single feature in the subset (testing $N - 1$ pairwise combinations rather than exhaustively testing $N * (N - 1)$ combinations), and the best performing pair of descriptors is chosen. Next the $N - 2$ remaining features are tested in combination with the

two in the subset, and the process is repeated until accuracy ceases to improve, or a desired accuracy or number of features is reached, etc.

For a comprehensive overview of current feature selection techniques, see [115]. Feature selection techniques are often associated with various methods of constructing ensembles - see chapter 6 for recent examples.

2.4.2 Feature Construction

> 'The conclusive majority of feature construction algorithms have been specifically designed to generate features of a rigidly predefined representation. Among the popular representations are simple Boolean expressions, M-of-N expressions, hyperplanes, logical rules and bit strings. Most construction algorithms employ special-purpose construction methods and heuristics that are especially suited to their underlying representation. Each of these representations was shown to be beneficial in specific classes of problems. For example, it was shown that M-of-N expressions are particularly useful for medical classification problems where expert systems make use of "criteria tables" that are essentially M-of-N concepts.' (Markovitch and Rosenstein, [120]).

Although the majority of feature construction algorithms are domain specific there remain a number of algorithms that construct new features in a more generic fashion. For instance Markovitch and Rosenstein created a grammar for describing a specification for methods of feature construction for a specific domain, and an algorithm that generates new features given such a specification [120] (though this approach does require input from a user with a certain level of domain knowledge to write the specification).

Bloedorn and Michalski took an exhaustive approach to constructing new features with AQ17-DCI [28], which "exhaustively searches all possible pairwise combinations of attributes only once, and ignores a large portion of the possible space by not considering multiple combinations" [189], using several operators including equivalence, greater than, addition, subtraction, difference, multiplication, division, maximum, minimum, and average. Piramathu and Sikora [144] recently took a similarly exhaustive approach, using standard arithmetic oper-

ators {+,-,*,/} to combine all possible pairs of original attributes (using each of the operators) and using the chi-squared feature selection technique to select from the combined set of original and constructed features. They then pass the selected features to a genetic algorithm for classification[21] (also testing it with C4.5 and an instance-based learner). The obvious problem with these exhaustive approaches being the exponential increase in effort required as the number of features increases.

Vilalta and Rendell developed DALI [193] which creates a decision tree in which each node uses a constructed feature obtained by conducting 'a beam search over the space of all boolean feature conjuncts'. They combined this with multiple-classifier techniques (bagging and boosting - see chapter 6) so that each node uses a combination of multiple constructed features (i.e. using bagging and boosting to create many subsamples of the data available at the node, treating DALI's beam search as the 'classifier').

An artificial neural network (ANN) can be considered to construct features at its hidden nodes. For instance, Liu and Setiono's BMDT algorithm [116] used a standard feed-forward ANN with original attributes as input nodes, classes as output nodes and a layer of hidden nodes. After training and pruning of the hidden nodes, the remaining hidden nodes are used as new features to be passed to C4.5 - these new features are hyperplanes in as many dimensions as there are original attributes. The drawback to this approach is the difficulty in understanding what relationships the new features are drawing between the original attributes.

Raymer et al. [159] have used Genetic Programming to create Automatically Defined Functions (ADFs) for feature extraction in conjunction with the k-nearest-neighbour algorithm, in which an original feature is replaced with the result from passing it through a functional mapping. In Raymer et al.'s approach "each feature is altered by a GP ADF tree, evolved for that feature only, with the aim of increasing

[21] Piramathu and Sikora use a GA for classification as follows: 'the representation of a concept or a classifier used by the GA is that of a disjunctive normal form. A Concept is represented as $\Omega = \delta_1 \vee \delta_2 \vee \ldots \vee \delta_p$, where each disjunct δ_i (also referred to as a rule) is a conjunction of conditions on the k attributes, $\delta_i = (\xi_{1,i} \wedge \ldots \wedge \xi_{k,i})$. In order to handle continuous attributes each condition $\xi_{j,i}$ is in the form of a closed interval $[a_j, b_j]$ ... Each member of the population in the GA is a single disjunct and the GA tries to find the best possible disjunct. At each generation it retains the best disjunct and replaces the rest through the application of the genetic operators. After the genetic algorithm converges, the best disjunct found is retained and the positive examples it covers are removed. The process is repeated until all the positive instances are covered. The final rule or concept is then the disjunct of all the disjuncts found.' [144]

the separation of pattern classes in the feature space" [4]; for problems with n features, individuals consist of n ADFs. Ahluwalia and Bull [4] built on the work of Raymer et al. by co-evolving an additional population of bit strings used to perform feature selection among the extracted features. Bot has similarly used GP to produce new features for k-nearest-neighbour [32], but with the express intention of reducing the number of features passed to the classifier (ideally to one or two, with the advantage that these can be plotted on a graph), using feature construction (though referred to as extraction) as a filter method (using a simple minimum distance to mean classifier to measure the fitness of functions). Otero et al. [137] also used Genetic Programming as a filter method to construct a single feature to pass to C4.5 in addition to the original attributes, using Information Gain Ratio as a fitness measure.

Vafie & DeJong [189, 191] combined a GA for feature selection and GP for feature construction to create a new set of features for use in image processing by ID3 (C4.5's forerunner). Krawiec [100] has used GP to construct a fixed number of new features to replace the original attributes for use by C4.5. Gavrilis et al. [75] later (2008) extended Krawiec's work by incorporating grammatical evolution (Backus Naur form, or BNF) allowing a context-free grammar to describe all the possible algebraic expressions of the original attributes (specified for the dataset at hand), using a genetic search function that encourages simple expressions and automatically normalizing the new features (keeping to Krawiec's design of constructing a fixed number of features). They constructed features for Multi-Layer Perceptron (MLP) and Radial Basis Function (RBF) neural networks and k-nearest-neighbour, evaluating their technique on a number of UCI datasets and finding an improvement over both the original attributes and those derived by Principal Component Analysis. The authors also report a previous variant of the algorithm was used to replace 15,000 original attributes with 20 new features while retaining a high level of accuracy.

2.5 Ensemble Classifiers

Traditional ensemble learning methods (such as Bagging [34] and Boosting [36, 68]) have concentrated on constructing ensembles of homogeneous classifiers (e.g. an ensemble consisting entirely of C4.5 classifiers).

The aim in both approaches is to construct multiple classifiers (generally 10-15 by default, though ensembles of hundreds or thousands have been used), varying the instances that they are trained on. In Bagging subsets of the data (or 'bootstrap replicates') are provided separately for each classifier, using random sampling with replacement (so that a given instance may appear many times or not at all in a particular subset, but the chance of being selected is the same for all instances). The output of the ensemble is then determined by majority voting.

In Boosting the ensemble is built in successive rounds, and the training data provided to the n^{th} classifier depends on the output of the $n-1^{th}$ classifier. Thus boosting is usually a sequential task while bagging is inherently parallelizable, however methods for parallelizing boosting have been demonstrated by Yu and Skillicorn [205]. Where the base classifier supports it, instances are assigned weights according to the ease with which they have been classified by previous classifiers (so that easy-to-classify instances are assigned low weight, hard instances much higher weights). Where the classifier requires unweighted instances each subset is assembled by random selection with replacement, but the probability of selection is weighted according to how hard the instance is. Thus boosting drives each successive classifier to form a different hypothesis from the previous one by concentrating its efforts on those instances previously misclassified. The output of the ensemble is decided differently according to the type of boosting: Arcing [36] uses simple majority voting; while the more popular AdaBoost [68] weights the vote of each classifier according to its accuracy on the training data.

Both bagging and boosting may be successfully applied to what Breiman [35] terms 'unstable' classifiers such as neural networks and decision tree learners; for which small changes in the training data may result in large differences in output (classification). Such unstable classifiers are characterised by having high variance and low bias; in contrast to stable classifiers such as nearest neighbour algorithms and linear discriminant analysis which tend to have low variance but high bias. The error of any classifier ψ has three components: [22]

$$Error(\psi) = Error(\psi^*) + Bias(\psi) + Variance(\psi)$$

[22] see [35, 205] for more details, including mathematical definitions of both bias and variance as they apply to numeric regression.

Where $Error(\psi^*)$ is the minimum error of a Bayes classifier, $Bias(\psi)$ is the systematic error associated with the technique and $Variance(\psi)$ is the error arising from the finite sample size of the training data. Both bagging and boosting reduce the variance of the base classifier, while boosting also reduces its bias (though it reduces variance to a lesser degree than bagging). Boosting can achieve a greater increase in accuracy than bagging, but this is associated with a high degree of susceptibility to noise in the training data [157].

Wagging [19] extends bagging by using the complete training set with each classifier, stochastically weighting each instance (i.e. they are not weighted according to hardness). Multiboosting [195] combines AdaBoost and wagging by applying wagging to a set of sub-committees (ensembles) formed by AdaBoost. Bagging and boosting have also been combined by maintaining separate bagged and boosted sub-ensembles in [164], combining their predictions using sum rule voting (in which each sub-ensemble gives a confidence value for each class - the class with the highest total confidence is selected). AveBoost:2 [139] extends AdaBoost by averaging the weights for the instances supplied to the current classifier with the weights supplied to the previous classifier. This averaging process leads to a worse training error bound than for AdaBoost but a better generalization error bound, i.e. it is not as accurate in the absence of noise but performs better than AdaBoost on noisy data. Redpath and Lebart suggested incorporating feature selection into the boosting algorithm, termed Boosting Feature Selection [161]. They found that their algorithm was competitive with AdaBoost despite the reduction in features for individual classifiers.

DECORATE [122] ('Diverse Ensemble Creation by Oppositional Relabeling of Artificial Training Examples') uses artificially constructed training samples to create a diverse ensemble of strong (highly accurate) base classifiers. Classifiers are added incrementally to the ensemble - the first uses the whole training set; additional artificial examples are added to the set for training subsequent classifiers. The attributes of these artificial instances are populated according to the distribution of the original dataset, but the class is selected to be maximally different from the current prediction of the ensemble.

Ensembles may also be constructed using methods other than manipulating the training data. Ho suggested the Random Subspace Method

[81] in which each of 500 decision trees in the ensemble is grown using a random subset (half) of the available features. Breiman expanded upon this and combined it with bagging to produce Random Forests [37]: an ensemble of unpruned decision trees, each of which is trained on a new bootstrap replicate, using a random feature subset to decide the split at each node of the tree; classifying an instance by majority vote. Tsymbal et al. [188] applied the random subspace method to create ensembles of simple Bayesian classifiers, refining the subspaces using a hill-climbing technique. Dietterich developed Randomized C4.5 [58], in which the 20 best tests are determined at each node in the tree - the actual test is randomly selected from these 20. Dietterich found that this approach was equivalent to or better than Bagging (and better than AdaBoost in noisy environments), without having to create multiple training sets (though it does have large memory requirements, especially with many continuous attributes that may have multiple splits to consider). Banfield et al. performed a comparison of seven ensemble methods [110] (bagging, boosting, randomised C4.5, random subspaces and three variants of random forests) on 34 public datasets and found that no single method had a statistically significant advantage over bagging, though boosting would have done had the noisiest dataset been excluded.

Brown et al. used an ensemble of Neural Networks to automatically extract multiple, diverse features from a high-dimensional image dataset [41] and pass the features to k-Nearest Neighbour for classification; though the features thus extracted suffer from the same human readability problems as all neural networks. Wang et al.[194] created a very different sort of ensemble by replacing a single n-class classifier (a *class-independent* classifier) with n 2-class (*class-dependent*) classifiers (e.g. for a dataset with 4 classes A,B,C,D 4 *class-dependent* classifiers are constructed; one classifies instances as A or not-A, another as B or not-B, and so on). They used a wrapper approach to perform feature selection separately for each of the classifiers (using SVM [53] as the base classifier and RELIEF, class separability & minimal-redundancymaximal-relevancy as the feature selection measures), assigning weights to the output of each classifier before determining which has the strongest prediction. Unlike Wang's approach, most SVM ensembles use data resampling to generate diversity in the ensemble, however Sun et al. [179] turned a drawback of SVM (that

kernel parameters must be tuned for the dataset at hand) into an advantage by using different kernel parameters to drive the diversity of the ensemble, thus also reducing the (human) effort required to construct it. They also examined a number of methods for pruning the ensemble. Valentini [192] used random aggregating (approximated using subsampled bagging, where the size of the bootstrap replicate is less than the original sample) to construct SVM ensembles on large datasets, using bias-variance analysis to explain why the approach can be successfully applied to large datasets.

More recent work has investigated the use of heterogeneous classifiers (e.g. Borji has combined a neural network, SVM, k nearest neighbour and a decision tree into an ensemble for network intrusion detection [31]). It has been shown that heterogeneous ensembles can provide a greater diversity, and therefore lower error rate, than homogeneous ensembles [22, 196] (though Bian and Wang [22] observe that 'the absolute value of [a diversity measure] alone is not always consistently proportional to the ensemble accuracy when the voting strategy is applied. The accuracy of individual models should also be taken into account').

2.5.1 Evolving Ensembles

Evolutionary learning has been used to create ensembles since Larry Fogel suggested combining the outputs of a population of Finite State Machines in Evolutionary Programming [64] during the 1960s, though most activity in evolving ensembles has occurred more recently. In the mid to late 1990s Yao and Liu created EPNet [202], based on evolutionary programming, to evolve both the architecture and node weights of an artificial neural network (ANN). They then [204] used a variety of combination methods to combine or select from the final population to construct an ensemble. Liu and Yao also proposed [117] using Negative Correlation Learning (NCL) to train ANN ensembles, which incorporates a measure of diversity from other ANNs into the backpropagation algorithm so that the ANNs are trained interactively on the same dataset (though unlike EPNet the structure of the individual ANNs is not evolved). The resulting ensemble contains the whole population, whose output is obtained by averaging the output of the individuals.

Liu et al. then proposed [118] incorporating fitness sharing[23] in addition to NCL in evolutionary ensembles with NCL (EENCL). The final ensemble was constructed either from the whole population, or from a single representative of each species in the population (identifying the different species using a k-means clustering algorithm), their output being combined using one of 3 combination methods (averaging, majority vote and winner-takes-all). Islam et al. [86] proposed 'a constructive algorithm for training cooperative neural network ensembles (CNNE), where both ensemble and individual ANN architectures are determined automatically' [203], incorporating NCL but building the ensemble in a fashion somewhat akin to boosting. A minimal ensemble (of 2 ANNs, each with a single hidden node) is incrementally expanded by adding more hidden nodes to an individual once the contribution of that individual has ceased improving. Once adding hidden nodes to existing ANNs no longer improves accuracy the existing ANNs are frozen and a new ANN added and the cycle repeated (so the only ANN subject to change is the newly added one in each epoch).

In 1999 Guerra-Salcedo and Whitley [78] used a GA to perform feature selection for a base classifier, running the GA 50 times, assembling an ensemble consisting of 50 classifiers each using the fittest feature subset from a single run. At about the same time Opitz [135] also used a GA for feature selection to determine the input features for an ensemble of neural networks, but constructed an ensemble from the population of a single run. He also incorporated a measure of diversity in the fitness measure: fitness = accuracy + λ diversity) where λ is a parameter determining the importance of diversity, and diversity is the average difference between the individuals' prediction and that of the ensemble. Kuncheva and Jain [103] used a GA to select both feature subsets and the type of classifier used by an individual[24], while the elitist selection procedure maintained both the classifier with the highest individual accuracy and those classifiers that have the highest accuracy in combination (by majority vote) - thus evolving both individuals and ensembles at the same time (the ensemble size being fixed in advance).

Kim et al. [90, 91] also use a GA to perform feature selection to

[23] In fitness sharing as employed in EENCL each of n classifiers that correctly classify an instance get $1/n$ incremental fitness, other classifiers getting 0. In this way individuals are rewarded for classifying instances others find difficult.

[24] This bears similarity to some aspects of the GAP algorithm but uses 3 different base classifiers: linear discriminant classifier, quadratic discriminant classifier and logistic classifier.

promote diversity among a population of neural networks, assigning greater reward to classifiers that correctly classify instances most other individuals in the population mis-classify. More interesting is their simultaneous evolution of a population of ensembles ('Meta-Evolutionary Ensembles') which compete for member classifiers and are rewarded according to the majority vote of their members. In this way they aim to create small, optimally diverse ensembles (unlike Kuncheva's work, genetic operators can cause the ensembles to grow and shrink). Oliveira et al. [133] have used a 'hierarchical multi-objective genetic algorithm approach' having separate layers to perform both feature selection for a set of base classifiers (Multi Layer Perceptron neural networks) and to combine them into an ensemble. Kondo et al. [127] have used a multi-objective approach to evolve an ensemble of RBF neural networks for pattern classification, using accuracy and complexity as the objectives and constructing the ensemble from the Pareto set in the final generation. Ahmadian et al. [5] have used multi-objective evolutionary techniques (NSGA II) with a bagging and boosting-like strategy for datasets with low numbers of features where feature selection may not be appropriate. Having evolved a population using as objectives the error rate for each class, they use another multi-objective GA to combine the population into an ensemble.

Reynolds et al. have evolved ensembles using the Cultural Algorithms Framework [162], which consists of a social population and a 'belief space' consisting of various forms of symbolic knowledge (Situational, Normative, Topographic, Historic/Temporal and Demographic). Individuals in a population of 200 belong to, and can move between, subcultures associated with each of the of the knowledge forms. Each subculture evolves a decision maker that in turn guides the next generation of the subculture. The ensemble is formed from a representative decision maker from each of the 5 subcultures.

Kanevskiy and Vorontsov created a cooperative coevolutionary algorithm [89] in which p populations are evolved simultaneously using an elitist GA for feature subset selection, constructing an ensemble of size p. The populations are independent except for the fact that the fitness of individuals is determined by their contribution to the current ensemble - which is made up of the individual under evaluation and the fittest classifiers from the other populations. The number of pop-

ulations can also grow and shrink: a population is removed when its contribution to the ensemble becomes too small; a new population is added when evolution has stagnated. Evolution is stopped when adding new populations ceases to improve the accuracy of the ensemble.

Bull et al. [44] have demonstrated ensembles of 10 Learning Classifier System (LCS) [43] individuals which are trained in parallel, migrating rules between them using a technique inspired by parallel GAs and combining the predictions of the population by majority vote. They found that this improves the learning speed - an important factor for large datasets. Bacardit and Krasnogor have used a form of LCS called GAssist to construct two types of ensemble [12]. The first is similar to bagging in that the ensemble is the result of running GAssist many times, but using the *same* dataset with different random seeds, using majority vote to decide the ensemble prediction. They found this provided a significant improvement over a single GAssist individual in 10 of 25 datasets from the UCI repository. The second form of ensemble applies only to problems of ordinal classification (datasets where the classes have a meaningful order), where they decomposed an N-classes dataset into $N-1$ binary datasets and trained an individual on each binary dataset then determined the ensemble prediction using a form of decision tree whose nodes are the individual predictions. They found that this reduced the error of mis-classifications (i.e. the predicted value, when wrong, was *close* to the actual value).

Park and Cho [141] combined seven different feature selection methods with 6 classifier types to create 42 heterogenous classifiers, and used a GA to select the most accurate ensemble, using both majority voting and weighted voting as combination methods.

Nojima and Ishibuchi used a two-stage multi-objective genetic algorithm to generate ensembles of fuzzy rule classifiers (of the form *if-then-prediction*) [128, 129]. They used an heuristic approach to generate an initial population of fuzzy rule candidates and used NSGA-II with two objectives to select a set of rules that is both accurate and compact. In the second stage NSGA-II is used again to generate ensemble classifiers from the candidate rules, where each individual in the second stage encodes a complete ensemble (within which each rule may appear in at most one classifier) using as objectives the classification accuracy and diversity of the ensembles. In [129] they evaluated many different

diversity measures and found that the choice of measure did have an impact on the accuracy of the final ensemble, citing the *measure of interrater agreement* as being the most useful of 6 diversity measures employed (not including the Q statistic used in chapter 6).

2.6 Putting the GAP algorithm into context

There are a large number of classification algorithms of varying complexity in machine learning, using a variety of different techniques. The focus in machine learning has moved from the simpler classifiers to more complex variants (whether entirely new forms such as SVM, or extensions to simpler classifiers such as multivariate trees or bayesian networks), though simpler classifiers continue to be used in conjunction with other techniques for instance as a black box in a wrapper technique such as used by the GAP algorithm presented here.

Evolutionary algorithms (predominantly GA's) have frequently been used to improve the performance of existing classifiers, either through parameter setting or through feature selection, and Genetic Programming has been used to create the classification algorithm itself, but relatively little work has been done using evolutionary techniques to perform feature construction. Where this has been done (such as work by Vafie & De Jong [189] or Krawiec [100]) it has almost always been specifically for C4.5, and has focused on improving accuracy without considering what may be learnt about the relationships discovered between features. Similarly while there has been much work using feature selection to produce ensemble classifiers there has been very little work on the use of feature *construction* to produce ensembles, or on the production of heterogenous ensembles. There has been much work in genetic programming to reduce the size of individuals, either through the inclusion of a parsimony measure in the fitness function or the use of multi-objective techniques. This tends to be an effort to combat bloat though, and relatively little has been done to investigate how to make solutions more easily understood by humans.

Through the work in the following chapters the GAP algorithm seeks to fill some of these gaps, by using a combination of genetic algorithms and genetic programming for feature construction and selection to improve the accuracy of a variety of simple classifiers (and indeed to se-

lect the most appropriate classifier/featureset combination) while providing explicit information about the relationships between attributes that have made the improvement possible. A number of techniques are investigated to highlight the relevant relationships between attributes and make them more readily visible and to distinguish between the impact of different relationships. The output of the algorithm is also used in the creation of heterogenous ensembles using multiple featureset/classifier combinations.

Chapter 3

The GAP Algorithm: Improving the performance of a classifier

3.1 Introduction

This chapter introduces the GAP algorithm (short for Genetic Algorithm and Programming) in it's initial, two-stage form (algorithm 3.1). Its ability to improve the accuracy of C4.5 will be evaluated on 10 datasets from the UCI machine learning repository [23], which are also briefly introduced here. The utility of the construction and selection stages will be compared against each other and against unaided C4.5, and analysis of the constructed features will be undertaken. The choice of initialisation technique will be examined and compared with the more standard Ramped Half and Half. The effect of running the create stage before the select stage will also be examined (and vice-versa), as will the effect of randomising the order of the training data between stages.

3.2 Motivation

Section 2.1.1 discussed the fact that the search through the hypothesis space is guided by the bias of the algorithm. There is another form of bias not yet mentioned: the language bias. The language bias restricts the hypotheses that can be expressed by the algorithm - it places limits on what hypotheses can be learned (a decision tree learning algorithm

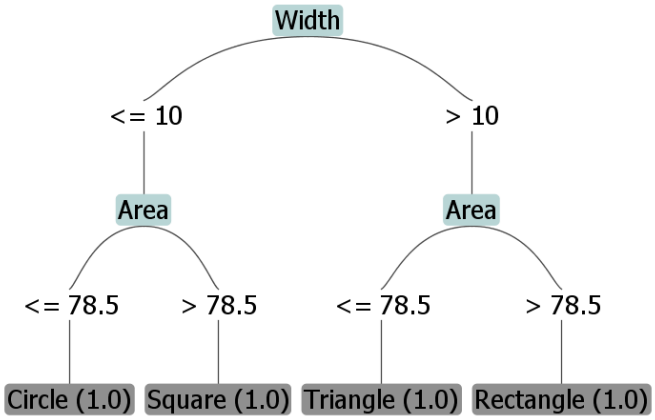


Figure 3.1: Sample 'shapes' decision tree

"searches only the space of finite trees that carry out axis-orthogonal splits" [207]).

An example illustrating the language bias in decision tree learning involves trying to distinguish between different shapes (this example was used to test the algorithm presented in this thesis during its earliest stages). The training examples consist of a set of three attributes {length, width, area} and a class with four possible values {square, rectangle, circle, triangle}:

Table 3.1: Sample 'shapes' dataset

Length	Width	Area	Class
10	10	100	Square
10	12	120	Rectangle
10	12	60	Triangle
10	10	78.54	Circle

Given this dataset C4.5 will produce the decision tree shown in figure 3.1[1] (as it is such a small dataset a parameter must be adjusted to allow leaf nodes to contain only a single instance. A larger dataset is likely to result in a much more complicated tree as C4.5 attempts to classify all instances with insufficient information). The decision tree could then be re-cast as a set of production rules such as:

```
'If (width<=10) and (area>78.54) then class = Square'
```

[1]Please note that the numbers used in figure 3.1 have been rounded (e.g. from 78.54 to 78.5).

While C4.5 may be able to construct a set of rules that accurately classify the training dataset, the rules would obviously not generalise well to unseen examples (e.g. the above rule will misclassify long narrow rectangles as squares). The algorithm is unable to learn to classify unseen shapes from the training dataset, as it is unable to construct rules that consider the relation of one attribute to another, e.g. rules of the form:

```
'If (area = (length * width)) and (length = width) then
class = Square'
```

With some knowledge of the subject domain we could transform the dataset into more appropriate features that would allow such generally accurate rules to be constructed. There is a class of machine learning techniques called Analytical Learning (e.g. Explanation Based Learning [57]) that uses domain knowledge to augment the information in training examples and avoid the limitations described above. But what if there is little or no domain knowledge at hand? The problem becomes one of constructing an algorithm to find new, useful features without any a-priori domain knowledge.

Given the above Shapes dataset (with 100 instances), the algorithm presented in this thesis suggested replacing the original attributes with two new features:

1. ((Length * Width) / Area)
2. (Width - Length)

These features are used by C4.5 to create a decision tree that perfectly classifies the dataset, as illustrated in figure 3.2.[2] Pre-processing the data is therefore an important (often essential) step in data mining and classification tasks. There are many reasons why data is pre-processed, foremost among which are:

- To transform the data into a form intelligible to the classifier. At its most basic level this can mean converting the raw data into the appropriate file format, but can also involve *discretization* of continuous variables where a classifier can only accept nominal

[2]Note that while the two new features can be used to determine the shape of any instance, the tree created by C4.5 mis-classifies rectangles whose (width-length) is between -5 and 0. This is because of the small size of the dataset: a larger set should allow C4.5 to produce better rules.

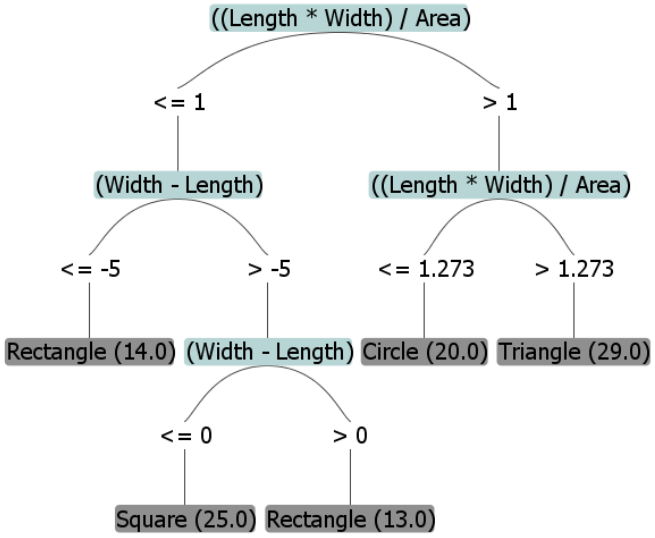


Figure 3.2: Sample 'shapes' decision tree with constructed features

or boolean inputs. This can involve domain-specific knowledge of the best way to discretize variables, or the use of measures such as Information Gain to automatically decide the best splits. Transformation may also require dealing with missing values. This can be handled by deleting data (either all instances with missing values or features with many missing values) or replacing the missing value with an estimate - usually an average value but sometimes the value of a similar instance (see [18] for further details).

- To improve the quality of the data, e.g. by detecting and removing outliers (instances whose values don't seem to fit with the rest of the data - often because they contain errors). This is sometimes performed manually but may also be performed automatically using statistical or clustering methods. See [82] for a survey of outlier detection methods. Many classifiers also require that data be normalised (e.g. transforming a continuous variable so that values in the training data always lie between 0 and 1), for instance to prevent features with a large range of values from swamping other features with a small range. This of course requires that all new instances to be classified also undergo the same normalisation.
- To reduce the volume of data to manageable levels. Often a requirement for large datasets, the aim is to select a smaller but

still representative subset of the data. This may be by random sampling or by actively selecting specific examples (e.g. representative instances or those on the boundary between one class and another). See [114] for details of instance selection methods.

- To improve classification accuracy and ability to generalise to unseen examples, for instance by constructing new features as illustrated above.
- To reduce overfitting, generally by removing noisy features (those that are not useful in predicting the class).
- To improve understanding of the dataset, by making it easier to see which features/values are useful in classification and how they relate to each other.

The last three items in the list are the ones concentrated on here, all of which can be achieved by transforming the feature space.

3.3 The GAP algorithm

The rest of this thesis concerns the development of an algorithm that uses Genetic Programming to construct new features for use by a classifier, and a Genetic Algorithm to select a subset of those constructed features; hence forming a 'Genetic Algorithm and Programming' (GAP) classifier[3]. This is a wrapper approach, in which the fitness of individuals is evaluated by performing 10-fold cross validation on the training data, using the same induction system (initially C4.5) as is used on the test data. While cross validation is very time consuming when compared to a filter approach (e.g. Otero's [137] use of the Information Gain Ratio when constructing a single feature) or even a single train and test set, it has been observed by a number of authors (e.g. [100, 194]) that the wrapper approach produces better classifiers. It is worth noting that during early development of this algorithm both a single train & test set and cross-validation were tried - cross validation produced much more accurate solutions. The use of a filter function such as Information Gain Ratio, closely associated with C4.5, is appropriate when attempting to create a single feature that is maximally

[3] 'GAP Classifier' is admittedly not the best name (in that it does not distinguish itself from any other combination of GA and GP), but it's been published now, so it stays.

useful over the whole dataset; but not when attempting to create a set of features which work well in conjunction with each other (such that some features may only be useful in combination with another or on a subset of the data).

The GAP algorithm bears some similarity to algorithms by both Krawiec [100] and Vafie & DeJong [191]; however there are notable differences. Krawiec breeds a fixed number of constructed features (4) for use by C4.5, rather than varying the genotype size according to the number of attributes in the dataset or through feature selection; performing feature selection only implicitly (i.e. any attributes not forming part of the 4 constructed features are excluded). He also protects against the loss of useful features by allowing individual trees to become 'hidden' from the processes of crossover and mutation (by comparison the GAP algorithm employs a form of elitism to maintain the single fittest genotype [rather than individual trees] between generations). The GAP algorithm shows greater similarity to Vafie & DeJong's approach (at least in its initial form - subsequent chapters will introduce further differences), but they construct a new featureset by performing multiple, successive rounds of construction and selection *starting with selection*; and they employ a single train and test set for evaluation rather than cross-validation. In both cases the mechanics of the construction of the initial population (and subsequent mutation) takes a different form to that presented here. This is also somewhat similar to the approach of Ahluwalia and Bull [4] who also used bit-strings to select feature *extraction* functions for use by k-nearest neighbour, though they co-evolved separate populations of bit-strings and extracted features rather than evolving constructed features in one stage and selecting between them in another or, as in chapter 4, performing construction and selection within a single population.

The approach consists of two phases: Feature Construction followed by Feature Selection (chapter 4 details the combining of these into a single stage). The GAP algorithm is implemented in the Java programming language and based upon classifiers from the Weka Data Mining software[4] [198]. The algorithm described in this chapter was first pub-

[4]'Waikato Environment for Knowledge Analysis': http://www.cs.waikato.ac.nz/ml/weka/. The version of Weka used for the GAP algorithm is somewhat out-of-date now (3.2, as opposed to the current stable version 3.6) - version 3.2 has been retained for compatibility with earlier results. Some of the implementations changed between versions 3.2 and 3.4 leading to statistically insignificant changes from the published results (e.g. for the C4.5 comparison) when run under the newer

lished in [172].

In this and subsequent chapters the performance of the GAP algorithm is evaluated using ten datasets from the well-known UCI repository[5] [23]. The datasets were selected because they consist entirely of numeric attributes (though the GAP algorithm can also deal with datasets containing nominal attributes - these are ignored and passed directly to the classifier; while binary attributes can be treated as numeric with values {0,1}). Details of the datasets are shown in table 3.2. The case study in chapter 7 uses a much larger dataset, the Bristol Vision dataset. Unless otherwise noted all results are obtained from twenty runs on each dataset using 90% train and 10% test data (i.e. two sets of ten-fold cross-validation, with the dataset ordered differently for each set).

Table 3.2: UCI dataset information

	Numeric attributes	**Classes**	**Instances**	**Instances with missing attributes**
BUPA Liver Disorder	6	2	345	0
Glass Identification	9	6	214	0
Ionosphere	34	2	351	0
New Thyroid	5	3	215	0
Pima Indians Diabetes	8	2	768	0
Sonar	60	2	208	0
Vehicle	18	4	846	0
Wine Recognition	13	3	178	0
Wisconsin Breast Cancer (New)	30	2	569	0
Wisconsin Breast Cancer (Original)	9	2	699	16

In data mining literature the terms attribute, feature and variable tend to be interchangeable when describing the attributes of a dataset, whether this be the original attributes or those induced by the algorithm. This thesis will attempt to adhere to the convention of referring to attributes present in the original dataset as *attributes*, while those constructed by the algorithm will be referred to as *features* (or sometimes *constructed features* where it seems necessary to make the distinction explicit).

version.

[5] http://mlearn.ics.uci.edu/MLRepository.html

The initial GAP algorithm is outlined in pseudocode in algorithm 3.1, and described in the following sections. The parameter settings shown in these sections are the settings selected after a series of experiments to determine the effect of varying the parameters, and are applicable to the two-stage version of the algorithm only. These experiments were performed over ten runs on two of the ten UCI datasets[6]: New Thyroid and Wisconsin Breast Cancer (New), which differ in the number of instances, attributes and classes. An initial set of parameters was decided on and each parameter was varied in turn (with the other values frozen, except in the case of tournament group size and chance of selecting the winner, which were also co-varied). These experiments showed that the GAP algorithm is robust when considering the values of individual parameters - in most cases the accuracy associated with a particular parameter setting is within a standard deviation of the accuracy for all other settings for that parameter; though the combination of all the new parameter settings did cause the algorithm to run for more generations and allow improvements in accuracy (and with reduced size) over the initial settings. The decision to search for incremental improvements by changing one parameter at a time rather than exploring changes to multiple parameters at once (e.g. using some kind of genetic algorithm to evolve the parameters of the GAP algorithm) was made because of time constraints - the time to explore changes to multiple parameters at once would have been prohibitive. It is entirely possible that a much improved combination of parameters could be found using a broader search.

The robustness to individual parameter settings helps to achieve one of the goals for the GAP algorithm: to improve performance on a dataset without requiring any prior domain knowledge. Once set, none of the parameter settings have been tailored for individual datasets. Changes to some of the parameters are made in chapter 4 to values more appropriate for a single stage algorithm. Changes are also made in chapter 4 to some of the mechanics of the algorithm to bring it into line with more standard approaches (e.g. replacing the two-point crossover used in this chapter with uniform crossover).

[6]Two datasets were used instead of all ten for performance reasons, a number of years ago when the available computing power was somewhat less than today.

Algorithm 3.1: The GAP Algorithm: two stage

```
1  fittest = null;
2  t = 0;
3  P_t = initialiseGPPopulation ();
4  evaluatePopulation (P_t) ; // 10-fold cross-validation on train
   data
5  fittest =findFittestGP (fittest, P_t);
6  while age (fittest) ≤ 6 or t ≤ 10 do
7      t = t + 1;
8      P_t = breedGPPopulation (P_{t-1});
9      evaluatePopulation (P_t);
10     fittest = findFittestGP (fittest, P_t);
11 t = 0;
12 P_t = initialiseGAPopulation (fittest);
13 evaluatePopulation (P_t);
14 fittest =findFittestGA (fittest, P_t);
15 while age (fittest) ≤ 6 or t ≤ 10 do
16     t = t + 1;
17     P_t = breedGAPopulation (P_{t-1});
18     evaluatePopulation (P_t);
19     fittest = findFittestGA (fittest, P_t);
20 return(fittest);
```

Procedure initialiseGPPopulation

Input: training dataset
Output: initial population of GP genotypes P

```
1  begin
2      i = 0;
       // first genotype has one tree for each numeric attribute:
3      foreach numericAttribute in training dataset do
4          P[i].trees[index] = numericAttribute ;
5      i = i + 1;
6      while i < 101 do
           // trees in remaining genotypes are created according to
              P_leaf equation:
7          foreach numericAttribute in training dataset do
8              P[i].trees[index] = createGpTree (training dataset) ;
9          i = i + 1;
10     return(P)
```

Procedure breedGPPopulation

Input: old generation of genotypes P_{t-1}
Output: new generation of genotypes P_t

```
1  begin
2      P_t[0] = findFittestGP (P_{t-1});
3      i = 1;
4      while i < 100 do
5          P_t[i+1] = P_t[i] = tournamentSelection (P_{t-1}); // group size 8,
           30% chance of selecting fittest
6          while P_t[i+1] = P_t[i] do
7              P_t[i+1] = tournamentSelection (P_{t-1});
8          twoPointCrossoverGP (P_t[i], P_t[i+1]);
9          mutate (P_t[i]);
10         mutate (P_t[i+1]);
11         invert (P_t[i]) ; // chance of two-point inversion is 0.2
12         invert (P_t[i+1]);
13         i = i + 2;
14     return(P_t)
```

Procedure twoPointCrossoverGP

Input: *genotype*1, *genotype*2

```
1  begin
2      if random (0,1) ≤ 0.6 then
3          a =random (0,numberOfTrees);
4          b =random (0,numberOfTrees);
           // ensure a < b
           // swap trees between a and b from genotype1 to genotype2
5      if random (0,1) ≤ 0.6 then
           // select a random tree (at the same locus) in each
              individual
           // exchange a random subtree between them
```

Procedure mutateGP

Input: *genotype*

```
1  begin
2      foreach tree in genotype do
3          foreach node in tree do
4              if random (0,1) ≤ 0.008 then
                   // replace node with a new subtree created according
                      to P_leaf equation with Depth = 1
5      if random (0,1) ≤ 0.2 then
           // invert (reverse) the order of trees between two
              randomly selected points
```

Procedure findFittestGP

Input: population of GP genotypes P
Output: fittest (most accurate) member of P

```
1  begin
2      fittest = P[0];
3      i = 1;
4      while i < 101 do
5          if P[i].accuracy > fittest.accuracy then
6              fittest = P[i];
7          else if P[i].accuracy = fittest.accuracy then
8              if P[i].age > fittest.age then
9                  fittest = P[i];
10         i = i + 1;
11     return(fittest)
```

Procedure initialiseGAPopulation

Input: fittest GP Genotype, training dataset
Output: initial population of GA genotypes P

```
1  begin
       // one feature in GA population for each active tree in
          fittest GP genotype:
2      features[] = empty array;
3      f = 0;
4      foreach tree in fittest GP Genotype do
5          features[f] = tree;
6          f = f + 1;
       // ensure that all numeric attributes from training dataset
          are present:
7      foreach numericAttribute in training dataset do
8          if numericAttribute not in features[] then
9              features[f] = numericAttribute;
10             f = f + 1;
11     i = 0;
       // first genotype has all features active:
12     foreach feature in features do
13         P[i].features[index].active = true ;
14     i = i + 1;
15     while i < 101 do
           // activity of features in remaining genotypes is random:
16         foreach feature in features do
17             P[i].features[index].active = randomBoolean ();
18         i = i + 1;
19     return(P)
```

Procedure breedGAPopulation

Input: old generation of genotypes P_{t-1}
Output: new generation of genotypes P_t

```
1  begin
2      P_t[0] = findFittestGA (P_{t-1});
3      i = 1;
4      while i < 100 do
5          P_t[i+1] = P_t[i] = tournamentSelection (P_{t-1}); // group size 8,
           30% chance of selecting fittest
6          while P_t[i+1] = P_t[i] do
7              P_t[i+1] = tournamentSelection (P_{t-1});
8          twoPointCrossover (P_t[i], P_t[i+1]);
9          mutateGA (P_t[i]);
10         mutateGA (P_t[i+1]);
11         i = i + 2;
12     return(P_t)
```

Procedure twoPointCrossoverGA

Input: *genotype*1, *genotype*2

```
1  begin
2      if random (0,1) ≤ 0.6 then
3          a =random (0,numberOfTrees);
4          b =random (0,numberOfTrees);
           // ensure a < b
           // swap flags between a and b from genotype1 to genotype2
```

Procedure mutateGA

Input: *genotype*

```
1  begin
2      foreach tree in genotype do
3          if random (0,1) ≤ 0.005 then
               // toggle this flag
       // if this causes all flags to be false, toggle a random flag
```

```
Procedure findFittestGA
  Input: population of GA genotypes P
  Output: fittest member of P
1 begin
2     fittest = P[0];
3     i = 1;
4     while i < 101 do
5         if P[i].accuracy > fittest.accuracy then
6             fittest = P[i];
7         else if P[i].accuracy = fittest.accuracy then
8             if P[i].countOfActiveTrees < fittest.countOfActiveTrees then
                  // countOfActiveTrees refers to the number of flags
                     set to true
9                 fittest = P[i];
10            else if P[i].length = fittest.length then
11                if P[i].age > fittest.age then
12                    fittest = P[i];
13        i = i + 1;
14    return(fittest)
```

3.3.1 Feature Construction

Initialise the population

A population of 101 genotypes is created at random[7]. Each genotype consists of n trees, where n is the number of numeric valued attributes in the dataset[8]. A tree can be either an original attribute or a compound feature (an Automatically Defined Function, or ADF[9], with the part of the result producing branch being played by C4.5). That is, a genotype consists of n GP trees, each of which may contain 1 or more nodes. The number of trees is tied to the number of attributes in the original dataset to allow the size of genotypes to vary in proportion to the dimensionality of the dataset without having to maintain a parameter specifying the genotype length. The chance of a node being a leaf node (a primitive attribute) is determined by equation 3.1:

[7]The reason for 101 rather than 100 individuals is purely historical. In a very early iteration of the algorithm, elitism (keeping the single fittest individual alive between generations) was turned on if the population size was odd. This is no longer the mechanism by which elitism is switched on, but the population size has stuck.

[8]Subject to a minimum of 7 trees. This minimum is chosen to ensure that, for datasets with a small number of numeric attributes, the initial population contains a enough compound features to give a good chance of success. This measure was introduced initially when the algorithm was being developed against a dataset with only 3 attributes and affects only the Liver Disorder and New Thyroid datasets, with 6 and 5 attributes respectively.

[9]It should be noted that these compound features cannot make calls on other features (or be called by them), unlike the ADFs discussed in section 2.2.2.

$$P_{leaf} = 1 - \frac{1}{(depth + 1)} \tag{3.1}$$

Where *depth* is the depth of the tree at the current node. Hence a root node will have a depth of 1, and therefore a probability of 0.5 of being a leaf node. Nodes at depth 2 will have a 0.67 probability of being a leaf node, and so on. If a node is a leaf node, it takes the value of one of the original attributes chosen at random. Otherwise, a function is randomly chosen from the set {*, /, +, -, %}[10] and two child nodes are generated. In this manner there is no absolute limit placed on the depth any one tree may reach but the average depth is limited to 1.96. The total number of nodes in a tree averages 2.92 and the frequency of node counts decreases on a logarithmic scale. Figure 3.3 shows the frequency of node counts (total number of nodes in a tree) and tree depths over a sample of 100 million trees created according to equation 3.1.

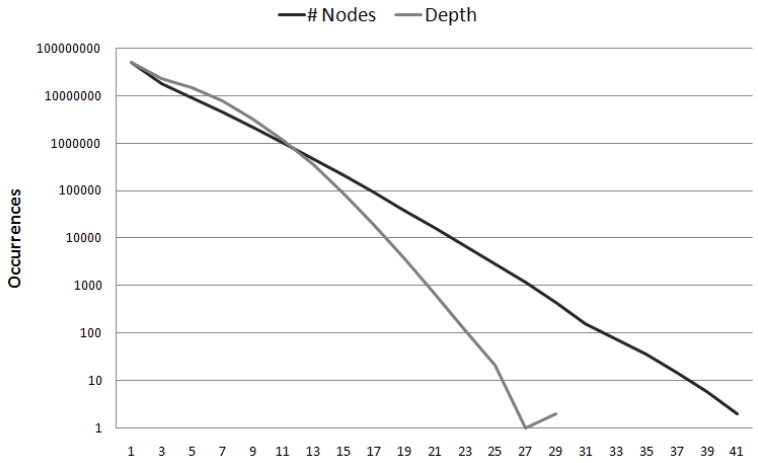


Figure 3.3: Node counts and tree depths in the initial population

During the initial creation no two trees in a single genotype are allowed to be alike (i.e. duplicate attributes/compound features are replaced), though this restriction is not enforced in later stages. Additionally, it is pointless for nodes with functions '–', '%' or '/' to have child nodes that are equal to each other (as they will always equate to 0 or 1). In order to prevent this child nodes within a function '*' or '+' are ordered alphabetically to enable comparison (e.g. [width + length]

[10] An additional function, the natural logarithm $log_e(x)$ was added later, in chapter 5 and onward.

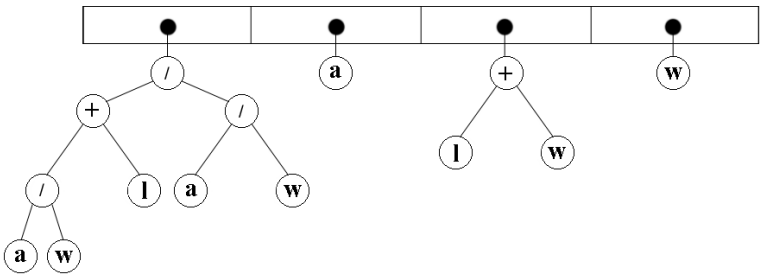


Figure 3.4: Example Genotype

will become [length + width]) - this allows the sub-trees of '–', '%' and '/' nodes to be accurately compared.

Genetic Programming algorithms frequently make use of random constants as node values; these are not employed by the GAP algorithm. This is partly for the sake of simplicity but also because such constants are usually employed for comparison (e.g. 'length less than 0.5') while the purpose of the trees in the genotype is to find useful relationships between attributes, leaving comparison and decision making to the classifier. Figure 3.4 shows a graphical depiction of an example genotype with 4 trees (the example images in this chapter are limited to four trees for space reasons, normally a genotype would consist of at least 7 trees). In the example **a**, **l** and **w** are the attributes Area, Length and Width of an artificial Shapes dataset.

Evaluation

An individual is evaluated by constructing a new dataset (from the training data) with one feature for each tree in the genotype. During calculation of instance values the result of any invalid calculation (e.g. division by 0, or a calculation involving a missing value) is replaced by zero. While this may suggest the loss of information (e.g. the difference between 0 and a missing value might be of use to C4.5) it is better than making the result of the entire function a missing value; which might in turn cause the loss of information about any other attributes in the function. Missing attributes that do not form part of a function are passed to the classifier without change (i.e. not converted to 0). The resulting dataset is then passed to a C4.5 classifier (using default

parameters), whose performance on the dataset is evaluated using 10-fold cross validation. The percentage correct is then assigned to the individual and used as the fitness score.

Once the initial population has been evaluated, several generations of selection, crossover, mutation and evaluation are performed. After each evaluation, if the fittest individual in the current generation is fitter than the fittest so far, a copy of it is set aside and the generation noted. A form of elitism is practised, whereby the fittest genotype is copied unchanged into the next generation.

Selection

Tournament Selection is used to select the parents of the next generation, with each parent being the fittest of a tournament group of 8 randomly selected individuals. The results in this chapter were obtained by applying a probability of 0.3 to the chance of selecting the winner of the tournament (otherwise the parent was selected at random from the group).[11]

Crossover

Two forms of crossover are practised in the GAP algorithm. First, copies are made of the parents - these are the offspring for the next generation. The offspring are then subject to GA and GP crossover.

GA Crossover: Two-point crossover is performed with a probability of 0.6, in which whole trees are exchanged between the offspring. Figure 3.5 shows GA crossover occurring between two randomly selected points shown by dotted lines. Tests using this algorithm have shown no significant difference between two-point crossover (with inversion) and the more common uniform crossover. Of ten datasets tested, 5 showed a higher fitness with two-point crossover and 5 a lower fitness, with no significant difference overall on a T-test at the 95% confidence level. (Similar results were also obtained comparing two-point crossover with inversion and uniform crossover with inversion).

GP Crossover: The offspring are also subject to GP crossover with a probability of 0.6 (this is separate to the chance for GA crossover, so a pair of offspring may be subject to both forms of crossover with a

[11] Please note that this probability was later replaced by certainty - the winner is always chosen, as is the more common approach (see chapter 4 for more details).

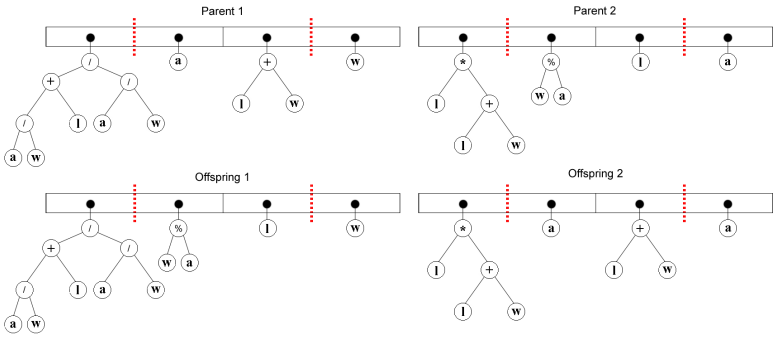


Figure 3.5: GA Crossover

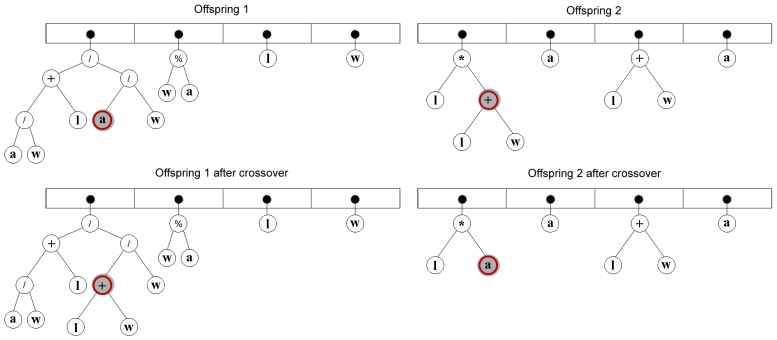


Figure 3.6: GP Crossover

probability of $0.6 * 0.6 = 0.36$). A random tree is selected (at the same position in both individuals), a random node is selected within the tree and sub-trees are exchanged between the offspring, as illustrated in figure 3.6.

Mutation

There is a small probability (0.008) of mutation for each node in every tree in the genotype, in which case the node is replaced by a new sub-tree (the new sub-tree is created according equation 3.1). This is illustrated in figure 3.7 (assuming a single node in the tree is mutated). There is additionally a chance (with probability 0.2) that the order of trees between two random points in an individual is inverted, as illustrated in figure 3.8. This does not directly affect the fitness of a genotype (as classifiers tend to ignore the order of features) but does allow trees to change location within the genotype (thus potentially allowing two trees in the same loci in different genotypes to appear in

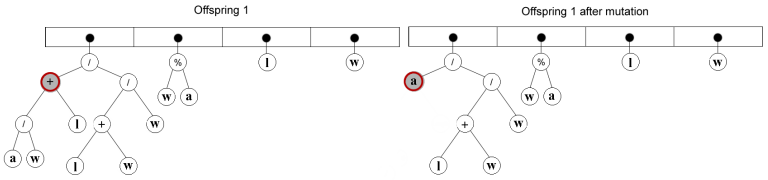


Figure 3.7: Mutation

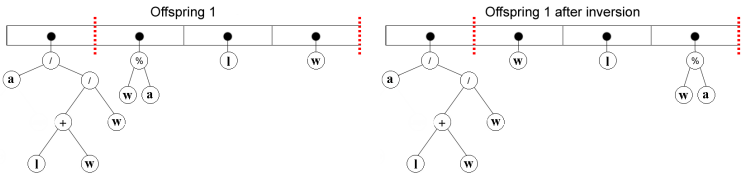


Figure 3.8: Inversion

the same genotype in a later generation).

Termination Criteria

Rather than specify a fixed number of generations the cycle of evaluation, selection, crossover and mutation continues until there has been no improvement in the accuracy of the fittest individual for six generations (subject to a minimum of ten generations in total). While there is no fixed limit the number of generations run remains quite low - varying between averages of 11.25 and 32 (for the Wine and Sonar datasets respectively) with an overall average of 19.3 (for the create stage - a similar count of generations applies for the select stage).

3.3.2 Feature Selection

The fittest individual from the feature creation stage is analysed to see if any original attributes are missing from the genotype. If there are any missing, they are added to the end of the genotype. This extended genotype (potentially twice as long as the winner of the feature creation stage - e.g. if the genotype consists solely of constructed features then all the original attributes will be reinstated) is used as the basis of the new population. In an attempt to reduce overfitting of the data, the order of the dataset is randomised at this point. This has the effect of providing a different split of the data for 10-fold cross validation during the selection stage, giving the algorithm a chance of recognising

attributes that over-fit the particular data partition in the creation stage. As a result of the randomisation, it is usually the case that the fitness score of individuals in the select stage is initially less than that of individuals in the create stage. The benefit of randomising the dataset is demonstrated at the end of this chapter.

Initialise the population

A new population of 101 bit strings is randomly created. The bit strings have the same number of bits as the seed genotype has trees. The last member of the population, the 101st bit string, is not randomly created but is initialised to all ones. This ensures that there are no missing alleles at the start of the selection stage (only once has this bit string been the fittest of the first generation, and it has never been the overall winner of the selection stage).

Evaluation

A bit string is evaluated by taking a copy of the parent dataset and removing every attribute that has a '0' in the corresponding position in the bit string. As in the feature creation stage, this dataset is then passed to a J48 classifier (J48 is the Weka implementation of C4.5) whose performance on the dataset is evaluated using 10-fold cross validation. If the fittest individual in the current generation has a higher fitness score than the fittest so far, or it has the same fitness score and fewer '1's, a copy of it is set aside and the generation noted.

Selection

As with the Construction stage, Tournament Selection is used to select the parents of the next generation, with each parent being the fittest of a tournament group of 8 randomly selected individuals (again with a probability of 0.3, otherwise the parent is selected randomly from the group).

Crossover

As with the GA crossover from the construction stage, two-point crossover is performed with a probability of 0.6.

Mutation

There is a probability of 0.005 for each bit in the string that it will be flipped to the opposite value - a standard GA mutation operator. If the genotype should have all trees deactivated (all bits '0') then a single random bit is flipped to '1'.

Termination Criteria

As before the cycle of evaluation, selection, crossover and mutation continues until there has been no improvement in the accuracy of the fittest individual for six generations (subject to a minimum of ten generations in total).

3.3.3 Results

At the end of the second stage the fittest genotype (the most accurate, taking the shortest (i.e. with fewest '1's) in case of ties) is evaluated on the test data and compared with C4.5. This is repeated 20 times for each dataset using 90% train and 10% test data and the results are shown in table 3.3. Results that are significant at the 95% confidence level using a paired T-test are shown in bold. The GAP algorithm out-performs C4.5(J48) on eight out of ten datasets, and provides a significant improvement on three (Glass Identification, New Thyroid, and Wisconsin Breast Cancer Original) - two of which are significant at the 99% confidence level. There are no datasets on which the GAP algorithm performs significantly worse than C4.5 alone. Comparative results for SVM and published results for other algorithms are shown in table 4.10 on page 108.

The standard deviation of the GAP algorithm's results do not differ greatly from that of C4.5 alone; there are five datasets where the GAP algorithms' standard deviation is greater and five where it is smaller. This is a pleasantly surprising aspect of the results, given that the GAP algorithm is nondeterministic[12] while C4.5 is deterministic.

[12]Strictly speaking, the GAP algorithm is a probabilistic deterministic algorithm, as all choices are determined by a (pseudo) random number generator.

Table 3.3: Improvement to performance of C4.5 provided by GAP

Dataset	GAP+C4.5	S.D.	C4.5 (J48)	S.D.	Paired t-test
LiverDisorder	65.97	(11.27)	**66.37**	(8.86)	-0.22
Glass	**73.74**	(9.86)	68.28	(8.86)	**3.39**
Ionosphere	89.38	(4.76)	**89.82**	(4.79)	-0.34
NewThyroid	**96.27**	(4.17)	92.31	(4.14)	**3.02**
Diabetes	**73.50**	(4.23)	73.32	(5.25)	0.19
Sonar	**73.98**	(11.29)	73.86	(10.92)	0.05
Vehicle	**72.46**	(4.72)	72.22	(3.33)	0.20
Wine	**94.68**	(5.66)	93.27	(5.70)	0.85
WBC 30	**95.62**	(2.89)	93.88	(4.22)	1.87
WBC 9	**95.63**	(1.58)	94.42	(3.05)	**2.11**
Overall Average	83.12		81.77		2.91

Comparing effects of Feature Construction and Selection

Table 3.4 shows the effects on C4.5 of running the individual feature construction and selection stages on their own, and comparing them with the full GAP algorithm (combined feature construction and selection, as shown in table 3.3). Results that show a significant improvement over C4.5 alone at the 95% confidence level (as indicated by a t-test) are shown in bold. Running the feature selection stage alone offers no real difference to C4.5 alone, and t-tests show no significant difference at the 95% confidence level on any dataset. This is unsurprising when you consider that C4.5 performs feature selection during construction of each node of the decision tree, though the feature selection stage should prove beneficial to other algorithms such as IBk (see 4.3 'Applying the algorithm to other classifiers').

Running the feature construction stage on its own gives better results than running both stages on most of the datasets. T-tests show a significant improvement over C4.5 on half of the datasets at the 95% confidence level, though on the glass dataset a t-test shows significantly worse performance at the 95% confidence level (compared with a significant *improvement* when running both stages). This suggests that in general C4.5 is better at feature selection than the GA used in the selection stage, though there are some datasets (notably glass) where including the feature selection stage gives a better performance. It is likely that features created in the construction stage have over-fit the training data (perhaps because of noise in the dataset), and re-ordering

the dataset before running the selection stage gives a chance to recognise and remove these features.

Table 3.4: Comparison of effects of Feature Construction and Selection

Dataset	FC + C4.5	S.D.	FS + C4.5	S.D.	FC+FS + C4.5	S.D.
LiverDisorder	**70.73**	(8.62)	66.41	(9.09)	65.97	(11.27)
Glass	60.56	(10.39)	66.91	(10.57)	**73.74**	(9.86)
Ionosphere	91.22	(4.39)	89.35	(6.07)	89.38	(4.76)
NewThyroid	**96.03**	(4.88)	92.56	(5.12)	**96.27**	(4.17)
Diabetes	75.58	(5.49)	72.46	(4.78)	73.50	(4.23)
Sonar	74.70	(8.49)	73.32	(9.63)	73.98	(11.29)
Vehicle	71.52	(4.99)	71.40	(3.49)	72.46	(4.72)
Wine	**96.63**	(4.26)	93.25	(5.70)	94.68	(5.66)
WBC 30	**96.85**	(3.12)	94.64	(3.52)	95.62	(2.89)
WBC 9	**96.21**	(1.70)	94.84	(2.73)	**95.63**	(1.58)
Overall Average	83.00		81.51		83.12	

Initialisation Techniques

It is normal in Genetic Programming to use the 'Ramped-half-and-half' initialisation technique rather than one such as outlined above (hereafter referred to as P_{leaf}). P_{leaf} was initially developed because the author set about the task with more enthusiasm than knowledge, but it has been retained because it is particularly suited to the situation. As can be seen in table 3.5, initialising the population with Ramped-half-and-half (with a depth limit of 6) provides improved fitness over P_{leaf} (with better fitness in 8 out of 10 datasets, shown in bold) but this results in a *lower* average accuracy on the test data (lower test accuracy on 7 of 10 datasets), and slightly *larger* standard deviation on most datasets. The final solutions obtained using ramped-half-and-half initialisation contain on average 0.88 more trees than P_{leaf} and the trees are larger, with an average 3.6 more nodes per tree. Those extra nodes seem to have the same effect as too many nodes in an artificial neural network; causing the solution to over-fit the training data.

Computational Effort

As this algorithm employs a wrapper approach it is computationally expensive when compared to C4.5 alone (though for an embarked application only the single best solution would be used, and the computa-

Table 3.5: Comparison of initialisation techniques

	Train		Test	
	P_{leaf}	**RH&H**	P_{leaf}	**RH&H**
LiverDisorder	75.04	**78.10**	65.97	**66.52**
	(2.37)	(1.59)	(11.27)	(9.26)
Glass	78.17	**80.12**	**73.74**	68.83
	(1.95)	(2.30)	(9.86)	(11.03)
Ionosphere	**95.99**	95.89	89.38	**89.62**
	(0.87)	(1.04)	(4.76)	(4.99)
NewThyroid	98.22	**98.94**	**96.27**	94.47
	(0.68)	(0.46)	(4.17)	(4.81)
Diabetes	78.15	**79.65**	73.50	**73.82**
	(0.96)	(0.92)	(4.23)	(4.43)
Sonar	**92.33**	92.23	**73.98**	73.16
	(1.27)	(2.68)	(11.29)	(11.94)
Vehicle	78.82	**79.33**	**72.46**	72.28
	(1.21)	(1.76)	(4.72)	(5.00)
Wine	98.47	**99.13**	**94.68**	93.86
	(0.80)	(0.68)	(5.66)	(7.49)
WBC 30	97.86	**98.55**	**95.62**	94.65
	(0.47)	(0.42)	(2.89)	(3.07)
WBC 9	97.65	**98.05**	**95.63**	95.63
	(0.38)	(0.28)	(1.58)	(2.28)
Overall Average	89.07	**90.00**	**83.12**	82.28

Standard deviations are shown in brackets below the average

tional effort would differ from C4.5 only in the time taken to transform the dataset). For instance on the New Thyroid dataset the algorithm runs for an average of 23.8 generations (this figure includes both create and select stages). With a population size of 101 (and assuming an unlikely worst case where every genotype is unfamiliar and requires fitness evaluation) this means 2404 genotypes requiring evaluation. As fitness evaluation involves 10-fold cross-validation on the training data, this equates to roughly 24,040 times the amount of computational effort of C4.5 alone[13] - something in the region of 3 minutes per run on a 1.4Ghz PC. See also chapter 6 'Ensembles with GAP' for comparisons with ensemble techniques such as Boosting.

Utility of Constructed Features

The results in Table 3.3 represent a total of two hundred individual solutions. In those two hundred individuals there are a total of 2,425

[13]The effort required to transform the dataset is minimal compared with 10-fold cross-validation, so it has been ignored for this calculation.

trees: 982 constructed features and 1,443 original attributes - a ratio of roughly two constructed features to three original attributes. All but two of the two hundred individuals (99%) contained at least one constructed feature. Table 3.6 gives details of the average number of ADFs per individual for each dataset (the number of original attributes used is not shown but may be deduced by subtracting the average count of ADFs from the count of trees).

Table 3.6: Counts of constructed features by dataset

Dataset	# Attributes in Dataset	Average # Trees	Average # ADFs	Average # Nodes	Minimum # ADFs
LiverDisorder	6	4.9	2.6	17.2	1
Glass	9	7.1	3.1	22.4	1
Ionosphere	34	19.7	7.6	54.3	4
NewThyroid	5	3.2	2.3	11.9	1
Diabetes	8	6.4	2.7	18.1	1
Sonar	60	38	13.6	95.7	5
Vehicle	18	16.3	5.5	39.3	2
Wine	13	5.2	2.1	13.6	0
WBC 30	30	15.7	7	44.8	4
WBC 9	9	5	2.9	18.3	1

While we cannot be certain that all the features in the solutions are of use to C4.5 (but see section 5.3 'Post-Processing' for ways to address this), we can be reasonably confident that by the end of the selection stage many of the attributes are useful in most of the solutions. This can be illustrated by looking in detail at a single solution. One of the most accurate solutions for the New Thyroid dataset has 3 trees, two of which are constructed features. The original dataset contains five attributes and the class:

a. T3-resin uptake test. (A percentage)

b. Total Serum thyroxin as measured by the isotopic displacement method.

c. Total serum triiodothyronine as measured by radioimmuno assay.

d. Basal thyroid-stimulating hormone (TSH) as measured by radioimmuno assay.

e. Maximal absolute difference of TSH value after injection of 200 micro grams of thyrotropin-releasing hormone as compared to the basal

value.

Class attribute. (normal, hyper, hypo)

The 3 trees of the chosen solution utilise four of the original attributes:

i. "e"

ii. "((a/d)*b)"

iii. "(b/a)"

The decision tree created by C4.5 using these constructed features is shown in figure 3.9 (the numbers after the class prediction indicate the count of instances correctly / incorrectly classified by that node).[14] C4.5 uses the constructed features to classify a large majority of the instances, only referring to an original attribute ("e") to classify 50 of 215 instances. The decision tree is also simpler than that created using the five original attributes (figure 3.10) with 5 decision nodes compared to 8. Out of 20 runs on the New Thyroid dataset, the ADF "(b/a)", or its inverse "(a/b)", appears in 15 separate runs and there is a similar feature in some of the other 5 runs (e.g. "((a+a)/b)" appears in one run, "(a/(b+b))" in another). This is evidently a useful relationship for classifying the New Thyroid dataset (it appears more often than the most common original attribute, "e", which appears 11 times).

We can further demonstrate that the constructed features are of use by comparing the results with those when performing feature selection only. The results in table 3.7 show the effect of C4.5 of running the feature selection stage only. The results show no significant difference at the 95% confidence level for any of the 10 datasets, and no significant difference overall. This contrasts to the 3 datasets for which a combination of construction and selection offered a significant improvement over C4.5 alone at the 95% confidence level, as shown previously in table 3.3.

[14] The images of decision trees were taken with an adaptation of the Weka PrefuseTree plugin, using the prefuse visualisation toolkit. For more details see `http://weka.wiki.sourceforge.net/Explorer+tree+visualization+plugins`.

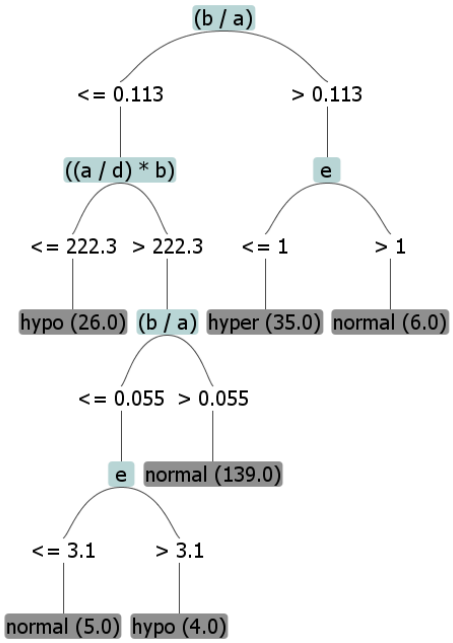


Figure 3.9: C4.5 Decision Tree for New Thyroid (constructed features)

Size of Constructed Features

One of the criticisms often levelled at Genetic Programming is that the size (usually measured by the depth) of the program trees grows uncontrollably as a result of crossover (code *bloat*). The initial population is generally limited to a fixed maximum depth but crossover can extend this without limit in subsequent generations - though this behaviour can be avoided with one-point crossover and point mutation [149]. This means it can become very difficult to make sense of what the tree is doing. Though there is no absolute limit on the depth of trees in the GAP algorithm, the P_{leaf} formula causes the initial population to tend toward shorter trees; though they are not restricted in size during crossover. However the GAP algorithm does not suffer from code bloat to any meaningful degree. In the 982 ADFs of the 200 solutions the average depth of a tree is 3.0, with a standard deviation of 1.16. The maximum depth of those trees is 9, and table 3.6 includes a column showing the average size (number of nodes) of the solutions (this includes function nodes, so 'Length * Width' is 3 nodes). There are three factors that act to limit the size of the trees:

1. The tendency toward smaller trees in the initial population.

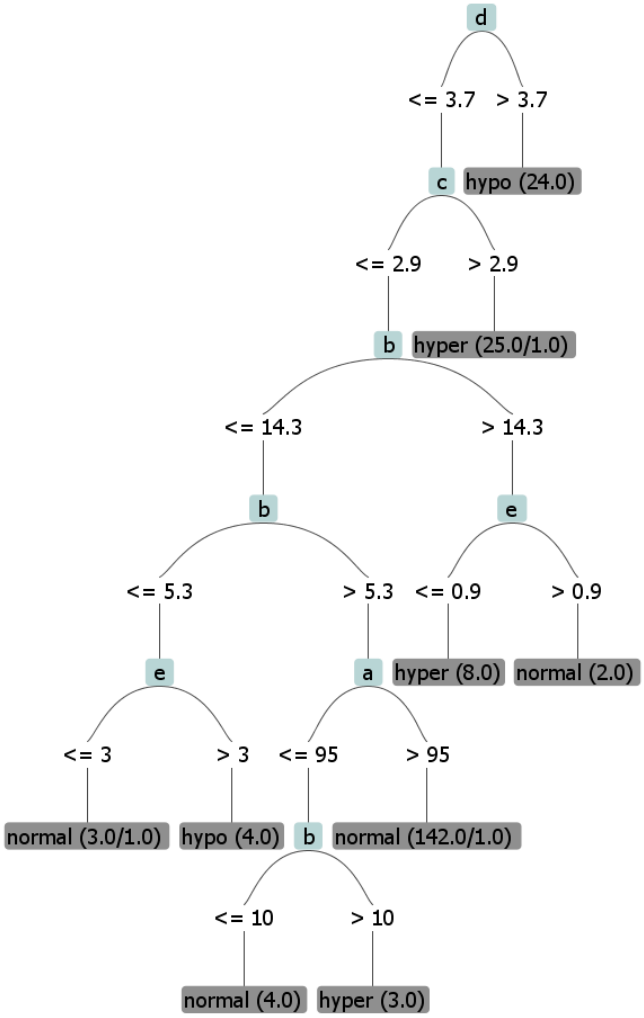


Figure 3.10: C4.5 Decision Tree for New Thyroid (original attributes)

Table 3.7: Feature selection with C4.5

	Feature Selection + C4.5	SD	C4.5	SD	t-test
LiverDisorder	66.41	9.09	**66.37**	8.86	0.03
Glass	66.91	10.57	**68.28**	8.86	-0.58
Ionosphere	89.35	6.07	**89.82**	4.79	-0.39
NewThyroid	**92.56**	5.12	92.31	4.14	0.21
Diabetes	72.46	4.78	**73.32**	5.25	-1.01
Sonar	73.32	9.63	**73.86**	10.92	-0.23
Vehicle	71.40	3.49	**72.22**	3.33	-0.93
Wine	93.25	5.70	**93.27**	5.70	-0.01
WBC 30	**94.64**	3.52	93.88	4.22	0.92
WBC 9	**94.84**	2.73	94.42	3.05	0.83
Overall Average	81.51		81.77		-0.61

2. GP Crossover is not as destructive an operation as in other GP algorithms. Although each individual is likely to be subject to GP crossover, it acts only on one of several program trees possessed by the individual so the number of program trees experiencing crossover remains relatively low. It also has less impact on the overall fitness of the individual than if it consisted of a single tree (the normal case in GP).
3. When a node in a program tree is mutated it is replaced by a new sub-tree, subject to the same restraints on depth as during the initial creation of the population. So mutation is likely to reduce the size of deep trees over time.

The order of the Create and Select stages

As noted earlier, Vafie and DeJong used a similar approach but employed selection before any construction occurred. Table 3.8 shows the negative effect on the GAP algorithm of putting the selection stage first. Although putting the select stage first does reduce the standard deviation of the accuracy of the solutions; putting the create stage first gives improved performance on 6 of ten datasets, there is significantly better performance at the 95% confidence level on 2 datasets and none significantly worse. This is due to the create stage having all attributes available. For instance, on the New Thyroid dataset the select stage will usually remove either attribute ‘a’ or attribute ‘b’ thus preventing

the create stage from being able to construct the feature 'b/a' shown to be useful in section 3.3.3 'Utility of Constructed Features'.

Table 3.8: Comparing the order of Create and Select stages

Dataset	Create 1^{st}	S.D.	Select 1^{st}	S.D.	Paired t-test
LiverDisorder	65.97	(11.27)	**67.42**	(8.23)	-0.95
Glass	**73.74**	(9.86)	68.75	(6.36)	**2.99**
Ionosphere	**89.38**	(4.76)	89.02	(4.62)	0.25
NewThyroid	**96.27**	(4.17)	93.67	(5.82)	**2.01**
Diabetes	73.50	(4.23)	**74.09**	(4.46)	-0.67
Sonar	**73.98**	(11.29)	73.16	(9.05)	0.32
Vehicle	**72.46**	(4.72)	71.22	(3.13)	1.19
Wine	94.68	(5.66)	**94.69**	(3.80)	-0.01
WBC 30	95.62	(2.89)	**95.88**	(3.15)	-0.37
WBC 9	**95.63**	(1.58)	95.13	(2.16)	1.04
Overall Average	83.12		82.30		

Effect of randomising the dataset between stages

It has been mentioned that the training data is randomly reordered before the second stage commenced, it was hoped that this would reduce over fitting and improve the performance on the test data. In order to test this the algorithm was retested without randomisation, results shown in table 3.9. The first impression is that there is no important difference - there are 5 datasets where not reordering gives a better result and 5 where it is worse. However, there are now only two (rather than three) datasets on which the algorithm provides a significant improvement over C4.5 (New Thyroid and Wisconsin Breast Cancer); and most importantly the t-test performed over the 200 runs from all datasets no longer shows a significant improvement at the 95% confidence level. (In table 3.9 the column for paired t-test shows the results for testing the algorithm *without* randomisation against C4.5):

3.4 Conclusion

This chapter has introduced the GAP algorithm, a pre-processing stage that aims to improve the accuracy of a simple classifier with the added benefit of being able to highlight the relationships between attributes that allow this improvement. We have shown that GP individuals consisting of multiple trees/ADFs can be used for effective feature creation

Table 3.9: Effect of randomising the dataset

Dataset	randomised	non-randomised	t-test v C4.5
LiverDisorder	65.97	**66.65**	0.17
Glass	**73.74**	69.74	0.61
Ionosphere	89.38	**89.77**	-0.04
NewThyroid	96.27	**97.22**	**3.99**
Diabetes	**73.5**	71.74	-1.37
Sonar	73.98	**75.22**	0.5
Vehicle	**72.46**	71.94	-0.28
Wine	**94.68**	94.08	0.55
WBC 30	**95.62**	94.56	0.71
WBC 9	95.63	**95.71**	**2.03**
Overall Average	83.12	82.66	1.82

and that solutions, combined with feature selection via a GA, can give significant improvements to the classification accuracy of C4.5. We have also shown that it is beneficial not to put feature selection before creation, and that randomising the order of the training data between stages is also beneficial in reducing overfitting.

Chapter 4

Extending the GAP algorithm

4.1 Introduction

This chapter presents three different versions of the GAP algorithm, of which the final version (4.3) is the base version used in future chapters. First the creation and selection stages are combined into a single stage (algorithm 4.1), then some amendments are made to improve performance (algorithm 4.2). This version is then tested with other simple classifiers, namely IBk, Naïve Bayes, and a linear classifier. Finally the algorithm is extended to select the most appropriate of 3 classifiers for the dataset, and some final amendments are made to this extended version (algorithm 4.3). Algorithm 4.3 is then tested against C4.5 again to confirm that there has been no loss in accuracy from the initial version, and performance is compared against both SVM and a survey of results from the literature.

4.2 Combining creation and selection in a single stage

Knowing that feature construction and selection work as separate stages (as long as selection does not come first) it seems desirable to combine both steps into a single stage so as to expand the search space for the selection part of the algorithm. With this in mind the algorithm was redesigned to include feature selection in the construction stage. When

operating in two stages, the GA has only a single set of features to select from - those of the final genotype from the construction stage. When the two stages are combined the selection algorithm can operate on the entire GP population and so has a much wider range of feature sets to select from. The combination of construction and selection in a single stage was first published in [173]. Feature construction occurs as before, but within each individual every tree now has a boolean flag associated with it to determine whether the tree is passed to C4.5 for evaluation. During crossover each tree retains its associated boolean flag, which is subject to the same chance of mutation as during the second stage (0.005). Pseudo-code for this is shown in algorithm 4.1.

Algorithm 4.1: The GAP Algorithm: Single stage (initial)

```
1  fittest = null;
2  t = 0;
3  P_t = initialisePopulation ();
4  evaluatePopulation (P_t) ; // 10-fold cross-validation on train
   data
5  fittest =findFittest (fittest, P_t);
6  while true do
7      t = t + 1;
8      P_t = breedPopulation (P_{t-1});
9      evaluatePopulation (P_t);
10     fittest = findFittest (fittest, P_t);
11     if age (fittest) ≥ 6 and t ≥ 10 then
12         return (fittest);
```

Procedure breedPopulation

```
   Input: old generation of genotypes P_{t-1}
   Output: new generation of genotypes P_t
1  begin
2      P_t[0] = findFittest (P_{t-1});
3      i = 1;
4      while i < 100 do
5          P_t[i + 1] = P_t[i] = tournamentSelection (P_{t-1}); // group size 8,
           30% chance of selecting fittest
6          while P_t[i + 1] = P_t[i] do
7              P_t[i + 1] = tournamentSelection (P_{t-1});
8          twoPointCrossover (P_t[i], P_t[i + 1]);
9          mutate (P_t[i]);
10         mutate (P_t[i + 1]);
11         i = i + 2;
12     return(P_t)
```

Procedure twoPointCrossover

```
   Input: genotype1, genotype2
1  begin
2      if random (0,1) ≤ 0.6 then
3          a =random (0,numberOfTrees);
4          b =random (0,numberOfTrees);
           // ensure a < b
           // swap trees between a and b from genotype1 to genotype2
5      if random (0,1) ≤ 0.6 then
           // select a random tree (at the same locus) in each
              individual
           // exchange a random subtree between them
```

Procedure mutate

```
   Input: genotype
1  begin
2      foreach tree in genotype do
3          foreach node in tree do
4              if random (0,1) ≤ 0.008 then
                   // replace node with a new subtree created according
                      to P_leaf equation with Depth = 1
5          if random (0,1) ≤ 0.005 then
               // toggle the activity of this tree
       // If no trees are active, randomly select a tree to make
          active
6      if random (0,1) ≤ 0.2 then
           // invert (reverse) the order of trees between two
              randomly selected points
```

```
Procedure findFittest
  Input: population of genotypes P
  Output: fittest member of P
1 begin
2     fittest = P[0];
3     i = 1;
4     while i < 101 do
5         if P[i].accuracy > fittest.accuracy then
6             fittest = P[i];
7         else if P[i].accuracy = fittest.accuracy then
8             if P[i].countOfActiveTrees < fittest.countOfActiveTrees then
9                 fittest = P[i];
10            else if P[i].countOfActiveTrees = fittest.countOfActiveTrees
              then
11                if P[i].age > fittest.age then
12                    fittest = P[i];
13        i = i + 1;
14    return(fittest)
```

Testing the single stage algorithm with the same parameter values as before gives a much shorter run time (not surprisingly, roughly half the time of the two stage algorithm) but with poorer results - an overall average of 82.20%[1] (though this is still an improvement over unaided C4.5).

There are three primary differences between the two versions of the algorithm that might account for this drop in performance:

1. With a single stage we are asking the algorithm to do the same amount of work in half the time.

2. There is no longer an opportunity to randomly reorder the dataset between stages.

3. There is no longer an opportunity to reintroduce any original attributes that have been dropped during the first stage.

There seems no immediately obvious way to address the third of these differences with a single stage approach[2], but the other two can be compensated for. Firstly we can change the termination criteria - by doubling both the minimum number of generations to 20 and the age

[1] The average result over all 10 datasets is taken as a useful indicator of the performance of the algorithm (and substantially briefer than showing a table with 10 sets of results).

[2] A genotype is still created with all original attributes in the initial population, but there is no re-introduction of original attributes. This could be achieved when reordering the dataset but as the results in table 4.1 indicate, it is not necessary.

of the fittest individual to 12 generations. Doing this does improve the result (to an overall average result of 82.88%), but not sufficiently to bring it into line with a two stage process.

Additionally we can randomly reorder the dataset. Two approaches were considered. One was to have two versions of the dataset from the start, with the same data but in a different order, and simply alternate between datasets when evaluating each generation (i.e. the first dataset was used to evaluate even numbered generations, the second to evaluate odd numbered) - this approach did not seem to improve the results (an average result of 82.24%). The more successful approach was to reorder the dataset once the termination criteria had been reached. That is, when the fittest individual reaches 12 generations old instead of stopping; randomly reorder the training dataset, re-evaluate the current generation and reset the fittest individual, then continue until the termination criteria are met again (the minimum age criteria only, the minimum generations of 20 having been already reached). Pseudo-code for this modified version is shown in algorithm 4.2 (only the main algorithm changes, sub procedures are the same as algorithm 4.1). The results obtained with the longer run time and randomly reordering the dataset part-way through are shown in table 4.1 (the column for paired t-test shows the results for testing the single stage algorithm against C4.5).

The extended termination criteria have provided an improvement in accuracy over the 2 stage results in 7 out of 10 datasets, and a slightly improved overall average. Although the single stage algorithm out-performs C4.5 on only one dataset at the 95% confidence level (a t-test of 1.96 or higher), as compared to three datasets for the two stage algorithm; it outperforms C4.5 on everything but the Vehicle dataset (and then loses only by a very small margin), resulting in an increase of the (already high) overall confidence of improvement over C4.5 (a confidence level of 99.9% that the algorithm offers an improvement over C4.5). This improvement has come at the cost of an increase in the number of overall generations run, to an average of 85 (up from a total of roughly 40 for the two separate stages); but with the added benefit of reducing the number of trees in the final solution from 12.2 for 2 stages to an average of 7.7 (with S.D. of 1.6). This is likely to be due to the fact that there is now greater scope for feature selection:

Algorithm 4.2: The GAP Algorithm: Single stage (amended)

```
1  canTerminate = false;
2  fittest = null;
3  t = 0;
4  P_t = initialisePopulation ();
5  evaluatePopulation (P_t) ; // 10-fold cross-validation on train
   data
6  fittest =findFittest (fittest, P_t);
7  while true do
8      t = t + 1;
9      P_t = breedPopulation (P_{t-1});
10     evaluatePopulation (P_t);
11     fittest = findFittest (fittest, P_t);
12     if age (fittest) ≥ 12 and t ≥ 20 then
13         if canTerminate then
14             return (fittest);
15         else
16             randomiseTrainingData ();
17             canTerminate = true;
```

the selection GA is no longer operating solely on the trees of a *single* solution from the construction stage; it now shares the more diverse population of the construction stage.

Introns

Combining feature creation and selection in a single stage means that each genotype is now carrying a number of non-functional trees (introns). Introns have been shown to protect individuals from the destructive effects of crossover not only in GP (by, among others, Nordin et al. [132]) as discussed in section 2.2.2, but also in a GA (by Levenick [111]). Additionally, Banzhaf et al. [15] have suggested that introns, having been introduced to protect against crossover, have subsequently been converted into useful code by the mutation operator. The non-functional trees could be similarly useful in the GAP algorithm: a tree that has been deactivated may undergo crossover and mutation[3] and subsequently be re-activated as a useful feature.

[3]Though in later versions of the algorithm they may only experience crossover as inactive trees will not experience mutation - see algorithm 4.3.

Table 4.1: Comparing effect on C4.5 of GAP Single stage with GAP 2 stage

Dataset	2 stage	S.D.	1 stage	S.D.	C4.5	S.D.	Paired t-test
LiverDisorder	65.97	11.27	**66.55**	8.10	66.37	8.86	0.11
Glass	**73.74**	9.86	71.84	10.26	68.28	8.86	1.78
Ionosphere	89.38	4.76	**90.69**	4.66	89.82	4.79	0.96
NewThyroid	96.27	4.17	**96.49**	3.98	92.31	4.14	**3.69**
Diabetes	73.50	4.23	**73.64**	5.11	73.32	5.25	0.24
Sonar	73.98	11.29	**75.89**	9.00	73.86	10.92	0.80
Vehicle	**72.46**	4.72	72.11	4.60	72.22	3.33	-0.09
Wine	94.68	5.66	**96.10**	4.08	93.27	5.70	1.76
WBC 30	**95.62**	2.89	95.71	4.39	93.88	4.22	1.71
WBC 9	95.63	1.58	**95.56**	2.52	94.42	3.05	1.62
Overall Average	83.12		83.46		81.77		3.62

4.3 Applying the algorithm to other classifiers

There is nothing inherent in the GAP algorithm that limits its use to C4.5. As the version of C4.5 used is part of the Weka package (J48), it is a simple matter to replace C4.5 with different classifiers and thus test the GAP algorithm with a number of different classification techniques. The following sections detail the outcome of replacing C4.5 with IBk (a k-nearest neighbour classifier with $k = 1$) and Naive Bayes (a probability based classifier). The application of the GAP algorithm to classifiers other than C4.5 was first published in [175].

An attempt was also made to apply the GAP algorithm to an artificial neural network[4], however the training time of an ANN proved prohibitive when performing cross-validation of a population of 101 individuals.

4.3.1 Applying the GAP algorithm to IBk

Breiman et al. [38] (as reported in [3]) list a number of problems experienced by nearest neighbour algorithms:

1. "they are computationally expensive classifiers since they save all training instances,

2. they are intolerant of attribute noise,

[4]Using the FastNeuralNetwork weka classifier provided by Aldebaro Klautau (a faster implementation than the gui-oriented implementation supplied with Weka), available at http://www.laps.ufpa.br/aldebaro/weka/index.html

3. they are intolerant of irrelevant attributes,
4. they are sensitive to the choice of the algorithm's similarity function,
5. there is no natural way to work with nominal-valued attributes or missing attributes, and
6. they provide little usable information regarding the structure of the data".

IBk is designed to address the first two issues. The feature selection activity of the GAP algorithm can help address the third issue (it also serves to reduce the storage requirements by reducing the feature count), while feature construction can to some extent address the fifth (recall that the immediate outcome of a calculation involving missing values is zero so that a feature involving multiple calculations may still produce useful information).

On its own IBk (with k=1) offers marginally better performance over the ten datasets than C4.5 (on average only - there are several individual datasets on which C4.5 performs better) and perhaps offers the GAP algorithm less scope for improvement. By applying the GAP algorithm there is no significant overall improvement over IBk at the 95% level and no improvement at all on half the datasets using a T-test, but there are two datasets on which GAP does offer a significant improvement (Glass and Ionosphere) and none where there is a significant drop in performance. Overall the results shown in table 4.2 are competitive with GAP using C4.5.

To determine whether the improvement was provided solely by feature selection rather than construction, the process was repeated with feature selection only, results are shown in table 4.3. The overall average is very similar to that with feature construction switched on, but there is only one dataset that shows a significant improvement at the 95% confidence level and now one dataset where feature selection has given a significantly worse result (which can only have been caused by selecting a subset of features that over-fit the training data).

Table 4.2: Applying the algorithm to IBk

Dataset	GAP + IBk	SD	IBk	SD	t-test
LiverDisorder	60.89	7.65	**62.62**	8.68	-0.86
Glass	**73.92**	9.10	68.79	9.75	**2.19**
Ionosphere	**91.38**	4.00	86.95	4.53	**3.62**
NewThyroid	95.36	3.28	**96.95**	3.73	-1.91
Diabetes	68.96	6.27	**69.90**	4.27	-0.64
Sonar	83.72	7.88	**86.65**	6.39	-1.39
Vehicle	**72.46**	5.42	70.03	4.10	1.87
Wine	**96.00**	4.88	95.44	5.81	0.52
WBC 30	94.82	3.25	**95.44**	2.92	-1.26
WBC 9	**95.64**	2.51	95.42	2.16	0.41
Overall Average	83.31		82.82		1.01

Table 4.3: Applying Feature Selection only to IBk

	Feature Selection	SD	IBk	SD	t-test
LiverDisorder	59.97	7.65	**62.62**	8.90	-1.29
Glass	**70.66**	8.03	68.79	10.00	0.83
Ionosphere	**90.63**	3.46	86.95	4.65	**3.17**
NewThyroid	96.27	4.19	**96.95**	3.82	-0.79
Diabetes	67.57	4.90	**69.90**	4.38	**-2.19**
Sonar	86.64	7.21	**86.65**	6.56	-0.01
Vehicle	**72.05**	3.92	70.03	4.20	1.65
Wine	**96.41**	4.79	95.44	5.96	0.73
WBC 30	**95.60**	2.65	95.44	3.00	0.22
WBC 9	95.41	2.23	**95.42**	2.22	-0.03
Overall Average	83.12		82.82		0.71

4.3.2 Applying the GAP algorithm to Naïve Bayes

While Naïve Bayes is often used as a bench mark comparison for GP algorithms relatively little work seems to have been done to combine GP with Naive Bayes. Langdon and Buxton [107] have combined GP and Naive Bayes by using the output of Naive Bayes (along with a number of other classification algorithms) as an input to GP, but not used GP to improve the performance of a Naive Bayes algorithm.

Most implementations of Naïve Bayes rely on the assumption that continuous attributes can be modelled using the normal distribution, but this may not be a valid assumption for many datasets. There are variants of the algorithm that address this issue, such as Flexible Bayes [88] that uses kernel density estimation[5]. These variants

[5]If you consider the Gaussian density function $g(x;\mu,\sigma)$ to be a single 'kernel', then kernel

are not employed here, but the GAP algorithm may be able to assist in overcoming the limitation by transforming or combining attributes so that they more closely resemble the normal distribution. It may also be able to assist in situations where the assumptions of the conditional independence of attributes and lack of hidden attributes do not hold true. For example it is possible to envisage a dataset with a pair of attributes (among others) that are not independent of one another and do not resemble the normal distribution, which can be combined so that the resulting function is both independent of other attributes and is more accurately modelled by the normal distribution than the original attributes. It is worth stressing that the single-stage version of GAP (algorithm 4.3) does not re-introduce the original attributes during feature selection (as occurs in the two-stage version), so no additional interdependence is introduced.

On its own, Naïve Bayes cannot compete with C4.5 or IBk over the ten datasets - it is approx. 6% worse on average (though again there are individual datasets where Naive Bayes performs better than the other two - e.g., New Thyroid, Wine). This relatively poor performance gives the GAP algorithm much greater scope for improvement. In fact, the GAP algorithm brings the results very closely into line with those achieved using C4.5 and IBk with a T-test showing an improvement significant at the 95% confidence level on 6 of the ten datasets (see table 4.4). Applying the GAP algorithm also allows Naive Bayes to outperform both IBk and C4.5 on their own when averaged over the ten datasets.

4.3.3 Applying the GAP algorithm to a Linear Classifier

The Weka library provides a simple linear classifier[6] that is based upon linear regression (modelling the relationship between a predicted variable y and one or more features X). An n-class problem is broken down into n two-class problems and a linear regression performed for each class value, where the predicted variable is the probability that the instance has that class value (this will be 0 or 1 for all training

density estimation provides an estimated density averaged over many kernels. It is also possible to use kernels with probability density functions other than the Gaussian, such as *triangle* or *cosine*. See `http://en.wikipedia.org/wiki/Kernel_(statistics)` for other examples.

[6] weka.classifiers.meta.ClassificationViaRegression, using weka.classifiers.functions.LinearRegression as the base classifier.

Table 4.4: Applying the algorithm to Naïve Bayes

Dataset	GAP + N.B.	SD	N.B.	SD	t-test
LiverDisorder	**71.35**	8.51	54.19	9.61	**5.82**
Glass	**61.45**	7.73	48.50	14.03	**4.23**
Ionosphere	**90.60**	4.38	82.37	7.31	**5.09**
NewThyroid	97.18	3.48	**97.20**	3.83	-0.02
Diabetes	**75.77**	4.83	75.13	5.00	0.59
Sonar	**77.64**	8.77	67.16	7.53	**5.80**
Vehicle	**69.03**	4.96	43.98	4.61	**20.69**
Wine	96.40	4.80	**97.99**	3.85	-1.47
WBC 30	**96.75**	2.20	93.26	4.79	**4.04**
WBC 9	95.92	2.16	**96.06**	1.74	-0.32
Overall Average	83.21		75.58		**9.68**

instances).

On it's own the Weka Linear Classifier performs slightly worse on average than IBk and C4.5 though better on some datasets and worse on others, and better on average than Naïve Bayes. Applying the GAP algorithm to this Linear Classifier brings it's average performance to very nearly the same level as GAP with the other classifiers, as shown in table 4.5. A t-test shows a significant improvement at the 95% confidence level over using the linear classifier alone over the 10 datasets, and a significant improvement on 3 of the 10 datasets with no datasets showing a significant drop in accuracy. Figures in bold show the highest accuracy for a given dataset.

Table 4.5: Applying the algorithm to a Linear Classifier

Dataset	GAP + LR	sd	Linear Regression	sd	t-test
LiverDisorder	**70.55**	(9.88)	67.39	(6.39)	1.70
Glass	**61.80**	(8.81)	54.61	(11.40)	**2.38**
Ionosphere	**89.06**	(6.76)	84.79	(5.75)	**3.11**
NewThyroid	**93.72**	(4.53)	86.26	(7.60)	**4.92**
Diabetes	76.43	(5.05)	**76.56**	(4.62)	-0.17
Sonar	72.87	(7.30)	**74.52**	(8.50)	-0.65
Vehicle	**75.18**	(3.63)	74.41	(3.35)	1.14
Wine	97.71	(3.45)	**98.58**	(2.53)	-1.32
WBC 30	**95.88**	(3.40)	95.80	(3.07)	0.18
WBC 9	95.42	(2.82)	**95.77**	(2.18)	-0.79
Overall Average	82.86		80.87		3.77

Please note that because the evaluation of the GAP algorithm with

a linear classifier was performed after other experiments the linear classifier is not included in results for the following sections.

4.4 Selecting the most effective classifier

While neither of the two new classifiers provide an improvement on GAP's overall result using C4.5, both results are competitive - regardless of the performance of the classifier on its own. It seems as if there is a ceiling on the overall results achievable with any one classifier, a finding in keeping with the No Free Lunch theorem [200]. While using any particular classifier GAP may perform well on some datasets and worse on others, the average seems to settle out at somewhere near 83% no matter which classifier is employed. That is to say that the GAP algorithm appears to provide a robustness to the classifier techniques used. It might also be used to select the most appropriate classifier type.

Given that the GAP algorithm has been successfully applied to 3 different types of classifier, each of which performs relatively well on some datasets and poorly on others (either with or without pre-processing by the GAP algorithm); it seems plausible that the accuracy of the solutions could be improved by allowing the algorithm to select the most appropriate classifier for a particular dataset based on the fitness score. The genotype is therefore extended to include the classifier type, which is set at random in the initial population. In subsequent generations the likelihood of mutating the classifier type was set at 0.2 (this being the same as the chance of inverting sections of the genotype). In each new generation the fittest genotype for every classifier is copied unchanged to the next generation (so instead of just the single fittest genotype being carried forward in each generation, there are now 3 genotypes being carried forward). The work described in this section was first published in [174].

As before, the algorithm was evaluated using two sets of 10-fold cross validation for each of 10 datasets, giving an average result of 84.01% - only a slight improvement over the algorithm with any single classifier. Looking in more detail at the results it emerged that C4.5 appears more prone to over-fitting than the other classifiers and C4.5 genotypes occasionally overshadow other, ultimately more accurate, genotypes.

The Liver Disorder dataset is a good example of this: the C4.5 genotype was selected in 16 of 20 runs and Naïve Bayes genotypes selected in the remaining 4 runs, giving an average result for the 20 runs of 65.7%. However if the Naïve Bayes genotype had been selected in every run the average result would have improved to 70.1%. One simple method to try to compensate for this is to randomly re-order the dataset at the end of the breeding process and re-test the fittest genotype for each classifier, then choose the genotype based on this 'train' score rather than the original fitness. Using this method a different genotype/classifier is chosen in 43 of the 200 runs (20 of which result in an improvement against 11 that are worse) for a small improvement of the overall average to 84.26%[7] - see table 4.6.

The performance of the algorithm can no longer reasonably be compared to that of a single classifier; so for each fold the 3 (unaided) classifier types[8] were evaluated on the training data and the most accurate was selected as the comparison for that fold ('Best CV').

Table 4.6: Allowing GAP to select the classifier - initial results

Dataset	GAP + C4.5/IBk/NB	SD	Best CV C4.5/IBk/NB	SD	t-test
LiverDisorder	**65.78**	8.03	64.76	8.98	0.61
Glass	**70.89**	11.02	68.77	11.46	0.71
Ionosphere	**89.9**	5.61	89.82	4.79	0.06
NewThyroid	94.87	4.48	**96.25**	4.19	**-2.78**
Diabetes	75.72	5.2	**76.04**	4.86	-0.34
Sonar	85.57	7.63	**86.64**	6.56	-0.55
Vehicle	**72.99**	3.74	72.22	3.32	0.64
Wine	95.44	5.11	**97.99**	3.84	**-2.38**
WBC 30	**96.14**	2.5	95.09	3.06	1.67
WBC 9	95.27	2.46	**95.99**	1.84	-1.32
Overall Average	84.26		84.36		**-3.04**

The results are obviously disappointing - the GAP algorithm performs significantly worse on two datasets and overall (using a T-test at the 95% confidence level) than simply selecting the most appropriate classifier type, at a heavy cost in computational effort. That the GAP algorithm provides a worse accuracy than selecting the best classifier

[7] This figure is directly comparable to the 84.01% figure given above as both sets of results were collected on the same runs.

[8] Application of the GAP algorithm to a linear classifier was done after results were obtained for selecting a classifier - it is not included in the set of classifiers the GAP algorithm can choose from.

based on cross-validation can only be due to over-fitting of the training data, particularly when selecting C4.5 as the base classifier (as noted in the previous paragraph).

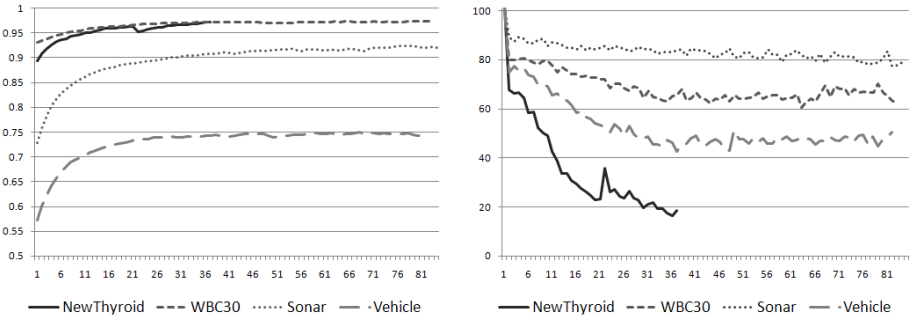
The following section details a number of changes that were introduced to the GAP algorithm to provide improvements over simply choosing the best classifier. See also chapter 6 for ensembles constructed from the various GAP-enhanced classifiers, and comparison with other forms of ensemble. As with the parameter settings described in section 3.3, the changes were made incrementally with one parameter setting (and/or change in technique) being established before the next change was explored, due to practical time constraints. Again, it is possible that a broader search of multiple changes might find a much better combination.

4.5 Inspecting the population, Further modifications

Inspecting the mechanics of the population it is possible to see that the algorithm functions differently for datasets with different numbers of attributes. Figure 4.1 shows the population history (the average accuracy of the population and the number of distinct individuals) for four of the UCI datasets: New Thyroid, Wisconsin Breast Cancer (30), Sonar and Vehicle; having 5, 30, 60 and 18 attributes respectively. (Due to the nature of the termination criteria the number of generations varies between runs - for each dataset the length of the line indicates the point at which half the runs completed. E.g. for New Thyroid, half the runs had finished by the 37th generation).

The average accuracy of the population follows broadly the same pattern for all datasets[9], but the number of distinct individuals in the population increases with the number of attributes in the dataset. On datasets with few attributes the population quickly settles down to many copies of a small number of individuals; while never settling at all for datasets with very many attributes. In small part this may be because with fewer attributes there are fewer possible combinations, but the main cause is the nature of the mutation operator. Recall that

[9] A dip in accuracy on the new thyroid dataset occurs shortly after generation 20, when reordering the dataset causes the average accuracy to dip sharply in four of the runs. This is accompanied by a subsequent increase in the number of distinct individuals in the population.



(a) Population average accuracy

(b) # Distinct individuals

Figure 4.1: Population history by generation - original settings

GA mutation occurs at random in each tree (the activity of the tree is flipped) with probability 0.005; while GP mutation occurs in each node with probability 0.008. Therefore in the offspring of the initial population the probability of an individual undergoing mutation is as follows (in the initial population there will be half as many active trees as attributes, each with an average of 2.92 nodes; subject to a minimum of 7 trees):

$$\begin{aligned} P_{mutation} &= 1 - (0.995^{trees} * 0.992^{nodes}) \\ &= 1 - (0.995^{attributes} * 0.992^{(\frac{2.92*attributes}{2})}) \end{aligned}$$

Thus the likelihood of a New Thyroid genotype in the first child generation undergoing mutation is $1 - (0.995^7 * 0.992^{(\frac{2.92*7}{2})}) = 0.11$, while the likelihood for a Sonar genotype $= 1-(0.995^{60}*0.992^{(\frac{2.92*60}{2})}) =$ 0.63; i.e. approximately 11% of the population will undergo mutation for the New Thyroid dataset compared to 63% of the population for Sonar[10]. This is obviously not ideal, particularly considering that one of the aims of the GAP algorithm is not to require any prior domain knowledge (and by extension parameters should not need to be changed to suit the dataset).

The following changes were made to the algorithm to make the mutation rate invariant with the number of attributes, and to take a more standard approach to some of the GP operations:

[10]The proportion undergoing mutation in generations after the first will change with the number of active trees and number of nodes in those trees, but the figures quoted here are accurate for the offspring of the initial population.

1. Each individual undergoes GP mutation with a probability of 0.24, in which a randomly selected node in a randomly selected (active) tree is replaced by a new sub-tree (the new sub-tree is created according equation 3.1 with an initial depth of 1). There is a similar chance (0.24) of the individual undergoing GA mutation, in which the activity of a randomly selected tree is flipped. If this results in the genotype having no active trees then a random (but different) tree is made active. This provides a consistent rate of mutation regardless of the number of attributes in the population.

2. Two-point crossover is replaced by uniform crossover (with a mixing ratio of 0.5, i.e. on average offspring are likely to receive half of their trees from each parent).

3. the chance of inversion (reversal of the order of a number of trees) is reduced from 0.2 to 0.002 (With uniform crossover the effects of and need for inversion are reduced).

4. The selection pressure is increased by increasing the chance of selecting the tournament group winner from 0.3 to certainty, while the group size is reduced slightly from 8 to 6.

5. The termination criteria are changed slightly so that instead of the count of active trees being used to break ties between genotypes of equal accuracy, the count of *nodes* in active trees is now used. While this indirectly drives the population toward fewer trees, it primarily drives it toward simpler individuals (with fewer nodes overall). This change was introduced to improve the human readability of genotypes for chapter 5, but is also in accord with the principle of Occam's razor.

6. The opportunity was also taken to introduce a new function, the natural logarithm $log_e(x)$[11]. As $log_e(x)$ takes a single parameter this has the effect of changing the tree structure from a binary tree to a less uniform structure.

[11] Logarithm was introduced on the grounds that it transforms multiplicative relationships into additive relationships (i.e. $log(xy) = log(x) + log(y)$), which allows statistical methods (and therefore classifiers) to find relationships more easily. "Working with the logarithms of data rather than the data themselves may have several advantages. Multiplicative relationships may become additive, skewed distributions may become symmetrical, and curves may become straight lines."[25]

The pseudo-code for the updated multi-classifier version of GAP are shown in algorithm 4.3.

Algorithm 4.3: The GAP Algorithm: Multiple classifiers (amended)

```
1  canTerminate = false;
2  fittest = null;
3  t = 0;
4  P_t = initialisePopulation ();
5  evaluatePopulation (P_t) ; // 10-fold cross-validation on train
   data
6  fittest =findFittest (fittest, P_t);
7  while true do
8      t = t + 1;
9      P_t = breedPopulation (P_{t-1});
10     evaluatePopulation (P_t);
11     fittest = findFittest (fittest, P_t);
12     if age (fittest) ≥ 12 and t ≥ 20 then
13         if canTerminate then
14             return (fittest);
15         else
16             randomiseTrainingData ();
17             canTerminate = true;
```

The population history for the same four datasets with the updated algorithm is shown in figure 4.2. Comparing this with figure 4.1 we can see that while the average accuracy of the population follows the same trend as before, the number of distinct individuals in the population is now much more consistent across datasets. After an initial period in which the population settles down the average number of distinct individuals tends to vary (very roughly) between 55 and 75. This point of equilibrium is reached more quickly on the New Thyroid dataset than on the others, but the number of distinct individuals is broadly similar (and, having fewer generations overall, the time taken to reach equilibrium as a proportion of the overall run time is quite similar to other datasets).

Figure 4.3 shows the average number of new individuals created in each generation (i.e. a combination of trees not previously seen in the population); and also the count of the number of copies of the most numerous single individual (subtracting the number of distinct individuals in 4.2(b) from the size of the population gives the total number of duplicates in the population, figure 4.3(b) shows how many of those are copies of the single most populous individual).

Procedure initialisePopulation

Input: training dataset
Output: initial population of genotypes P

```
1  begin
2      i = 0;
       // first genotype has one active tree for each numeric
          attribute:
3      foreach numericAttribute in training dataset do
4          P[i].trees[index] = numericAttribute ;
5          P[i].trees[index].active = true ;
6      P[i].classifier = randomClassifier ();
7      i = i + 1;
8      while i < 101 do
           // trees in remaining genotypes are created according to
              P_leaf equation:
9          foreach numericAttribute in training dataset do
10             P[i].trees[index] = createGpTree (training dataset) ;
11             P[i].trees[index].active = randomBoolean () ;
12             P[i].classifier = randomClassifier ();
13         i = i + 1;
14     return(P)
```

Procedure evaluatePopulation

Input: training dataset, current generation of genotypes
Output: current generation of genotypes, with updated fitness values P

```
1  begin
2      i = 0;
3      while i < 101 do
4          if P[i].fitness is unknown then
               // evaluate fitness of P[i] using 10-fold CV
5              P[i].accuracy = accuracy using 10-fold cross-validation ;
6              P[i].fitness = P[i].accuracy ;
7          i = i + 1;
8      return(P)
```

Procedure breedPopulation

Input: old generation of genotypes P_{t-1}
Output: new generation of genotypes P_t

```
1  begin
2      i = 0;
3      foreach classifier in (C4.5, IBK, Naïve Bayes) do
           // copy the fittest genotype for each classifier into the
              next generation:
4          P_t[i] = findFittestWithClassifier (P_{t-1}, classifier);
5          i = i + 1;
6      while i < 100 do
7          P_t[i+1] = P_t[i] = tournamentSelection (P_{t-1}); // group size 6,
           100% chance of selecting fittest
8          while P_t[i+1] = P_t[i] do
9              P_t[i+1] = tournamentSelection (P_{t-1});
10         uniformCrossover (P_t[i], P_t[i+1]);
11         mutate (P_t[i]);
12         mutate (P_t[i+1]);
13         i = i + 2;
14     return(P_t)
```

Procedure uniformCrossover

Input: $genotype1, genotype2$

```
1  begin
2      if random (0,1) ≤ 0.6 then
3          foreach tree in genotype1.trees do
4              if random (0,1) ≤ 0.5 then
5                  genotype1.trees[index] = genotype2.trees[index];
6                  genotype2.trees[index] = tree;
                   // (activity switch remains with the tree)
7              index = index + 1;
           // if all trees in a genotype are now inactive, toggle
              activity of a random tree
8      if random (0,1) ≤ 0.6 then
           // select a random tree (at the same locus) in each
              individual
           // exchange a random subtree between them
```

Procedure mutate

Input: *genotype*

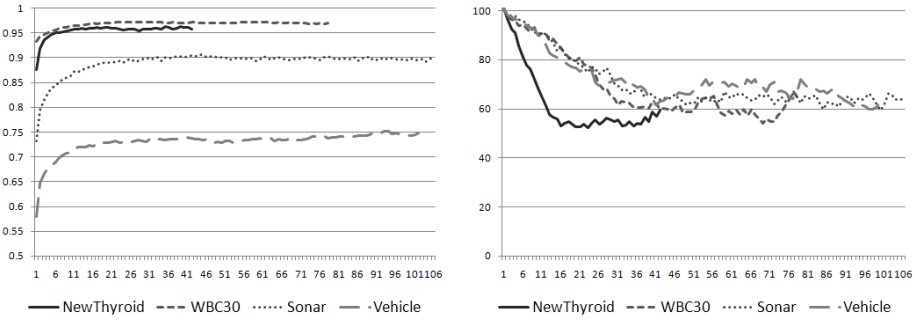
```
1  begin
2      if random (0,1) ≤ 0.2 then
           // replace classifier with a random, different classifier
              type
3      if random (0,1) ≤ 0.24 then
           // toggle the activity of a single tree in the genotype
           // if this causes all trees to be inactive, toggle
              activity of a random tree
4      if random (0,1) ≤ 0.24 then
           // replace single random subtree (in an active tree) with
              a new subtree created according to P_leaf equation with
              Depth = 1
5      if random (0,1) ≤ 0.02 then
           // invert (reverse) the order of trees between two
              randomly selected points
```

Procedure findFittest

Input: population of genotypes P

Output: fittest member of P

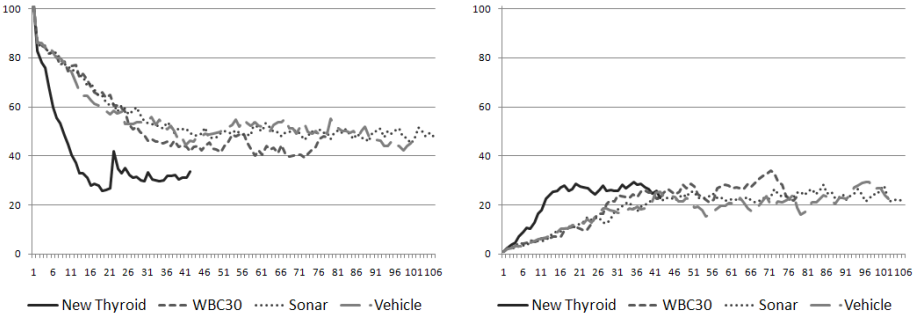
```
1  begin
2      fittest = P[0];
3      i = 1;
4      while i < 101 do
5          if P[i].accuracy > fittest.accuracy then
6              fittest = P[i];
7          else if P[i].accuracy = fittest.accuracy then
8              if P[i].countOfNodes < fittest.countOfNodes then
                   // countOfNodes refers to the count of nodes in
                      active trees
9                  fittest = P[i];
10             else if P[i].countOfNodes = fittest.countOfNodes then
11                 if P[i].age > fittest.age then
12                     fittest = P[i];
13         i = i + 1;
14     return(fittest)
```



(a) Population average accuracy

(b) # Distinct individuals

Figure 4.2: Population history by generation - updated settings



(a) New individuals

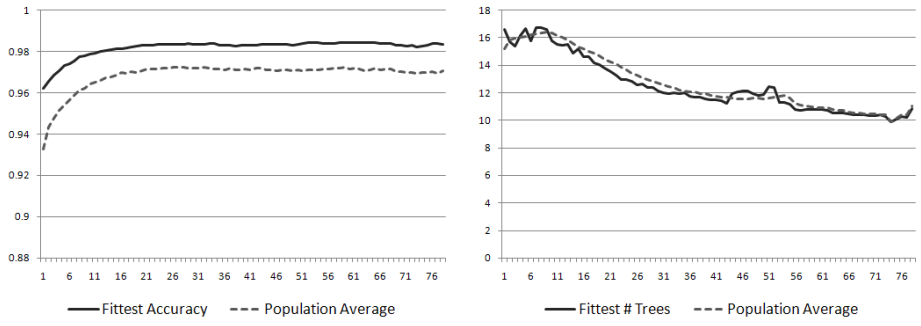
(b) Max count of single individual

Figure 4.3: Counts of individuals by generation - updated settings

These graphs show that for the most part approximately half the individuals of each generation are novel in some way, while 20-25% (roughly half the remainder) are copies of a single individual. The New Thyroid dataset does show a difference in the number of new individuals discovered at each generation[12] - perhaps because the smaller number of attributes offers less scope for the novel combination of trees.

It seems plausible to conclude from figures 4.2 and 4.3 that after the first few generations the single fittest genotype has started to multiply within the population and that many of the new individuals are variants of this genotype. This is confirmed by viewing the way the population average tracks the accuracy, and particularly the size, of the fittest individual in figure 4.4. It is further illustrated in figure 4.5,

[12]The spike in the count of new individuals after generation 20 occurs when the dataset was reordered in a large number of the runs. This highlights the fact that for many runs on the new thyroid dataset it is the minimum number of generations (20) rather than the minimum age of the fittest individual that determines when the dataset is reordered.



(a) Accuracy

(b) Count of Trees

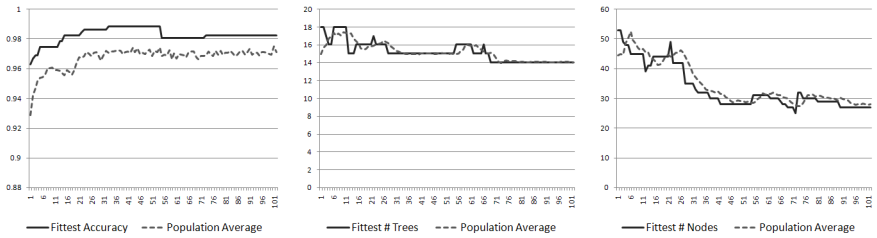
Figure 4.4: Accuracy and size by generation on WBC30

which tracks accuracy, count of trees and count of nodes within a single population (i.e. not averaged over 20 runs). It is clear that once a point of (relative) equilibrium is reached the algorithm has settled on an area of the search space dominated by a single individual and is subsequently exploring variations on a theme. Re-ordering the dataset can nudge the population out of a local maxima, generally replacing the existing fittest genotype with one of its variants rather than selecting an individual with a completely different set of trees.

Now that ties between equally accurate individuals are broken by counting the nodes in active trees instead of the active trees themselves we have approximately the same number of trees as before (an average of 9.2 in the fittest individual, compared with an average of 9.7 when selecting the classifier type with the original settings[13]), but many fewer nodes in the trees (23 compared with 34). Thus the trees are about a third less complex than previously - this reduction in size of the solution will prove useful in aiding human readability (see chapter 5).

The effect of these changes on the success of the algorithm is shown in table 4.7, while table 4.8 shows the type of classifier most often selected by the algorithm for each dataset. This is an obvious improvement over the results with the original settings (table 4.6) - there is an overall improvement over the comparison classifier instead of a significant loss of accuracy, two datasets with significant improvement and only one significantly worse (compared with previously no datasets with a significant improvement and two significantly worse, all determined

[13]Both figures are an increase in the number of trees over using C4.5 alone, this is because the algorithm produces more trees with IBk and Naïve Bayes.



(a) Accuracy (b) Count of Trees (c) Count of Nodes

Figure 4.5: Accuracy and size by generation on WBC30 (single run)

using a T-test with 95% confidence level); this is accomplished using the fittest genotype without having to resort to retesting the fittest individual for each classifier type at the end of the process.

As shown in table 4.8 the GAP algorithm usually selects the classifier which has the best unaided performance on the dataset, though this is not always the classifier with the best performance when aided by features created by the GAP algorithm. There is still some evidence that C4.5 is overfitting the dataset to a sufficient degree that it is being selected over a more accurate classifier (e.g. selecting Naïve Bayes every time on the WBC 9 dataset instead of choosing C4.5 ten times would have improved the accuracy), but not to the same degree as before and not sufficiently to retest individuals at the end of the run. [14]

To demonstrate that the modifications listed in this section have not had a negative impact on the accuracy for a single classifier, the algorithm was re-run with just C4.5. The results are compared to those for the pre-modification results (from section 4.2), as shown in table 4.9. While the overall average accuracy is down very slightly (with an improvement over C4.5 on only 7 datasets instead of 9) and the t-test score (against C4.5) is reduced, the improvement over unaided C4.5 remains significant at the 99% confidence level, and there is a significant improvement over unaided C4.5 on 3 datasets rather than the 1 with pre-modification results (as shown by a t-test at the 95% confidence level).

[14] One possible approach to dealing with this overfitting would be to use only some of the training data, by randomly selecting a proportion (say 70%) of the data at the start for use when cross-validating individuals in the population; then repeating the random selection of a different 70% when the termination criteria are met the first time; finally cross-validating the population on the whole training set when selecting the fittest individual after termination criteria are met the second time. This has not been attempted here, but the use of a random subset of the data is employed when dealing with a larger dataset (see chapters 5 and 6).

Table 4.7: Allowing GAP to select the classifier - updated results

Dataset	GAP + C4.5/IBk/NB	SD	Best CV C4.5/IBk/NB	SD	t-test
LiverDisorder	**68.71**	7.98	64.76	8.99	**2.44**
Glass	**73.86**	9.09	68.78	11.47	**2.16**
Ionosphere	88.08	6.69	**89.82**	4.79	-1.10
NewThyroid	**96.52**	3.63	96.26	4.19	0.32
Diabetes	74.55	5.28	**76.05**	4.87	-1.12
Sonar	**88.30**	7.66	86.65	6.56	0.95
Vehicle	71.93	4.79	**72.22**	3.33	-0.20
Wine	96.32	4.27	**97.99**	3.85	**-2.29**
WBC 30	94.32	3.71	**95.09**	3.06	-0.93
WBC 9	95.70	2.60	**95.99**	1.84	-0.74
Overall Average	84.83	5.57	84.36	5.29	1.03

Table 4.8: Frequency of classifiers selected by GAP

Dataset	Classifier Most Used	count	Best unaided Classifier	Best with GAP
LiverDisorder	C4.5	16	C4.5	NaiveBayes
Glass	IBk	20	IBk	IBk
Ionosphere	C4.5	15	C4.5	IBk
NewThyroid	NaiveBayes	11	NaiveBayes	NaiveBayes
Diabetes	NaiveBayes	14	NaiveBayes	NaiveBayes
Sonar	IBk	20	IBk	IBk
Vehicle	C4.5	12	C4.5	IBk
Wine	NaiveBayes	11	NaiveBayes	NaiveBayes
WBC 30	IBk	9	IBk	NaiveBayes
WBC 9	C4.5	10	NaiveBayes	NaiveBayes

4.6 Comparison with other algorithms

Table 4.10 shows a comparison between the amended algorithm (4.3) and both SVM and other results from the literature (values for GAP and Best CV are the same as in table 4.7). The accuracy figures for SVM in table 4.10 have been obtained using the Weka wrapper for LibSVM [48], and values for the *cost* and γ parameters have been tuned for each dataset using the parameter selection tool available with LibSVM (which uses a grid search technique to identify a good combination of parameters). Of the 200 runs, there were 105 where SVM provided better accuracy, and 95 where GAP provided better accuracy. The overall t-test result comparing GAP (+ C4.5/IBk/Naïve Bayes) with SVM for the 200 runs is 1.78. Thus, while SVM provides greater overall

Table 4.9: Comparing the effects of modifications on a single classifier

Dataset	pre-modifications	S.D.	post-modifications	S.D.	t-test vs C4.5
LiverDisorder	**66.55**	8.10	65.50	6.68	-0.52
Glass	**71.84**	10.26	67.74	6.88	-0.25
Ionosphere	**90.69**	4.66	89.39	4.85	-0.47
NewThyroid	**96.49**	3.98	96.31	4.11	**3.03**
Diabetes	73.64	5.11	**73.89**	4.51	0.51
Sonar	75.89	9.00	**79.11**	7.47	1.80
Vehicle	72.11	4.60	**72.23**	4.47	0.00
Wine	**96.10**	4.08	95.75	4.07	**2.37**
WBC 30	**95.71**	4.39	95.63	3.25	1.86
WBC 9	95.56	2.52	**95.99**	2.41	**2.32**
Overall Average	83.46		83.15		2.83

accuracy, the difference is not significant at the 95% confidence level.

Values in the final column of table 4.10 have been obtained from the following sources: LiverDisorder, Ionosphere: [70]; Glass, Diabetes, Wine, WBC9: [20]; NewThyroid, Vehicle: [183]; Sonar: [76]; WBC30: [1].

Table 4.10: Comparison of SVM and published results with GAP-assisted C4.5, Naïve Bayes and IBk

Dataset	GAP + C4.5/IBk/NB	SD	Best CV C4.5/IBk/NB	SD	SVM	SD	Lit. Search
LiverDisorder	68.71	7.98	64.76	8.99	**73.02**	6.89	70.9
Glass	**73.86**	9.09	68.78	11.47	66.80	8.10	70.8
Ionosphere	88.08	6.69	89.82	4.79	**93.73**	4.95	94.70
NewThyroid	**96.52**	3.63	96.26	4.19	95.80	3.71	94.22
Diabetes	74.55	5.28	**76.05**	4.87	73.12	5.27	75.8
Sonar	**88.30**	7.66	86.65	6.56	82.96	9.20	90.40
Vehicle	71.93	4.79	72.22	3.33	**83.80**	4.17	78.36
Wine	96.32	4.27	**97.99**	3.85	97.16	3.49	97.8
WBC 30	94.32	3.71	95.09	3.06	**97.19**	1.83	97.50
WBC 9	95.70	2.60	95.99	1.84	**96.78**	1.60	96.7
Overall Average	84.83	5.57	84.36	5.29	86.04	4.92	

While improved, the results still suggest that using the GAP algorithm offers a relatively small improvement over selecting the most appropriate classifier type, at a high cost in the time taken. However, the GAP algorithm is probabilistic in nature and will generally produce a different set of features every time it is run - some of which will be better than others. All the results shown so far have been obtained

from 20 runs per dataset, every run providing a distinct set of features and the result an average over the 20 feature sets (or feature/classifier combinations) - each set tested on a single data fold.

In a practical situation we would not be interested in the average utility of a number of feature sets, but in using the best available feature set. Evaluating each of the 20 feature set/classifier combinations on all of the data folds shows that the best set of features offers a marked improvement over the average performance, and over the performance of an unaided classifier. For 8 of the 10 datasets there were at least 2 (often 5 or more) feature sets with results more than a standard deviation better than the best unaided classifier. (There was one such feature set for Diabetes and none for New Thyroid). That is to say that run 20 times the GAP algorithm is highly likely to produce at least one, and often many more, feature sets that offer a marked improvement over an unaided classifier (when evaluated using cross-validation on the whole dataset). As the whole dataset is used to choose between the feature sets it is not possible to perform a statistically valid comparison, however the case study in chapter 7 deals with a larger dataset from which some of the instances have been held back for just that purpose.

4.7 Conclusion

In this chapter we have demonstrated that it is possible to combine the creation and selection stages, providing the selection algorithm with a greater range of features to combine, without adversely affecting the accuracy of the classifier. We have shown that the GAP algorithm, in its single stage form, can be successfully applied to other classifiers, providing a similar boost in accuracy as for C4.5. We have also shown that the algorithm can be extended to select the type of classifier most appropriate to the dataset, providing a small improvement on average over an unaided version of the most appropriate of the 3 classifier types. Finally, we have stated that of the 20 runs performed on each dataset there are a number of runs that provide an accuracy a standard deviation or more better than the average, and this is followed up in the case study in chapter 7.

Chapter 5

Improving the Human Readability of Derived Features

5.1 Introduction

This chapter looks at a number of methods to reduce the complexity (and count) of the derived features without substantially affecting the accuracy of the classifier, in an effort to highlight the relevant relationships between attributes and make them easier for people to see and understand. Three approaches to this are considered: modifying the fitness function to include a measure of parsimony, modifying the algorithm to use a multi-objective approach (algorithm 5.1), and post-processing to reduce the complexity of the single fittest genotype produced by the standard algorithm (4.3).

The work described in this chapter (except section 5.4) was first published in [171].

5.2 Parsimony

One of the advantages of Genetic Programming is that it has the potential to produce solutions that can be read and understood by humans. The results can be made easier to read by reducing their complexity, either through post-processing (removing redundant sub-trees, or *introns*) or during their evolution (e.g. by introducing a fitness function

that encourages simplicity). Simpler (more parsimonious) solutions can also reduce the well known problem of over-fitting as, while a simple and a complex rule may provide equal accuracy on training data, the simpler rule is more likely to be generally applicable and maintain that accuracy on unseen test data.

GA and GP communities both make use of the term parsimony, in GA it tends to refer to the number of attributes while in GP it refers to the size of a tree. While we are interested in reducing the number of features (by filtering out irrelevant and unhelpful features we allow investigation to concentrate on those features that do improve accuracy), the primary concern here is to reduce the complexity of derived features so as to make them more intelligible. It is the useful constructed features we are aiming to highlight and simplify, not the classifier itself (as the new features are what the GAP algorithm brings to the table, though there is some evidence in section 3.3.3 'Utility of Constructed Features' that the constructed features can simplify the classifier as well). The hope here is that we may learn something of the domain; to be able to say what relationships between attributes are useful in prediction, without harming the accuracy of the solutions.

The great majority of the work on enforcing simplicity in GP has focused not on improving readability, but on controlling bloat in the population. The most common, and simplest, method for this is to specify a fixed maximum size for the GP trees and rejecting any that grow past this point - usually by setting a maximum depth, or by setting a maximum number of nodes. The problem with this approach is that it can be difficult to tell in advance what the appropriate maximum size is. Set the maximum too low and individuals in the population will be too small to achieve a solution (to take a trivial example, a quadratic equation $[ax^2 + bx + c]$ cannot be solved using a tree with a maximum depth of two); set the maximum too high and the exponential growth of introns will cause the population to stagnate. Thus the appropriate maximum must be determined individually for each problem; or face a choice between a limited solution space and allowing bloat. There are techniques for dynamically adjusting the maximum size of gp programs during a run ([169, 170]), they are not relevant here because the GAP algorithm applies no such limit.

Two alternative approaches to combating bloat are common in GP:

Modifying the fitness function to incorporate parsimony pressure (section 5.2.1); and treating the twin goals of fitness and simplicity as separate objectives to be solved using multi-objective optimisation (section 5.4). A different approach that focuses on reducing the size of the final solution, without attempting to address code bloat, involves post-processing or pruning (section 5.3). A further approach is to modify the genetic operators to reduce the factors that cause bloat (see section 2.2.2). This form of approach is more invasive as it modifies the genetic operators and is concerned only with the control of bloat. It is not treated here, where the primary goal is to improve human readability.

Most theories that explain bloat in decision trees rest on the fact that "it is easier to add code to a program than it is to remove it. That is, it is quite difficult to remove code from an effectively functioning program without heavily impacting on that functionality and thus reducing its fitness" [21], so that crossover and mutation are much more likely to be destructive operators than they are constructive. As this does not apply in the same way to constructed features (a sub-tree can be removed from a single feature without dramatically reducing overall fitness[1]) bloat does not pose such a problem (for example figure 4.5 on page 106 shows that the average number of nodes actually *decreases* over time[2]), and instead we concentrate here on improving readability. In doing so we adapt the algorithm used by Bojarczuk et al. [30], who have applied a fitness function that combines predictive accuracy and simplicity to medical datasets, with the primary intention of improving human readability.

Bojarczuk et al. use a form of genetic programming using classification rules inspired by Learning Classifier Systems[3] where each rule has

[1] While removing a subtree may reduce the utility of the feature in which it appears, this is only one feature among many, so the impact is much less than for a standard GP individual that consists only of a single tree.

[2] The decrease in average size occurs because a strong selection pressure is combined with a fitness measure that breaks ties between equally accurate individuals by considering the number of nodes. While this is a form of parsimony pressure it is entirely subservient to accuracy and thus does not have any negative impact on accuracy the way other parsimony measures have sometimes been found to do [130].

[3] Learning Classifier Systems evolve populations of rules in the form 'if ⟨*condition*⟩ then ⟨*action*⟩' with an associated fitness value. In very simplified form an LCS will choose the action of the fittest of those rules whose conditions apply (though it will also take into account the expected cost/reward of an action). There are two broad categories of LCS: Pittsburgh and Michigan. A Pittsburgh-style LCS has a population of *sets* of rules (i.e. each individual in the population consists of multiple rules and a GA operates to find the single best individual); the more common Michigan-style (including 'ZCS' and 'XCS') has a population of individual rules (and the GA operates to evolve the best combination of individuals). An LCS learns using a combination of Reinforcement Learning and Genetic Algorithms. For more information see [43].

a condition and a classification ('if $\langle condition \rangle$ then $\langle classification \rangle$') and the solution consists of many rules. Bojarczuk et al. term their approach a hybrid Pittsburgh/Michigan approach because an individual consists of a set of rules, constrained so that all rules in a single individual predict the same class; while a complete solution consists of k individuals each predicting a different class, where k is the number of classes in the dataset. Rather than running the algorithm k times, each with a population all predicting the same class, the population consists of individuals which may have different rule consequents, so that a complete solution may be bred during a single run of the algorithm. Partial elitism is employed so that the fittest individual predicting each of k classes is maintained across generations. There are then 3 scenarios when classifying a new instance: a) the instance matches the rule set of exactly one of k individuals, the prediction is that of the individual; b) the instance matches the rule sets of two or more individuals, the prediction is that of the fittest individual; c) the instance matches the rule set of none of the individuals, the prediction is the default (majority) class.

As Bojarczuk et al. are concerned with the classification of medical datasets the fitness measure originally employed in [29] rested on the notions of *Sensitivity* and *Specificity*, which are defined in appendix C. Fitness is then the product of sensitivity and specificity. This function was modified to incorporate a measure of *Simplicity*:

$$Simplicity = \frac{maxnodes - 0.5 \cdot numnodes - 0.5}{maxnodes - 1} \tag{5.1}$$

"Where *numnodes* is the current number of nodes (functions and terminals) of an individual (tree), and *maxnodes* is the maximum allowed size of a tree (set to 45)" [30]. Simplicity ranges between 0.5 (when the individual has the maximum number of nodes) and 1.0 (when the individual has just a single node). The fitness function is modified to maximise sensitivity and specificity while minimising the size of the individual:

$$Fitness = Sensitivity \times Specificity \times Simplicity \tag{5.2}$$

Bojarczuk et al. evaluated their algorithm on 5 datasets, four of which are available in the UCI repository. They found no overall dif-

ference between the accuracy of their algorithm and C4.5 (significantly worse on two datasets, better on two and no significant difference on one) but did show an improved ability to generate rules comprehensible to an expert.

5.2.1 Applying Parsimony to the GAP Algorithm

As individuals in the GAP algorithm must be able to classify an instance as being one of many classes, rather than always being a binary classifier as is the case for Bojarczuk et al., the fitness function defined in equation 5.2 is not directly applicable (we cannot calculate single values for simplicity and specificity in cases where there are more than 2 classes). Instead we use the product of accuracy and simplicity:

$$Fitness = Accuracy \times Simplicity \tag{5.3}$$

Simplicity is defined as in equation 5.1, however *maxnodes* must still be defined. The GAP algorithm has no maximum size for an individual. It is therefore defined, somewhat arbitrarily, as twice the size of the largest individual in the initial population and fixed for the life of the population. (It is not desirable to redefine *maxnodes* in every generation (so that it always represents the size of the largest individual) as this would cause the strength of the Simplicity parameter to vary unpredictably between generations, such that a single large outlier can have a dramatic effect on the fitness of other genotypes, causing the fitness of other individuals to change from one generation to the next).[4]

With simplicity defined as in equation 5.1, simplicity quickly overwhelms predictive accuracy. Without the simplicity measure the average number of nodes in the fittest individual is approximately 23 nodes in 9 trees; with the simplicity measure applied it drops to roughly 7 nodes in 6 trees, i.e. feature construction is almost completely eliminated. The overall performance also drops, from 84.3% to 83.3%. This is because Bojarczuk et al.'s measure is designed to reduce the complexity of a GP classifier, not reduce the complexity of features passed

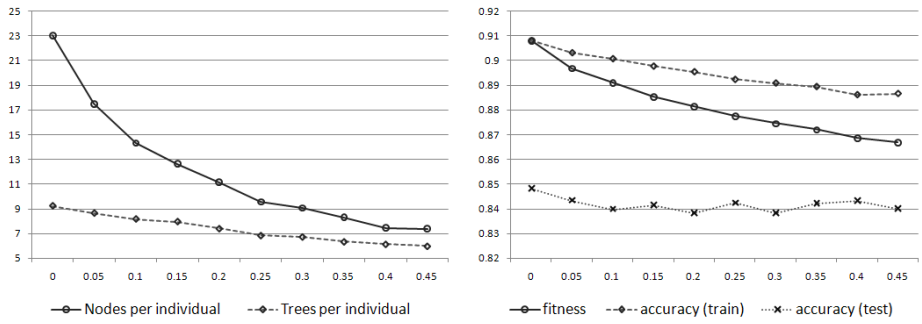
[4]While measures for adaptive parsimony pressure such as that developed by Zhang and Mühlenbein [182] have been successfully applied, these act to increase parsimony pressure as the improvement in accuracy of the fittest individual tends to zero, or allowing pressure to slacken in response to growth of the *fittest* individual; but not changing in response to the size of the *largest* individual.

to a classifier. A GP classifier with just a single node is doomed to failure while a classifier such as C4.5 may still perform well given just one feature (at least to the extent that the loss in accuracy is less than the gain from simplicity).

Bojarczuk et al.'s equation can be modified to replace the constant 0.5 with a strength parameter, as shown in equation 5.4:

$$Simplicity = \frac{maxnodes - \alpha numnodes - (1 - \alpha)}{maxnodes - 1} \tag{5.4}$$

With α set at 0.5 the simplicity measure is the same as employed by Bojarczuk et al., if set at 0 the simplicity measure is eliminated and fitness is based entirely on accuracy. Figure 5.1 shows the effects on the final solution of varying α between 0 and 0.45 in steps of 0.05. All values are averaged over all ten UCI datasets, with 20 runs per dataset. The size, fitness and accuracy on training data of the fittest individuals drop quite fast as α rises, while accuracy on the test data drops as alpha reaches 0.1 then remains steadily low. It is evident that a low value of α is required.



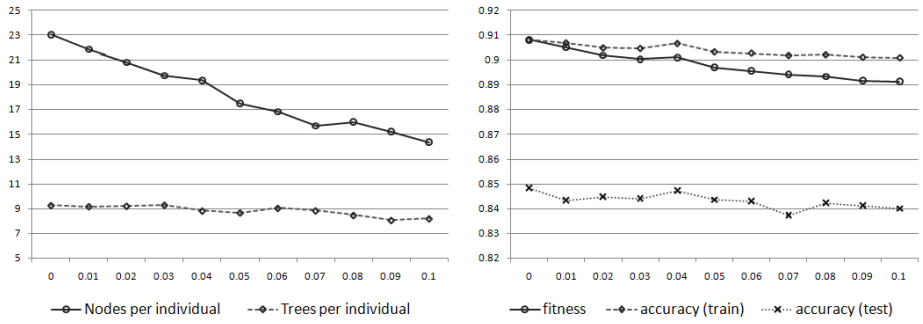
(a) Node & Tree Counts

(b) Fitness & Accuracy

Figure 5.1: Simplicity Strength 0 - 0.45

Further tests were then performed with α in the range 0 - 0.1 in steps of 0.01, as shown in figure 5.2. A value of 0.04 was finally chosen for α, because it successfully simplifies the individuals (reducing the average number of nodes per tree from roughly 2.5 to just over 2) while allowing feature construction to continue playing a meaningful part in the algorithm. It also occupies a slight hump in the graph of predictive accuracy for both train and test scores (figure 5.2(b)).

Table 5.1 shows the results for the algorithm including parsimony



(a) Node & Tree Counts

(b) Fitness & Accuracy

Figure 5.2: Simplicity Strength 0 - 0.1

on the UCI datasets. The figures are very similar to those without the parsimony measure. Results for the Liver dataset are no longer significant but otherwise there is no significant change in the performance (based on T-tests at the 95% confidence level), while the average number of nodes in the output has been reduced by 16%.[5] The accuracy on the test set remains almost exactly the same when setting simplicity strength to 0.04 (as compared to no parsimony pressure - see table 4.7), while the average standard deviation drops slightly from 5.6% to 5.0%. The average number of generations drops by just under 11%, from 72.5 to 64.7 generations.

Table 5.1: Performance with parsimony on UCI datasets

Dataset	GAP	(sd)	Comparison	(sd)	t-test
LiverDisorder	66.64	(9.77)	64.76	(8.99)	0.80
Glass	**73.46**	(8.50)	68.78	(11.47)	**2.18**
Ionosphere	89.34	(4.68)	89.82	(4.79)	-0.37
NewThyroid	96.28	(3.22)	96.26	(4.19)	0.03
Diabetes	75.13	(4.39)	76.05	(4.87)	-1.26
Sonar	87.74	(5.85)	86.65	(6.56)	0.88
Vehicle	71.82	(4.98)	72.22	(3.33)	-0.31
Wine	94.90	(4.83)	**97.99**	(3.85)	**-2.90**
WBC 30	95.88	(2.14)	95.09	(3.06)	1.42
WBC 9	96.36	(1.68)	95.99	(1.84)	1.05
Overall Average	84.75	(5.01)	84.36	(5.29)	0.92

[5]The average of 16% reduction in node counts hides some variation between datasets, some of which are associated with greater reduction than others. One might expect more nodes to removed from datasets with many attributes (and correspondingly many possible trees) than on those with few attributes, however this is not necessarily the case - for example parsimony pressure has little effect on the Sonar dataset (with 60 attributes). See table 5.4 on page 127 for a comparison of node counts on each dataset.

Applying parsimony pressure appears to have reduced the size of the final solution while providing greater consistency (lowering the standard deviation of the accuracy on test data), as we would expect in line with Occam's Razor.

5.3 Post-Processing

Even with parsimony pressure applied by the simplicity function there is no guarantee that the final genotype does not contain redundant or unhelpful nodes that obscure useful information. A number of techniques have been developed for Genetic Programming to prune trees, either during evolution ('pre-pruning', which differs from applying parsimony pressure in that nodes are actively cut from the tree, and is generally employed to stop trees overfitting the training data) or afterward ('post-pruning', for instance Reduced Error Pruning [42, 124] and Grow [51]), as applied here.

> Pre-pruning techniques are more efficient (faster) than post-pruning techniques, but post-pruning usually finds models with higher accuracy and simpler rules than pre-pruning. Intuitively, this is due to the fact that post-pruning has more information (the complete learned model) available to make decisions, and so it tends to be less "shortsighted" than pre-pruning. In any case, many post-pruning techniques are still greedy, by removing one condition at a time from a rule. [140]

There are two deterministic techniques that we apply to clean the final genotype. Both techniques have been employed using the full dataset and have not been applied to the genotypes used to obtain the results shown above. The full dataset is used to give the pruning process as much data as possible to work on and reduce overfitting. Note that as the post-processing techniques involve the full dataset it is impossible to give a fair estimate of the change in predictive accuracy, as the cleaned genotypes have effectively 'seen' the test data (some of the UCI datasets are too small to usefully keep back a separate validation set), but impact can be tested using the larger Bristol Vision dataset (see the case study in chapter 7).

The first technique is to identify any sub-trees that evaluate to a constant and replace them with that constant. In this regard a sub-tree is said to evaluate to a constant if it returns the same value for every instance in the dataset. Constants do not appear in the final genotype very often - in fewer than 1 in 20 runs, but this step certainly aids human readability when they do occur.

The second is to identify and remove (filter) any trees or sub-trees that do not contribute to the predictive accuracy of the genotype (backward elimination, measuring simple change in accuracy rather than a statistically significant change). This has the same purpose as the simplicity measure but operates in a deterministic fashion, and is similar to the reduced error pruning technique applied to decision trees [124] except using cross-validation instead of separate validation data, also to work by Garcia-Almanza and Tsang [74]. First, the genotype is evaluated by ten-fold cross validation to provide a baseline accuracy. Then each active tree in turn is deactivated and the genotype is re-evaluated. If the new accuracy is the same as or better than the baseline then the tree remains deactivated and the new accuracy becomes the baseline. Otherwise the tree is re-activated. An alternative approach would be a form of forward selection, to start with the single most predictive tree and stepwise add in trees that most improve the accuracy. This approach runs the risk of excluding trees that individually appear irrelevant but are useful in combination. Performing stepwise addition it is possible that neither tree would appear sufficiently useful to be added and so they would never be evaluated in combination, but by removing trees they both start out present and removing either one will show a drop in accuracy.

After whole trees have been evaluated, sub-trees in the (possibly smaller) genotype are similarly evaluated. This is done by replacing the top node of each sub-tree by each of its operands in turn, re-evaluating the genotype at each step. Once again if the new accuracy is the same as or better than the baseline then the function is permanently replaced by its operand. Otherwise the process continues in a depth-first, left-right fashion. For example, if a sub-tree consists of the function (A + B) then the function will be replaced by A (so that the function is replaced by a leaf node) and the genotype re-evaluated. If using A instead of (A + B) offers the same or improved performance then

the function will be replaced by A. Otherwise the function is instead replaced by B and the genotype re-evaluated again.

Post-processing reduces the number of nodes in the final genotype by an average of 6.9 nodes (or 30% of the total) when no parsimony pressure is applied, and 5.3 nodes (27.4%) with parsimony pressure. With a simplicity strength of 0.04 there is evidently still scope for the post-processing to remove a fairly large proportion of redundant nodes from the final genotype. Conversely, after cleaning the parsimonious genotypes remain slightly smaller than their counterparts (an average of 14.1 nodes compared with 16.1) indicating that the parsimony pressure may be eliminating some potentially useful nodes. The lower standard deviation of the parsimonious solutions suggests that some of those nodes may be over-fitting the training data, however.

5.4 Multi-Objective techniques: Adapting POPE-GP for the GAP algorithm

In order to evaluate multi-objective optimisation for improving human readability we have made modifications to the GAP algorithm to implement a multi-objective optimisation technique based on NSGA-II and POPE-GP (see section 2.3.1). In this variant the following changes have been made:

1. The single fitness objective (that considered both accuracy and size when applying parsimony pressure; or considered size only when comparing genotypes with equal accuracy in the normal case) has been replaced by two objectives: an objective to maximise accuracy and another to minimise size (minimise the number of nodes in a genotype).

2. Fitness is computed according to the POPE-GP algorithm by which genotypes are divided into non-dominated fronts, within which genotypes are sorted according to the crowding comparison operator. This is achieved by using the non-dominated front q that the individual belongs to as the integer part of the fitness score, and the crowding distance as the decimal part. [6]

[6] As crowding distance forms the decimal part of the fitness value, if an individual does not have a neighbour on one side (i.e. it is on the edge of the front) then its crowding distance is deemed to be one instead of infinity (in order to ensure a correct sort order individuals on the leading edge

3. The partial elitism of the standard GAP algorithm (in which the fittest genotype for each classifier type in the parent generation was persisted into the next generation) has been replaced by full elitism whereby the parent and child generations are combined and the fittest half selected for the next generation.

4. The termination criteria remain the same (the 'fittest' genotype remains unchanged for 12 generations, at which point the dataset is randomly resampled and evolution continues until the fittest genotype has remained unchanged for a further 12 generations); while the definition of 'fittest' genotype for this purpose treats the accuracy objective as paramount, considering the size objective only when comparing genotypes with equal accuracy. This is a form of Lexicographic Ordering [49] which is an alternative approach to the notion of the Pareto front and requires that the user provide an order of importance for the objectives. However it is used here purely to determine the termination criterion (and to select the solution from the first non-dominated front in the final generation), not to replace the notion of the Pareto front.

5. the 'fittest' genotype of the final generation is put forward as the single solution to be evaluated against the test data. (Unlike POPE-GP in which the entire first non-dominated front is put forward).

The changes to the GAP algorithm are shown in algorithm 5.1 on page 123. Note that this is an extension of algorithm 4.3 - the pseudocode for procedures such as breedPopulation is the same unless otherwise indicated.

Initial results on the 10 UCI datasets show a large reduction in the size of individuals (to an average of 10 nodes over 6 trees) at a relatively high cost in accuracy - average accuracy drops to 83.85%. These poor results may be explained by the early convergence of the algorithm (completing in an average of 53 generations as compared with 72 when no parsimony measures are applied), as illustrated by the fact that for the majority of each run (after the first few generations) the first non-dominated front consists of the entire population. At this time the

of the front are assigned a crowding distance of 1, while those on the trailing edge are assigned a distance of 0.9999). The crowding distance is then normalised so that $0 <$ distance < 1, to ensure that an individual from one front is always less fit than an individual from an earlier front.

population consists largely of multiple copies of a few individuals.

This problem was also experienced by Parrott et al [142] when implementing POPE-GP. They therefore extended POPE-GP so that 'only one individual is allowed at a (*fitness; size*) point in objective space. Where more than one solution occurs at a point, a solution is chosen at random to be maintained and the others deleted.' This restriction is applied at the end of a generation which means that the population may be smaller than the specified size p, though the next child generation will be the full size.

The restriction on duplicates was adapted for the GAP algorithm - instead of allowing a single genotype at a (*fitness; size*) point in objective space, we allow only a single instance of a phenotype (a set of trees and a classifier type). This measure is applied after the parent and child generations have been combined and sorted but before the top p individuals are selected, so that the combined population will be less than $2*p$ but hopefully greater than p. If there are fewer than p individuals then duplicates are re-introduced to the population in fitness order until it reaches size p. A constant population size p is maintained rather than allowing the population to shrink between generations as in the extended POPE-GP.

Removing duplicates from the population slightly increases both the accuracy (to 84.2%) and the size (11 nodes over 7 trees) in the same number of generations (an average of 51). The major difference is the size of the first non-dominated front, now generally no more than a quarter of the population. When compared to the GAP algorithm without any parsimony a large reduction in the size of the final genotype has been achieved at some cost to its accuracy. We seem to have the same problem here as with Bojarczuk et al.'s parsimony measure - the simplicity objective has been too aggressive and removed some nodes that improve classification accuracy. Two potential routes have been investigated to overcome this:

1. A third objective was introduced to drive the algorithm toward the discovery of new features. This objective attempts to improve the ratio of constructed features to original attributes in a genotype by maximising *(trees with constructed features)/(number of trees)*.

2. The Simplicity objective was modified to take the error rate of an

individual into account, so that instead of simply minimising the size it minimises *Size*Error Rate*.

The two approaches were also combined - the constructed features objective used in conjunction with the *Size*Error Rate* objective. The results are shown in table 5.2 in which the first two rows show results from the GAP algorithm with and without parsimony (as previously discussed), later rows show results with differing multiple objectives. All results are (as before) averages of 20 runs on each of 10 UCI datasets.

Table 5.2: Comparing effects of various forms of Multi-Objective GAP on UCI datasets

	Accuracy	(S.D.)[7]	Nodes	Trees	Generations
No Parsimony	84.83	(5.57)	23.1	9.2	72.5
Parsimony	84.75	(5.01)	19.5	8.8	65.2
Accuracy, Simplicity	84.17	(5.59)	11.0	6.9	51.1
Accuracy, (Size x Error)	84.25	(5.49)	12.9	7.7	56.0
Accuracy, Simplicity, Constructed Features	84.55	(5.96)	19.6	6.8	63.7
Accuracy, (Size x Error), Constructed Features	83.99	(5.54)	19.7	7.1	63.4

The highest accuracy of the multiple objective approaches is achieved by combining the Accuracy, Simplicity and Constructed Features objectives, though it falls slightly short of the accuracy achieved with the parsimony measure and has a higher standard deviation (see table 5.2). The accuracy achieved using these three objectives is not significantly different (on a T-test at the 95% confidence level) from that achieved without any parsimony measure. Accordingly these three objectives were used when carrying out further experiments with multi-objective optimisation. Performance on the individual UCI datasets is shown in table 5.3, where the comparison ('Best CV') is determined for each fold by evaluating the 3 (unaided) classifier types on the training data and selecting the most accurate as the comparison for that fold.

The number of nodes in the final solution is in line with the parsimony measure but there is greater potential for knowledge discovery here as the nodes are spread over fewer trees. (As you would expect from including an objective to maximise the number of constructed

[7]As a single value for standard deviation over 10 different datasets doesn't make sense, the values for S.D. are the averages of the standard deviation for the 10 datasets.

Algorithm 5.1: The GAP Algorithm: Multi-Objective

```
1  Objectives = array of Objectives;
2  Objectives[0] = accuracy Objective ; // maximise accuracy
3  Objectives[1] = size Objective ; // minimise nodes in genotype
4  Objectives[2] = constructed features Objective ; // maximise (trees with
   depth>1) /(trees)
5  canTerminate = false;
6  fittest = null;
7  t = 0;
8  P_t = initialisePopulation ();
9  evaluatePopulation (P_t) ; // 10-fold cross-validation on train
   data
10 assignFitness (null, P_t);
11 fittest =findFittest (fittest, P_t);
12 while true do
13     t = t + 1;
14     childGeneration = breedPopulation (P_{t-1});
15     evaluatePopulation (childGeneration);
16     assignFitness (P_{t-1}, childGeneration);
17     P_t = chooseSurvivors (P_{t-1}, childGeneration);
18     fittest = findFittest (fittest, P_t);
19     if age (fittest) ≥ 12 and t ≥ 20 then
20         if canTerminate then
21             return (fittest);
22         else
23             randomiseTrainingData ();
24             evaluatePopulation (P_t);
25             assignFitness (null, P_t);
26             canTerminate = true;
```

Procedure evaluatePopulation

Input: training dataset, generation of genotypes G

Output: specified generation of genotypes, with updated accuracy values

```
1  begin
2      for i = 0 to 100 do
3          if G[i].accuracy is unknown then
               // evaluate accuracy of G[i] using 10-fold CV
4              G[i].accuracy = accuracy using 10-fold cross-validation ;
5      return(G)
```

Procedure assignFitness

Input: parent generation of genotypes, child generation of genotypes
Output: specified generations of genotypes, with updated fitness values

```
1  begin
2      combinedPopulation = parentGeneration + childGeneration;
       // calculate fitness for each objective:
3      for j = 0 to 2 do
4          for i = 0 to 100 do
5              P[i].objectiveFitness[j] = Objectives[j].calculateFitness(P[i]);
           // normalise population fitness values for O[j] in the
              range 0 to 1
       // determine dominance in population:
       // initialise dominatedBy count to 0 for all members of P
6      for i = 0 to 99 do
7          for j = i + 1 to 100 do
8              if P[i].dominates(P[j]) then
9                  P[j].dominatedBy = P[j].dominatedBy + 1;
10             else if P[j].dominates(P[i]) then
11                 P[i].dominatedBy = P[i].dominatedBy + 1;
12     i = 0;
13     nonDominatedFront[i] = all members of P with dominatedBy = 0;
14     dominatedGenotypes = all members of P with dominatedBy > 0;
15     while dominatedGenotypes is not empty do
16         for j = 0 to dominatedGenotypes.size − 1 do
17             dominatedGenotypes[j].dominatedBy =
               dominatedGenotypes[j].dominatedBy −
               nonDominatedFront[i].size;
18         i = i + 1;
19         nonDominatedFront[i] =all members of dominatedGenotypes with
           dominatedBy = 0;
20     countOfFronts = i + 1;
21     for i = 0 to (countOfFronts − 1) do
22         currentFront = nonDominatedFront[i];
23         determineCrowdingDistance (currentFront);
24         for j = 0 to currentFront.size do
25             currentFront[j].fitness =
               (countOfFronts − i) + currentFront[j].crowdingDistance;
```

Procedure genotype.dominates

Input: *other* genotype, to compare with this genotype
Output: true if this genotype dominates the other genotype, false otherwise

```
1  begin
2      isDominant = false;
3      for j = 0 to 2 do
4          if this.objectiveFitness[j] < other.objectiveFitness[j] then
5              return(false);
6          else if this.objectiveFitness[j] > other.objectiveFitness[j] then
7              isDominant = true;
       // if both genotypes have equal fitness on all objectives
          then isDominant = false
8      return(isDominant)
```

Procedure determineCrowdingDistance

Input: *currentFront*: array of genotypes in a single front

```
1  begin
       // initialise crowdingDistance to 0 for all members of
          currentFront
2      for j = 0 to 2 do
3          sort (currentFront,Objectives[j]); // sort currentFront
           according to fitness for Objectives[j]
           // update crowdingDistance for leading and trailing edge:
4          currentFront[currentFront.size − 1].crowdingDistance =
           currentFront[currentFront.size − 1].crowdingDistance + 1;
5          currentFront[0].crowdingDistance =
           currentFront[0].crowdingDistance + 0.99999;
           // update crowdingDistance for non-edge genotypes:
6          for i = 1 to currentFront.size − 2 do
7              objectiveDistance = currentFront[i + 1].objectiveFitness[j] −
               currentFront[i − 1].objectiveFitness[j];
8              currentFront[i].crowdingDistance =
               currentFront[i].crowdingDistance + objectiveDistance;
           // normalise the crowding distance (0 <= distance < 0.99):
9          for i = 0 to currentFront.size − 1 do
10             currentFront[i].crowdingDistance =
               (currentFront[i].crowdingDistance/3) ∗ 0.99;
11     return(isDominant)
```

Procedure chooseSurvivors

Input: parent generation of genotypes, child generation of genotypes
Output: survivors of combined parent and child populations

```
1  begin
2      combinedPopulation = parentGeneration + childGeneration;
3      sort(combinedPopulation) ; // sort by descending fitness
4      P = empty array;
5      duplicateGenotypes = empty array;
6      knownPhenotypes = empty set ; // phenotype is combination of
       features and classifier type
7      i = 0; j = 0;
8      foreach genotype in combinedPopulation do
9          if knownPhenotypes.contains(genotype.phenotype) then
10             duplicateGenotypes[j] = genotype;
11             j = j + 1;
12         else
13             P[i] = genotype;
14             i = i + 1;
15             knownPhenotypes.add(genotype.phenotype);
16         if i >= 101 then
17             break;
18     j = 0;
19     while i < 101 do
           // add duplicates until min. pop. size reached
20         P[i] = duplicateGenotypes[j];
21         i = i + 1; j = j + 1;
22     return(P)
```

Procedure findFittest

Input: Population of genotypes P
Output: single fittest genotype

```
1  begin
2      fittestGenotype = P[0];
3      for i = 0 to 100 do
4          for j = 0 to 2 do
5              if P[i].objectiveFitness[j] > fittestGenotype.objectiveFitness[j]
               then
6                  fittestGenotype = P[i];
7              else if
               P[i].objectiveFitness[j] < fittestGenotype.objectiveFitness[j]
               then
8                  break; // return to outer (i) loop
9      return(fittestGenotype)
```

Table 5.3: Performance on UCI datasets of classifiers assisted by Multi-objective GAP

Dataset	Accuracy	(sd)	Best CV	(sd)	t-test
LiverDisorder	65.01	(10.08)	64.76	(8.99)	0.16
Glass	78.58	(8.93)	68.78	(11.47)	4.29
Ionosphere	88.89	(5.83)	89.82	(4.79)	-0.64
NewThyroid	96.05	(3.11)	96.26	(4.19)	-0.26
Diabetes	75.12	(4.73)	76.05	(4.87)	-0.83
Sonar	82.51	(9.68)	86.65	(6.56)	-1.64
Vehicle	72.29	(5.84)	72.22	(3.33)	0.05
Wine	95.78	(5.42)	97.99	(3.85)	-1.95
WBC 30	95.61	(3.55)	95.09	(3.06)	0.86
WBC 9	95.63	(2.41)	95.99	(1.84)	-0.80
Overall Average	84.55	(5.96)	84.36	(5.29)	0.36

features, there is a higher ratio of constructed features in the final solution). There is evidence of greater variability in the count of nodes removed from the fittest genotype between one dataset and the next using a multi-objective approach than using parsimony pressure. This is perhaps because the fittest genotype is selected using the accuracy objective first then the simplicity objective, so the simplicity objective has less of a direct effect on the fittest individual than when parsimony pressure is made part of a single the fitness measure. However there is a high variance in the node counts of individual runs, so perhaps not too much should be read into this.

Table 5.4: Comparing GAP genotype size between UCI datasets

	No parsimony		Parsimony		Multi-Objective	
Dataset	Nodes	Trees	Nodes	Trees	Nodes	Trees
LiverDisorder	23.6	4.7	18.1	4.7	13.3	3.7
Glass	16.6	7.2	17.6	6.6	12.3	5.2
Ionosphere	23.7	9.1	20.6	8.8	21.2	6.8
NewThyroid	7.3	3.4	5.3	3.5	6.4	2.8
Diabetes	21.6	5.5	11.9	4.7	15.9	4.4
Sonar	52.3	28.1	52.2	26.0	57.8	20.2
Vehicle	34.0	11.7	32.0	11.6	24.3	8.5
Wine	10.5	7.1	8.8	6.8	10.9	5.0
WBC 30	24.5	11.0	17.3	10.5	20.9	7.7
WBC 9	16.8	4.6	10.3	4.9	12.7	3.9
Overall Average	23.1	9.2	19.4	8.8	19.6	6.8

Post-processing the multi-objective solutions reduces the number of nodes in the final genotype by an average of 6.1 nodes, leaving 13.5 nodes spread over 5.9 trees. This is roughly in line with the post-processing of the final genotypes obtained using the parsimony measure (which had an average of 14.1 nodes after cleaning - compared with 16.1 for genotypes obtained without any parsimony measure). The final solution is slightly smaller again than the parsimonious genotypes, suggesting that the multi-objective approach may also be eliminating potentially useful information (perhaps eliminating more information than the parsimony measure).

The use of multi-objective optimisation to enforce parsimony, during pre-processing of data for feature construction, does not appear to offer any improvement over the application of parsimony pressure within a single objective. This is in contrast to previously published results using multi-objective optimisation when evolving a *complete solution* [21, 26, 54]. Both parsimony pressure and multi-objective optimisation seem to run additional risk of removing useful information from the final solution when compared with a post-processing approach.

5.5 Reading the derived features

Having determined that post-processing rather than modification of the algorithm is the better approach to reducing the complexity and number of constructed features, can we show that post processing improves intelligibility? As an example, take the most accurate of the 20 solutions to the Wisconsin Diagnostic Beast Cancer dataset (WBC30) - a Naïve Bayes classifier with (initially) 9 derived features (trees).

Post-processing removed a total of only four nodes across 3 trees, including one complete tree (an original attribute). The other two changes served to remove extraneous information from derived features: removing a redundant logarithm from one feature, removing an attribute that obscured the relationship in another (the useful relationship was the difference between the standard error of the area and maximum value of the perimeter, obscured by the addition of mean texture). Table 5.5 shows these derived features (in the order in which they appear in the genotype), together with estimates of their utility, and indicating where post-processing has removed nodes.

Table 5.5: Derived features for WBC30

Feature	(Accuracy with) - (accuracy without) this feature	Accuracy using only this feature
StdErr_Texture	1.05	72.06
(StdErr_Area % Mean_Area)	2.29	91.39
Mean_ConcavePoints	0.88	90.51
(log:StdErr_Area)	1.05	92.27
Replaces (log:(log:StdErr_Area))		
StdErr_Texture	1.05	72.06
(StdErr_Texture * Max_ConcavePoints)	2.29	90.51
(StdErr_Smoothness * StdErr_Area)	0.53	86.64
(StdErr_Area - Max_Perimeter)	0.88	72.23
Replaces ((StdErr_Area - Max_Perimeter) + Mean_Texture)		

A more extreme example can be seen in the most accurate of the 20 final individuals for the Diabetes dataset. Post-processing removed a total of 22 nodes, mostly from a single derived feature. Most of the (rather lengthy) equation for this feature was removed, leaving only the left-hand side of the equation. The remaining equation is responsible for a reduction in error of just over 0.5% (as with the second column of table 5.5, this is the drop in accuracy of the classifier when the feature is removed).

$$(((BodyMassIndex\%Insulin) + (BodyMassIndex\%SkinFold)) + (BodyMassIndex\%(log : ((((BodyMassIndex\%SkinFold) + (TimesPregnant - BodyMassIndex)) + (BodyMassIndex\% (log : (GlucoseConcentration * Insulin)))) * Insulin))))$$

Was reduced to:

$$((BodyMassIndex\%Insulin) + (BodyMassIndex\%SkinFold))$$

5.6 Conclusion

Unlike standard GP algorithms where the tree being evolved is a classifier, applying parsimony to the GAP algorithm has a stronger than

desired effect on the accuracy of the client classifier (compared to the accuracy of an unaided classifier). The difference remains that during feature construction the 'benefit' to the algorithm of finding solutions with few nodes outweighs the 'cost' of the relatively small drop in accuracy. As has been noted earlier, the use of post-processing techniques to reduce the size of the final genotypes offers the same improvement in robustness and human readability (measured here by the reduced count and size of features), without the risk of losing useful information during the run.

Chapter 6

Ensembles with GAP

6.1 Introduction

This chapter explores ways to create ensembles of individuals using sets of features constructed with the GAP algorithm. It uses the Q-statistic as a diversity measure to determine the membership and size of an ensemble, comparing this with an ensemble of all members of the final population (de-duplicated). It also explores modifications to the GAP algorithm to increase diversity in the population - both a modification to the fitness function and a diversity objective for the multi-objective version of the algorithm (5.1).

With the recent increase in computing power (particularly the increasing number of processors available in off-the-shelf machines) has come rising interest in combining multiple classifiers into ensembles. In ensemble learning the individual predictions of a collection of diverse classifiers are combined (e.g. by majority voting) into a single prediction that has a greater probability of being accurate than the prediction of any one of the constituent classifiers. It has been shown that with a sufficiently diverse population of classifiers the error rate of the ensemble will reach an arbitrarily low level as the number of classifiers rises (as long as the classifiers are 'weak learners', that is as long as their accuracy is at least slightly greater than random guessing) [61]. Here two classifiers are crudely defined as being diverse if they make different predictions for a given instance - the higher the proportion of predictions on which they disagree, the more diverse a pair of classifiers is.

6.2 Calculating Diversity

As diversity is the key factor in an ensembles ability to out-perform a single individual (though the accuracy of individuals is also important - an ensemble of individuals only slightly better than random guessing would need to grow fairly large before it could compete with a single highly accurate individual), deciding how to measure diversity can be an important factor in the success of an algorithm. The understanding of diversity and its effects on ensemble accuracy is more mature for regression tasks than it is for classification tasks - it has been studied in financial forecasting research for a number of years, for example. See the survey by Brown, Wyatt, Harris and Yao [40] for more information.

Measures for determining the diversity of an ensemble come in two forms: pairwise and non-pairwise. A pairwise statistic measures the differences between a pair of classifiers - the greater the difference in their predictions the greater their diversity. For a pairwise statistic the diversity of the ensemble is the average of all possible classifier pairs in the ensemble. A non-pairwise statistic measures the diversity of the whole ensemble without considering combinations of pairs - thus offering improved (polynomial) performance. There are many diversity measures of each kind - an example of each is provided here.

6.2.1 A pairwise diversity measure - the Q statistic

The Q statistic [102] for two classifiers i and k is

$$Q_{i,k} = \frac{N^{11}N^{00} - N^{01}N^{10}}{N^{11}N^{00} + N^{01}N^{10}} \tag{6.1}$$

Where

N^{00} = classifier i is incorrect, classifier k is incorrect
N^{01} = classifier i is incorrect, classifier k is correct
N^{10} = classifier i is correct, classifier k is incorrect
N^{11} = classifier i is correct, classifier k is correct

If i and k are statistically independent Q will be zero, for positively correlated classifiers (those that tend to make mistakes on the same instances) $0 < Q <= 1$, while for negatively correlated classifiers (those that tend to make mistakes on different instances) $-1 <= Q < 0$. The

Q statistic for the ensemble, Q_{avg}, is obtained by averaging the Q value for each possible pair of classifiers in the ensemble $\mathcal{L}$ (see [13]):

$$Q_{avg} = \frac{2}{\mathcal{L}(\mathcal{L}-1)} \sum_{i=1}^{\mathcal{L}-1} \sum_{k=i+1}^{\mathcal{L}} \frac{N^{11}N^{00} - N^{01}N^{10}}{N^{11}N^{00} + N^{01}N^{10}} \tag{6.2}$$

6.2.2 A non-pairwise diversity measure - Coincident Failure Diversity

Coincident Failure Diversity (CFD) was proposed by Krzanowski and Partridge [101] as a modification of Generalised Diversity (presented in the same paper). Let p_i be the probability that of L classifiers, i are incorrect on a randomly selected instance, i.e. that the proportion of classifiers making an incorrect prediction on a random instance is $\frac{i}{L}$. Let $p(i)$ be the probability that i randomly selected classifiers will fail on a randomly selected instance. Considering two randomly chosen classifiers, maximum diversity occurs when the failure of one classifier is always accompanied by the success of the other, such that $p(2) = 0$. Minimum diversity occurs when they always fail together, at which point the probability of two classifiers failing is the same as that of a single classifier failing, $p(1)$.

$$p(1) = \sum_{i=1}^{L} \frac{i}{L} p_i$$

$$p(2) = \sum_{i=1}^{L} \frac{i}{L} \frac{(i-1)}{(L-1)} p_i$$

Generalised Diversity is then:

$$GD = 1 - \frac{p(2)}{p(1)}$$

The Coincident Failure Diversity measure is designed so that it has a minimum value of zero when all classifiers always misclassify the same instances (or are all always correct), and a maximum value of 1 when at most one classifier will fail on any given instance:

$$CFD = \begin{cases} 0, & p_0 = 1.0 \\ \frac{1}{1-p_0} \sum_{i=1}^{L} \frac{L-i}{L-1} p_i, & p_0 < 1.0 \end{cases}$$

6.2.3 Selecting a diversity measure

Kuncheva and Whitaker [104] investigated ten measures for determining the diversity of an ensemble, six pairwise and four non-pairwise, recommending the pairwise Q statistic [102] on the grounds of simplicity and ease of use. Bian and Wang [22], investigating the relationship between diversity and the accuracy of an ensemble by correlating the results of ten different diversity measures with ensemble accuracy using ten different learning algorithms over fifteen datasets, found that CFD was the most effective diversity measure 'when the diversity measure value is used as a factor for choosing base models for building an ensemble'. However, a pairwise diversity measure is preferred for application to the GAP algorithm as it enables the incremental building of an ensemble by iteratively adding the single classifier with the highest pairwise diversity from those classifiers already in the ensemble (see 6.4.1 'Ensemble Selection and combination'). Hence the Q statistic will be used to measure the diversity of ensembles created with the GAP algorithm.

6.3 Combining the classifiers

Another factor in the success of an ensemble is how the output of the individual classifiers is combined into a collective prediction. The simplest and probably most common approach is majority voting, which treats all individuals as being equally valid (this is the method used in Bagging). This can be extended by weighting the vote of each classifier, for instance according to its accuracy on the training data (weighted average or rank based linear combination, the method used in Boosting). Wolpert [199] introduced the concept of *stacking* learning algorithms (or generalizers in Wolpert's terminology) by using the cross-validated output of one set of generalizers (*level-0*) as the input to another set of generalizers (meta-learners (*level-1*), which may use a different algorithm to those in *level-0*), potentially adding as many levels as desired (majority voting can thus be seen as a specialised form of stacked generalisation). Canuto and Abreu [46] have explored the use of fuzzy sets and neural networks (also combining the two: fuzzy neural networks) as combination methods for ensembles with varying

levels of diversity. Abraham et al. [2] have used two forms of evolutionary multi-objective optimisation (NSGA-II and Pareto Archive Evolution Strategy) to determine appropriate weights for combining a heterogenous ensemble (using Multi expression programming[1], a neural network, SVM, a neuro-fuzzy model and a difference-boosting neural network) for predicting stock market movement, using four different performance (error) measures as the objectives. Saerens and Fousse [165] have used a maximum entropy approach to combining classifier outputs, where information such as the reliability of the classifiers and the correlations between them are expressed as linear constraints. These are combined to obtain a joint probability density, where 'the maximum entropy density is the density that is "least informative" while satisfying all the constraints; i.e. it does not introduce "extra ad hoc information" that is not relevant to the problem. Once this joint density is found, [they] compute the a posteriori probability of the event by averaging all the possible situations that can be encountered, i.e. by computing the marginal $P(y = i|x)$' [165]. A number of researchers [6, 8, 119] have used the Dempster-Shafer theory of evidence [167] (which can represent uncertainty and lack of knowledge) to calculate the weights to assign to each classifier for different predictions.

Yao and Islam [203] reported on four different combination methods for neural network ensembles (rank based, majority voting, RLS[2] and winner-takes-all[3]), with results indicating that the most effective combination method varied with the dataset at hand. Torres-Sospedra et al. [186] have compared 14 different combination methods for ensembles of Radial Basis Function (RBF) neural networks, concluding that the three best combination methods are Borda count[4], weighted average and majority voting; more recently they have investigated alternative combination methods for use with Boosting [187].

[1] Multi Expression Programming is a form of GP developed by Oltean and Dumitrescu [134], in which complex programs may be expressed in linear form rather than as trees. Notably MEP individuals may contain multiple solutions, the best of which is usually selected for fitness assignment.

[2] RLS: Recursive Least Squares, which minimises a weighted least squares error.

[3] Winner-takes-all is a neural network-specific method in which the output of the ensemble is decided by the individual with the highest output activation.

[4] Borda count is a voting method by which the individuals in the ensemble rank predicted classes in order of likelihood. Each class is then assigned a number of points according to the position in which it is ranked by each individual.

6.4 GP Ensembles

Genetic Programming relies on having a diverse population (at least in early generations) in order to find solutions, and often has measures to maintain that diversity (e.g. in the GAP algorithm elitism retains the fittest genotype *for each classifier type*, thus maintaining sub-populations [of at least one individual] suited to different classifiers), and so seems a good fit for generating ensembles. GP has been used to create both heterogeneous and homogeneous ensembles. Heterogeneous GP ensembles consist of individuals created with from different GP techniques, e.g. Grosan and Abraham [77] created an ensemble of two individuals to model stock market movements - one using Multi-Expression Programming and the other Linear GP (the weighting between the individuals determined by the NSGA-II multi-objective technique). Johansson et al. [87] have taken a different approach and used GP to combine a number of neural networks into ensembles of varying sizes, ultimately using GP to create 'an ensemble of neural network ensembles'.[5]

The more common approach to GP ensembles is to construct an homogeneous ensemble, either by selecting the best individuals from a number of populations or, less frequently, by constructing an ensemble from the final population of a single run. While the resulting ensemble is homogeneous by definition (having been created using a single learning algorithm), it can reasonably be expected to be more diverse than, say, an ensemble of bagged C4.5 classifiers (indeed Zhang and Bhattacharyya have attributed the improved performance of a GP ensemble over a decision tree ensemble at least partly to the greater diversity in a GP ensemble, of both the structure of the individuals and the variables selected as inputs [206]). Hong and Cho [84] have used GP to create an ensemble of rules for classifying gene expression data, first using feature selection to reduce dimensionality before breeding a population of classification rules then selecting diverse rules from the final population by direct distance calculation, and finally combining the rules by majority voting. Folino et al. [65] constructed an ensemble from the output of multiple populations using parallel cellular GP,

[5] As with the GAP algorithm, Johansson et al. cite the use of a number of 'micro techniques' as contributing to the improved performance, singling out the resampling of data during training, using a technique called tombola training, as being particularly effective.

each population trained on a different subset of the data in a fashion similar to bagging (though not sampling with replacement), combining the classifiers using a majority vote. They found that such an ensemble outperformed a single classifier trained on the entire dataset. Folino et al. [66] went on to combine the island and cellular GP models and apply clustering techniques to select a diverse set of classifiers, then apply pruning techniques to reduce the size (and memory requirements) of the resulting ensemble.

6.4.1 Searching for a free lunch

Gagné et al. [72] took advantage of the similarities between evolutionary computing and ensemble learning in their use of classifier diversity to explore strategies for constructing an ensemble from a single population (the intent being to produce an ensemble with no additional computational effort over evolving the population). In doing so they constructed two ensemble frameworks to investigate two interrelated areas: *i*) enforcing both predictive accuracy and population diversity, *ii*) selecting the ensemble classifiers. In Offline Evolutionary Ensemble Learning (*Off*-EEL) the ensemble is constructed from the final population only, while in Online Evolutionary Ensemble Learning (*On*-EEL) some classifiers are added to the ensemble at each generation. In either case the decision of the ensemble is the majority vote of the classifiers. Because *Off*-EEL constructs an ensemble from the final population it can easily be applied to any other evolutionary approach with a relatively minor modification to the fitness function to encourage population diversity (happily it also outperforms *On*-EEL in the results presented by Gagné et al.). A version of *Off*-EEL has therefore been applied to the GAP algorithm. For further details of *On*-EEL the reader is directed to [72].

Diversity Enforcing Fitness Measure

In order to enforce diversity while maintaining accuracy Gagné et al. extended a fitness measure previously employed by Liu et al. [118], weighting instances according to how difficult they are for a set of reference classifiers to classify. This approach is inspired by co-evolutionary learning [80], in which the classifier is presented with increasingly diffi-

cult examples to continuously to drive improvement. The 'reference classifiers' here are the current population of classifiers - thus the weighting of instances is continuously updated.

The weight $w_{(i)}$ of a single instance i is derived from a set of reference classifiers $\mathcal{Q}$ as follows:

$$w_i = \frac{1}{|\mathcal{Q}|} \sum_{h \in \mathcal{Q}} \ell(h(\mathrm{x}_i), y_i)$$

Where the loss function $\ell(h(\mathrm{x}), y)$ defines the error cost of classifier h predicting value x instead of the true value y. The loss function used here is the step loss function (where $\ell(h(\mathrm{x}), y) = 0$ if $\mathrm{x} = y$, 1 otherwise).

The fitness of classifier h is defined as the sum, over all instances correctly classified by h, of each weight w_i raised to the power γ:

$$\mathcal{F}(h) = \sum_{h(\mathrm{x}_i)=y_i}^{i=1\ldots n} w_i^{\gamma}$$

The parameter γ governs the importance of the weights and can be used to compensate for noise in the dataset. The combination of the step loss function ℓ and γ=1 causes the fitness function to be the same as that used by Liu et al. [118], and is the fitness function used here.

Due to the nature of the termination criteria employed in the GAP algorithm it is not possible to re-evaluate the weights of instances at every generation[6]. Instead the weights are evaluated against the current population at generation 0 and again when the instances are randomly reselected, leaving the weights constant at other generations. This has practical advantages in that surviving genotypes need not be re-evaluated in successive generations, but does mean that the co-evolutionary nature of the algorithm intended by Gagné et al. is somewhat diminished.

Ensemble Selection and combination

Gagné et al. observe that the selection of individuals to form an ensemble is formally equivalent to a feature selection algorithm, and also note that feature selection is one of the hardest tasks in machine learning.

[6]The termination criteria of the GAP algorithm is that a single individual must remain the fittest in the population for n generations. Re-evaluating instance weights causes the fitness of individuals to vary from generation to generation, leading the algorithm to cycle indefinitely.

They therefore opt for a simple greedy selection process, using a selection criterion that minimises the error margin of the ensemble. Given a population of classifiers $\mathcal{H} = \{h_1, \ldots, h_T\}$, T possible ensembles are considered. Starting with a ensemble $\mathcal{L}_0$ consisting of a single classifier - the one with the smallest error rate - each new candidate ensemble $\{\mathcal{L}_1 \ldots \mathcal{L}_{T-1}\}$ is constructed by adding the classifier that optimises the margin of the new ensemble, combining the output of the classifiers by majority vote.

As used by Gagné et al, *Margin* can be defined as follows. For each instance (x_i, y_i) find the most popular incorrect prediction, i.e. find y'_i, the class most frequently predicted by the classifiers in $\mathcal{L}$ "such that y'_i is different from the true class y_i. Let c_i (respectively c'_i) denote the number of classifiers in $\mathcal{L}$ associating class y_i (resp. y'_i) to x_i. Then margin m_i is defined as c_i - c'_i" [72]. If the majority vote of the ensemble predicts the class correctly the margin will be positive, otherwise it will be negative. The initial selection process aimed to maximise the minimum margin. Gagné et al. provide Pseudo-code for the margin based selection procedure (see algorithm 6.1).

Algorithm 6.1 Gagné's margin-based Ensemble Selection

1. Let $t = 1$, and $\mathcal{H}_1$ be the set $\mathcal{H}$ with duplicate individuals removed
2. While $\mathcal{H}_t$ is not empty:
   (a) Let $h^*_t = \mathrm{argmax}_{h \in \mathcal{H}_t}(\mathcal{L}_{t-1} \cup \{h\})$ (find h^*_t that optimises the margin function for the new ensemble)
   (b) Let $\mathcal{H}_t + 1 = \mathcal{H}_t \backslash \{h^*_t\}$ (remove h^*_t from $\mathcal{H}_t$)
   (c) Let $\mathcal{L}_t = \mathcal{L}_{t-1} \cup \{h^*_t\}$ (and add it to $\mathcal{L}_t$)
   (d) $t = t + 1$
3. Return $\mathcal{L}^*$, the classifier ensemble in $\{\mathcal{L}_0 \ldots \mathcal{L}_{t-1}\}$ that achieves the lowest error rate on the training set $\mathcal{E}$, selecting the smallest ensemble in case of ties.

Gagné et al. found that their initial choice of minimum margin (as shown in algorithm 6.1 for the selection criteria was not fine-grained enough - there were too many ties - so they developed a more complex criterion based on a margin histogram (see [72] for more details).

However as the power of ensembles lies in the diversity of the classifiers, a different criteria was used for the GAP algorithm: maximising the pairwise diversity of the new classifier with those of the current ensemble (specifically to minimise the average Q-score as measured be-

tween the candidate classifier and those in the current ensemble)[7]. The use of a pairwise diversity measure is important here as it allows the identification of a single classifier with the greatest pairwise diversity to add to the current ensemble, rather than having to consider multiple combinations as would be the case for a non-pairwise measure. Pseudo-code for the diversity based selection procedure is shown in algorithm 6.2. As Banfield et al. [13] note, the Q statistic is vulnerable to divide-by-zero errors - these are handled here by the simple expedient of setting the divisor to 1 instead of 0. (Given that for the divisor to be zero both $N^{11}N^{00}$ and $N^{01}N^{10}$ must be zero (see equation 6.1) this causes no problems because the dividend will also be 0 and so the result should be 0 regardless of the divisor's value).

Algorithm 6.2 GAP diversity-based Ensemble Selection

1. Let $\mathcal{H}_1$ be the set $\mathcal{H}$ with duplicate individuals removed
2. Let $t = 2$, let $\mathcal{L}_1$ consist of the fittest individual h_1^*, and $\mathcal{H}_t$ be the set $\mathcal{H}_1$ with h_1^* removed
3. While $\mathcal{H}_t$ is not empty:
   (a) Let $h_t^* = \text{argmin}_{h \in \mathcal{H}_t}(\frac{1}{t-1}\sum_{i=1}^{\mathcal{L}_{t-1}} \mathcal{Q}_{i,h})$ (find h_t^* possessing the smallest average pairwise diversity with members of the current ensemble $\mathcal{L}_{t-1}$)
   (b) Let $\mathcal{H}_t + 1 = \mathcal{H}_t \backslash \{h_t^*\}$ (remove h_t^* from $\mathcal{H}_t$)
   (c) Let $\mathcal{L}_t = \mathcal{L}_{t-1} \cup \{h_t^*\}$ (and add it to $\mathcal{L}_t$)
   (d) $t = t + 1$
4. Return $\mathcal{L}^*$, the classifier ensemble in $\{\mathcal{L}_1 \ldots \mathcal{L}_{t-1}\}$ that achieves the lowest error rate on the training set $\mathcal{E}$, selecting the smallest ensemble in case of ties.

6.5 Ensembles using the GAP algorithm

Gagné et al.'s approach to ensemble building has been applied to the GAP algorithm as described above. The Ensemble selection method was applied first, then combined with the diversity-enforcing selection criteria. Both steps were evaluated against the ten UCI datasets.

[7] One potential disadvantage of the Q statistic is that is designed primarily for binary classes and considers only correct/incorrect classifications. An area for future study would be to use a pairwise diversity measure that is designed to take account of multiple classes (i.e. to take advantage of the notion that different incorrect classifications are also an indicator of diversity).

6.5.1 Selecting the ensemble

After breeding a population using the GAP algorithm as previously set out (non-multi-objective) in algorithm 4.3, two ensembles were created from the final population. The first consists of the entire population, de-duplicated (the 'whole ensemble' in table 6.1, with an average size of 56 individuals). The second uses the ensemble selection method outlined in algorithm 6.2 (the 'selected ensemble' in table 6.1). Both forms of ensemble are heterogenous as the individual genotypes code for the classifier type to use (and as the form of elitism employed maintains the fittest genotype *for each classifier type* all 3 classifier types are guaranteed to be represented in the whole ensemble). As previously, results on the UCI datasets are obtained over 20 runs, splitting the instances into 90% train and 10% test data. The standard deviation shown in table 6.1 is the *average* of the standard deviations over the 10 datasets. Train and test accuracy of the fittest genotype are included for reference (once again, the comparison classifier (the highest-scoring of the 3 classifiers on the train set) achieves an accuracy of 84.36%).

Table 6.1: GAP-assisted ensemble performance on UCI datasets - without weighting

	fittest genotype		**whole ensemble**		**selected ensemble**	
	average	**(sd)**[8]	**average**	**(sd)**	**average**	**(sd)**
train accuracy	89.29	(1.43)	89.16	(1.40)	89.45	(1.30)
test accuracy	84.39	(5.64)	85.12	(5.47)	85.15	(5.63)
Q score			0.65	(0.30)	0.55[9]	(0.38)
ensemble size			55.77	(8.70)	16.86	(19.21)

As measured by the Q score, the average diversity of the whole final population (once duplicates are removed) is 0.65, with the Q score being calculated against the test data for each fold. The Q score varies widely between datasets, from an average of 0.27 for Wine to 0.90 for Wisconsin Breast Cancer (original). A low average Q score seems to be slightly correlated with datasets with both few instances and high population accuracy (this means that there are relatively few mis-classified instances), but this is by no means a conclusive relationship - it may

[8] As a single value for standard deviation over 10 different datasets doesn't make sense, the values for S.D. are the averages of the standard deviation over the 10 datasets.

[9] The selected ensemble consisted of a single classifier in 58 of the 200 runs, Q scores are averaged over the remaining 142 runs.

simply be that the new thyroid and wine datasets are particularly open to classification by diverse means.

Applying the selection criteria reduces the size of the ensemble dramatically (to less than a third that of the whole ensemble, to 17 individuals - this is in part due to the fact that the ensemble consists of only a single genotype in a quarter of cases[10]), and returns a lower Q score for the selected ensemble while very slightly improving the accuracy. In one run on the new thyroid dataset the selected ensemble even managed to achieve a Q score of -1 and accuracy of 100% (though this only applied on the 10% test data fold and subsequent cross-validation on the full dataset saw the accuracy drop to 98.6%. The Q-score of -1 is a much due to the fact that there are very few instances mis-classified by any of the classifiers as it is to their diversity.). Thus applying Gagné et al's selection criteria (modified to maximise diversity rather than margin) does have a positive effect on the ensuing ensemble, identifying a smaller number of more diverse classifiers that offer slightly improved accuracy.

6.5.2 Enforcing diversity in the population

The experiments outlined in section 6.5.1 were repeated with the inclusion of the modified Diversity Enforcing fitness measure outlined in section 6.4.1 - the results of which are shown in table 6.2 (as with table 6.1, the standard deviation shown in table 6.2 is an average of the standard deviations of the 10 datasets).

As can be seen by comparison with table 6.1, applying weights to the instances to enforce population diversity does not work well with the GAP algorithm. It does increase the diversity of the population on average (reducing the Q score from 0.65 to 0.61) with a corresponding increase in the number of unique individuals in the final population, but at the expense of population fitness. The fittest individual in the population suffers a marked drop in accuracy on both train and test sets, and this is mirrored in the accuracy of the ensembles. As the drop in accuracy occurs on train as well as test sets we can say that the drop in accuracy is not due to increased overfitting, rather that

[10]The prevalence of single-classifier 'ensembles' is a good argument for setting a minimum ensemble size of 3 classifiers, this being the practical minimum for a majority voting system. This has not been explored further at this point.

the increased emphasis on hard-to-classify instances hampers overall accuracy. Comparing the drop in accuracy of the fittest genotype with that of the ensemble (the average accuracy of the fittest genotype drops by 1.65% with weighting applied, and by slightly less (1.22% and 1.34%) for the whole and selected ensembles respectively) suggests that the increase in the diversity of the population does improve the accuracy of the ensemble, but not nearly enough to counter the effects of the reduced accuracy of the individual classifiers. Comparing the number of generations the algorithm runs for on average (69 generations without weighting, 97 generations with weighting) tells us that this drop in performance has not come about through prematurely reaching the termination criteria - rather the algorithm has worked harder to achieve poorer results.

Table 6.2: GAP-assisted ensemble performance on UCI datasets - with weighting

	fittest genotype		**whole ensemble**		**selected ensemble**	
	average	**sd**	**average**	**sd**	**average**	**sd**
train accuracy	86.67	(2.37)	87.13	(2.08)	87.94	(1.71)
test accuracy	82.74	(5.85)	83.90	(5.30)	83.81	(5.77)
Q score			0.61	(0.19)	0.51[11]	(0.28)
ensemble size			58.99	(8.83)	16.40	(12.12)

6.5.3 Enforcing diversity with Multi-Objective optimisation

Adapting a co-evolutionary model to the GAP algorithm is evidently an unsuccessful approach to enforcing diversity. Perhaps a more suitable route would be to use the multi-objective techniques from section 5.4, treating diversity as a specific objective (i.e. minimise the average pairwise q-score between an individual and the rest of the population) rather than relying on encouraging diversity using the mechanics of the multi-objective algorithm alone [12]. This is similar to DIVACE ('diverse and accurate ensemble learning algorithm') developed by Chandra and Yao [47], which also uses explicit accuracy and diversity objectives[13] in

[11] The selected ensemble consisted of a single classifier in 30 of the 200 runs, Q scores are averaged over the remaining 170 runs.

[12] The diversity-encouraging mechanism (which removes duplicates from the combined population of parent and child generations - see section 5.4 for further details) is retained in addition to the diversity objective.

[13] The diversity objective used in DIVACE is not the Q-statistic but the correlation penalty function from Negative Correlation Learning which seeks, for each instance, to maximise the

a multi-objective approach (also using a non-dominated sorting algorithm) to evolving an ensemble of neural networks, taking the Pareto set rather than the whole population as the final ensemble.

A new Diversity Objective is created that minimises the average of the Q-scores obtained between an individual and all other members of the population (this measures the Q-score for the individual rather than the whole ensemble). We must also create a new objective to replicate the current fitness function - to maximise accuracy, breaking ties by minimising the number of nodes (an 'AccuracyThenNodeCount' objective). (Section 5.4 employed separate Accuracy and Simplicity objectives, which are not appropriate here). This is achieved by converting the accuracy of an individual to an integer value (multiplying by 100,000) and using the decimal portion of the number to represent size[14] by adding $1/$*(number of nodes)* (multiplied by 0.99 to ensure that the number of nodes can never affect the integer portion of fitness in cases where the genotype has only a single node). The result is used as the fitness for this objective.

The inclusion of a Diversity objective requires an amendment to the mechanics of the termination criteria to prevent the algorithm from cycling indefinitely. Recall that for termination purposes the objectives are considered in order, so that in section 5.4 the Simplicity objective is only used to break ties from the Accuracy objective. The Diversity objective cannot be used in this fashion, as over time a highly accurate genotype will produce more offspring, which will make similar classifications, thus reducing its diversity as measured by the Q-score. This means that the success of an accurate genotype actually *reduces* its fitness over time; leading to a situation in which the fittest genotype may be replaced by one with exactly the same accuracy and size (but slightly lower Q-score), only to regain its position a few generations later once the newly fittest has spread offspring through the population. To prevent this the Diversity objective is excluded from consideration when determining termination criteria - effectively leaving the same termination criteria as the non-MO version of the algorithm.

difference between the output of the ensemble and that of the individual (as measured by the activation level of the output node of a neural network).

[14]This introduces the theoretical possibility that a smaller genotype might have greater fitness than a larger, more accurate one if it has the same accuracy to within one instance in 100,000 - however this is many more instances than are used during training so is impossible for practical purposes.

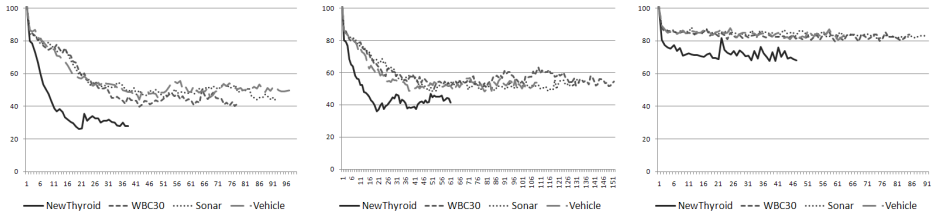
Table 6.3: GAP-assisted ensemble performance on UCI datasets - multi-objective

	fittest genotype		**whole ensemble**		**selected ensemble**	
	average	**sd**	**average**	**sd**	**average**	**sd**
train accuracy	89.20	(1.46)	91.10	(1.27)	91.36	(1.26)
test accuracy	83.91	(5.27)	85.62	(4.98)	85.53	(5.25)
Q score			0.44	(0.26)	0.38	(0.30)
ensemble size			101.00	(0.00)	48.17	(23.13)

Table 6.3 shows the results of the multi-objective approach on the UCI datasets. It is immediately obvious that this is a more effective method of increasing diversity than applying weights to the dataset - the Q-score is reduced from 0.61 with weighting to 0.44, without the devastating effect on accuracy. While the accuracy of the single fittest genotype has dropped somewhat compared with the standard algorithm[15] (table 6.1) from 84.39% to 83.91%, there is an equivalent *improvement* in the accuracy of the ensembles (from 85.15% to 85.62% for the whole ensemble). It is noteworthy that the whole ensemble now always consists of the entire population - the combination of the diversity objective and the diversity-encouraging mechanism adapted from POPE-GP mean that throughout all generations each individual in the population is unique. This does not have any impact on the number of generations run (an average of 67 with multi-objective code compared to 69 with the standard algorithm). There is however an increase in the number of new individuals evaluated at each generation, as shown in figure 6.1. This means that there is extra effort required to train the ensemble, as well as extra effort at run-time (due to the increased size of the ensemble).

The increase in the number of new individuals is associated with an increase in the diversity of the population (a more diverse population naturally means that offspring are less likely to resemble individuals already present in the population). Figure 6.2 shows a comparison of Q score by generation for the three methods. The multi-objective code shows a steady decrease in Q score over time, and is the only method to do so. While weighting the instances causes the initial population

[15]Note however that this is only evident on the test accuracy - the train scores are almost identical. Either the multi-objective version of the code has somehow resulted in a reduced ability to generalise or the difference in test scores is just down to random chance.



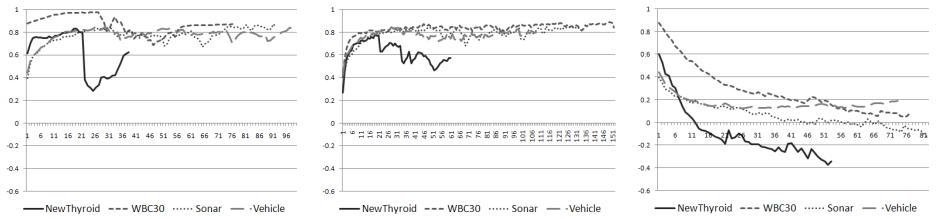
(a) Standard

(b) Weighted. Note that there are more generations for the weighted runs.

(c) Multi-objective

Figure 6.1: # New individuals by generation - standard v. multi-objective

to show greater diversity than with either of the other methods, this does not have any effect in the long term - the Q score shows a rapid increase until it reaches roughly the same level as with the standard version.



(a) Standard

(b) Weighted.

(c) Multi-objective

Figure 6.2: Q score by generation - standard v. multi-objective

When applied to the multi-objective population, the ensemble selection algorithm cuts the average size of the ensemble by just over half, but now results in only a very small increase in diversity and a small *drop* in accuracy (see table 6.3). This is perhaps because the whole ensemble is already relatively diverse - the small increase in diversity is not sufficient to offset the loss of half the classifier votes. The larger size of the multi-objective ensemble, both whole and selected, also means that the ensemble will require more effort to classify each instance. It might therefore be appropriate to use a different mechanism to select the members of the ensemble, e.g. a multi-objective GA similar to that employed by Park and Cho [141] or Nojima and Ishibuchi [129]. (This has not been investigated any further here).

An attempt was made to reduce the size of the ensemble and improve the accuracy of the fittest individual by removing the constraint

requiring individuals in the population be unique (this constraint was originally introduced to promote diversity and prevent premature convergence - now that there is a diversity objective the constraint may no longer be necessary). While the accuracy of the fittest individual does increase slightly, the size of the ensemble drops rather too much and is much smaller than the ensembles created with the standard version while possessing a higher Q score. Results are shown in table 6.4. The uniqueness constraint was therefore retained.

Table 6.4: GAP-assisted ensemble performance on UCI datasets - multi-objective without uniqueness constraint

	fittest genotype		**whole ensemble**		**selected ensemble**	
	average	**sd**	**average**	**sd**	**average**	**sd**
train accuracy	89.51	1.53	89.40	2.01	89.73	1.55
test accuracy	84.06	5.23	84.33	5.43	84.72	5.30
Q score			0.68	0.28	0.71	0.34
ensemble size			15.38	7.89	7.07	5.96

Table 6.5: Comparison of ensemble performance on UCI datasets

Dataset	AdaBoost (C4.5, 56 iterations)	GAP whole ensemble	GAP selected ensemble	Off-EEL	AdaBoost (decision stumps)	Random Forests
LiverDisorder	69.88	67.22	67.80	**70.80**	69.60	68.71
	(10.21)	(8.85)	(8.13)	(7.40)	(5.40)	(7.36)
Glass	77.16	76.93	77.87			**79.06**
	(9.68)	(7.95)	(9.31)			(9.41)
Ionosphere	93.74	93.24	93.62			**93.77**
	(3.75)	(3.70)	(3.63)			(3.81)
NewThyroid	94.46	**96.30**	96.29			95.13
	(4.10)	(3.54)	(3.54)			(4.08)
Diabetes	74.73	75.12	75.00	**76.00**	71.90	73.82
	(5.54)	(5.10)	(5.75)	(4.00)	(5.00)	(4.04)
Sonar	**85.50**	84.30	82.88			80.17
	(7.76)	(7.91)	(8.86)			(8.88)
Vehicle	**78.79**	75.30	75.89			75.59
	(4.43)	(4.22)	(3.80)			(3.71)
Wine	95.75	96.83	94.85			**97.73**
	(5.72)	(4.38)	(5.58)			(3.88)
WBC 30	**96.83**	95.17	95.35			95.70
	(2.60)	(2.24)	(2.06)			(2.83)
WBC 9	**96.70**	95.78	95.71	96.60	94.70	95.71
	(1.16)	(1.89)	(1.85)	(1.70)	(2.00)	(1.81)
Average	86.35	85.62	85.53			85.54

Results for the whole and selected ensembles are obtained using the multi-objective variant of the GAP algorithm (5.1, with the uniqueness constraint enforced).

6.5.4 Comparison with AdaBoost and Random Forest

Gagné et al. compared the performance of their ensembles with AdaBoost, using Decision Stumps[16] as a base classifier with the number of iterations set at 2000. We also use AdaBoost as a comparison, but with C4.5 as the base classifier and the number of iterations set at 10 (the Weka default), 17 (average size of a selected ensemble) and 56 (average size of the whole ensemble). By comparing the results in table 6.5 on the three UCI datasets in common with Gagné et al. we can see that AdaBoost using C4.5 provides greater accuracy than AdaBoost using Decision Stumps, even with a greatly reduced number of iterations. We also provide comparison with another frequently used ensemble technique, Random Forests, using the Weka default of 10 trees.

Table 6.5 shows that, while not completely outclassed (the selected and whole ensembles achieve a greater accuracy than AdaBoost on 3 and 4 datasets respectively), ensembles created from a single run of the GAP algorithm do not perform as well as a more traditional ensemble using AdaBoost with C4.5[17], and are roughly on a par with random forests using the weka default of 10 trees. The Off-EEL ensembles created by Gagné et al. also achieve a greater accuracy than the GAP ensembles on the three datasets they have in common.

If modifying the GAP algorithm to produce an ensemble from a single run is not a competitive approach, what about an ensemble of 20 individuals created from the fittest individual from each run? (This is a similar approach to that suggested in section 4.4 'Selecting the most effective classifier' - but instead of selecting the most accurate individual from 20 runs we are using all 20 individuals). The resulting ensemble will consist of individuals with a much higher average accuracy than the

[16] Decision Stumps are simple binary classifiers that classify instances using a threshold on a single feature of the dataset and are thus the simplest possible linear classifiers.

[17] The effect of using AdaBoost to create an ensemble from the single fittest genotype produced by the GAP algorithm was also investigated, using C4.5 as the only classifier type as C4.5 seems so particularly suited to boosting. The average accuracy on test data of the individual classifiers was 83.62% (slightly less than the results obtained in chapter 4), while the AdaBoosted ensemble achieved only slightly greater average accuracy of 83.18%, 84.04% and 84.11% with 10, 20 and 56 iterations respectively. The lack of improvement in accuracy with increasing iterations may be due to the fact that AdaBoost can be prone to overfitting, particularly in the presence of noisy data, because at each iteration it focuses on the misclassified instances [45]. As the GAP algorithm is also prone to overfitting we may asking too much by placing one algorithm atop the other (this is borne out by the fact that, on some folds, the ensemble built from 10 or 20 iterations out-performs the 56-iteration ensemble). This is supported by research by Wickramaratna et al. , who demonstrated that 'boosting productivity increases, peaks and then falls as the strength of the underlying learner increases'[197]. In short, the GAP algorithm is not a good candidate for producing classifiers for AdaBoost.

final population of a single run, and should have greater diversity. As we have not kept back a subset of the UCI data to test the ensemble on we cannot draw any statistically valid conclusions from the UCI datasets. However, this can be tested on a larger dataset from which we have held back a suitable amount of test data: the Bristol Vision dataset (see the case study in chapter 7).

6.6 Conclusion

In this chapter we have shown that the final population of individuals produced by the GAP algorithm is suited to the creation of a diverse ensemble, and that a diversity-based selection method can reduce the size of the ensemble without affecting accuracy to any meaningful degree. We have also shown that using multi-objective techniques are much more effective at promoting diversity in the population than modifying the fitness function, producing more diverse and more accurate ensembles than the standard algorithm (though at the cost of a less accurate single fittest individual). We have also speculated that an ensemble of the fittest individuals from the 20 runs would be both more accurate and more diverse than an ensemble from the final population of a single run.

Chapter 7

Case Study: the Bristol Vision dataset

7.1 Introduction

This chapter introduces a new, large, real world dataset. The different versions of the algorithm from the previous three chapters (the basic algorithm 4.3, the parsimonious fitness function and multi-objective approach 5.1, the ensemble selection mechanism 6.2), are all evaluated against this dataset. We explore an approach to highlight both the similarities and differences between the features employed by different individuals in an ensemble. The larger size of the dataset also allows us to confirm that the single most accurate individual (or ensemble) from 20 runs (as identified on the training data) is indeed more accurate than the average individual on the test data.

7.2 The Bristol Vision Dataset

The Bristol Vision dataset is a machine vision dataset consisting of measurements of video data feed from a camera in a controlled environment. The aim is to identify and provide false colouring for various objects in a room on a heads-up display worn by a partially sighted user as they maneuver through the environment. The dataset consists of 9963 instances, 31 real-number attributes and 6 classes, without normalisation. The attributes are Luminance, Red, Green, Blue, and 27 Haar Wavelet coefficients (at 9 size scales and 3 orientations). (A Haar Wavelet is a discrete oscillation where values can be 0, 1 or -1. A Haar

Wavelet Transformation can be used to detect edges in an image by producing edge maps of the horizontal, vertical or diagonal edges, as illustrated in figure 7.1.[1]) The class values are Obstacle, Boundary, Floor, Chair, Ball and Box. The target accuracy is 93%, as achieved by a Multi-Layer Perceptron (MLP) Artificial Neural Network in a previous study at Bristol University (unpublished).

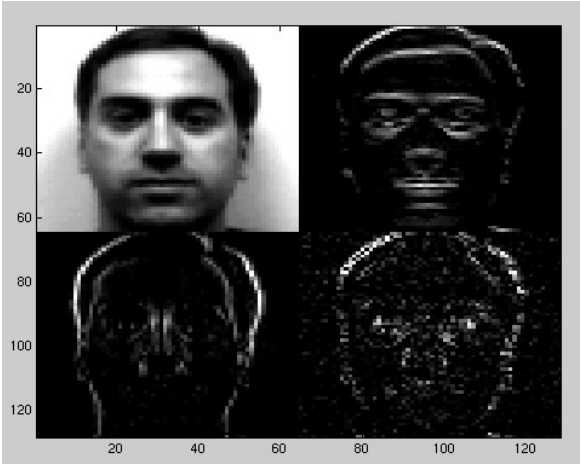


Figure 7.1: Example of a Haar Wavelet transformed image (original and horizontal, vertical and diagonal edge maps)

The dataset was divided into a training set of 6676 randomly selected instances (67%) and a test set of the remaining 3287 instances. The training set was then treated as the source for 20 runs of the algorithm, each using 90% of the data for training and 10% for test. The performance of the unaided classifiers on the training set is shown in table 7.1 (accuracy on the train set is from cross-validation, followed by standard deviation in brackets). The accuracy figure in the final column was obtained from a single run on the test data. Of the three classifiers, IBk (using the default settings in Weka) is the most accurate by a convincing margin, but falls short of the performance achieved by the MLP neural network.

7.3 Applying The GAP algorithm

The GAP algorithm (4.3) was run with and without parsimony on the training data, the results of which are shown in table 7.2 (at this point

[1]Haar Wavelet transformed image reproduced from `http://www.owlnet.rice.edu/~elec539/Projects97/morphjrks/moredge.html`.

Table 7.1: Performance of unaided classifiers on Bristol Vision dataset

Classifier	Train	(sd)	Test
C4.5	87.43	(1.06)	88.56
IBk	90.14	(0.84)	90.57
Naive Bayes	65.36	(2.05)	65.14

no post-processing has been done, and genotypes not evaluated on the test data). The t-test values are obtained by comparing the accuracy against that of IBk on the training data.

Due to the performance implications of using a large dataset the actual number of instances used for fitness evaluation during the evolution of the population was limited to 2000 (randomly selected).[2]

Table 7.2: Performance of GAP-assisted classifiers on Bristol Vision dataset

	Accuracy (S.D.)	Paired t-test
No Parsimony	91.56 (1.66)	4.26
Parsimony	91.42 (1.26)	4.53
Multi-Objective	90.90 (1.62)	1.49

The accuracy is virtually the same with or without parsimony, but applying parsimony pressure reduces the number of nodes by approximately 48% and in doing so reduces the standard deviation (see table 7.3 for node counts). Applying parsimony pressure therefore provides some robustness to the algorithm by reducing variation between solutions with minimal cost to accuracy (thus improving the t-test value). Despite having roughly comparable performance on the 10 UCI datasets, the multi-objective code ceases to achieve a significant improvement over C4.5 on the Bristol Vision dataset (while the performance of the multi-objective code no longer offers a significant improvement over IBk it is not significantly worse than the GAP algorithm [with or without parsimony] at the 95% confidence level).

Despite the fact that on its own IBk is clearly the best of the three classifiers for this dataset, over 25% of the final genotypes used C4.5 as the base classifier, rising to just over 50% when parsimony pressure is

[2]At the point where the termination criteria have been met the first time and the dataset would be randomly reordered, it was instead re-sampled to obtain a different random selection of 2000 instances from the training set. Note that this means random shuffling of the instances has been replaced by random subsampling in cases where the training dataset is larger than 2000 instances.

applied.

Although the algorithm provides a significant improvement over an unaided classifier at the 95% confidence level, it still falls short of the accuracy of the MLP neural network used in the previous study. However we have not yet performed any post-processing to clean the genotypes, and the presence of a separate test set allows us to evaluate the best single genotype (as identified by cross-validation on the training set).

7.3.1 Post-processing

Post-processing to clean the final genotypes reduces the number of nodes as shown in table 7.3 - figures show the count of nodes before and after post-processing (averaged over the 20 runs), followed by accuracy using cross-validation on the training set and finally accuracy on the test set of 3287 previously unseen instances. As post-processing uses the entire (training) set the train accuracy cannot be fairly compared to the accuracy of the initial genotypes shown in table 7.2, although it is indicative of improvement. All three sets of genotypes show an improvement in accuracy from train to test data.

Table 7.3: Effect of post-processing derived features on performance on Bristol Vision dataset

	# Nodes (before)	# Nodes (after)	Train	(sd)	Test	(sd)
Without Parsimony	52.2	28.6	92.17	(0.93)	93.04	(0.88)
With Parsimony	27.2	21.2	91.52	(1.06)	92.41	(1.02)
Multi-Objective	31.7	19.7	91.30	(1.21)	91.86	(0.80)

The cleaned genotypes produced without parsimony pressure are over 25% larger than the cleaned genotypes with parsimony applied (and still larger than the parsimonious genotypes before post-processing), but they are more accurate (by over half a percent) and the standard deviation is now smaller than that with parsimony. Comparing the number of nodes after post-processing, it appears that applying parsimony pressure is causing useful information to be lost from the fittest individual.

Accuracy on the test data confirms that the additional nodes in the non-parsimonious solutions are providing genuinely useful information and are not simply over-fitting the training data. A paired t-test per-

formed on the difference in accuracy on the test data (between the normal and parsimonious methods) gives a score of 1.93 - just short of showing a significant improvement at the 95% confidence level.

The Multi-objective code (algorithm 5.1) produces genotypes with slightly more nodes than genotypes produced by parsimony, but the position is reversed after post-processing (see table 7.3). From the accuracy on the training set after post-processing it appears that the multi-objective optimisation is also causing useful information to be lost from the final solution, even more so than is the case by combining parsimony pressure with accuracy in a single objective. This is borne out when we look at the performance of the best-of-run genotype on the validation dataset - see table 7.4.

7.4 Selecting the best-of-run individual

The individuals with the best accuracy using cross-validation on the training set (after post-processing) were applied to the validation set to obtain the results shown in Table 7.4 (one individual from the non-parsimonious genotypes, one from the parsimonious, one from the multi-objective approach). All three individuals offer a marked improvement over the average shown in table 7.3.

Table 7.4: Performance of classifiers assisted by Best-of-run GAP genotype on Bristol Vision dataset

	Accuracy (train)	Accuracy (validation)
No Parsimony	94.01	94.83
Parsimony	93.68	94.07
Multi-Objective	93.26	93.31

The best of run genotype without parsimony is three quarters of a percent better than that with parsimony (both of which use IBk as the base classifier), one and a half percent better than the multi-objective genotype, and indeed now offers an almost two percent improvement over the baseline performance of the MLP neural network. To determine the impact of post-processing on the overall accuracy we also evaluated the same genotypes as they were before post-processing on the test data. In both cases the cleaned genotype performed better on the

test data than the original genotype (the original without parsimony obtained 94.34% and the original with parsimony obtained 93.7%). That is, post-processing successfully reduces over-fitting as well as improving human readability. That the parsimonious solution (before post-processing) achieved a greater accuracy than the non-parsimonious solution indicates that applying parsimony pressure does successfully remove nodes that have over-fit the training data; however the results after post-processing show that it also removes nodes that are genuinely useful. It is possible to say that for the GAP algorithm post-processing is a more effective method for reducing the size of the solution than applying parsimony pressure during evolution (either through modification of the fitness measure or through application of multi-objective techniques).

7.5 Extracting Meaning from the Genotypes

Visual inspection of a textual representation of the best single genotype, while easier with the cleaned genotype than the original, did not provide any immediately useful information to a subject matter expert. This is perhaps because the genotype still contains 18 features (of which 7 are constructed features, the remaining 11 are original attributes) using 17 of the original attributes in total, and it is not clear which of the features are the most useful in classification (though we can immediately state that the 14 the coefficients not present in the genotype are not required for classification). More useful information may be obtained using a process that estimates the utility of each feature (by removing each one in turn and determining the accuracy of the tree without it, also noting any features that occur more than once).[3] Using this yardstick 3 constructed features were identified as being the most important features: Blue/Luminance, Green/Luminance and Log(Red), along with one of the original coefficients. Dividing a colour by its luminance was identified by the expert as a method by which the true colour of an object may be determined (by factoring out the level of light falling on it). Though this relationship is not new to an expert, it was not known to the algorithm (or the author) beforehand and

[3]Because the genotype has been processed to remove any part that does not contribute toward accuracy, we can be confident that IBk is benefiting from having extra copies of each of two features: Blue/Luminance and log(Red); and one of the coefficients.

demonstrates that the GAP algorithm is capable of finding meaningful and useful relationships between attributes in real-world datasets.

Additional use was made of the human readability of the decision trees created by C4.5. These were created using the features of the both the best of run genotype (based on IBk) and of the best genotype based on C4.5 - see figures 7.2 and 7.3, which show the top sections of the decision tree only, including the shortest path to a classification.[4] The presence of ratios of colour/luminance at the top of both decision tree hierarchies adds weight to their importance in identifying objects in the images. See appendix A.1 and A.2 for textual representations of the full decision trees.

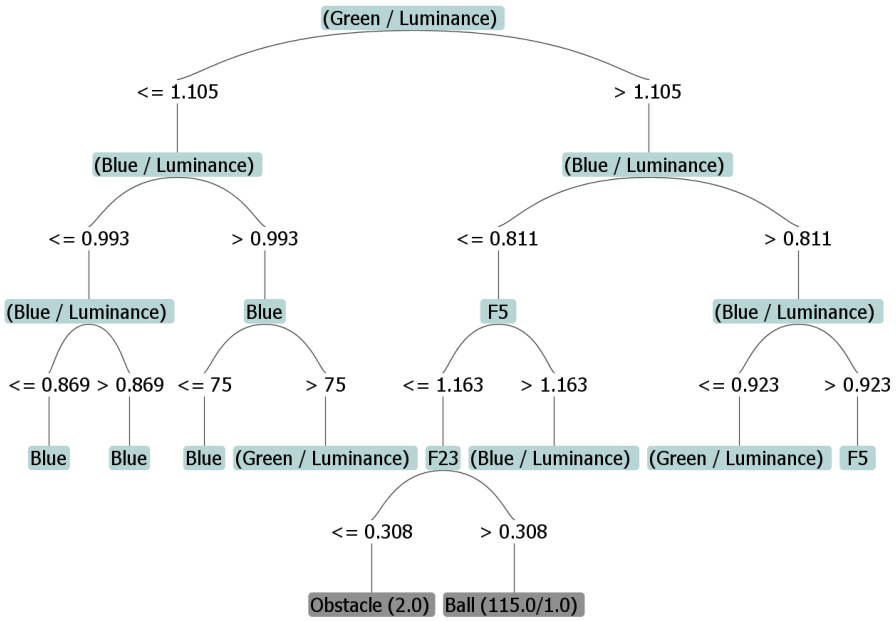


Figure 7.2: Top section of the C4.5 decision tree for the best-of-run (IBk) genotype. (Complete tree has 425 nodes).

Analysis of the frequency with which attributes appear in all of the final (cleaned) genotypes also provides some indication of how useful they are in classification. The 3 colours and luminosity are the most useful of the original attributes (they always appear in all 20 of the final genotypes regardless of parsimony pressure, and are the only attributes to do so). These are followed by 2 of the coefficients which appear

[4]While a C4.5 decision tree can serve to illustrate the features used by IBk to make classifications, and can highlight those features likely to be most informative in the classification, it should be remembered that the decision tree is *not* how IBk makes classifications.

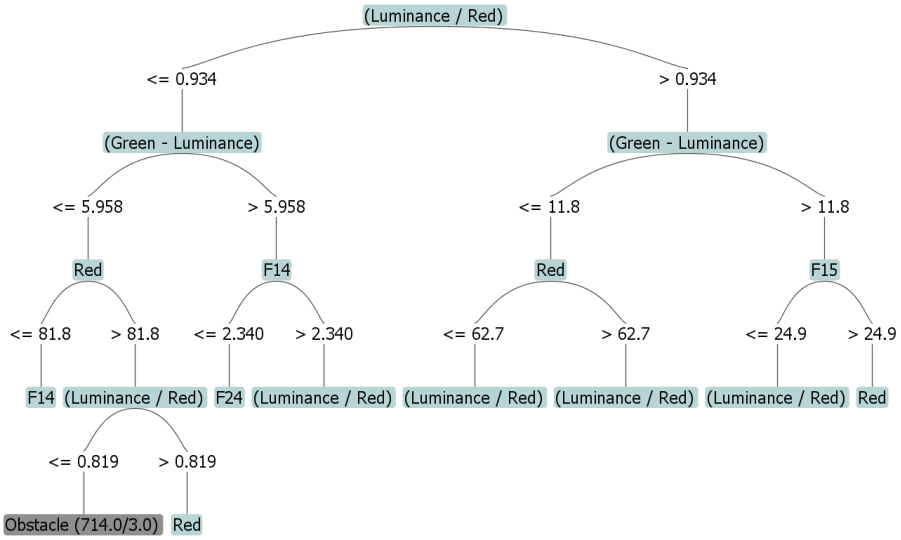


Figure 7.3: Top section of the C4.5 decision tree for the best C4.5 genotype. (Complete tree has 381 nodes).

in over 75% of the final genotypes. Similar analysis of the frequency of constructed features confirms the utility of the 3 features identified above (which appear in at least a quarter of the final genotypes), and also the ratios of Green and Blue to Red (features that did not appear in the best of run individual).

Figure 7.4 shows the top section of a C4.5 decision tree based on the best-of-run multi-objective genotype (which uses IBk rather than C4.5), illustrating those features/attributes likely to be most important for classification and showing the shortest paths to a classification (see appendix A.3 for a textual representation of the complete tree). As with the decision trees shown previously (figures 7.2 and 7.3) constructed features are used in the majority of the decision nodes of the top few levels of the tree, though the decision tree derived from the multi-objective code has a slightly more complicated feature at the first node, with more easily understood ratios appearing in the second and third levels.

7.6 Ensembles on the Bristol Vision dataset

Preliminary ensemble results are presented in table 7.5, using the GAP algorithm 4.3 (the final generation forming the 'whole ensemble') and

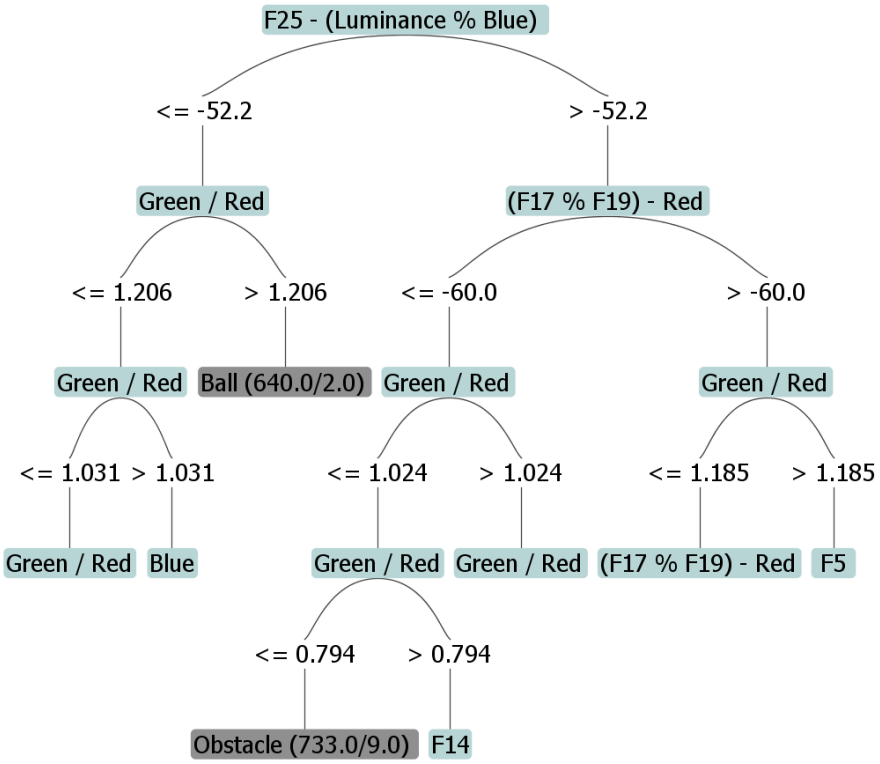


Figure 7.4: Top section of the C4.5 decision tree for the best-of-run (IBk) genotype using multi-objectives. (Complete tree has 431 nodes).

the ensemble selection algorithm 6.2 to choose the 'selected ensemble' from the final generation. The comparison classifier (IBk) achieved an average accuracy of 90.28%, which was improved on by the fittest individual genotypes with a t-test score of 2.63.

Table 7.5: Preliminary GAP-assisted ensemble performance on Bristol Vision dataset (Standard version)

	fittest genotype		**whole ensemble**		**selected ensemble**	
	average	**sd**	**average**	**sd**	**average**	**sd**
train accuracy	91.64	(1.05)	92.23	(0.83)	93.51	(0.62)
test accuracy	91.37	(1.80)	91.97	(1.69)	93.75	(1.47)
Q score			0.90	(0.02)	0.85	(0.03)
ensemble size			60.10	(11.63)	31.70	(10.10)

The Q-score of the whole ensemble (as measured on the test data [i.e. 10% of the 6676 instances]) is relatively high - as high as the highest Q-score for any of the UCI datasets (Wisconsin Breast Cancer) and with a small standard deviation - this is a consistently high Q-score. The average ensemble size is slightly larger than that of the UCI datasets when using the whole population, and much larger for the selected ensemble: the smallest of the selected ensembles consists of 9 genotypes (compared with a quarter of the UCI ensembles having just a single genotype) - the larger ensemble size may explain why the selected ensemble provides so much more of an improvement over the whole ensemble than is the case for the UCI datasets (see tables 6.1 and 6.5 for comparison values). The accuracy of the selected ensembles suggests that an ensemble can benefit from even a modest degree of diversity if the accuracy of the individual classifiers is high (this is a corollary to the demonstration by Esposito [61] that a successful ensemble may consist of an arbitrarily large number of highly diverse but relatively inaccurate classifiers - it may also consist of a smaller number of highly accurate classifiers [as long as at least some diversity is maintained]).

The selected ensemble, unlike the whole ensemble or the fittest individual, actually increases in accuracy on the test data (the benefit of having additional instances during training is greater than the level of overfitting, which is to say that the selected ensemble does not over fit the training data). However the performance of the selected ensemble is put into shade by that of an AdaBoost ensemble of C4.5 classifiers: with

a maximum number of iterations of 60 (the same as the average size of the whole ensemble) AdaBoost achieves a test accuracy of 95.62% (s. d. 0.72%)[5]. As with the UCI datasets, selecting an ensemble from the final population provides a real improvement over the performance of the single best genotype, but not enough to match that of AdaBoosted C4.5. So what of the performance on unseen test data?

Table 7.6: GAP-assisted ensemble performance on Bristol Vision test data

Classifier	**Train**	**Test**	**Q-score**
ensemble (fittest 20 genotypes)	96.42	96.87	0.886
AdaBoosted C4.5 (max iterations = 60)	95.64	96.78	
selected ensemble (19 genotypes)	96.45	96.84	0.884
most accurate single genotype	94.13	94.16	
most accurate whole ensemble (74 individuals)	93.15	93.52	0.941
most accurate selected ensemble (31 individuals)	94.44	95.04	0.887
most diverse whole ensemble (82 individuals)	92.72	93.76	0.886
most diverse selected ensemble (23 individuals)	93.71	94.04	0.842

The first row of table 7.6 shows the performance of an ensemble of the fittest individuals from each of twenty runs (after applying the post-processing techniques presented in section 5.3), the second the performance of AdaBoosted C4.5. The GAP ensemble offers an improvement over AdaBoost on both train and test data[6], though the improvement on the test data is admittedly marginal. A GAP ensemble can at least match the performance of an AdaBoosted C4.5 ensemble, and at the same cost in effort - both ensembles took about 28 minutes to complete both the cross-validation of the training set and train-and-test on the test data[7].

To show that this result is not just limited to the Bristol Vision dataset, an ensemble of the fittest individuals on UCI datasets (from the 20 runs on each of the 10 datasets used to produce the results shown

[5] an AdaBoost ensemble of the most accurate individual classifier, IBk, offers no real improvement over a single IBk classifier as, unlike C4.5, it is a stable classifier and so not suited to boosting.

[6] the train score is a result of 10-fold cross-validation of the training data, the test score is a single result on the test data, thus there is no value for standard deviation.

[7] this performance is for an embarked ensemble, i.e. after determining the individual genotypes that make up the ensemble. That process takes 7-10 days on the same hardware.

in table 6.1 - i.e. algorithm 4.3) has an average Q-score of 0.40 (greater diversity than an ensemble of the final population from a single run), and an average accuracy of 95.73% (this compares with the average accuracy of the individual classifiers of 84.39%). This is not a statistically meaningful result as the ensembles have effectively 'seen' the test data during training, and we have not kept back a subset of the data to test the ensemble with, but it does suggest that creating an ensemble of the fittest individuals from many runs is a more promising approach than constructing an ensemble from a single run, and is applicable to more datasets than just Bristol Vision.

The final four rows of table 7.6 show the performance of ensembles taken from a single population - the most accurate and most diverse of both whole and selected ensembles. The most diverse (whole) ensembles out-perform their more accurate (on the training data) counterparts - their greater diversity makes them more able to generalise to unseen data. Of the two selected ensembles however, it is the most accurate (on train data), rather than the most diverse, that gives the best performance on the test data. The selected ensemble of 19 genotypes in the third row is the result of applying the diversity based selection algorithm 6.2 to the fittest individuals from 20 runs, reducing the ensemble size by 1. This reduces the Q-score and improves the performance in the training data by a small fraction, but the loss of a single genotype reduces the performance on the test set by a small fraction. All of which reinforces the notion that the diversity of an ensemble *enhances* its accuracy but is not a substitute for it. The selection algorithm performs well when it is applied to an ensemble of individuals of low or mixed accuracy, but may not be appropriate when the ensemble consists of roughly comparable individuals with a high level of accuracy.

7.7 Extracting meaning from the Ensemble

Using an ensemble of the fittest genotypes offers comparable accuracy to AdaBoosted C4.5 (with maximum iterations of 60), though at the cost of much additional effort to determine the individual genotypes. There is something extra that the GAP algorithm can offer to make this additional effort worthwhile: the data-mining techniques introduced in chapter 5 'Improving the Human Readability of Derived Features'.

Chapter 5 concentrated on identifying the similarities between the final genotypes (i.e. those attributes and combinations of attributes that were repeatedly found to be useful) but of course the primary aspect of an ensemble is its diversity, so we must find a way of usefully describing the differences as well as the similarities. One approach is to revisit the twin notions of *sensitivity* and *specificity* defined in appendix C. These are normally used to measure binary classifications, particularly for medical datasets (i.e. how good a classifier is at picking up whether a person has a particular condition, and how well it avoids raising false alarms), however they can be adapted to multi-class datasets by measuring sensitivity and specificity for each class in turn (e.g. for an n–class dataset considering the sensitivity and specificity for class A and not-A, then for class B and not-B, etc. in a similar fashion to Wang et al.[194]). Grouping the members of an ensemble by class in this fashion provides an opportunity to say how individuals within groups are similar, and how they differ between groups. The output from such an analysis can be seen in appendix B and the sensitivity and specificity counts are summarised in table 7.7 - showing the average sensitivity/specificity of the individuals and the number of individuals most sensitive/specific to a class. All figures are based on 10-fold cross-validation of the training data.

Table 7.7: Breakdown of Classifications made by Bristol Vision Ensemble

Class	**# Instances**	**Sensitivity**		**Specificity**	
		average	**# Indiv.**	**average**	**# Indiv.**
Obstacle	1127	0.881	0	0.981	0
Boundary	1113	0.910	0	0.983	0
Floor	1115	0.901	0	0.973	0
Chair	1113	0.961	10	0.992	11
Ball	1098	0.930	0	0.989	1
Box	1110	0.959	10	0.991	8

Immediately we know that the Obstacle, Boundary and Floor classes are more difficult to classify than the other 3 classes, and Ball only slightly easier than they. Most individuals are best at classifying either Chairs or Boxes. Most classifiers that are most sensitive to a particular class are also most specific to the same class but as specificity is very high this is a close run thing and two of the classifiers most sensitive to Boxes are most specific to Chairs or Balls instead. The 20 individuals

in the ensemble are made up of 14 IBk genotypes and 6 C4.5 genotypes. The 6 C4.5 genotypes are all most specific to Chairs, and all but one are also most sensitive to Chairs.

When comparing the sensitivity/specificity of groups of classifiers the differences are much more distinct when comparing a group to other groups rather than to the average. There are two main groups to compare - those classifiers better at identifying (most sensitive to) Chairs (those most *specific* to Chairs are almost exactly the same group) and those better at identifying (most sensitive to) Boxes ('Chair group' and 'Box group' respectively in the discussion that follows). The Chair group is more sensitive to Obstacles than is the Box group while the Box group is more sensitive to the other four classes, particularly to Boundaries. Specificity is much closer between the two groups across all classes - there is little to distinguish them.

Both groups make consistent use of the colours Red and Green - these attributes appear in all the genotypes (mostly as attributes in their own right, though occasionally only as part of a larger function) but they are made greatest use of by the Chair group in which they appear more often than any other attributes. Luminance tends to appear more as part of a function than as an attribute in its own right (in both groups), and appears in combination with Green in the majority of the Chair group: the comparison Green-Luminance (or its inverse) or the ratio Green/Luminance (or the inverse) appear as a function in over half the classifiers. This relationship appears with much greater frequency in the Chair group than in the Box group. The Box group, on the other hand, makes greater use of the Blue attribute than does the Chair group, both as an attribute and as part of a function.

The single fittest individual, a member of the Box group using IBk, makes extensive use of the feature Red-Green (it appears 3 times in the one individual even after post-processing, and a fourth time as part of a larger function) and the only member of the Box group using C4.5 uses its inverse, Green-Red (this is the individual that is most specific to Chairs but most sensitive to Boxes - it seems to straddle both groups and also uses the function Green-Luminance). None of the Chair group make use of the relationship Green-Red.

Figures 7.5 and 7.6 show the top sections of C4.5 decision trees for the best individuals from the Box and Chair groups, respectively. It

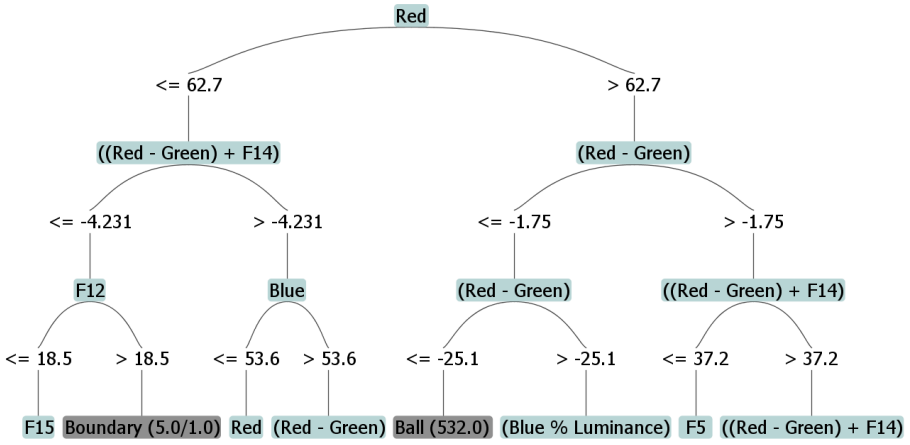


Figure 7.5: Top section of the C4.5 decision tree for the best genotype (IBk) from the Box group. (Complete tree has 463 nodes).

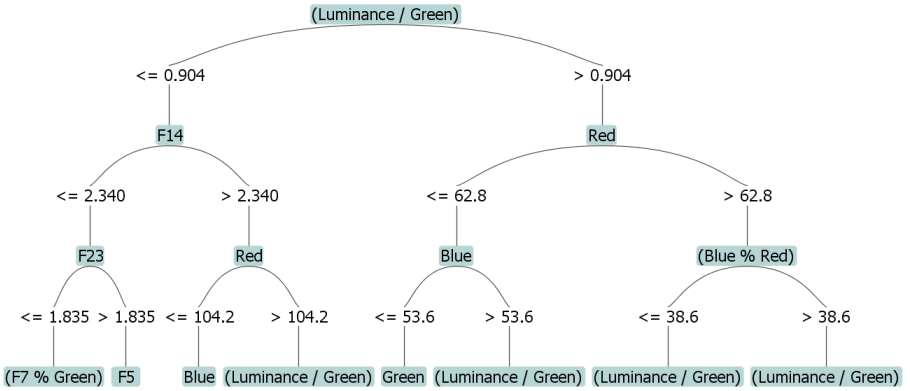


Figure 7.6: Top section of the C4.5 decision tree for the best genotype (IBk) from the Chair group. (Complete tree has 465 nodes).

should be borne in mind that because these are decision trees created with features generated for IBk they are illustrative only and do not reflect the actual decision making process of the genotypes.

7.8 Conclusion

Evaluation of the algorithm against the vision dataset has allowed us to confirm conclusions drawn against the UCI datasets in previous chapters: that highlighting relationships between attributes is best performed by post-processing rather than any measures to include parsimony within the algorithm itself; that of 20 runs on a dataset there will be one or more individuals (or ensembles) with accuracy approxi-

mately a single standard deviation greater than the average, and that this increase in accuracy is maintained on a separate test set; and that an ensemble created from the fittest individual from each run is indeed more accurate than an ensemble created from the final population of any single run. We have also been able to demonstrate a way to analyse the similarities and differences among individuals in an ensemble by grouping them according to sensitivity and specificity to each class.

Chapter 8

Conclusion

The algorithm presented in this thesis uses a combination of genetic programming and genetic algorithms to pre-process data to improve the performance of classifiers for data mining (the GAP algorithm). It does this by performing a combination of feature construction and feature selection to replace the original attributes of the dataset with a novel set of features. The initial hypothesis, that the algorithm could be used to sufficiently improve the accuracy of a simple classifier to make it competitive with a more complex classifier such as SVM, while making explicit the feature construction implicitly performed by the more complex classifier, has been upheld by the results in section 4.6 (with the GAP algorithm selecting the most appropriate combination of features and classifier type). Sections 3.3.3, 5.5, 7.5 and 7.7 discuss how it is possible to extract useful information from the results.

It has emerged over the course of experiments that the most accurate individual of the final solutions from 20 runs is almost always a standard deviation more accurate than the average solution. Running the algorithm many times and selecting the most accurate individual (or an ensemble of final solutions) is thus a promising avenue for finding improvements in the accuracy of a classifier (and related data mining benefits in being able to say how), though it may only be appropriate for larger datasets (both to provide a large enough training set to reduce the risk of overfitting, and allow a set of validation data to be held back to confirm the additional improvement).

8.1 Summary

In chapter 3 we introduced the GAP algorithm, including a novel formula for determining the size of a GP tree in the initial population (P_{leaf}, described in section 3.3.1), that seems particularly suited to the construction of multiple features within a single genotype. In section 3.3.3 'Utility of Constructed Features' we demonstrated that the GAP algorithm can produce genuinely useful new features that can reduce the size of a C4.5 decision tree as well as improve the classification performance on a number of datasets from the UCI ML repository. We have shown that previous work combining feature construction and selection ran the risk of removing potentially useful information by performing feature selection first; removing apparently redundant attributes that later prove to be useful in combination with another attribute (see section 3.3.3 'The order of the Create and Select stages'). We have also shown that randomising the dataset part-way through the algorithm (either between stages in a two-stage process, or at a suitable stopping point in a combined construction/selection stage) can help offset the risks of over-fitting the dataset (see section 3.3.3 'Effect of randomising the dataset between stages'), which seems to be one of the major risks when constructing new features.

Chapter 4 demonstrated that construction and selection can be combined within a single stage without negatively affecting classification accuracy. We also showed that the GAP algorithm can be successfully applied to classifiers other than C4.5, and that it can be extended to select the most appropriate classifier (in combination with the feature set).

Chapter 5 showed that, while a GP algorithm that constructs new features does not suffer from bloat in the same fashion as does a GP algorithm that performs classification, various measures can be taken to reduce the size of the final solution to improve human readability and highlight those features useful in improving classification. Of the three methods considered (incorporating a parsimony measure into the fitness function; treating fitness and size as separate objectives to be simultaneously optimised; post-processing the final solution), post-processing was shown to be able to reduce the size of the solution while best retaining the ability to discover useful relationships between at-

tributes. Applying the algorithm to the larger Bristol Vision dataset (and retaining a number of instances from the initial runs) allowed us to show that, while the performance of the GAP algorithm can vary between runs, selecting the single best solution from a number of runs can provide a marked improvement over the average result.

Chapter 6 showed that the GAP algorithm naturally produces heterogenous ensembles, and that while measures can be taken to increase the diversity of the population so as to produce an ensemble from a single run, the best ensembles are produced by combining the fittest individuals from multiple runs. We also showed that an ensemble of the fittest individuals is comparable in performance to an Ada-boosted ensemble, while offering the opportunity to explain the nature of the diversity of the ensemble. This is done by grouping the individuals according to their sensitivity and specificity to each of the classes and analysing the similarity of features within groups and the differences in features between groups.

8.2 Future Directions

8.2.1 Hardware acceleration

A key limitation of the GAP algorithm is the amount of computation required. This limitation can be overcome to some extent by performing more calculations in parallel, taking advantage of the parallelisation made possible by General Purpose computing on Graphics Processing Units (GPGPU). It would be programmatically simpler to take advantage of multiple CPUs through Grid computing[1] using Apache Hadoop and/or Amazon EC2[2] (the GAP algorithm is already able to take advantage of multiple cores on a single computer and could be extended to use Hadoop without porting from Java); however GPGPU allows parallelisation *on a single PC*, which would make use of the GAP algorithm much more practical.

Implementing the GAP algorithm using GPGPU would require not

[1] Grid computing is a form of distributed computing where the workload is distributed across a network of computers (that network being anything from a fixed collection of PCs on a local area network to a variable number of computers cooperating over the internet).

[2] The Apache Hadoop project provides open source software for distributed computing, using the Java language. See `http://hadoop.apache.org/`.
Amazon EC2 (Elastic Compute Cloud) is a web service that provides computing capacity over the internet, and is compatible with Hadoop. EC2 provides the ability to rent a computing cluster by the hour, rather than having to operate one yourself. See `http://aws.amazon.com/ec2/`.

only porting from Java to a C-like language (perhaps OpenCL[3]) but also recasting the problem in the GPU programming model (In essence segmenting the application into a number of independent sections [kernels] that can be run in parallel and splitting the data into streams that are operated on by the kernels). GPU programming is particularly applicable to processing two-dimensional arrays, which is the essential structure of a data mining dataset, so there is reason to think that this approach would work. For instance the k-nearest neighbour classification technique has already been implemented using GPGPU by Garcia, Debreuve and Barlaud [73].

> "... Effective GPGPU programming is not simply a matter of learning a new language. Instead, the computation must be recast into graphics terms by a programmer familiar with the design, limitations, and evolution of the underlying hardware... But despite the programming challenges, the potential benefits - a leap forward in computing capability, and a growth curve much faster than traditional CPUs - are too large to ignore."
> Owens et. al, "A Survey of General-Purpose Computation on Graphics Hardware" [138].

The faster operation enabled by parallelisation would allow expansion in two directions: operating on larger datasets (without the self-imposed limit of 2,000 instances per fold); and using more expensive classification algorithms such as SVM and Neural Networks.

8.2.2 Preselection of operators and classifiers

There are many operators that could be employed by the Gap algorithm in addition to the existing set {+,-,*,/,%,log} (such as exponent, x^y, square or cube root, conversion of degrees to radians, sine, cosine etc.), however each addition expands the search space and runs the risk of making it more difficult to find a solution. This risk might be overcome by performing preselection: providing a large list of operators to a small population that is bred for a fixed number of generations (say between

[3]OpenCL (Open Computing Language) is a framework (based on C) for the parallel programming of multiple processors. Unlike APIs such as Nvidia's CUDA, OpenCL is intended as a cross-platform environment, and will shortly be supported by both Nvidia and AMD. See `http://www.khronos.org/opencl/`.

2 and 10 generations), then selecting a subset of the operators by examining which are present in the active trees of the fittest individuals (say the top 10% - 20% of the final generation, after post-processing to remove redundant sub-trees). As some operators might be particularly suited for use by one classifier and not another, it may be necessary to include the operators used by the fittest individual for each classifier type, as well as those used by the fittest individuals overall. The same approach could be taken with the selection of classifiers, though here it would be more appropriate to rank the classifiers by the average fitness of the genotypes using them and take the top n classifiers; as it is entirely possible that the top 20% of the population all use the same classifier type. (It may be that this is a better approach to take with operators, too).

8.2.3 Evolving classifier settings

The classifiers used by genotypes in the GAP algorithm all use default settings. This is partly for simplicity, partly because the GAP algorithm is intended for use in situations where there is no expert knowledge of the subject matter. It would be possible to extend the genotype to include settings for the classifier, in such a way that the number and type of these settings is variable (so that the same genotype format can be used for classifiers with different settings). This would require adding mutation and crossover operators for integers and floating point numbers. It might be necessary to prevent changes to the classifier settings until the population has broadly settled on a set of features - perhaps evolving the settings in a separate stage, or only after the population has met the termination criteria the first time.

8.2.4 Extraction of common modules

It might be possible to improve performance by identifying common modules (sub-trees) that have been independently evolved by many individuals/trees, and making these modules available to the rest of the population. (This would be akin to using ADFs within a GP population, but without using parameters). A second set of attributes ('modules') would be introduced alongside the initial attributes of the dataset. This set would be initially empty but would be added to every

few generations by identifying sub-trees that occur in multiple individuals (modules could also be removed from the set if no longer used by the population). These modules would then be introduced to the rest of the population by making them available during mutation (to be treated as if they were original attributes, and not subject to mutation themselves) - perhaps increasing their distribution by replacing the least fit individual of each generation with a randomly created one. As well as reducing the time required for individuals to discover the (presumably) useful sub-tree; this could reduce computation time by calculating values for the module once and caching rather than having to recalculate for each individual.

8.2.5 Variable length genotypes

One performance-related limitation that has not been discussed so far is the number of attributes in a dataset (it has been assumed that the classifiers are capable of operating on all the attributes in the original dataset, which is the maximum number of attributes present in a genotype). However large datasets may possess not only many thousands of instances, but thousands of attributes too. It would be difficult to classify instances in such datasets without performing some sort of feature selection first, but as was discussed in section 3.3.3 'Utility of Constructed Features', performing feature selection before feature construction runs the risk of removing potentially useful features. The GAP algorithm can be adapted to overcome this limitation by using variable length genotypes: replacing a fixed number of trees and associated activity flags by a variable number of trees (without the activity flag). Genotypes in the initial population would be created with a random, but limited, number of trees (perhaps using a distribution weighted so that most of the population possessed a small but non-trivial number of trees while allowing some individuals to contain a relatively large number of trees). The size of individuals would naturally grow and shrink during crossover by randomly assigning trees from each parent to one offspring or the other, and perhaps a new mutation operator could be introduced to add or remove trees. The size of individuals might best be controlled by combining this with the parsimony methods discussed in sections 5.2.1 and 5.4.

Appendix A

C4.5 Decision Trees

Textual representations of C4.5 decision trees place one decision node on each line, indicating hierarchy and the depth of the node by vertical bars. Leaf nodes appear on the same line as their parent decision node, separated by a colon.

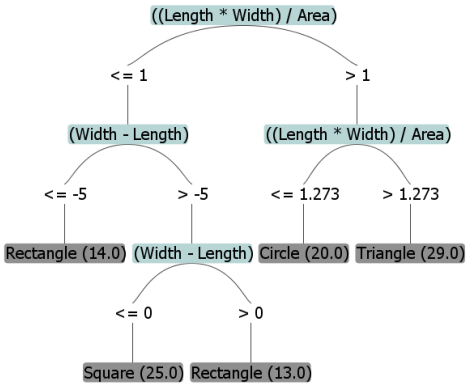


Figure A.1: Sample decision tree

The sample decision tree shown above would be represented by the following text: (The numbers after a leaf node indicate the number of instances correctly/incorrectly classified at that node)

```
    .
((Length * Width) / Area) <= 1
| (Width - Length) <= -5:  Rectangle (14.0)
| (Width - Length) > -5
| | (Width - Length) <= 0:  Square (25.0)
| | (Width - Length) > 0:  Rectangle (13.0)
((Length * Width) / Area) > 1
| ((Length * Width) / Area) <= 1.27324:  Circle (20.0)
| ((Length * Width) / Area) > 1.27324:  Triangle (29.0)
```

A.1 C4.5 decision tree for the best-of-run (IBk) genotype

Representation of the full decision tree, the top section of which was illustrated in figure 7.2. Please note that the genotype uses IBk to make decisions and that the C4.5 decision tree is provided for illustrative purposes only.

```
     Features
------------------

     [F23] [F8] [F15] [Blue] [F9] [F6] [(F28 - (F6 - (F16 + F28)))] [F13] [((log:(log:F11)) + F22)] [F5] [(Blue / Luminance)] [(Green / Luminance)]
[(Blue / Luminance)] [F19] [F5] [(log:Red)] [(log:Red)] [F14]

     .
C4.5 pruned tree
------------------

     Number of Leaves :  213
     Size of the tree :  425
     .
(Green / Luminance) <= 1.105606
| (Blue / Luminance) <= 0.993243
| | (Blue / Luminance) <= 0.86959
| | | Blue <= 125.125
| | | | (Green / Luminance) <= 0.988764
| | | | | (log:Red) <= 4.405194
| | | | | | F13 <= 5:  Obstacle (21.0/1.0)
| | | | | | F13 > 5:  Boundary (6.0/1.0)
| | | | | (log:Red) > 4.405194
| | | | | | Blue <= 110.125:  Obstacle (686.0)
| | | | | | Blue > 110.125
| | | | | | | F15 <= 17.0135:  Obstacle (15.0)
| | | | | | | F15 > 17.0135:  Box (2.0)
| | | | (Green / Luminance) > 0.988764
| | | | | F13 <= 9.4167
| | | | | | (log:Red) <= 4.267948
| | | | | | | F14 <= 7.3005
| | | | | | | | F6 <= 7.4818:  Obstacle (3.0)
| | | | | | | | F6 > 7.4818:  Box (4.0)
| | | | | | | F14 > 7.3005
| | | | | | | | (F28 - (F6 - (F16 + F28))) <= 34.8361:  Boundary (7.0)
| | | | | | | | (F28 - (F6 - (F16 + F28))) > 34.8361:  Obstacle (3.0/1.0)
| | | | | | (log:Red) > 4.267948
| | | | | | | F5 <= 1.1632
| | | | | | | | F8 <= 3.9896:  Obstacle (2.0/1.0)
| | | | | | | | F8 > 3.9896:  Ball (5.0)
| | | | | | | F5 > 1.1632
| | | | | | | | F9 <= 5.8542
| | | | | | | | | F6 <= 12.0661
| | | | | | | | | | (Blue / Luminance) <= 0.780457:  Obstacle (8.0)
| | | | | | | | | | (Blue / Luminance) > 0.780457
| | | | | | | | | | | (F28 - (F6 - (F16 + F28))) <= 37.0804
| | | | | | | | | | | | (Green / Luminance) <= 0.990744:  Obstacle (2.0)
| | | | | | | | | | | | (Green / Luminance) > 0.990744:  Floor (11.0)
| | | | | | | | | | | (F28 - (F6 - (F16 + F28))) > 37.0804:  Obstacle (3.0)
| | | | | | | | | F6 > 12.0661
| | | | | | | | | | F23 <= 1.6962:  Box (10.0/1.0)
| | | | | | | | | | F23 > 1.6962
| | | | | | | | | | | (Blue / Luminance) <= 0.792453:  Obstacle (5.0)
| | | | | | | | | | | (Blue / Luminance) > 0.792453:  Box (2.0)
| | | | | | | | F9 > 5.8542
| | | | | | | | | F9 <= 42.5938
| | | | | | | | | | F5 <= 12.2298
| | | | | | | | | | | F5 <= 10.883:  Obstacle (33.0/3.0)
| | | | | | | | | | | F5 > 10.883
| | | | | | | | | | | | (Blue / Luminance) <= 0.805242
| | | | | | | | | | | | | F6 <= 14.3828:  Box (5.0/1.0)
| | | | | | | | | | | | | F6 > 14.3828:  Obstacle (4.0)
| | | | | | | | | | | | (Blue / Luminance) > 0.805242
| | | | | | | | | | | | | F6 <= 24.5767:  Floor (4.0/1.0)
| | | | | | | | | | | | | F6 > 24.5767:  Obstacle (2.0/1.0)
| | | | | | | | | | F5 > 12.2298
| | | | | | | | | | | (log:Red) <= 4.718499
| | | | | | | | | | | | F15 <= 6.1842
| | | | | | | | | | | | | F13 <= 3.5:  Obstacle (5.0/1.0)
| | | | | | | | | | | | | F13 > 3.5:  Ball (5.0)
| | | | | | | | | | | | F15 > 6.1842:  Obstacle (6.0)
| | | | | | | | | | | (log:Red) > 4.718499:  Obstacle (25.0/1.0)
| | | | | | | | | F9 > 42.5938
| | | | | | | | | | F8 <= 34.3737:  Box (4.0)
| | | | | | | | | | F8 > 34.3737:  Obstacle (2.0)
| | | | | F13 > 9.4167
| | | | | | F6 <= 15.9969
| | | | | | | F15 <= 6.8385:  Ball (5.0)
| | | | | | | F15 > 6.8385:  Boundary (4.0/1.0)
| | | | | | F6 > 15.9969:  Box (5.0)
| | | Blue > 125.125:  Box (20.0)
| | (Blue / Luminance) > 0.86959
| | | Blue <= 54.625
| | | | F15 <= 0.4785
| | | | | F5 <= 14.3828
| | | | | | F23 <= 1.1589:  Floor (10.0/2.0)
| | | | | | F23 > 1.1589:  Boundary (3.0)
| | | | | F5 > 14.3828:  Obstacle (3.0)
```

```
| | | | F15 > 0.4785
| | | | | F23 <= 0.3408
| | | | | | Blue <= 48.25:  Boundary (6.0)
| | | | | | Blue > 48.25:  Floor (8.0/1.0)
| | | | | F23 > 0.3408
| | | | | | (Blue / Luminance) <= 0.877462
| | | | | | | F8 <= 5.5794:  Boundary (3.0/1.0)
| | | | | | | F8 > 5.5794:  Floor (3.0/1.0)
| | | | | | (Blue / Luminance) > 0.877462:  Boundary (172.0/15.0)
| | | Blue > 54.625
| | | | (log:Red) <= 4.984462
| | | | F13 <= 4.6667
| | | | | F19 <= 6.7708
| | | | | | F15 <= 15.9668
| | | | | | | F14 <= 13.9045
| | | | | | | | Blue <= 59.375
| | | | | | | | | F9 <= 4.7188:  Floor (9.0)
| | | | | | | | | F9 > 4.7188
| | | | | | | | | | F6 <= 9.8437:  Boundary (5.0)
| | | | | | | | | | F6 > 9.8437:  Floor (4.0/1.0)
| | | | | | | | Blue > 59.375
| | | | | | | | | (Green / Luminance) <= 1.031447
| | | | | | | | | | (F28 - (F6 - (F16 + F28))) <= 27.6748
| | | | | | | | | | | ((log:(log:F11)) + F22) <= 3.508264:  Floor (17.0)
| | | | | | | | | | | ((log:(log:F11)) + F22) > 3.508264
| | | | | | | | | | | | F14 <= 3.6174:  Floor (6.0)
| | | | | | | | | | | | F14 > 3.6174:  Obstacle (6.0/1.0)
| | | | | | | | | | (F28 - (F6 - (F16 + F28))) > 27.6748
| | | | | | | | | | | F8 <= 10.9125:  Boundary (3.0/1.0)
| | | | | | | | | | | F8 > 10.9125:  Obstacle (2.0)
| | | | | | | | | (Green / Luminance) > 1.031447
| | | | | | | | | | F9 <= 19.3802
| | | | | | | | | | | (Blue / Luminance) <= 0.97999:  Floor (681.0/14.0)
| | | | | | | | | | | (Blue / Luminance) > 0.97999
| | | | | | | | | | | | (log:Red) <= 4.246708:  Ball (4.0)
| | | | | | | | | | | | (log:Red) > 4.246708:  Floor (16.0)
| | | | | | | | | | F9 > 19.3802
| | | | | | | | | | | (log:Red) <= 4.771743
| | | | | | | | | | | | F6 <= 32.3097:  Floor (44.0/1.0)
| | | | | | | | | | | | F6 > 32.3097:  Obstacle (2.0)
| | | | | | | | | | | (log:Red) > 4.771743
| | | | | | | | | | | | F23 <= 3.5459:  Obstacle (6.0)
| | | | | | | | | | | | F23 > 3.5459:  Boundary (2.0/1.0)
| | | | | | | | F14 > 13.9045
| | | | | | | | | F15 <= 2.9625:  Boundary (6.0)
| | | | | | | | | F15 > 2.9625:  Floor (11.0)
| | | | | | | F15 > 15.9668
| | | | | | | | F5 <= 11.242:  Floor (46.0/2.0)
| | | | | | | | F5 > 11.242
| | | | | | | | | F6 <= 6.819
| | | | | | | | | | F15 <= 16.6941:  Boundary (4.0)
| | | | | | | | | | F15 > 16.6941
| | | | | | | | | | | F15 <= 57.7687
| | | | | | | | | | | | (Blue / Luminance) <= 0.957198:  Floor (20.0)
| | | | | | | | | | | | (Blue / Luminance) > 0.957198
| | | | | | | | | | | | | F23 <= 3.6634:  Boundary (2.0)
| | | | | | | | | | | | | F23 > 3.6634:  Floor (3.0)
| | | | | | | | | | | | F15 > 57.7687:  Boundary (2.0)
| | | | | | | | | F6 > 6.819
| | | | | | | | | | ((log:(log:F11)) + F22) <= 3.320147:  Boundary (9.0)
| | | | | | | | | | ((log:(log:F11)) + F22) > 3.320147
| | | | | | | | | | | (Green / Luminance) <= 1.058629:  Boundary (2.0/1.0)
| | | | | | | | | | | (Green / Luminance) > 1.058629:  Floor (3.0)
| | | | | | F19 > 6.7708
| | | | | | | (Green / Luminance) <= 1.014012
| | | | | | | | F9 <= 7.1771
| | | | | | | | | Blue <= 74.5:  Obstacle (4.0/1.0)
| | | | | | | | | Blue > 74.5:  Floor (3.0)
| | | | | | | | F9 > 7.1771:  Obstacle (6.0)
| | | | | | | (Green / Luminance) > 1.014012
| | | | | | | | F19 <= 14.6458
| | | | | | | | | Blue <= 113.25
| | | | | | | | | | F13 <= 3.5833
| | | | | | | | | | | F23 <= 6.7023
| | | | | | | | | | | | (Blue / Luminance) <= 0.915672
| | | | | | | | | | | | | F13 <= 2.75
| | | | | | | | | | | | | | Blue <= 69.125:  Obstacle (3.0/1.0)
| | | | | | | | | | | | | | Blue > 69.125:  Floor (7.0)
| | | | | | | | | | | | | F13 > 2.75:  Obstacle (5.0)
| | | | | | | | | | | | (Blue / Luminance) > 0.915672
| | | | | | | | | | | | | (log:Red) <= 4.19908
| | | | | | | | | | | | | | F13 <= 2.5:  Floor (6.0/1.0)
| | | | | | | | | | | | | | F13 > 2.5
| | | | | | | | | | | | | | | F15 <= 4.9681:  Ball (2.0)
| | | | | | | | | | | | | | | F15 > 4.9681:  Boundary (2.0)
| | | | | | | | | | | | | | (log:Red) > 4.19908:  Floor (45.0/2.0)
| | | | | | | | | | | F23 > 6.7023
| | | | | | | | | | | | ((log:(log:F11)) + F22) <= 6.291594
| | | | | | | | | | | | | F13 <= 2:  Floor (3.0)
| | | | | | | | | | | | | F13 > 2:  Boundary (3.0/1.0)
| | | | | | | | | | | | ((log:(log:F11)) + F22) > 6.291594:  Boundary (3.0)
| | | | | | | | | | F13 > 3.5833
| | | | | | | | | | | (Blue / Luminance) <= 0.934185
| | | | | | | | | | | | (F28 - (F6 - (F16 + F28))) <= 35.4251:  Boundary (4.0/1.0)
| | | | | | | | | | | | (F28 - (F6 - (F16 + F28))) > 35.4251:  Floor (3.0)
| | | | | | | | | | | (Blue / Luminance) > 0.934185:  Chair (2.0)
| | | | | | | | | Blue > 113.25
| | | | | | | | | | F14 <= 5.9689:  Obstacle (6.0)
| | | | | | | | | | F14 > 5.9689
| | | | | | | | | | | F9 <= 13.1302:  Floor (6.0/1.0)
| | | | | | | | | | | F9 > 13.1302:  Chair (2.0)
| | | | | | | | F19 > 14.6458
| | | | | | | | | F13 <= 3.9167
| | | | | | | | | | Blue <= 113.75
| | | | | | | | | | | F23 <= 3.2513
| | | | | | | | | | | | F15 <= 0.645:  Ball (3.0/1.0)
| | | | | | | | | | | | F15 > 0.645
```

```
| | | | | | | | | | | | | | | F15 <= 11.2077:  Floor (23.0)
| | | | | | | | | | | | | | | F15 > 11.2077
| | | | | | | | | | | | | | | | (Green / Luminance) <= 1.057367:  Boundary (8.0/1.0)
| | | | | | | | | | | | | | | | (Green / Luminance) > 1.057367:  Floor (7.0/1.0)
| | | | | | | | | | | | | F23 > 3.2513
| | | | | | | | | | | | | | F9 <= 19.5052
| | | | | | | | | | | | | | | F5 <= 7.1353
| | | | | | | | | | | | | | | | Blue <= 68.25:  Boundary (2.0)
| | | | | | | | | | | | | | | | Blue > 68.25:  Floor (4.0)
| | | | | | | | | | | | | | F5 > 7.1353
| | | | | | | | | | | | | | | F5 <= 24.415:  Boundary (15.0/1.0)
| | | | | | | | | | | | | | | F5 > 24.415:  Floor (3.0/1.0)
| | | | | | | | | | | | | | F9 > 19.5052
| | | | | | | | | | | | | | | F14 <= 7.754:  Boundary (4.0)
| | | | | | | | | | | | | | | F14 > 7.754:  Ball (2.0)
| | | | | | | | | | | Blue > 113.75
| | | | | | | | | | | F23 <= 1.1589:  Obstacle (2.0)
| | | | | | | | | | | F23 > 1.1589:  Boundary (9.0)
| | | | | | | | | F13 > 3.9167
| | | | | | | | | | F23 <= 3.6237:  Boundary (3.0/1.0)
| | | | | | | | | | F23 > 3.6237:  Obstacle (4.0/1.0)
| | | | | F13 > 4.6667
| | | | | | F9 <= 6.6771
| | | | | | | F13 <= 9.25
| | | | | | | | F14 <= 3.9115:  Floor (16.0/2.0)
| | | | | | | | F14 > 3.9115
| | | | | | | | | ((log:(log:F11)) + F22) <= 5.446224
| | | | | | | | | F8 <= 2.2513:  Obstacle (2.0)
| | | | | | | | | F8 > 2.2513:  Floor (4.0/1.0)
| | | | | | | | | ((log:(log:F11)) + F22) > 5.446224:  Obstacle (7.0)
| | | | | | | F13 > 9.25:  Boundary (3.0/2.0)
| | | | | | F9 > 6.6771
| | | | | | | F5 <= 7.7382
| | | | | | | | F6 <= 15.5126
| | | | | | | | | F15 <= 6.7023:  Obstacle (3.0)
| | | | | | | | | F15 > 6.7023:  Floor (2.0)
| | | | | | | | F6 > 15.5126:  Chair (3.0)
| | | | | | | F5 > 7.7382
| | | | | | | | ((log:(log:F11)) + F22) <= 14.39254
| | | | | | | | (log:Red) <= 4.553877
| | | | | | | | | F15 <= 0.7595:  Ball (2.0)
| | | | | | | | | F15 > 0.7595
| | | | | | | | | | F14 <= 4.2705
| | | | | | | | | | | F5 <= 12.7679:  Boundary (3.0)
| | | | | | | | | | | F5 > 12.7679:  Obstacle (5.0/1.0)
| | | | | | | | | | F14 > 4.2705:  Boundary (12.0)
| | | | | | | | (log:Red) > 4.553877
| | | | | | | | | F19 <= 43.7917
| | | | | | | | | | (Green / Luminance) <= 1.083045:  Obstacle (36.0/4.0)
| | | | | | | | | | (Green / Luminance) > 1.083045
| | | | | | | | | | F15 <= 7.1248:  Ball (4.0)
| | | | | | | | | | F15 > 7.1248:  Obstacle (4.0/1.0)
| | | | | | | | | F19 > 43.7917:  Boundary (3.0)
| | | | | | | | ((log:(log:F11)) + F22) > 14.39254:  Boundary (8.0/1.0)
| | | | (log:Red) > 4.984462
| | | | | (log:Red) <= 5.420535
| | | | | | F15 <= 12.8589
| | | | | | | F5 <= 22.4978
| | | | | | | | F8 <= 52.474
| | | | | | | | | F9 <= 11.125
| | | | | | | | | | (F28 - (F6 - (F16 + F28))) <= 52.4631:  Floor (3.0/1.0)
| | | | | | | | | | (F28 - (F6 - (F16 + F28))) > 52.4631:  Obstacle (4.0/1.0)
| | | | | | | | | F9 > 11.125:  Obstacle (19.0)
| | | | | | | | F8 > 52.474:  Boundary (4.0/1.0)
| | | | | | | F5 > 22.4978
| | | | | | | | Blue <= 171.5:  Floor (6.0)
| | | | | | | | Blue > 171.5:  Boundary (2.0/1.0)
| | | | | | F15 > 12.8589
| | | | | | | (Green / Luminance) <= 1.032657
| | | | | | | | F23 <= 4.2765:  Box (4.0)
| | | | | | | | F23 > 4.2765:  Boundary (2.0)
| | | | | | | (Green / Luminance) > 1.032657
| | | | | | | | (F28 - (F6 - (F16 + F28))) <= 74.1918
| | | | | | | | | F13 <= 2.6667:  Boundary (46.0)
| | | | | | | | | F13 > 2.6667
| | | | | | | | | | F6 <= 10.883:  Boundary (10.0/2.0)
| | | | | | | | | | F6 > 10.883:  Obstacle (3.0)
| | | | | | | | (F28 - (F6 - (F16 + F28))) > 74.1918
| | | | | | | | | F23 <= 4.5365:  Chair (3.0)
| | | | | | | | | F23 > 4.5365:  Obstacle (2.0/1.0)
| | | | | (log:Red) > 5.420535
| | | | | | F23 <= 2.7571:  Box (31.0)
| | | | | | F23 > 2.7571:  Boundary (4.0/1.0)
| (Blue / Luminance) > 0.993243
| | Blue <= 75
| | | Blue <= 53.625
| | | | F19 <= 8.6458
| | | | | (log:Red) <= 3.688879:  Boundary (453.0)
| | | | | (log:Red) > 3.688879
| | | | | | F14 <= 3.6174
| | | | | | | F14 <= 1.0827:  Boundary (21.0)
| | | | | | | F14 > 1.0827
| | | | | | | | F5 <= 7.5902:  Ball (2.0)
| | | | | | | | F5 > 7.5902
| | | | | | | | | F23 <= 5.6874:  Floor (9.0/1.0)
| | | | | | | | | F23 > 5.6874:  Boundary (3.0)
| | | | | | F14 > 3.6174:  Boundary (144.0/3.0)
| | | | F19 > 8.6458
| | | | | (Green / Luminance) <= 1.030952:  Boundary (40.0)
| | | | | (Green / Luminance) > 1.030952
| | | | | | F23 <= 6.1362:  Ball (8.0)
| | | | | | F23 > 6.1362:  Boundary (2.0)
| | | Blue > 53.625
| | | | (Green / Luminance) <= 1.051318
| | | | F14 <= 5.2689
| | | | | F19 <= 4.1458:  Floor (17.0)
| | | | | F19 > 4.1458
```

```
| | | | | | | Blue <= 61.25:  Boundary (3.0/1.0)
| | | | | | | Blue > 61.25:  Chair (3.0)
| | | | | F14 > 5.2689
| | | | | | F15 <= 0.4305:  Ball (3.0/1.0)
| | | | | | F15 > 0.4305
| | | | | | | F5 <= 16.3968
| | | | | | | | (Blue / Luminance) <= 1.08
| | | | | | | | | F23 <= 4.5365
| | | | | | | | | | F5 <= 4.84:  Floor (3.0/1.0)
| | | | | | | | | | F5 > 4.84:  Boundary (14.0)
| | | | | | | | | F23 > 4.5365:  Floor (3.0)
| | | | | | | | (Blue / Luminance) > 1.08:  Chair (3.0)
| | | | | | | F5 > 16.3968:  Boundary (22.0)
| | | | (Green / Luminance) > 1.051318
| | | | | (Blue / Luminance) <= 1.003068
| | | | | | F5 <= 10.1456:  Boundary (2.0)
| | | | | | F5 > 10.1456:  Floor (3.0/1.0)
| | | | | (Blue / Luminance) > 1.003068
| | | | | | F5 <= 17.387:  Ball (24.0)
| | | | | | F5 > 17.387:  Boundary (3.0/1.0)
| | Blue > 75
| | | (Green / Luminance) <= 1.061851
| | | | (log:Red) <= 5.438079
| | | | | (Blue / Luminance) <= 1.019999
| | | | | | (log:Red) <= 4.539297
| | | | | | | F23 <= 4.196
| | | | | | | | | (F28 - (F6 - (F16 + F28))) <= 35.0143
| | | | | | | | | | F15 <= 1.5743:  Boundary (2.0/1.0)
| | | | | | | | | | F15 > 1.5743:  Floor (14.0)
| | | | | | | | | (F28 - (F6 - (F16 + F28))) > 35.0143
| | | | | | | | | (Green / Luminance) <= 1.028721:  Boundary (2.0)
| | | | | | | | | (Green / Luminance) > 1.028721:  Chair (3.0)
| | | | | | | F23 > 4.196:  Boundary (5.0/1.0)
| | | | | | (log:Red) > 4.539297
| | | | | | | F9 <= 89.2969
| | | | | | | | F19 <= 57.7292:  Chair (37.0/1.0)
| | | | | | | | F19 > 57.7292
| | | | | | | | | F8 <= 24.4583:  Boundary (4.0/1.0)
| | | | | | | | | F8 > 24.4583:  Chair (3.0)
| | | | | | | F9 > 89.2969:  Obstacle (2.0/1.0)
| | | | | (Blue / Luminance) > 1.019999
| | | | | | (Green / Luminance) <= 0.96956
| | | | | | | (Green / Luminance) <= 0.867712:  Obstacle (3.0)
| | | | | | | (Green / Luminance) > 0.867712:  Chair (15.0)
| | | | | | (Green / Luminance) > 0.96956:  Chair (1026.0/4.0)
| | | | (log:Red) > 5.438079
| | | | | F5 <= 14.1588:  Box (9.0)
| | | | | F5 > 14.1588
| | | | | | F23 <= 2.2309:  Chair (5.0)
| | | | | | F23 > 2.2309:  Boundary (5.0/1.0)
| | | (Green / Luminance) > 1.061851:  Ball (23.0/1.0)
(Green / Luminance) > 1.105606
| (Blue / Luminance) <= 0.811201
| | F5 <= 1.1632
| | | F23 <= 0.3083:  Obstacle (2.0)
| | | F23 > 0.3083:  Ball (115.0/1.0)
| | F5 > 1.1632
| | | (Blue / Luminance) <= 0.607203
| | | | F14 <= 2.3401
| | | | | F23 <= 4.5365
| | | | | | F23 <= 1.3534:  Ball (52.0/1.0)
| | | | | | F23 > 1.3534:  Obstacle (2.0)
| | | | | F23 > 4.5365:  Box (28.0)
| | | | F14 > 2.3401
| | | | | F15 <= 0.7359
| | | | | | F13 <= 8.5:  Box (43.0/1.0)
| | | | | | F13 > 8.5
| | | | | | | F15 <= 0.1886:  Box (6.0)
| | | | | | | F15 > 0.1886
| | | | | | | | F23 <= 2.7571:  Ball (18.0)
| | | | | | | | F23 > 2.7571:  Box (2.0)
| | | | | F15 > 0.7359
| | | | | | F23 <= 4.1603:  Box (789.0/8.0)
| | | | | | F23 > 4.1603
| | | | | | | F5 <= 14.3828:  Box (39.0/2.0)
| | | | | | | F5 > 14.3828:  Ball (6.0)
| | | (Blue / Luminance) > 0.607203
| | | | ((log:(log:F11)) + F22) <= 5.893939
| | | | | F8 <= 33.8568
| | | | | | Blue <= 49.875
| | | | | | | (F28 - (F6 - (F16 + F28))) <= 6.6199:  Box (9.0)
| | | | | | | (F28 - (F6 - (F16 + F28))) > 6.6199
| | | | | | | | F15 <= 3.2513:  Boundary (2.0)
| | | | | | | | F15 > 3.2513:  Obstacle (5.0)
| | | | | | Blue > 49.875
| | | | | | | (Green / Luminance) <= 1.203422:  Obstacle (143.0/9.0)
| | | | | | | (Green / Luminance) > 1.203422:  Ball (2.0)
| | | | | F8 > 33.8568:  Box (9.0/1.0)
| | | | ((log:(log:F11)) + F22) > 5.893939
| | | | | (Green / Luminance) <= 1.202669
| | | | | | F14 <= 2.3401
| | | | | | | F23 <= 0.6909:  Ball (16.0)
| | | | | | | F23 > 0.6909:  Obstacle (2.0/1.0)
| | | | | | F14 > 2.3401
| | | | | | | Blue <= 101.25
| | | | | | | | F13 <= 8.8333
| | | | | | | | | (F28 - (F6 - (F16 + F28))) <= 59.3196
| | | | | | | | | | (Blue / Luminance) <= 0.741329
| | | | | | | | | | | F9 <= 9.901:  Box (9.0/1.0)
| | | | | | | | | | | F9 > 9.901
| | | | | | | | | | | | F6 <= 20.8499:  Obstacle (6.0)
| | | | | | | | | | | | F6 > 20.8499:  Box (2.0)
| | | | | | | | | | (Blue / Luminance) > 0.741329:  Ball (3.0/1.0)
| | | | | | | | | (F28 - (F6 - (F16 + F28))) > 59.3196:  Floor (2.0)
| | | | | | | | F13 > 8.8333
| | | | | | | | | (log:Red) <= 4.945207:  Ball (8.0/1.0)
| | | | | | | | | (log:Red) > 4.945207:  Box (4.0/1.0)
```

```
| | | | | | | Blue > 101.25
| | | | | | | | F23 <= 4.1603:  Box (78.0/1.0)
| | | | | | | | F23 > 4.1603
| | | | | | | | | F8 <= 12.8099:  Ball (4.0)
| | | | | | | | | F8 > 12.8099:  Box (3.0)
| | | | | (Green / Luminance) > 1.202669:  Ball (28.0)
| (Blue / Luminance) > 0.811201
| | (Blue / Luminance) <= 0.92381
| | | (Green / Luminance) <= 1.130459
| | | | (log:Red) <= 4.114964:  Boundary (7.0)
| | | | (log:Red) > 4.114964
| | | | | F8 <= 8.8144:  Floor (31.0/2.0)
| | | | | F8 > 8.8144
| | | | | | F23 <= 3.6634
| | | | | | | F8 <= 28.1172
| | | | | | | | F14 <= 1.884:  Floor (2.0)
| | | | | | | | F14 > 1.884:  Boundary (3.0)
| | | | | | | F8 > 28.1172:  Ball (2.0/1.0)
| | | | | | F23 > 3.6634:  Obstacle (5.0/1.0)
| | | (Green / Luminance) > 1.130459
| | | | (F28 - (F6 - (F16 + F28))) <= 129.6922
| | | | | F15 <= 24.114:  Ball (45.0/1.0)
| | | | | F15 > 24.114:  Boundary (2.0)
| | | | (F28 - (F6 - (F16 + F28))) > 129.6922:  Boundary (3.0)
| | (Blue / Luminance) > 0.92381
| | | F5 <= 18.6987:  Ball (671.0/1.0)
| | | F5 > 18.6987
| | | | F6 <= 11.6091:  Boundary (2.0)
| | | | F6 > 11.6091:  Ball (12.0)
```

A.2 C4.5 decision tree for the best C4.5 genotype

Representation of the full decision tree, the top section of which was illustrated in figure 7.3.

```
     Features
------------------

     [Red] [F24] [Red] [(Green - Luminance)] [F15] [F20] [F14] [(Luminance / Red)] [(log:(log:F5))] [F11]
     Number of Leaves :  191
     Size of the tree :  381
     .
C4.5 pruned tree
------------------
.
(Luminance / Red) <= 0.934798
| (Green - Luminance) <= 5.9583
| | Red <= 81.875
| | | F14 <= 5.6438
| | | | (Luminance / Red) <= 0.898762
| | | | | (Green - Luminance) <= 0.7917:  Obstacle (20.0)
| | | | | (Green - Luminance) > 0.7917
| | | | | | (Green - Luminance) <= 2.625
| | | | | | | (Green - Luminance) <= 2.125:  Box (2.0)
| | | | | | | (Green - Luminance) > 2.125:  Obstacle (3.0)
| | | | | | (Green - Luminance) > 2.625:  Box (4.0)
| | | | (Luminance / Red) > 0.898762
| | | | | (log:(log:F5)) <= 0.874792:  Floor (5.0)
| | | | | (log:(log:F5)) > 0.874792
| | | | | | Red <= 75.125:  Obstacle (3.0/1.0)
| | | | | | Red > 75.125:  Ball (2.0/1.0)
| | | F14 > 5.6438
| | | | F15 <= 23.1663
| | | | | Red <= 65.75
| | | | | | F11 <= 28.5:  Boundary (8.0/1.0)
| | | | | | F11 > 28.5:  Obstacle (2.0/1.0)
| | | | | Red > 65.75:  Obstacle (13.0/2.0)
| | | | F15 > 23.1663:  Boundary (14.0/1.0)
| | Red > 81.875
| | | (Luminance / Red) <= 0.819642:  Obstacle (714.0/3.0)
| | | (Luminance / Red) > 0.819642
| | | | Red <= 166
| | | | | F11 <= 7.25
| | | | | | F24 <= 1.6099:  Floor (4.0)
| | | | | | F24 > 1.6099
| | | | | | | F24 <= 30.9906:  Obstacle (39.0/7.0)
| | | | | | | F24 > 30.9906
| | | | | | | | F24 <= 37.0798:  Floor (2.0)
| | | | | | | | F24 > 37.0798:  Obstacle (2.0/1.0)
| | | | | F11 > 7.25:  Obstacle (45.0/6.0)
| | | | Red > 166:  Box (3.0)
| (Green - Luminance) > 5.9583
| | F14 <= 2.3401
| | | F24 <= 3.9466
| | | | F15 <= 7.0834:  Ball (140.0/1.0)
| | | | F15 > 7.0834:  Obstacle (4.0)
| | | F24 > 3.9466
| | | | (log:(log:F5)) <= 0.45547
| | | | | F14 <= 1.0827:  Ball (46.0)
| | | | | F14 > 1.0827
| | | | | | F24 <= 17.0135:  Box (29.0)
| | | | | | F24 > 17.0135:  Obstacle (2.0)
| | | | (log:(log:F5)) > 0.45547:  Obstacle (25.0/1.0)
| | F14 > 2.3401
| | | (Luminance / Red) <= 0.822046
| | | | F15 <= 0.7359
```

```
| | | | | F24 <= 11.3244:  Ball (19.0)
| | | | | F24 > 11.3244:  Box (49.0)
| | | | F15 > 0.7359
| | | | | (log:(log:F5)) <= 1.056752
| | | | | | F14 <= 5.2689:  Box (620.0/1.0)
| | | | | | F14 > 5.2689
| | | | | | | (Luminance / Red) <= 0.760656
| | | | | | | | (Green - Luminance) <= 15.4167
| | | | | | | | | Red <= 142.125:  Box (10.0)
| | | | | | | | | Red > 142.125:  Obstacle (3.0)
| | | | | | | | (Green - Luminance) > 15.4167:  Box (130.0)
| | | | | | | (Luminance / Red) > 0.760656
| | | | | | | | F14 <= 7.5731:  Box (19.0/3.0)
| | | | | | | | F14 > 7.5731
| | | | | | | | | F14 <= 9.2138:  Ball (3.0)
| | | | | | | | | F14 > 9.2138
| | | | | | | | | | (log:(log:F5)) <= 0.974687:  Box (17.0)
| | | | | | | | | | (log:(log:F5)) > 0.974687:  Ball (2.0)
| | | | | (log:(log:F5)) > 1.056752
| | | | | | F15 <= 3.4222:  Ball (4.0)
| | | | | | F15 > 3.4222:  Box (31.0/1.0)
| | | (Luminance / Red) > 0.822046
| | | | Red <= 208
| | | | | (Green - Luminance) <= 30.5417
| | | | | | F20 <= 8.75
| | | | | | | Red <= 86.375
| | | | | | | | F20 <= 1.75:  Obstacle (4.0)
| | | | | | | | F20 > 1.75:  Box (8.0/1.0)
| | | | | | | Red > 86.375
| | | | | | | | (log:(log:F5)) <= 0.985133
| | | | | | | | | F14 <= 2.4617:  Box (2.0)
| | | | | | | | | F14 > 2.4617
| | | | | | | | | | (log:(log:F5)) <= 0.974687:  Obstacle (100.0/15.0)
| | | | | | | | | | (log:(log:F5)) > 0.974687:  Box (2.0)
| | | | | | | | (log:(log:F5)) > 0.985133
| | | | | | | | | F14 <= 11.9182:  Obstacle (50.0/1.0)
| | | | | | | | | F14 > 11.9182
| | | | | | | | | | F15 <= 9.1627:  Ball (3.0)
| | | | | | | | | | F15 > 9.1627:  Obstacle (4.0)
| | | | | F20 > 8.75
| | | | | | Red <= 149.375
| | | | | | | F20 <= 42.8333
| | | | | | | | F15 <= 10.9213
| | | | | | | | | (log:(log:F5)) <= 0.715969:  Obstacle (3.0/1.0)
| | | | | | | | | (log:(log:F5)) > 0.715969
| | | | | | | | | | F14 <= 7.1248
| | | | | | | | | | | F15 <= 2.1148:  Ball (4.0)
| | | | | | | | | | | F15 > 2.1148:  Obstacle (4.0)
| | | | | | | | | | F14 > 7.1248:  Ball (6.0)
| | | | | | | | F15 > 10.9213
| | | | | | | | | (Luminance / Red) <= 0.898902
| | | | | | | | | | (log:(log:F5)) <= 1.104081:  Obstacle (4.0)
| | | | | | | | | | (log:(log:F5)) > 1.104081:  Box (2.0)
| | | | | | | | | (Luminance / Red) > 0.898902:  Floor (2.0)
| | | | | | | F20 > 42.8333
| | | | | | | | Red <= 134.625:  Boundary (2.0)
| | | | | | | | Red > 134.625:  Box (3.0/1.0)
| | | | | | Red > 149.375:  Box (19.0/1.0)
| | | | (Green - Luminance) > 30.5417:  Ball (10.0/1.0)
| | | Red > 208:  Box (101.0/1.0)
(Luminance / Red) > 0.934798
| (Green - Luminance) <= 11.8333
| | Red <= 62.75
| | | (Luminance / Red) <= 1.159419
| | | | Red <= 44.5:  Boundary (724.0/13.0)
| | | | Red > 44.5
| | | | | (Luminance / Red) <= 1.057337
| | | | | | F14 <= 3.9115
| | | | | | | (log:(log:F5)) <= 0.957253
| | | | | | | | (log:(log:F5)) <= -1.907709:  Ball (3.0)
| | | | | | | | (log:(log:F5)) > -1.907709
| | | | | | | | | (Green - Luminance) <= 1.8333
| | | | | | | | | | (log:(log:F5)) <= 0.874792:  Floor (5.0/2.0)
| | | | | | | | | | (log:(log:F5)) > 0.874792:  Boundary (3.0)
| | | | | | | | | (Green - Luminance) > 1.8333:  Boundary (10.0)
| | | | | | | (log:(log:F5)) > 0.957253:  Floor (18.0/1.0)
| | | | | | F14 > 3.9115
| | | | | | | Red <= 49.75:  Boundary (52.0/1.0)
| | | | | | | Red > 49.75
| | | | | | | | (log:(log:F5)) <= 0.45547:  Floor (9.0)
| | | | | | | | (log:(log:F5)) > 0.45547
| | | | | | | | | (log:(log:F5)) <= 1.028578
| | | | | | | | | | (Green - Luminance) <= 0.1667:  Boundary (11.0)
| | | | | | | | | | (Green - Luminance) > 0.1667
| | | | | | | | | | | (log:(log:F5)) <= 0.764385:  Boundary (14.0/1.0)
| | | | | | | | | | | (log:(log:F5)) > 0.764385
| | | | | | | | | | | | (log:(log:F5)) <= 0.883626:  Floor (13.0/2.0)
| | | | | | | | | | | | (log:(log:F5)) > 0.883626
| | | | | | | | | | | | | (log:(log:F5)) <= 0.994936
| | | | | | | | | | | | | | F15 <= 2.7839
| | | | | | | | | | | | | | | (Luminance / Red) <= 1.026217:  Floor (6.0/1.0)
| | | | | | | | | | | | | | | (Luminance / Red) > 1.026217:  Boundary (2.0)
| | | | | | | | | | | | | | F15 > 2.7839:  Boundary (21.0/2.0)
| | | | | | | | | | | | | | (log:(log:F5)) > 0.994936:  Floor (3.0)
| | | | | | | | | | (log:(log:F5)) > 1.028578
| | | | | | | | | | | F15 <= 2.7839:  Floor (2.0/1.0)
| | | | | | | | | | | F15 > 2.7839:  Boundary (34.0)
| | | | | (Luminance / Red) > 1.057337
| | | | | | (log:(log:F5)) <= 1.074475
| | | | | | | (Green - Luminance) <= 1.3333
| | | | | | | | Red <= 52.125
| | | | | | | | | Red <= 47.5:  Boundary (3.0/1.0)
| | | | | | | | | Red > 47.5:  Floor (2.0)
| | | | | | | | Red > 52.125:  Chair (6.0)
| | | | | | | (Green - Luminance) > 1.3333
| | | | | | | | (Green - Luminance) <= 3.4167
| | | | | | | | | F14 <= 8.5628
```

```
| | | | | | | | | | F20 <= 3.5833:  Floor (5.0/1.0)
| | | | | | | | | | F20 > 3.5833:  Ball (4.0)
| | | | | | | | | F14 > 8.5628:  Boundary (3.0)
| | | | | | | | (Green - Luminance) > 3.4167
| | | | | | | | | F15 <= 24.114:  Ball (19.0/1.0)
| | | | | | | | | F15 > 24.114:  Boundary (2.0)
| | | | | | (log:(log:F5)) > 1.074475:  Boundary (11.0)
| | | (Luminance / Red) > 1.159419
| | | | (Green - Luminance) <= 3.3333
| | | | | F24 <= 4.2765
| | | | | | Red <= 34.375:  Boundary (2.0)
| | | | | | Red > 34.375:  Ball (4.0)
| | | | | F24 > 4.2765:  Boundary (11.0)
| | | | (Green - Luminance) > 3.3333:  Ball (97.0)
| | Red > 62.75
| | | (Luminance / Red) <= 1.04916
| | | | Red <= 150.625
| | | | | F11 <= 7.5833
| | | | | | F20 <= 10.5833
| | | | | | | (Luminance / Red) <= 1.038447
| | | | | | | | F15 <= 14.8401
| | | | | | | | | F24 <= 36.1374
| | | | | | | | | | F14 <= 13.9045
| | | | | | | | | | | (Luminance / Red) <= 0.967649
| | | | | | | | | | | | F20 <= 4.5:  Floor (24.0/4.0)
| | | | | | | | | | | | F20 > 4.5:  Obstacle (6.0/2.0)
| | | | | | | | | | | (Luminance / Red) > 0.967649
| | | | | | | | | | | | Red <= 129.875:  Floor (786.0/17.0)
| | | | | | | | | | | | Red > 129.875
| | | | | | | | | | | | | F15 <= 2.3531:  Obstacle (4.0)
| | | | | | | | | | | | | F15 > 2.3531:  Floor (26.0/1.0)
| | | | | | | | | | F14 > 13.9045
| | | | | | | | | | | F15 <= 2.9625:  Boundary (5.0)
| | | | | | | | | | | F15 > 2.9625:  Floor (12.0)
| | | | | | | | | F24 > 36.1374
| | | | | | | | | | Red <= 81.75:  Floor (9.0)
| | | | | | | | | | Red > 81.75
| | | | | | | | | | | F24 <= 39.7494:  Boundary (6.0/1.0)
| | | | | | | | | | | F24 > 39.7494:  Floor (2.0/1.0)
| | | | | | | | F15 > 14.8401
| | | | | | | | | Red <= 133.125
| | | | | | | | | | (log:(log:F5)) <= 0.883626:  Floor (61.0/3.0)
| | | | | | | | | | (log:(log:F5)) > 0.883626
| | | | | | | | | | | (Green - Luminance) <= 1.7083
| | | | | | | | | | | | Red <= 68.625:  Boundary (2.0)
| | | | | | | | | | | | Red > 68.625:  Obstacle (2.0)
| | | | | | | | | | | (Green - Luminance) > 1.7083
| | | | | | | | | | | | F20 <= 1.9167
| | | | | | | | | | | | | F20 <= 1.5833
| | | | | | | | | | | | | | F20 <= 0.3333:  Boundary (4.0)
| | | | | | | | | | | | | | F20 > 0.3333:  Floor (18.0/6.0)
| | | | | | | | | | | | | F20 > 1.5833:  Boundary (4.0/1.0)
| | | | | | | | | | | | F20 > 1.9167:  Floor (30.0/3.0)
| | | | | | | | | Red > 133.125
| | | | | | | | | | F14 <= 13.5899:  Boundary (12.0)
| | | | | | | | | | F14 > 13.5899:  Floor (3.0/1.0)
| | | | | | | | (Luminance / Red) > 1.038447
| | | | | | | | | F14 <= 8.1281
| | | | | | | | | | F11 <= 5:  Floor (16.0/2.0)
| | | | | | | | | | F11 > 5:  Chair (2.0)
| | | | | | | | | F14 > 8.1281
| | | | | | | | | | F24 <= 5.6958:  Chair (3.0)
| | | | | | | | | | F24 > 5.6958:  Boundary (4.0)
| | | | | | F20 > 10.5833
| | | | | | | F24 <= 7.568
| | | | | | | | Red <= 67.625
| | | | | | | | | F24 <= 2.4893:  Boundary (2.0)
| | | | | | | | | F24 > 2.4893:  Ball (3.0)
| | | | | | | | Red > 67.625
| | | | | | | | | F11 <= 3.5833:  Floor (14.0)
| | | | | | | | | F11 > 3.5833
| | | | | | | | | | (Green - Luminance) <= 7.4583:  Floor (3.0/1.0)
| | | | | | | | | | (Green - Luminance) > 7.4583:  Ball (2.0)
| | | | | | | F24 > 7.568
| | | | | | | | F15 <= 1.6423:  Floor (5.0)
| | | | | | | | F15 > 1.6423
| | | | | | | | | (log:(log:F5)) <= 0.67552
| | | | | | | | | | F24 <= 10.5847:  Boundary (2.0)
| | | | | | | | | | F24 > 10.5847:  Floor (5.0)
| | | | | | | | | (log:(log:F5)) > 0.67552
| | | | | | | | | | F20 <= 13.0833
| | | | | | | | | | | F15 <= 7.1553:  Boundary (2.0)
| | | | | | | | | | | F15 > 7.1553:  Floor (3.0)
| | | | | | | | | | F20 > 13.0833:  Boundary (33.0/3.0)
| | | | | F11 > 7.5833
| | | | | | F20 <= 18.25
| | | | | | | Red <= 100.875
| | | | | | | | F11 <= 33.3333
| | | | | | | | | F15 <= 5.9286
| | | | | | | | | | F24 <= 37.0798
| | | | | | | | | | | F11 <= 11.75
| | | | | | | | | | | | F15 <= 4.2765:  Floor (8.0/1.0)
| | | | | | | | | | | | F15 > 4.2765
| | | | | | | | | | | | | F15 <= 5.5617
| | | | | | | | | | | | | | (Luminance / Red) <= 1.032918:  Ball (3.0)
| | | | | | | | | | | | | | (Luminance / Red) > 1.032918:  Floor (2.0/1.0)
| | | | | | | | | | | | | F15 > 5.5617:  Obstacle (2.0)
| | | | | | | | | | | | F11 > 11.75
| | | | | | | | | | | | | (Luminance / Red) <= 1.016344:  Obstacle (8.0)
| | | | | | | | | | | | | (Luminance / Red) > 1.016344
| | | | | | | | | | | | | F24 <= 6.1362:  Obstacle (3.0/1.0)
| | | | | | | | | | | | | F24 > 6.1362:  Ball (3.0)
| | | | | | | | | | | F24 > 37.0798:  Chair (3.0/1.0)
| | | | | | | | | | F15 > 5.9286
| | | | | | | | | | | F11 <= 23.4167
| | | | | | | | | | | | (log:(log:F5)) <= 0.874792:  Floor (4.0)
| | | | | | | | | | | | (log:(log:F5)) > 0.874792:  Boundary (8.0/1.0)
```

```
| | | | | | | | | | F11 > 23.4167:  Obstacle (3.0)
| | | | | | | | | F11 > 33.3333:  Boundary (8.0/1.0)
| | | | | | | Red > 100.875
| | | | | | | | (Green - Luminance) <= 4.2917
| | | | | | | | | (log:(log:F5)) <= 1.035056:  Chair (16.0/1.0)
| | | | | | | | | (log:(log:F5)) > 1.035056:  Obstacle (4.0/1.0)
| | | | | | | | (Green - Luminance) > 4.2917
| | | | | | | | | Red <= 107:  Floor (3.0/1.0)
| | | | | | | | | Red > 107
| | | | | | | | | | F15 <= 14.8401
| | | | | | | | | | | F20 <= 2.75
| | | | | | | | | | | | (Green - Luminance) <= 9.9583:  Floor (4.0/1.0)
| | | | | | | | | | | | (Green - Luminance) > 9.9583:  Obstacle (4.0)
| | | | | | | | | | | F20 > 2.75:  Obstacle (13.0)
| | | | | | | | | | F15 > 14.8401
| | | | | | | | | | | F15 <= 18.2829:  Chair (4.0)
| | | | | | | | | | | F15 > 18.2829:  Obstacle (9.0)
| | | | | | F20 > 18.25
| | | | | | | (Luminance / Red) <= 1.034581
| | | | | | | | Red <= 132.25:  Boundary (13.0)
| | | | | | | | Red > 132.25
| | | | | | | | | Red <= 141.25:  Obstacle (2.0)
| | | | | | | | | Red > 141.25:  Boundary (2.0)
| | | | | | | (Luminance / Red) > 1.034581:  Chair (7.0/1.0)
| | | | Red > 150.625
| | | | | (log:(log:F5)) <= 1.339759
| | | | | | Red <= 226.125
| | | | | | | (Luminance / Red) <= 1.014591
| | | | | | | | F15 <= 12.8589
| | | | | | | | | F24 <= 2.6748:  Boundary (3.0/1.0)
| | | | | | | | | F24 > 2.6748:  Obstacle (9.0)
| | | | | | | | F15 > 12.8589
| | | | | | | | | F20 <= 4.3333
| | | | | | | | | | F11 <= 13.0833:  Boundary (9.0)
| | | | | | | | | | F11 > 13.0833:  Obstacle (2.0)
| | | | | | | | | F20 > 4.3333:  Box (3.0/1.0)
| | | | | | | (Luminance / Red) > 1.014591
| | | | | | | | F20 <= 1.5
| | | | | | | | | (log:(log:F5)) <= 0.912443
| | | | | | | | | (Green - Luminance) <= 4.375:  Chair (2.0)
| | | | | | | | | (Green - Luminance) > 4.375:  Boundary (7.0)
| | | | | | | | | (log:(log:F5)) > 0.912443
| | | | | | | | | Red <= 157.375:  Floor (2.0)
| | | | | | | | | Red > 157.375:  Obstacle (4.0)
| | | | | | | | F20 > 1.5
| | | | | | | | | F20 <= 81.0833:  Chair (23.0/1.0)
| | | | | | | | | F20 > 81.0833:  Boundary (3.0/1.0)
| | | | | | Red > 226.125
| | | | | | | F14 <= 10.2727:  Box (33.0/1.0)
| | | | | | | F14 > 10.2727
| | | | | | | | F14 <= 11.0709:  Obstacle (2.0)
| | | | | | | | F14 > 11.0709:  Chair (7.0)
| | | | | (log:(log:F5)) > 1.339759:  Boundary (25.0)
| | | (Luminance / Red) > 1.04916
| | | | (Green - Luminance) <= 6.5833
| | | | | Red <= 89.125
| | | | | | (Green - Luminance) <= 3.6667
| | | | | | | F15 <= 25.9926
| | | | | | | | (Luminance / Red) <= 1.065194
| | | | | | | | | F14 <= 6.8385:  Floor (3.0/1.0)
| | | | | | | | | F14 > 6.8385:  Chair (6.0)
| | | | | | | | (Luminance / Red) > 1.065194:  Chair (102.0)
| | | | | | | F15 > 25.9926
| | | | | | | | Red <= 81.75:  Obstacle (2.0)
| | | | | | | | Red > 81.75:  Boundary (2.0/1.0)
| | | | | | (Green - Luminance) > 3.6667
| | | | | | | F15 <= 1.425:  Ball (4.0)
| | | | | | | F15 > 1.425
| | | | | | | | F11 <= 7.0833:  Floor (7.0/2.0)
| | | | | | | | F11 > 7.0833:  Chair (2.0)
| | | | | Red > 89.125:  Chair (882.0)
| | | | (Green - Luminance) > 6.5833
| | | | | Red <= 118.375
| | | | | | F15 <= 6.3406:  Ball (37.0/1.0)
| | | | | | F15 > 6.3406
| | | | | | | (Luminance / Red) <= 1.075038:  Obstacle (3.0)
| | | | | | | (Luminance / Red) > 1.075038
| | | | | | | | Red <= 86.125:  Ball (4.0)
| | | | | | | | Red > 86.125:  Boundary (2.0)
| | | | | Red > 118.375
| | | | | | (Green - Luminance) <= 9.9583:  Chair (39.0)
| | | | | | (Green - Luminance) > 9.9583:  Obstacle (2.0)
| (Green - Luminance) > 11.8333
| | F15 <= 24.9329
| | | (Luminance / Red) <= 1.057165
| | | | Red <= 215.5
| | | | | F20 <= 3.75
| | | | | | F11 <= 5.5833
| | | | | | | (Luminance / Red) <= 1.034835
| | | | | | | | F24 <= 9.2792:  Floor (11.0)
| | | | | | | | F24 > 9.2792
| | | | | | | | | (Green - Luminance) <= 12.625:  Floor (4.0/1.0)
| | | | | | | | | (Green - Luminance) > 12.625:  Boundary (6.0/1.0)
| | | | | | | (Luminance / Red) > 1.034835:  Obstacle (3.0)
| | | | | | F11 > 5.5833
| | | | | | | F14 <= 2.6173:  Ball (4.0)
| | | | | | | F14 > 2.6173
| | | | | | | | (Green - Luminance) <= 16:  Obstacle (7.0)
| | | | | | | | (Green - Luminance) > 16:  Ball (3.0/1.0)
| | | | | F20 > 3.75
| | | | | | F14 <= 7.3005
| | | | | | | (Green - Luminance) <= 14.8333
| | | | | | | | Red <= 128.875:  Ball (6.0/1.0)
| | | | | | | | Red > 128.875:  Boundary (3.0/1.0)
| | | | | | | (Green - Luminance) > 14.8333:  Ball (40.0)
| | | | | | F14 > 7.3005
| | | | | | | F14 <= 11.0709
```

```
| | | | | | | | F20 <= 17.1667:  Obstacle (2.0/1.0)
| | | | | | | | F20 > 17.1667:  Boundary (3.0)
| | | | | | | F14 > 11.0709:  Ball (2.0)
| | | | Red > 215.5:  Box (8.0)
| | | (Luminance / Red) > 1.057165:  Ball (598.0)
| | F15 > 24.9329
| | | Red <= 214:  Boundary (20.0)
| | | Red > 214:  Box (4.0)
```

A.3 C4.5 decision tree for the best Multi-Objective genotype (IBk)

Representation of the full decision tree, the top section of which was illustrated in figure 7.4. Please note that the genotype uses IBk to make decisions and that the C4.5 decision tree is provided for illustrative purposes only.

```
     Features
------------------

     [F14] [(F25 - (Luminance % Blue))] [F12] [((F17 % F19) - Red)] [F5] [Blue] [F22] [(Green / Red)] [F5] [F31] [(log:F15)] [(((F10 / F30)
- (F29 / Luminance)) + F11)]
     Number of Leaves :  216
     Size of the tree :  431
     .

     C4.5 pruned tree ------------------ .

     F25 - (Luminance % Blue) <= -52.2721
| Green / Red <= 1.206593
| | Green / Red <= 1.031773
| | | Green / Red <= 0.866024:  Obstacle (11.0)
| | | Green / Red > 0.866024
| | | | (F17 % F19) - Red <= -154.75
| | | | | F5 <= 1.2697:  Ball (11.0)
| | | | | F5 > 1.2697
| | | | | | Blue <= 188.125
| | | | | | | F14 <= 2.2886
| | | | | | | | log(F15) <= 1.154835:  Ball (6.0)
| | | | | | | | log(F15) > 1.154835:  Box (6.0)
| | | | | | | F14 > 2.2886:  Box (111.0/1.0)
| | | | | | Blue > 188.125
| | | | | | | F14 <= 10.9213:  Box (6.0/1.0)
| | | | | | | F14 > 10.9213:  Chair (7.0)
| | | | (F17 % F19) - Red > -154.75
| | | | | F5 <= 16.3968
| | | | | | F12 <= 6.5833:  Floor (2.0)
| | | | | | F12 > 6.5833:  Chair (5.0)
| | | | | F5 > 16.3968:  Boundary (5.0)
| | Green / Red > 1.031773
| | | Blue <= 98.625
| | | | (F17 % F19) - Red <= -106.9453:  Ball (6.0)
| | | | (F17 % F19) - Red > -106.9453
| | | | | Green / Red <= 1.136141
| | | | | | F14 <= 7.754
| | | | | | | F12 <= 3.5:  Floor (13.0/2.0)
| | | | | | | F12 > 3.5
| | | | | | | | F14 <= 3.4074
| | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 2.594126:  Floor (3.0)
| | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 2.594126
| | | | | | | | | | F12 <= 9.3333:  Chair (9.0)
| | | | | | | | | | F12 > 9.3333:  Floor (3.0/1.0)
| | | | | | | | F14 > 3.4074
| | | | | | | | | F5 <= 8.3625:  Chair (8.0)
| | | | | | | | | F5 > 8.3625
| | | | | | | | | | F31 <= 4:  Chair (6.0)
| | | | | | | | | | F31 > 4:  Boundary (2.0)
| | | | | | F14 > 7.754
| | | | | | | Green / Red <= 1.061776:  Boundary (2.0/1.0)
| | | | | | | Green / Red > 1.061776:  Chair (40.0)
| | | | | Green / Red > 1.136141
| | | | | | Blue <= 83.125:  Ball (6.0)
| | | | | | Blue > 83.125:  Chair (8.0/1.0)
| | | Blue > 98.625:  Chair (996.0/7.0)
| Green / Red > 1.206593:  Ball (640.0/2.0)
F25 - (Luminance % Blue) > -52.2721
| (F17 % F19) - Red <= -60.0039
| | Green / Red <= 1.024316
| | | Green / Red <= 0.794643:  Obstacle (733.0/9.0)
| | | Green / Red > 0.794643
| | | | F14 <= 2.4585
| | | | | F22 <= 5.9167
| | | | | | F5 <= 4.84
| | | | | | | F5 <= 3.5868
| | | | | | | | Blue <= 58.75:  Ball (10.0)
| | | | | | | | Blue > 58.75
| | | | | | | | | F25 - (Luminance % Blue) <= -6.974:  Obstacle (2.0)
| | | | | | | | | F25 - (Luminance % Blue) > -6.974:  Floor (3.0)
| | | | | | | F5 > 3.5868
```

```
| | | | | | | | log(F15) <= 1.738253:  Box (19.0/1.0)
| | | | | | | | log(F15) > 1.738253:  Obstacle (3.0)
| | | | | | F5 > 4.84
| | | | | | | F12 <= 14.1667
| | | | | | | | F25 - (Luminance % Blue) <= 14.82
| | | | | | | | | (F17 % F19) - Red <= -66.5769
| | | | | | | | | | F22 <= 5.4167:  Obstacle (43.0/4.0)
| | | | | | | | | | F22 > 5.4167:  Floor (2.0)
| | | | | | | | | (F17 % F19) - Red > -66.5769:  Floor (3.0)
| | | | | | | | F25 - (Luminance % Blue) > 14.82:  Floor (5.0)
| | | | | | | F12 > 14.1667
| | | | | | | | F14 <= 1.4376:  Boundary (3.0)
| | | | | | | | F14 > 1.4376:  Chair (2.0/1.0)
| | | | | F22 > 5.9167
| | | | | | log(F15) <= 1.453135:  Ball (66.0)
| | | | | | log(F15) > 1.453135
| | | | | | | log(F15) <= 1.559996:  Box (6.0/1.0)
| | | | | | | log(F15) > 1.559996
| | | | | | | | F31 <= 5:  Obstacle (3.0)
| | | | | | | | F31 > 5
| | | | | | | | | (F17 % F19) - Red <= -142.4544:  Ball (33.0)
| | | | | | | | | (F17 % F19) - Red > -142.4544
| | | | | | | | | Blue <= 69.875:  Ball (3.0)
| | | | | | | | | Blue > 69.875:  Obstacle (2.0)
| | | | F14 > 2.4585
| | | | | Blue <= 47.5
| | | | | | (F17 % F19) - Red <= -80.5221
| | | | | | | log(F15) <= -0.737099
| | | | | | | | F5 <= 7.6855:  Box (23.0)
| | | | | | | | F5 > 7.6855
| | | | | | | | | log(F15) <= -2.358098:  Box (4.0)
| | | | | | | | | log(F15) > -2.358098:  Ball (5.0)
| | | | | | | log(F15) > -0.737099
| | | | | | | | F5 <= 17.7611:  Box (629.0)
| | | | | | | | F5 > 17.7611
| | | | | | | | | F5 <= 18.9032:  Ball (3.0)
| | | | | | | | | F5 > 18.9032:  Box (23.0)
| | | | | | (F17 % F19) - Red > -80.5221
| | | | | | | F5 <= 6.995:  Obstacle (4.0)
| | | | | | | F5 > 6.995:  Box (25.0/1.0)
| | | | | Blue > 47.5
| | | | | | (F17 % F19) - Red <= -171.6775
| | | | | | | F5 <= 15.6652
| | | | | | | | log(F15) <= -1.002121
| | | | | | | | | F5 <= 12.7679:  Ball (6.0)
| | | | | | | | | F5 > 12.7679:  Box (9.0)
| | | | | | | | log(F15) > -1.002121:  Box (150.0/1.0)
| | | | | | | F5 > 15.6652
| | | | | | | | (F17 % F19) - Red <= -224.7642:  Boundary (6.0)
| | | | | | | | (F17 % F19) - Red > -224.7642
| | | | | | | | | log(F15) <= 1.902451:  Ball (5.0)
| | | | | | | | | log(F15) > 1.902451
| | | | | | | | | | F14 <= 5.352:  Box (6.0)
| | | | | | | | | | F14 > 5.352:  Obstacle (10.0/1.0)
| | | | | | (F17 % F19) - Red > -171.6775
| | | | | | | F31 <= 6
| | | | | | | | Green / Red <= 0.996337
| | | | | | | | | Blue <= 62.625
| | | | | | | | | | (F17 % F19) - Red <= -103.6682:  Box (25.0/1.0)
| | | | | | | | | | (F17 % F19) - Red > -103.6682
| | | | | | | | | | log(F15) <= 3.052372
| | | | | | | | | | | log(F15) <= -0.71703:  Floor (4.0/1.0)
| | | | | | | | | | | log(F15) > -0.71703
| | | | | | | | | | | | Blue <= 59.875
| | | | | | | | | | | | | F14 <= 3.7587:  Box (3.0)
| | | | | | | | | | | | | F14 > 3.7587
| | | | | | | | | | | | | | F12 <= 1.9167:  Box (2.0)
| | | | | | | | | | | | | | F12 > 1.9167:  Obstacle (26.0/4.0)
| | | | | | | | | | | | Blue > 59.875:  Floor (2.0)
| | | | | | | | | | | log(F15) > 3.052372
| | | | | | | | | | | | (F17 % F19) - Red <= -69.7005:  Obstacle (3.0)
| | | | | | | | | | | | (F17 % F19) - Red > -69.7005:  Boundary (4.0)
| | | | | | | | | Blue > 62.625
| | | | | | | | | | (F17 % F19) - Red <= -86.5361
| | | | | | | | | | | F12 <= 26.0833
| | | | | | | | | | | | F22 <= 9.5
| | | | | | | | | | | | | Green / Red <= 0.908374
| | | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 6.63366
| | | | | | | | | | | | | | F25 - (Luminance % Blue) <= -3.8509:  Obstacle (11.0/1.0)
| | | | | | | | | | | | | | F25 - (Luminance % Blue) > -3.8509:  Floor (3.0)
| | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 6.63366:  Obstacle (26.0)
| | | | | | | | | | | | | Green / Red > 0.908374
| | | | | | | | | | | | | | Green / Red <= 0.931232
| | | | | | | | | | | | | | | F12 <= 3.75:  Floor (2.0)
| | | | | | | | | | | | | | | F12 > 3.75
| | | | | | | | | | | | | | | | F25 - (Luminance % Blue) <= -19.9531:  Box (7.0)
| | | | | | | | | | | | | | | | F25 - (Luminance % Blue) > -19.9531:  Obstacle (5.0/1.0)
| | | | | | | | | | | | | | Green / Red > 0.931232
| | | | | | | | | | | | | | | F22 <= 3.6667:  Obstacle (65.0/3.0)
| | | | | | | | | | | | | | | F22 > 3.6667
| | | | | | | | | | | | | | | | F31 <= 5
| | | | | | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 11.415842
| | | | | | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 10.544039
| | | | | | | | | | | | | | | | | | Green / Red <= 0.975522:  Obstacle (12.0)
| | | | | | | | | | | | | | | | | | Green / Red > 0.975522
| | | | | | | | | | | | | | | | | | | F14 <= 14.3457
| | | | | | | | | | | | | | | | | | | | F14 <= 9.3442:  Obstacle (4.0/1.0)
| | | | | | | | | | | | | | | | | | | | F14 > 9.3442:  Box (3.0)
| | | | | | | | | | | | | | | | | | | F14 > 14.3457:  Obstacle (2.0/1.0)
| | | | | | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 10.544039:  Box (3.0)
| | | | | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 11.415842:  Obstacle (15.0)
| | | | | | | | | | | | | | | F31 > 5:  Box (2.0)
| | | | | | | | | | | | | F22 > 9.5
| | | | | | | | | | | | | | (F17 % F19) - Red <= -139.5638:  Box (2.0)
| | | | | | | | | | | | | | (F17 % F19) - Red > -139.5638
| | | | | | | | | | | | | | (F17 % F19) - Red <= -110.5104:  Obstacle (6.0)
| | | | | | | | | | | | | | (F17 % F19) - Red > -110.5104:  Ball (2.0)
```

```
| | | | | | | | | | | F12 > 26.0833:  Boundary (3.0/1.0)
| | | | | | | | | | (F17 % F19) - Red > -86.5361
| | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 10.719449
| | | | | | | | | | | | F14 <= 9.3442:  Floor (8.0/1.0)
| | | | | | | | | | | | F14 > 9.3442:  Obstacle (2.0/1.0)
| | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 10.719449
| | | | | | | | | | | | Blue <= 73.125:  Obstacle (8.0/1.0)
| | | | | | | | | | | | Blue > 73.125:  Boundary (2.0)
| | | | | | | | Green / Red > 0.996337
| | | | | | | | | F25 - (Luminance % Blue) <= -6.1274:  Obstacle (30.0/4.0)
| | | | | | | | | F25 - (Luminance % Blue) > -6.1274
| | | | | | | | | F12 <= 5.6667
| | | | | | | | | | F25 - (Luminance % Blue) <= 33.2448:  Floor (37.0/2.0)
| | | | | | | | | | F25 - (Luminance % Blue) > 33.2448:  Boundary (2.0)
| | | | | | | | | F12 > 5.6667
| | | | | | | | | | F25 - (Luminance % Blue) <= 0.6556:  Obstacle (3.0/2.0)
| | | | | | | | | | F25 - (Luminance % Blue) > 0.6556
| | | | | | | | | | | Green / Red <= 1.011494:  Obstacle (5.0)
| | | | | | | | | | | Green / Red > 1.011494:  Boundary (4.0/1.0)
| | | | | | | F31 > 6
| | | | | | | | F25 - (Luminance % Blue) <= -13.9043
| | | | | | | | | (F17 % F19) - Red <= -111.3242
| | | | | | | | | | F12 <= 11:  Box (6.0)
| | | | | | | | | | F12 > 11
| | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 20.582827:  Ball (8.0)
| | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 20.582827
| | | | | | | | | | | | (F17 % F19) - Red <= -133.5651:  Box (7.0)
| | | | | | | | | | | | (F17 % F19) - Red > -133.5651:  Ball (3.0/1.0)
| | | | | | | | | (F17 % F19) - Red > -111.3242
| | | | | | | | | | Blue <= 69.125:  Ball (6.0/1.0)
| | | | | | | | | | Blue > 69.125:  Floor (3.0)
| | | | | | | | F25 - (Luminance % Blue) > -13.9043
| | | | | | | | | F14 <= 3.9115:  Box (10.0)
| | | | | | | | | F14 > 3.9115
| | | | | | | | | | Blue <= 64.75:  Box (4.0/1.0)
| | | | | | | | | | Blue > 64.75:  Obstacle (10.0/1.0)
| | Green / Red > 1.024316
| | | Green / Red <= 1.139161
| | | | F25 - (Luminance % Blue) <= -11.9302
| | | | | (F17 % F19) - Red <= -79.625
| | | | | | F31 <= 5.6667
| | | | | | | Blue <= 142.125
| | | | | | | | F14 <= 2.6173:  Ball (9.0/1.0)
| | | | | | | | F14 > 2.6173
| | | | | | | | | F12 <= 6.25:  Floor (5.0/1.0)
| | | | | | | | | F12 > 6.25:  Obstacle (16.0)
| | | | | | | Blue > 142.125:  Box (4.0/1.0)
| | | | | | F31 > 5.6667
| | | | | | | log(F15) <= 3.284128:  Ball (42.0/2.0)
| | | | | | | log(F15) > 3.284128:  Box (2.0)
| | | | | (F17 % F19) - Red > -79.625
| | | | | | F14 <= 2.2886:  Floor (4.0/1.0)
| | | | | | F14 > 2.2886
| | | | | | | log(F15) <= 2.645494:  Chair (9.0/1.0)
| | | | | | | log(F15) > 2.645494:  Boundary (4.0)
| | | | F25 - (Luminance % Blue) > -11.9302
| | | | | Blue <= 127.125
| | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 9.757119
| | | | | | | F22 <= 9
| | | | | | | | log(F15) <= 3.244037
| | | | | | | | | F25 - (Luminance % Blue) <= 27.6875
| | | | | | | | | | F14 <= 13.5899
| | | | | | | | | | | Blue <= 109.125
| | | | | | | | | | | | (F17 % F19) - Red <= -71.7134:  Floor (665.0/8.0)
| | | | | | | | | | | | (F17 % F19) - Red > -71.7134
| | | | | | | | | | | | | log(F15) <= -0.85496
| | | | | | | | | | | | | | F12 <= 2.5833:  Floor (4.0)
| | | | | | | | | | | | | | F12 > 2.5833:  Ball (4.0)
| | | | | | | | | | | | | log(F15) > -0.85496
| | | | | | | | | | | | | | Green / Red <= 1.033958
| | | | | | | | | | | | | | | F14 <= 5.6438:  Floor (4.0/1.0)
| | | | | | | | | | | | | | | F14 > 5.6438:  Boundary (2.0)
| | | | | | | | | | | | | | Green / Red > 1.033958:  Floor (82.0/5.0)
| | | | | | | | | | | Blue > 109.125
| | | | | | | | | | | | Green / Red <= 1.098859
| | | | | | | | | | | | | F25 - (Luminance % Blue) <= 14.9238:  Floor (76.0/4.0)
| | | | | | | | | | | | | F25 - (Luminance % Blue) > 14.9238
| | | | | | | | | | | | | | F14 <= 1.0827:  Boundary (2.0)
| | | | | | | | | | | | | | F14 > 1.0827:  Floor (2.0)
| | | | | | | | | | | | | Green / Red > 1.098859
| | | | | | | | | | | | | F5 <= 12.0583
| | | | | | | | | | | | | | F25 - (Luminance % Blue) <= -4.5163:  Boundary (2.0/1.0)
| | | | | | | | | | | | | | F25 - (Luminance % Blue) > -4.5163:  Obstacle (4.0)
| | | | | | | | | | | | | F5 > 12.0583:  Floor (7.0)
| | | | | | | | | | | | F14 > 13.5899
| | | | | | | | | | | | | log(F15) <= 0.962143:  Boundary (5.0)
| | | | | | | | | | | | | log(F15) > 0.962143:  Floor (14.0)
| | | | | | | | | | | F25 - (Luminance % Blue) > 27.6875
| | | | | | | | | | | | F5 <= 12.0661
| | | | | | | | | | | | | F14 <= 0.315:  Floor (3.0)
| | | | | | | | | | | | | F14 > 0.315
| | | | | | | | | | | | | | log(F15) <= 1.410938:  Boundary (6.0)
| | | | | | | | | | | | | | log(F15) > 1.410938
| | | | | | | | | | | | | | | F14 <= 3.9115:  Boundary (3.0/1.0)
| | | | | | | | | | | | | | | F14 > 3.9115:  Floor (2.0)
| | | | | | | | | | | | | F5 > 12.0661:  Floor (13.0/1.0)
| | | | | | | | | log(F15) > 3.244037
| | | | | | | | | | F5 <= 11.3671:  Floor (16.0/1.0)
| | | | | | | | | | F5 > 11.3671
| | | | | | | | | | | F14 <= 7.3005:  Boundary (8.0)
| | | | | | | | | | | F14 > 7.3005
| | | | | | | | | | | | F22 <= 2.75:  Boundary (3.0)
| | | | | | | | | | | | F22 > 2.75:  Floor (9.0/1.0)
| | | | | | | F22 > 9
| | | | | | | | Green / Red <= 1.040816
| | | | | | | | | F12 <= 8.8333:  Obstacle (3.0/1.0)
| | | | | | | | | F12 > 8.8333:  Ball (2.0)
```

```
| | | | | | | | Green / Red > 1.040816
| | | | | | | | | log(F15) <= -0.306661:  Chair (2.0)
| | | | | | | | | log(F15) > -0.306661
| | | | | | | | | | F5 <= 13.0292
| | | | | | | | | | | log(F15) <= 2.451169
| | | | | | | | | | | | (F17 % F19) - Red <= -92.8659:  Floor (4.0/1.0)
| | | | | | | | | | | | (F17 % F19) - Red > -92.8659:  Boundary (9.0)
| | | | | | | | | | | log(F15) > 2.451169:  Floor (5.0)
| | | | | | | | | | F5 > 13.0292:  Boundary (6.0/1.0)
| | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 9.757119
| | | | | | | F5 <= 1.2697
| | | | | | | | F14 <= 0.6083:  Ball (3.0)
| | | | | | | | F14 > 0.6083
| | | | | | | | | Blue <= 76.125:  Ball (5.0)
| | | | | | | | | Blue > 76.125:  Floor (2.0)
| | | | | | | F5 > 1.2697
| | | | | | | | F12 <= 3.4167
| | | | | | | | | Blue <= 113.75:  Floor (27.0/4.0)
| | | | | | | | | Blue > 113.75:  Boundary (4.0/1.0)
| | | | | | | | F12 > 3.4167
| | | | | | | | | F22 <= 4.6667
| | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 14.631055
| | | | | | | | | | F14 <= 5.6438:  Floor (10.0/1.0)
| | | | | | | | | | F14 > 5.6438
| | | | | | | | | | | (F17 % F19) - Red <= -90.9596
| | | | | | | | | | | | F25 - (Luminance % Blue) <= -7.3255:  Floor (2.0)
| | | | | | | | | | | | F25 - (Luminance % Blue) > -7.3255:  Obstacle (4.0)
| | | | | | | | | | | (F17 % F19) - Red > -90.9596:  Boundary (3.0)
| | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 14.631055
| | | | | | | | | | F31 <= 3.8333:  Obstacle (24.0/2.0)
| | | | | | | | | | F31 > 3.8333
| | | | | | | | | | | F31 <= 4.5
| | | | | | | | | | | | F25 - (Luminance % Blue) <= 8.0768:  Obstacle (4.0/1.0)
| | | | | | | | | | | | F25 - (Luminance % Blue) > 8.0768:  Chair (3.0)
| | | | | | | | | | | F31 > 4.5:  Ball (3.0/1.0)
| | | | | | | | | F22 > 4.6667
| | | | | | | | | | log(F15) <= 1.811203
| | | | | | | | | | | F31 <= 3.6667:  Boundary (3.0)
| | | | | | | | | | | F31 > 3.6667
| | | | | | | | | | | | F25 - (Luminance % Blue) <= 12.1116:  Ball (11.0/3.0)
| | | | | | | | | | | | F25 - (Luminance % Blue) > 12.1116:  Obstacle (2.0/1.0)
| | | | | | | | | | log(F15) > 1.811203
| | | | | | | | | | | F22 <= 5:  Floor (2.0)
| | | | | | | | | | | F22 > 5
| | | | | | | | | | | | F14 <= 2.6173:  Obstacle (2.0/1.0)
| | | | | | | | | | | | F14 > 2.6173
| | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 36.114522:  Boundary (13.0/1.0)
| | | | | | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 36.114522:  Chair (2.0)
| | | | | Blue > 127.125
| | | | | | log(F15) <= 2.554036
| | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 6.306645
| | | | | | | | log(F15) <= 0.855734
| | | | | | | | | F14 <= 10.006:  Obstacle (5.0)
| | | | | | | | | F14 > 10.006:  Boundary (2.0)
| | | | | | | | log(F15) > 0.855734:  Floor (21.0/2.0)
| | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 6.306645
| | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 6.916465:  Boundary (2.0)
| | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 6.916465
| | | | | | | | | F12 <= 8.0833:  Obstacle (14.0)
| | | | | | | | | F12 > 8.0833
| | | | | | | | | | F5 <= 12.2298
| | | | | | | | | | | F14 <= 0.6083:  Chair (2.0)
| | | | | | | | | | | F14 > 0.6083:  Boundary (3.0/1.0)
| | | | | | | | | | F5 > 12.2298:  Obstacle (6.0)
| | | | | | log(F15) > 2.554036
| | | | | | | F14 <= 14.3457
| | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 14.352685
| | | | | | | | | Blue <= 222.75:  Boundary (64.0)
| | | | | | | | | Blue > 222.75:  Chair (2.0/1.0)
| | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 14.352685
| | | | | | | | | F22 <= 6.5:  Obstacle (7.0/1.0)
| | | | | | | | | F22 > 6.5
| | | | | | | | | | F25 - (Luminance % Blue) <= 15.1172:  Boundary (4.0/1.0)
| | | | | | | | | | F25 - (Luminance % Blue) > 15.1172:  Chair (3.0)
| | | | | | | F14 > 14.3457:  Floor (3.0/1.0)
| | | Green / Red > 1.139161
| | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 38.425087
| | | | | Green / Red <= 1.163297
| | | | | | F31 <= 5
| | | | | | | log(F15) <= 1.453135
| | | | | | | | F12 <= 3:  Floor (3.0/1.0)
| | | | | | | | F12 > 3:  Ball (6.0)
| | | | | | | log(F15) > 1.453135:  Floor (3.0)
| | | | | | F31 > 5:  Ball (12.0)
| | | | | Green / Red > 1.163297:  Ball (110.0)
| | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 38.425087
| | | | | F14 <= 3.1174:  Ball (4.0)
| | | | | F14 > 3.1174
| | | | | | F12 <= 28.0833:  Obstacle (3.0)
| | | | | | F12 > 28.0833:  Boundary (2.0)
| (F17 % F19) - Red > -60.0039
| | Green / Red <= 1.185233
| | | (F17 % F19) - Red <= -48.4537
| | | | Blue <= 43.375
| | | | | F22 <= 4.75
| | | | | | F22 <= 3.25:  Box (2.0)
| | | | | | F22 > 3.25:  Obstacle (3.0)
| | | | | F22 > 4.75
| | | | | | F14 <= 4.2705:  Ball (2.0)
| | | | | | F14 > 4.2705:  Boundary (3.0)
| | | | Blue > 43.375
| | | | | F5 <= 17.387
| | | | | | F14 <= 10.2436
| | | | | | | F5 <= 14.1551
| | | | | | | | F25 - (Luminance % Blue) <= 4.8067
| | | | | | | | | F25 - (Luminance % Blue) <= 3.5762
| | | | | | | | | | F22 <= 4.8333
```

```
| | | | | | | | | | | F22 <= 3.25
| | | | | | | | | | | | log(F15) <= 2.048209
| | | | | | | | | | | | | Blue <= 51.875:  Floor (2.0)
| | | | | | | | | | | | | Blue > 51.875:  Ball (3.0)
| | | | | | | | | | | | log(F15) > 2.048209:  Floor (5.0)
| | | | | | | | | | | F22 > 3.25
| | | | | | | | | | | | (F17 % F19) - Red <= -56.5846:  Boundary (3.0)
| | | | | | | | | | | | (F17 % F19) - Red > -56.5846:  Floor (9.0/1.0)
| | | | | | | | | | F22 > 4.8333
| | | | | | | | | | | Green / Red <= 1.090123:  Boundary (2.0)
| | | | | | | | | | | Green / Red > 1.090123:  Ball (2.0)
| | | | | | | | | | F25 - (Luminance % Blue) > 3.5762:  Obstacle (3.0/1.0)
| | | | | | | | | F25 - (Luminance % Blue) > 4.8067
| | | | | | | | | | F5 <= 4.84:  Floor (6.0/1.0)
| | | | | | | | | F5 > 4.84
| | | | | | | | | | F12 <= 1.4167
| | | | | | | | | | | F5 <= 11.3671:  Boundary (2.0)
| | | | | | | | | | | F5 > 11.3671:  Floor (2.0)
| | | | | | | | | | F12 > 1.4167:  Boundary (19.0)
| | | | | | | | F5 > 14.1551
| | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 8.071426:  Floor (17.0/1.0)
| | | | | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 8.071426:  Ball (2.0)
| | | | | | F14 > 10.2436
| | | | | | | log(F15) <= -0.759714:  Floor (2.0)
| | | | | | | log(F15) > -0.759714:  Boundary (34.0/3.0)
| | | | | F5 > 17.387:  Boundary (26.0/1.0)
| | | (F17 % F19) - Red > -48.4537
| | | | F5 <= 1.1632
| | | | | F14 <= 5.1957:  Ball (5.0)
| | | | | F14 > 5.1957:  Boundary (45.0)
| | | | F5 > 1.1632
| | | | | F12 <= 7.8333:  Boundary (777.0/22.0)
| | | | | F12 > 7.8333
| | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 <= 48.439514:  Boundary (10.0)
| | | | | | ((F10 / F30) - (F29 / Luminance)) + F11 > 48.439514
| | | | | | | F14 <= 4.9016:  Floor (2.0)
| | | | | | | F14 > 4.9016:  Obstacle (2.0/1.0)
| | Green / Red > 1.185233
| | | F5 <= 16.0342
| | | | log(F15) <= 3.182793
| | | | | F5 <= 9.4512:  Ball (28.0)
| | | | | F5 > 9.4512
| | | | | | F5 <= 11.3671:  Boundary (2.0)
| | | | | | F5 > 11.3671:  Ball (9.0)
| | | | log(F15) > 3.182793:  Boundary (3.0)
| | | F5 > 16.0342
| | | | F25 - (Luminance % Blue) <= -41.0078:  Ball (2.0)
| | | | F25 - (Luminance % Blue) > -41.0078:  Boundary (9.0)
```

Appendix B

Ensemble Analysis

B.1 Analysis of genotypes grouped by class specificity / sensitivity

(results based on 10-fold evaluation on train dataset)

Genotype count: 20
Accuracy: 0.9233 S.D.: 0.01
Nodes: 30.8 Trees: 16
QScore: 0.8807

Classifier counts:
weka.classifiers.IBk: 14
weka.classifiers.j48.J48: 6

Classification breakdown (averages):

Class	sensitivity	specificity
Obstacle	0.8806	0.981
Boundary	0.9099	0.9825
Floor	0.9006	0.9728
Chair	0.9608	0.992
Ball	0.9296	0.9886
Box	0.9589	0.9911

—-
Genotypes most SPECIFIC to 'Obstacle':

There are no genotypes more specific to this class than any other

Genotypes most SENSITIVE to 'Obstacle':
There are no genotypes more sensitive to this class than any other
—-
Genotypes most SPECIFIC to 'Boundary':

There are no genotypes more specific to this class than any other

Genotypes most SENSITIVE to 'Boundary':
There are no genotypes more sensitive to this class than any other
—-

Genotypes most SPECIFIC to 'Floor':
There are no genotypes more specific to this class than any other

Genotypes most SENSITIVE to 'Floor':
There are no genotypes more sensitive to this class than any other
—-

Genotypes most SPECIFIC to 'Chair':
Genotype count: 11
Accuracy: 0.9202 S.D.: 0.0116
Nodes: 29.9091 Trees: 14.3636
QScore: 0.8814

Classifier counts:
weka.classifiers.IBk: 5
weka.classifiers.j48.J48: 6

Classification breakdown (averages):

Class	sensitivity	specificity
Obstacle	0.887	0.9794
Boundary	0.8996	0.9798
Floor	0.8981	0.9736
Chair	0.9599	0.9928
Ball	0.923	0.9884
Box	0.9544	0.9902

Analysis of nodes appearing in >1 genotype:

node	genotypes	entire trees	nodes	average accuracy
Red	11	13	31	0.92023
Green	11	6	27	0.92023
F14	10	11	14	0.921929
F11	10	8	10	0.919578
Blue	9	9	21	0.920977
Luminance	9	1	16	0.920994
F23	8	6	9	0.922296
F5	7	6	10	0.923393
F13	7	5	7	0.92044
F18	7	4	8	0.919413
F24	7	2	10	0.923521
F31	6	5	7	0.919862
F15	5	5	5	0.924566
F12	5	3	6	0.926273
F21	5	3	6	0.917825
F8	5	3	5	0.924386
F9	5	2	5	0.927621
F26	5	1	5	0.916417
F20	4	2	5	0.927839
F22	4	2	4	0.914582
F6	4	1	4	0.929187
(Green - Luminance)	3	3	3	0.919613
F27	3	2	3	0.920811
F25	3	1	5	0.923357
F10	3	1	4	0.908328
F19	3	0	4	0.919563
F7	2	3	4	0.937313
(F19 / Red)	2	2	2	0.922259
(Luminance - Green)	2	2	2	0.916267
(Luminance / Red)	2	2	2	0.916043
(log:F14)	2	1	2	0.934017

genotypes:
weka.classifiers.IBk: [Blue] [F26] [(Green / Red)] [Red] [F31] [F11] [F21] [F15] [F18] [F27] [F20] [F5] [F14] [Red] [F23] [(((F9 % Red) + Red) / Green)] [(Blue - F29)] [(Green % Blue)] [Blue]
weka.classifiers.j48.J48: [((F10 * F9) * (log:F22))] [Blue] [F14] [F13] [(Green - Red)] [Green] [(F18 / (F8 / (Blue - F26)))] [F11] [(Green - Luminance)] [F23] [F20]
weka.classifiers.j48.J48: [(F11 * F12)] [(Blue / Red)] [(F10 - (F24 % F26))] [F14] [F22] [F6] [Green] [(Green / Luminance)] [F31]
weka.classifiers.IBk: [((F24 - F16) * F23)] [Red] [(Green % Red)] [((Luminance % Blue) - (F6 - (log:(F26 - F12))))] [F12] [F31] [F9] [F8] [F21] [F13] [F5] [(log:F14)] [(Blue / Luminance)] [F23] [(Green % Red)]
weka.classifiers.IBk: [F18] [Red] [F11] [(F20 + Green)] [F5] [F22] [(F7 - F24)] [(Green - Luminance)] [F12] [F14] [F7] [((log:(F6 % F5)) + F23)] [Red] [(Luminance % Green)] [F9] [(Blue % Green)] [F8] [F13] [Blue]
weka.classifiers.j48.J48: [F24] [(Luminance - Green)] [F11] [F14] [((log:F18) % (F12 - F25))] [(Red / Luminance)] [Green] [F31] [((((F5 / F21) + F21) / F31)]
weka.classifiers.IBk: [Red] [F10] [Red] [F11] [F13] [Green] [Red] [Green] [(((F22 / (F10 - Green)) + F23)] [(F21 + F26)] [Blue] [(F19 / Red)] [F5] [Blue] [Blue]
weka.classifiers.j48.J48: [(Green - Luminance)] [Red] [F11] [F24] [(Red - Luminance)] [F23] [F14] [F14] [F18] [Red] [Red] [F14] [F15]
weka.classifiers.j48.J48: [F31] [((F24 + F25) + F25)] [(F8 - F24)] [F14] [((F30 / (F18 * Red)) / (log:F13))] [F11] [F5] [(Luminance / Red)] [Luminance] [F15] [F18] [((Blue / Green) % F5)] [(((F19 + F5) + F27)] [F21]
weka.classifiers.IBk: [(((F25 - F24) + (F6 - (F24 % (F19 * F24))))] [F14] [Red] [((F20 + F28) + (F20 + F28))] [Red] [F5] [F11] [F13] [F12] [(Blue % Luminance)] [F7] [(Red - Green)] [(Red - Green)] [Blue] [F8] [((log:F14) * F9)] [F25] [((Red - Green) + F14)] [Blue] [(F19 / Red)] [(Red - Green)] [F7] [F15] [F23]
weka.classifiers.j48.J48: [F27] [(F31 - (F11 * Luminance))] [(Luminance / Red)] [((F13 / Blue) / Blue)] [F14] [(Luminance - Green)] [F23] [(Blue + F18)] [F15] [Green]

Genotypes most SENSITIVE to 'Chair':
Genotype count: 10
Accuracy: 0.9195 S.D.: 0.0085
Nodes: 26.3 Trees: 14.1
QScore: 0.8725

Classifier counts:
weka.classifiers.IBk: 5
weka.classifiers.j48.J48: 5

Classification breakdown (averages):

Class	sensitivity	specificity
Obstacle	0.8818	0.9796
Boundary	0.8985	0.9803
Floor	0.896	0.973
Chair	0.9648	0.9921
Ball	0.9234	0.9878
Box	0.953	0.9905

Analysis of nodes appearing in >1 genotype:

node	genotypes	entire trees	nodes	average accuracy
Red	10	12	24	0.919458
Green	10	10	21	0.919458
F14	8	9	10	0.920929
Blue	8	8	18	0.920106
Luminance	8	1	14	0.920742
F5	7	8	11	0.920312
F31	7	6	8	0.919734
F23	6	5	7	0.921085
F15	6	5	6	0.922434
F11	6	4	6	0.91407
F24	6	2	7	0.920686
F12	5	3	7	0.923247
F18	5	3	6	0.917945
F13	5	2	5	0.919233
F26	5	1	5	0.920971
F25	5	0	6	0.922439
F10	4	3	5	0.919151
F21	4	2	5	0.914694
F22	4	2	4	0.916979
F8	4	2	4	0.922558
F19	4	1	4	0.918851
F6	3	2	3	0.922009
F7	3	1	3	0.927801
(Green / Luminance)	2	2	2	0.919638
(Luminance - Green)	2	2	2	0.916267
(Luminance / Red)	2	2	2	0.916043
(Red % Blue)	2	2	2	0.924655
F9	2	2	2	0.92533
(log:F13)	2	1	2	0.924131
F27	2	1	2	0.916043
F30	2	0	2	0.924131

genotypes:
weka.classifiers.j48.J48: [(F11 * F12)] [(Blue / Red)] [(F10 - (F24 % F26))] [F14] [F22] [F6] [Green] [(Green / Luminance)] [F31]
weka.classifiers.IBk: [Green] [Red] [Green] [Blue] [(log:F13)] [F5] [(F7 % Green)] [F12] [(Luminance / Green)] [F10] [(log:F24)] [(F25 + F30)] [F14] [F15] [Blue] [(log:F26)] [F5] [F23] [Red] [F19] [(Blue % Red)]
weka.classifiers.IBk: [((F24 - F16) * F23)] [Red] [(Green % Red)] [((Luminance % Blue) - (F6 - (log:(F26 - F12))))] [F12] [F31] [F9] [F8] [F21] [F13] [F5] [(log:F14)] [(Blue / Luminance)] [F23] [(Green % Red)]
weka.classifiers.j48.J48: [F24] [(Luminance - Green)] [F11] [F14] [((log:F18) % (F12 - F25))] [(Red / Luminance)] [Green] [F31] [(((F5 / F21) + F21) / F31)]
weka.classifiers.IBk: [Red] [F10] [Red] [F11] [F13] [Green] [Red] [Green] [((F22 / (F10 - Green)) + F23)] [(F21 + F26)] [Blue] [(F19 / Red)] [F5] [Blue] [Blue]
weka.classifiers.j48.J48: [(Green - Luminance)] [Red] [F11] [F24] [(Red - Luminance)] [F23] [F14] [F14] [F18] [Red] [Red] [F14] [F15]
weka.classifiers.j48.J48: [F31] [((F24 + F25) + F25)] [(F8 - F24)] [F14] [((F30 / (F18 * Red)) / (log:F13))] [F11] [F5] [(Luminance / Red)] [Luminance] [F15] [F18] [((Blue / Green) % F5)] [((F19 + F5) + F27)] [F21]
weka.classifiers.j48.J48: [F27] [(F31 - (F11 * Luminance))] [(Luminance / Red)] [((F13 / Blue) / Blue)] [F14] [(Luminance - Green)] [F23] [(Blue + F18)] [F15] [Green]
weka.classifiers.IBk: [F7] [F18] [Red] [F10] [F6] [Green] [(F22 * F25)] [Blue] [(Red % Blue)] [F5] [(Green / Luminance)] [(F19 * F8)] [F9] [F31] [F20] [F15]
weka.classifiers.IBk: [F26] [F5] [F31] [(log:(log:F25))] [F14] [F8] [F22] [F5] [Red] [(Red % Blue)] [F23] [Green] [F12] [Blue] [Green] [Red] [Blue] [(log:F15)] [(F7 - F12)]

—-

Genotypes most SPECIFIC to 'Ball':
Genotype count: 1
Accuracy: 0.9299 S.D.: ?
Nodes: 26 Trees: 14

Classifier counts:
weka.classifiers.IBk: 1

Classification breakdown (averages):

Class	sensitivity	specificity
Obstacle	0.8713	0.9845
Boundary	0.9308	0.9863
Floor	0.9291	0.9727
Chair	0.9578	0.9908
Ball	0.9299	0.991
Box	0.9613	0.9905

genotypes:
weka.classifiers.IBk: [(Luminance / Red)] [Blue] [F5] [Red] [(F26 / Luminance)] [((F9 - F27) - F25)] [F11] [(F22 * F24)] [F13] [F18] [F23] [F14] [(Blue / Luminance)] [Green]

Genotypes most SENSITIVE to 'Ball':
There are no genotypes more sensitive to this class than any other
—-

Genotypes most SPECIFIC to 'Box':
Genotype count: 8
Accuracy: 0.9267 S.D.: 0.0067
Nodes: 32.625 Trees: 18.5
QScore: 0.9041

Classifier counts:
weka.classifiers.IBk: 8

Classification breakdown (averages):

Class	sensitivity	specificity
Obstacle	0.873	0.9827
Boundary	0.9215	0.9859
Floor	0.9004	0.9717
Chair	0.9625	0.991
Ball	0.9388	0.9885
Box	0.9649	0.9923

Analysis of nodes appearing in >1 genotype:

node	genotypes	entire trees	nodes	average accuracy
Blue	8	13	22	0.926678
F5	8	13	15	0.926678
Red	8	11	25	0.926678
Green	8	10	18	0.926678
F23	7	7	8	0.927074
F14	7	6	7	0.927074
F15	6	4	6	0.92503
F25	6	1	7	0.928001
F7	5	4	6	0.92891
F12	5	3	7	0.927112
F9	5	2	5	0.925075
F26	5	1	6	0.926693
Luminance	5	0	9	0.929329
F31	4	4	4	0.927127
F13	4	3	5	0.93016
F11	4	2	5	0.924356
F20	4	2	4	0.923195
F16	4	0	4	0.924356
F30	4	0	4	0.926228
F18	3	3	3	0.929049
F27	3	2	3	0.928201
F28	3	2	3	0.930597
F8	3	1	5	0.920711
F22	3	1	3	0.920711
F24	3	1	3	0.930048
(Red % Blue)	2	2	2	0.924655
F10	2	2	2	0.928999
(log:F25)	2	1	2	0.928026
F19	2	1	2	0.928999
F6	2	1	2	0.92825

genotypes:
weka.classifiers.IBk: [F5] [Blue] [(Red - F30)] [Green] [(F16 * F24)] [Blue] [F23] [(F15 + Red)] [(F11 + F9)] [(F27 + Red)] [((F20 + F25) * F25)] [(Luminance - Red)] [F13] [F14]
weka.classifiers.IBk: [Green] [Red] [Green] [Blue] [(log:F13)] [F5] [(F7 % Green)] [F12] [(Luminance / Green)] [F10] [(log:F24)] [(F25 + F30)] [F14] [F15] [Blue] [(log:F26)] [F5] [F23] [Red] [F19] [(Blue % Red)]
weka.classifiers.IBk: [F9] [Blue] [Blue] [Blue] [F15] [F5] [Green] [Green] [F20] [Red] [(F8 / (log:F5))] [((F11 / ((F11 / F12) / F26)) / F26)] [Red] [F14] [Blue] [((F8 % ((F16 - (F22 + F8)) * F30)) / (log:F5))] [F23]
weka.classifiers.IBk: [Red] [F5] [F7] [F13] [F5] [F11] [Red] [Blue] [F14] [F15] [(F26 % F16)] [F5] [Blue] [F23] [(Green % Red)] [F25] [F27] [F28] [F5] [Green] [F31]
weka.classifiers.IBk: [(Blue - Red)] [(Blue % Luminance)] [(Blue - Red)] [(Green % Luminance)] [F5] [(Green % Luminance)] [(log:F25)] [Red] [((log:F16) * F29)] [F11] [F23] [((log:F9) * F12)] [(F29 / (F23 / F28))] [F18] [(Green % Luminance)] [F27] [Red] [(Red % F20)] [F31] [F24] [F14]
weka.classifiers.IBk: [F7] [F18] [Red] [F10] [F6] [Green] [(F22 * F25)] [Blue] [(Red % Blue)] [F5] [(Green / Luminance)] [(F19 * F8)] [F9] [F31] [F20] [F15]
weka.classifiers.IBk: [F12] [F7] [F18] [F5] [F28] [F23] [(log:Red)] [(Green % Blue)] [(F13 + F9)] [Green] [F13] [(Luminance / Red)] [(Blue - F30)] [((log:F17) + F17)] [(Luminance / Red)] [((F26 - F6) + F12)] [(log:Blue)] [F7] [(log:F14)]
weka.classifiers.IBk: [F26] [F5] [F31] [(log:(log:F25))] [F14] [F8] [F22] [F5] [Red] [(Red % Blue)] [F23] [Green] [F12] [Blue] [Green] [Red] [Blue] [(log:F15)] [(F7 - F12)]

Genotypes most SENSITIVE to 'Box':
Genotype count: 10
Accuracy: 0.9271 S.D.: 0.0102
Nodes: 35.3 Trees: 17.9
QScore: 0.9043

Classifier counts:
weka.classifiers.IBk: 9
weka.classifiers.j48.J48: 1

Classification breakdown (averages):

Class	sensitivity	specificity
Obstacle	0.8794	0.9824
Boundary	0.9214	0.9848
Floor	0.9052	0.9726
Chair	0.9569	0.9918
Ball	0.9359	0.9893
Box	0.9649	0.9916

Analysis of nodes appearing in >1 genotype:

node	genotypes	entire trees	nodes	average accuracy
Blue	10	15	27	0.927127
Red	10	13	34	0.927127
F14	10	9	12	0.927127
F23	10	9	11	0.927127
Green	10	7	25	0.927127
F5	9	12	15	0.929432
F11	9	7	10	0.92652
F9	9	2	9	0.926969
F13	7	7	8	0.928208
F20	7	3	8	0.925747
Luminance	7	0	14	0.928507
F18	6	5	6	0.927202
F26	6	1	7	0.923432
F15	5	4	5	0.927681
F12	5	3	6	0.930138
F27	5	3	5	0.928969
F25	5	2	7	0.931156
F24	5	1	7	0.932115
F7	4	6	7	0.933942
F8	4	2	6	0.923457
F28	4	2	5	0.933268
F22	4	1	4	0.920611
F16	4	0	4	0.924356
F31	3	3	3	0.929848
F30	3	0	3	0.923607
F6	3	0	3	0.93574
(Luminance / Red)	2	3	3	0.931246
(Blue % Luminance)	2	2	2	0.935965
(Green % Blue)	2	2	2	0.931471
(Green - Luminance)	2	2	2	0.919862
(log:F14)	2	1	2	0.936938
F29	2	0	3	0.930497

genotypes:

weka.classifiers.IBk: [Blue] [F26] [(Green / Red)] [Red] [F31] [F11] [F21] [F15] [F18] [F27] [F20] [F5] [F14] [Red] [F23] [(((F9 % Red) + Red) / Green)] [(Blue - F29)] [(Green % Blue)] [Blue]

weka.classifiers.j48.J48: [((F10 * F9) * (log:F22))] [Blue] [F14] [F13] [(Green - Red)] [Green] [(F18 / (F8 / (Blue - F26)))] [F11] [(Green - Luminance)] [F23] [F20]

weka.classifiers.IBk: [F5] [Blue] [(Red - F30)] [Green] [(F16 * F24)] [Blue] [F23] [(F15 + Red)] [(F11 + F9)] [(F27 + Red)] [((F20 + F25) * F25)] [(Luminance - Red)] [F13] [F14]

weka.classifiers.IBk: [F18] [Red] [F11] [(F20 + Green)] [F5] [F22] [(F7 - F24)] [(Green - Luminance)] [F12] [F14] [F7] [((log:(F6 % F5)) + F23)] [Red] [(Luminance % Green)] [F9] [(Blue % Green)] [F8] [F13] [Blue]

weka.classifiers.IBk: [F9] [Blue] [Blue] [Blue] [F15] [F5] [Green] [Green] [F20] [Red] [(F8 / (log:F5))] [((F11 / ((F11 / F12) / F26)) / F26)] [Red] [F14] [Blue] [((F8 % ((F16 - (F22 + F8)) * F30)) / (log:F5))] [F23]

weka.classifiers.IBk: [Red] [F5] [F7] [F13] [F5] [F11] [Red] [Blue] [F14] [F15] [(F26 % F16)] [F5] [Blue] [F23] [(Green % Red)] [F25] [F27] [F28] [F5] [Green] [F31]

weka.classifiers.IBk: [(Blue - Red)] [(Blue % Luminance)] [(Blue - Red)] [(Green % Luminance)] [F5] [(Green % Luminance)] [(log:F25)] [Red] [((log:F16) * F29)] [F11] [F23] [((log:F9) * F12)] [(F29 / (F23 / F28))] [F18] [(Green % Luminance)] [F27] [Red] [(Red % F20)] [F31] [F24] [F14]

weka.classifiers.IBk: [((F25 - F24) + (F6 - (F24 % (F19 * F24))))] [F14] [Red] [((F20 + F28) + (F20 + F28))] [Red] [F5] [F11] [F13] [F12] [(Blue % Luminance)] [F7] [(Red - Green)] [(Red - Green)] [Blue] [F8] [((log:F14) * F9)] [F25] [((Red - Green) + F14)] [Blue] [(F19 / Red)] [(Red - Green)] [F7] [F15] [F23]

weka.classifiers.IBk: [F12] [F7] [F18] [F5] [F28] [F23] [(log:Red)] [(Green % Blue)] [(F13 + F9)] [Green] [F13] [(Luminance / Red)] [(Blue - F30)] [((log:F17) + F17)] [(Luminance / Red)] [((F26 - F6) + F12)] [(log:Blue)] [F7] [(log:F14)]

weka.classifiers.IBk: [(Luminance / Red)] [Blue] [F5] [Red] [(F26 / Luminance)] [((F9 - F27) - F25)] [F11] [(F22 * F24)] [F13] [F18] [F23] [F14] [(Blue / Luminance)] [Green]

Appendix C

Sensitivity and Specificity

The notions of *Sensitivity* and *Specificity* rest on the following definitions for binary classification (e.g. where a patient either has a condition or not):

1. *True Positive*: A correct positive prediction. (A patient is correctly diagnosed as having the condition).
2. *False Positive*: An incorrect positive prediction. (A patient is diagnosed as having the condition when they do not).
3. *True Negative*: A correct negative prediction. (A patient is correctly diagnosed as being healthy).
4. *False Negative*: An incorrect negative prediction. (A patient is diagnosed as being healthy when they actually have the condition).

$$Sensitivity = \frac{true\,positives}{true\,positives + false\,negatives}$$

Sensitivity measures the ratio of people correctly diagnosed with the condition to the total number of people with the condition. A test that is 100% sensitive will always recognise a sick person, but may incorrectly diagnose healthy people.

$$Specificity = \frac{true\,negatives}{true\,negatives + false\,positives}$$

Specificity measures the ratio of people correctly diagnosed as healthy to the total number of healthy people. A test that is 100% specific will always recognise healthy people as being healthy, but may incorrectly diagnose sick people as healthy.

Bibliography

[1] H. A. Abbass. An evolutionary artificial neural networks approach for breast cancer diagnosis. *Artificial Intelligence in Medicine*, 25:265–281, 2002.

[2] A. Abraham, C. Grosan, S. Y. Han, and E. Gelbukh. Evolutionary multiobjective optimization. In A. Gelbukh, editor, *Proceedings of the Fourth Mexican International Conference on Artificial Intelligence, Mexico, Lecture Notes in Computer Science*, pages 673–681. Springer Verlag, 2005.

[3] D. W. Aha, D. Kibler, and M. K. Albert. Instance-based learning algorithms. *Machine Learning*, 6(1):37–66, January 1991.

[4] M. Ahluwalia and L. Bull. Coevolving functions in genetic programming: Classification using K-nearest-neighbour. In W. Banzhaf, J. Daida, A. E. Eiben, M. H. Garzon, V. Honavar, M. Jakiela, and R. E. Smith, editors, *Proceedings of the Genetic and Evolutionary Computation Conference*, volume 2, pages 947–952, Orlando, Florida, USA, 13-17 July 1999. Morgan Kaufmann.

[5] K. Ahmadian, A. Golestani, M. Analoui, and M. R. Jahed. Evolving ensemble of classifiers in low-dimensional spaces using multi-objective evolutionary approach. In *6th Annual IEEE/ACIS International Conference on Computer and Information Science (ICIS 2007), 11-13 July 2007, Melbourne, Australia*, pages 217–222. IEEE Computer Society, 2007.

[6] A. Al-Ani and M. Deriche. A new technique for combining multiple classifiers using the dempster-shafer theory of evidence. *Journal of Artificial Intelligence Research*, 17:333–361, 2002.

[7] L. Altenberg. The schema Theorem and Price's Theorem. In D. L. Whitley and M. D. Vose, editors, *Foundations of Genetic*

Algorithms 3, pages 23–49, Estes Park, Colorado, USA, 1995. Morgan Kaufmann.

[8] H. Altınçay. A dempster-shafer theoretic framework for boosting based ensemble design. *Pattern Anal. Appl.*, 8(3):287–302, 2005.

[9] P. J. Angeline. Genetic programming and emergent intelligence. In K. E. Kinnear, Jr., editor, *Advances in Genetic Programming*, chapter 4, pages 75–98. MIT Press, Cambridge, MA, USA, 1994.

[10] P. J. Angeline. An investigation into the sensitivity of genetic programming to the frequency of leaf selection during subtree crossover. In J. R. Koza, D. E. Goldberg, D. B. Fogel, and R. L. Riolo, editors, *Genetic Programming 1996: Proceedings of the First Annual Conference*, pages 21–29, Stanford University, CA, USA, 28–31 July 1996. MIT Press.

[11] P. J. Angeline and J. B. Pollack. Competitive environments evolve better solutions for complex tasks. In *Proceedings of the 5th International Conference on Genetic Algorithms*, pages 264–270, San Francisco, CA, USA, 1993. Morgan Kaufmann Publishers Inc.

[12] J. Bacardit and N. Krasnogor. Empirical evaluation of ensemble techniques for a pittsburgh learning classifier system. In *Learning Classifier Systems: 10th International Workshop, IWLCS 2006, Seattle, MA, USA, and 11th International Workshop, IWLCS 2007, London, UK, Revised Selected Papers*, pages 255–268, Berlin, Heidelberg, 2007. Springer-Verlag.

[13] R. E. Banfield, L. O. Hall, K. W. Bowyer, and W. P. Kegelmeyer. A new ensemble diversity measure applied to thinning ensembles. In T. Windeatt and F. Roli, editors, *Multiple Classifier Systems, 4th International Workshop, Guildford, UK, 11-13 June 2003*, pages 306–316, 2003.

[14] W. Banzhaf. Genetic programming for pedestrians. In *Proceedings of the 5th International Conference on Genetic Algorithms*, page 628, San Francisco, CA, USA, 1993. Morgan Kaufmann Publishers Inc.

[15] W. Banzhaf, F. D. Francone, and P. Nordin. The effect of extensive use of the mutation operator on generalization in genetic

programming using sparse data sets. In *PPSN IV: Proceedings of the 4th International Conference on Parallel Problem Solving from Nature*, pages 300–309, London, UK, 1996. Springer-Verlag.

[16] W. Banzhaf, P. Nordin, R. E. Keller, and F. D. Francone. *Genetic Programming – An Introduction; On the Automatic Evolution of Computer Programs and its Applications*. Morgan Kaufmann, San Francisco, CA, USA, Jan. 1998.

[17] N. Barricelli. Esempi numerici di processi di evoluzione. *Methodos*, pages 45–68, 1954.

[18] G. Batista and M. C. Monard. An analysis of four missing data treatment methods for supervised learning. *Applied Artificial Intelligence*, 17:519–533, 2003.

[19] E. Bauer and R. Kohavi. An empirical comparison of voting classification algorithms: Bagging, boosting, and variants. *Machine Learning*, 36(1-2):105–139, 1999.

[20] E. Bernad, X. Llor, J. M. Garrell, E. Arquitectura, L. Salle, and G. Significantly. Xcs and gale: a comparative study of two learning classifier. In *In Proceedings of the 4th International Workshop on Learning Classifier Systems (IWLCS-2001*, pages 115–132. Springer-Verlag, 2001.

[21] Y. Bernstein, X. Li, V. Ciesielski, and A. Song. Multiobjective parsimony enforcement for superior generalisation performance. In *Proceedings of the 2004 IEEE Congress on Evolutionary Computation*, pages 83–89, Portland, Oregon, 20-23 June 2004. IEEE Press.

[22] S. Bian and W. Wang. On diversity and accuracy of homogeneous and heterogeneous ensembles. *Int. J. Hybrid Intell. Syst.*, 4(2):103–128, 2007.

[23] C. L. Blake and C. J. Merz. UCI repository of machine learning databases. http://mlearn.ics.uci.edu/MLRepository.html, 1998.

[24] J. M. Bland and D. G. Altman. Modelling genetic algorithms with markov chains. *Annals of Mathematics and Artificial Intelligence*, 5:79–88, 1992.

[25] J. M. Bland and D. G. Altman. Statistics Notes: Logarithms. *BMJ*, 312(7032):700–, 1996.

[26] S. Bleuler, M. Brack, L. Thiele, and E. Zitzler. Multiobjective genetic programming: Reducing bloat using SPEA2. In *Proceedings of the 2001 Congress on Evolutionary Computation CEC2001*, pages 536–543, COEX, World Trade Center, 159 Samseong-dong, Gangnam-gu, Seoul, Korea, 27-30 May 2001. IEEE Press.

[27] T. Blickle. Evolving compact solutions in genetic programming: A case study. In H.-M. Voigt, W. Ebeling, I. Rechenberg, and H.-P. Schwefel, editors, *Parallel Problem Solving From Nature IV. Proceedings of the International Conference on Evolutionary Computation*, volume 1141, pages 564–573, Berlin, Germany, 22-26 1996. Springer-Verlag.

[28] E. Bloedorn and R. S. Michalski. The aq17-dci system for data-driven constructive induction and its application to the analysis of world economics. In *ISMIS '96: Proceedings of the 9th International Symposium on Foundations of Intelligent Systems*, pages 108–117, London, UK, 1996. Springer-Verlag.

[29] C. Bojarczuk, H. Lopes, and A. Freitas. Data Mining with Constrained-syntax Genetic Programming: Applications in Medical Data Sets. In *Proc Intelligent Data Analysis in Medicine and Pharmacology - a workshop at MedInfo-2001*, London, September 2001.

[30] C. C. Bojarczuk, H. S. Lopes, A. A. Freitas, and E. L. Michalkiewicz. A constrained-syntax genetic programming system for discovering classification rules: application to medical data sets. *Artificial Intelligence in Medicine*, 30(1):27–48, Jan. 2004.

[31] A. Borji. Combining heterogeneous classifiers for network intrusion detection. In I. Cervesato, editor, *Advances in Computer Science - ASIAN 2007. Computer and Network Security, 12th Asian Computing Science Conference, Doha, Qatar, December 9-11, 2007, Proceedings*, volume 4846 of *Lecture Notes in Computer Science*, pages 254–260. Springer, 2007.

[32] M. C. J. Bot. Feature extraction for the k-nearest neighbour classifier with genetic programming. In *EuroGP '01: Proceedings of the 4th European Conference on Genetic Programming*, pages 256–267, London, UK, 2001. Springer-Verlag.

[33] M. Brameier. *On Linear Genetic Programming*. PhD thesis, Fachbereich Informatik, Universität Dortmund, Germany, Feb. 2004.

[34] L. Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[35] L. Breiman. Bias, variance, and arcing classifiers. Technical Report 460, Statistics Department, University of California, 1996.

[36] L. Breiman. Arcing classifiers. *Annals of Statistics*, 26(3):801–845, 1998.

[37] L. Breiman. Random forests. *Machine Learning*, 45(1):5–32, October 2001.

[38] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Statistics/Probability Series. Wadsworth Publishing Company, Belmont, California, U.S.A., 1984.

[39] C. E. Brodley and P. E. Utgoff. Multivariate decision trees. *Mach. Learn.*, 19(1):45–77, 1995.

[40] G. Brown, J. Wyatt, R. Harris, and X. Yao. Diversity creation methods: A survey and categorisation. *Journal of Information Fusion*, 6(1):5–20, March 2005.

[41] G. Brown, X. Yao, J. Wyatt, H. Wersing, and B. Sendhoff. Exploiting ensemble diversity for automatic feature extraction. In *Proc. of the 9th International Conference on Neural Information Processing (ICONIP'02)*, pages 1786–1790, November 2002.

[42] C. Brunk and M. Pazzani. An investigation of noise-tolerant relational concept learning algorithms. In L. Birnbaum and G. Collins, editors, *Proceedings of the 8th International Workshop on Machine Learning*, pages 389–393. Morgan Kaufmann, 1991.

[43] L. Bull and T. Kovacs. *Foundations of Learning Classifier Systems: An Introduction*, volume 183, pages 1–17. Springer, June 2005.

[44] L. Bull, M. Studley, T. Bagnall, and I. Whittley. On the use of rule-sharing in learning classifier system ensembles. In *Evolutionary Computation, 2005. The 2005 IEEE Congress on*, volume 1, pages 612–617 Vol.1, 2005.

[45] T. Bylander and L. Tate. Using validation sets to avoid overfitting in adaboost. In G. Sutcliffe and R. Goebel, editors, *FLAIRS Conference*, pages 544–549. AAAI Press, 2006.

[46] A. M. P. Canuto and M. C. C. Abreu. Using fuzzy, neural and fuzzy-neural combination methods in ensembles with different levels of diversity. In J. M. de Sá, L. A. Alexandre, W. Duch, and D. P. Mandic, editors, *Artificial Neural Networks - ICANN 2007, 17th International Conference, Porto, Portugal, September 9-13, 2007, Proceedings, Part I*, volume 4668 of *Lecture Notes in Computer Science*, pages 349–359. Springer, 2007.

[47] A. Chandra and X. Yao. Ensemble learning using multi-objective evolutionary algorithms. *Journal of Mathematical Modelling and Algorithms*, 5(4):417–445, December 2006.

[48] C.-C. Chang and C.-J. Lin. *LIBSVM: a library for support vector machines*, 2001. Software available at `http://www.csie.ntu.edu.tw/~cjlin/libsvm`.

[49] C. A. C. Coello. A comprehensive survey of evolutionary-based multiobjective optimization techniques. *Knowledge and Information Systems*, 1(3):129–156, 1999.

[50] C. A. C. Coello. An updated survey of evolutionary multiobjective optimization techniques: State of the art and future trends. In P. J. Angeline, Z. Michalewicz, M. Schoenauer, X. Yao, and A. Zalzala, editors, *Proceedings of the Congress on Evolutionary Computation*, volume 1, pages 3–13, Mayflower Hotel, Washington D.C., USA, 6-9 1999. IEEE Press.

[51] W. W. Cohen. Efficient pruning methods for separate-and-conquer rule learning systems. In R. Bajcsy, editor, *Proceedings of*

the 13th International Joint Conference on Artificial Intelligence. Chambry, France, August 28 -September 3, 1993, pages 988–994. Morgan Kaufmann, 1993.

[52] T. F. Covoes, E. R. Hruschka, L. N. de Castro, and Á. M. Santos. A cluster-based feature selection approach. In E. Corchado, X. Wu, E. Oja, Á. Herrero, and B. Baruque, editors, *Hybrid Artificial Intelligence Systems, 4th International Conference, HAIS 2009*, volume 5572 of *Lecture Notes in Computer Science*, pages 169–176. Springer, 2009.

[53] N. Cristianini and J. Shawe-Taylor. *An Introduction to Support Vector Machines and Other Kernel-based Learning Methods.* Cambridge University Press, March 2000.

[54] E. D. de Jong, R. A. Watson, and J. B. Pollack. Reducing bloat and promoting diversity using multi-objective methods. In L. Spector, E. D. Goodman, A. Wu, W. B. Langdon, H.-M. Voigt, M. Gen, S. Sen, M. Dorigo, S. Pezeshk, M. H. Garzon, and E. Burke, editors, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2001)*, pages 11–18, San Francisco, California, USA, 7-11 July 2001. Morgan Kaufmann.

[55] K. Deb. *Multi-Objective Optimization using Evolutionary Algorithms.* Wiley-Interscience Series in Systems and Optimization. John Wiley & Sons, Chichester, 2001.

[56] K. Deb, S. Agrawal, A. Pratab, and T. Meyarivan. A Fast Elitist Non-Dominated Sorting Genetic Algorithm for Multi-Objective Optimization: NSGA-II. In M. Schoenauer, K. Deb, G. Rudolph, X. Yao, E. Lutton, J. J. Merelo, and H.-P. Schwefel, editors, *Proceedings of the Parallel Problem Solving from Nature VI Conference*, pages 849–858, Paris, France, 2000. Springer. Lecture Notes in Computer Science No. 1917.

[57] G. DeJong. Toward robust real-world inference: A new perspective on explanation-based learning. In J. Fürnkranz, T. Scheffer, and M. Spiliopoulou, editors, *ECML 2006, 17th European Conference on Machine Learning, Berlin, Germany, September 18-22, 2006, Proceedings*, volume 4212 of *Lecture Notes in Computer Science*, pages 102–113. Springer, 2006.

[58] T. Dietterich. An experimental comparison of three methods for constructing ensembles of decision trees: Bagging, boosting, and randomization. In *Bagging, boosting, and randomization. Machine Learning*, volume 40, pages 139–157, 1998.

[59] S. Dignum and R. Poli. Generalisation of the limiting distribution of program sizes in tree-based genetic programming and analysis of its effects on bloat. In *GECCO '07: Proceedings of the 9th annual conference on Genetic and evolutionary computation*, pages 1588–1595, London, England, 2007. ACM.

[60] A. Ekart and A. Markus. Using genetic programming and decision trees for generating structural descriptions of four bar mechanisms. *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, 17(3):205–220, 2003.

[61] R. Esposito and L. Saitta. Monte Carlo Theory as an Explanation of Bagging and Boosting. In G. Gottlob and T. Walsh, editors, *Proceeding of the Eighteenth International Joint Conference on Artificial Intelligence*, pages 499–504. Morgan Kaufman Publishers, 2003.

[62] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. From data mining to knowledge discovery in databases. *AI Magazine*, 17:37–54, 1996.

[63] U. M. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. From data mining to knowledge discovery: an overview. In U. M. Fayyad, G. Piatetsky-Shapiro, P. Smyth, and R. Uthurusamy, editors, *Advances in knowledge discovery and data mining*, pages 1–34. American Association for Artificial Intelligence, Menlo Park, CA, USA, 1996.

[64] L. J. Fogel, A. J. Owens, and M. J. Walsh. *Artificial Intelligence through Simulated Evolution*. John Wiley, New York, USA, 1966.

[65] G. Folino, C. Pizzuti, and G. Spezzano. Ensemble techniques for parallel genetic programming based classifiers. In C. Ryan, T. Soule, M. Keijzer, E. Tsang, R. Poli, and E. Costa, editors, *Genetic Programming, Proceedings of EuroGP'2003*, volume 2610 of *LNCS*, pages 59–69, Essex, 14-16 Apr. 2003. Springer-Verlag.

[66] G. Folino, C. Pizzuti, and G. Spezzano. Training distributed gp ensemble with a selective algorithm based on clustering and pruning for pattern classification. *IEEE Trans. Evolutionary Computation*, 12(4):458–468, 2008.

[67] C. M. Fonseca and P. J. Fleming. An overview of evolutionary algorithms in multiobjective optimization. *Evolutionary Computation*, 3(1):1–16, 1995.

[68] Y. Freund and R. E. Schapire. Experiments with a new boosting algorithm. In *International Conference on Machine Learning*, pages 148–156, 1996.

[69] N. Friedman, D. Geiger, M. Goldszmidt, G. Provan, P. Langley, and P. Smyth. Bayesian network classifiers. In *Machine Learning*, pages 131–163, 1997.

[70] G. Fung, M. Dundar, J. Bi, and B. Rao. A fast iterative algorithm for fisher discriminant using heterogeneous kernels. In *ICML '04: Proceedings of the twenty-first international conference on Machine learning*, page 40, New York, NY, USA, 2004. ACM.

[71] D. B. F.Z. Brill and W. Martin. Fast genetic selection of features for neural network classifiers. *IEEE Transactions on Neural Networks*, 3(2):324–328, 1992.

[72] C. Gagné, M. Sebag, M. Schoenauer, and M. Tomassini. Ensemble learning for free with evolutionary algorithms? In *GECCO '07: Proceedings of the 9th annual conference on Genetic and evolutionary computation*, pages 1782–1789, London, England, 2007. ACM.

[73] V. Garcia, E. Debreuve, and M. Barlaud. Fast k nearest neighbor search using gpu. In *CVPR Workshop on Computer Vision on GPU*, Anchorage, Alaska, USA, June 2008.

[74] A. L. Garcia-Almanza and E. P. K. Tsang. Simplifying decision trees learned by genetic programming. In *Proceedings of the 2006 IEEE Congress on Evolutionary Computation*, pages 7906–7912, Vancouver, 6-21 July 2006. IEEE Press.

[75] D. Gavrilis, I. G. Tsoulos, and E. Dermatas. Selecting and constructing features using grammatical evolution. *Pattern Recogn. Lett.*, 29(9):1358–1365, 2008.

[76] R. P. Gorman and T. J. Sejnowski. Analysis of hidden units in a layered network trained to classify sonar targets. *Neural Networks*, 1:75, 1988.

[77] C. Grosan and A. Abraham. Stock market modeling using genetic programming ensembles. In N. Nedjah, A. Abraham, and L. de Macedo Mourelle, editors, *Genetic Systems Programming: Theory and Experiences*, volume 13 of *Studies in Computational Intelligence*, pages 133–148. Springer, Germany, 2006.

[78] C. Guerra-Salcedo and D. Whitley. Genetic approach to feature selection for ensemble creation. In *In Proc. of Genetic and Evolutionary Computation Conference*, pages 236–243. Morgan Kaufmann, 1999.

[79] I. Guyon, S. Gunn, M. Nikravesh, and L. A. Zadeh. *Feature Extraction: Foundations and Applications (Studies in Fuzziness and Soft Computing)*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 2006.

[80] W. D. Hillis. Co-evolving parasites improve simulated evolution as an optimization procedure. *Physica D: Nonlinear Phenomena*, 42(1-3):228–234, 1990.

[81] T. K. Ho. The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20(8):832–844, 1998.

[82] V. J. Hodge and J. Austin. A survey of outlier detection methodologies. *Artificial Intelligence Review*, 22:2004, 2004.

[83] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[84] J.-H. Hong and S.-B. Cho. Ensemble genetic programming for classifying gene expression data. In R. I. Mckay and S.-B. Cho, editors, *Proceedings of The Second Asian-Pacific Workshop on Genetic Programming*, Cairns, Australia, 6-7 Dec. 2004.

[85] K. B. Irani, J. Cheng, U. M. Fayyad, and Z. Qian. Applying machine learning to semiconductor manufacturing. *IEEE Expert: Intelligent Systems and Their Applications*, 8(1):41–47, 1993.

[86] M. M. Islam, X. Yao, and K. Murase. A constructive algorithm for training cooperative neural network ensembles. *IEEE Transactions on Neural Networks*, 14:820–834, 2003.

[87] U. Johansson, T. Lofstrom, R. Konig, and L. Niklasson. Building neural network ensembles using genetic programming. In G. G. Yen, L. Wang, P. Bonissone, and S. M. Lucas, editors, *Proceedings of the 2006 IEEE Congress on Evolutionary Computation*, pages 2239–2244, Vancouver, 6-21 July 2006. IEEE Press.

[88] G. H. John and P. Langley. Estimating continuous distributions in bayesian classifiers. In P. Besnard and S. Hanks, editors, *Proceedings of the Eleventh Annual Conference on Uncertainty in Artificial Intelligence, August 18-20, 1995, Montreal, Quebec, Canada*, pages 338–345, 1995.

[89] D. Kanevskiy and K. Vorontsov. Cooperative coevolutionary ensemble learning. In M. Haindl, J. Kittler, and F. Roli, editors, *MCS*, volume 4472 of *Lecture Notes in Computer Science*, pages 469–478. Springer, 2007.

[90] Y. Kim, W. N. Street, and F. Menczer. Feature selection in data mining. In *Data mining: opportunities and challenges*, pages 80–105. IGI Publishing, Hershey, PA, USA, 2003.

[91] Y. Kim, W. N. Street, and F. Menczer. Optimal ensemble construction via meta-evolutionary ensembles. *Expert Syst. Appl.*, 30(4):705–714, 2006.

[92] K. E. Kinnear, Jr. Evolving a sort: Lessons in genetic programming. In *Proceedings of the 1993 International Conference on Neural Networks*, pages 881–888. IEEE Press, 1993.

[93] K. E. Kinnear, Jr. Fitness landscapes and difficulty in genetic programming. In *Proceedings of the 1994 IEEE World Conference on Computational Intelligence*, volume 1, pages 142–147, Orlando, Florida, USA, 27-29 June 1994. IEEE Press.

[94] R. Kohavi and G. H. John. Wrappers for feature subset selection. *Artif. Intell.*, 97(1-2):273–324, 1997.

[95] I. Kononenko. Estimating attributes: Analysis and extensions of RELIEF. In F. Bergadano and L. D. Raedt, editors, *Machine Learning: ECML-94, European Conference on Machine Learning, Catania, Italy, April 6-8, 1994, Proceedings*, pages 171–182. Springer Verlag, 1994.

[96] J. Koza. *Genetic programming II: Automatic discovery of reusable programs.* Massachusetts Institute of Technology, Cambridge, Massachusetts, 1994.

[97] J. R. Koza. *Genetic Programming: On the Programming of Computers by Means of Natural Selection.* MIT Press, Cambridge, MA, USA, 1992.

[98] J. R. Koza. Evolving the architecture of a multi-part program in genetic programming using architecture-altering operations. In *Evolutionary Programming IV: Proceedings of the Fourth Annual Conference on Evolutionary Programming*, pages 695–717. The MIT Press, 1995.

[99] J. R. Koza, M. A. Keane, M. J. Streeter, W. Mydlowec, J. Yu, and G. Lanza. *Genetic Programming IV: Routine Human-Competitive Machine Intelligence.* Kluwer Academic Publishers, 2003.

[100] K. Krawiec. Genetic programming-based construction of features for machine learning and knowledge discovery tasks. *Genetic Programming and Evolvable Machines*, 3(4):329–343, 2002.

[101] W. Krzanowski and D. Partridge. Software diversity: practical statistics for its measurement and exploitation. *Information and Software Technology*, 39:39–707, 1997.

[102] L. Kuncheva, C. Whitaker, C. Shipp, and R. Duin. Is independence good for combining classifiers. In *15th International Conference on Pattern Recognition*, pages 168–171, 2000.

[103] L. I. Kuncheva and L. C. Jain. Designing classifier fusion systems by genetic algorithms. *IEEE Transactions On Evolutionary Computation*, 4:327–336, 2000.

[104] L. I. Kuncheva and C. J. Whitaker. Measures of diversity in classifier ensembles and their relationship with the ensemble accuracy. *Machine Learning*, 51(2):181–207, May 2003.

[105] K. Lakshmi and S. Mukherjee. An improved feature selection using maximized signal to noise ratio technique for TC. In *ITNG '06: Proceedings of the Third International Conference on Information Technology: New Generations*, pages 541–546, Washington, DC, USA, 2006. IEEE Computer Society.

[106] W. B. Langdon. Size fair and homologous tree genetic programming crossovers. In *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO 1999), 13-17 July 1999, Orlando, Florida, USA*, pages 95–119. Morgan Kaufmann, 1999.

[107] W. B. Langdon and B. F. Buxton. Genetic programming for improved receiver operating characteristics. In *MCS '01: Proceedings of the Second International Workshop on Multiple Classifier Systems*, pages 68–77, London, UK, 2001. Springer-Verlag.

[108] W. B. Langdon and R. Poli. Fitness causes bloat. In P. K. Chawdhry, R. Roy, and R. K. Pan, editors, *Second On-line World Conference on Soft Computing in Engineering Design and Manufacturing*, pages 13–22. Springer-Verlag London, July 1997.

[109] W. B. Langdon, T. Soule, R. Poli, and J. A. Foster. The evolution of size and shape. In L. Spector, W. B. Langdon, U.-M. O'Reilly, and P. J. Angeline, editors, *Advances in Genetic Programming 3*, chapter 8, pages 163–190. MIT Press, Cambridge, MA, USA, June 1999.

[110] R. B. Lawrence, L. O. Hall, K. W. Bowyer, D. Bhadoria, and W. P. K. Andsteveneschrich. A comparison of ensemble creation techniques. In *In The Fifth International Conference on Multiple Classifier Systems*, pages 223–232. Springer, 2004.

[111] J. R. Levenick. Inserting introns improves genetic algorithm success rate: Taking a cue from biology. In *Proceedings of the Fourth International Conference on Genetic Algorithms*, pages 123–127. Morgan Kaufmann, 1991.

[112] H. Liu, J. Li, and L. Wong. A comparative study on feature selection and classification methods using gene expression profiles and proteomic patterns. *Genome Informatics*, 13:51–60, 2002.

[113] H. Liu and H. Motoda. Feature transformation and subset selection. *IEEE Intelligent Systems*, 13(2):26–28, 1998.

[114] H. Liu and H. Motoda. *Instance Selection and Construction for Data Mining*. Kluwer Academic Publishers, Norwell, MA, USA, 2001.

[115] H. Liu and H. Motoda, editors. *Computational Methods of Feature Selection*. Data Mining and Knowledge Discovery. Chapman & Hall/CRC, 1 edition, October 2007.

[116] H. Liu and R. Setiono. Feature transformation and multivariate decision tree induction. In *Proc. of the First International Conference on Discovery Science (DS'98*, pages 279–290. Springer Verlag, 1998.

[117] Y. Liu and X. Yao. Ensemble learning via negative correlation. *Neural Networks*, 12:1399–1404, 1999.

[118] Y. Liu, X. Yao, and T. Higuchi. Evolutionary ensembles with negative correlation learning. *IEEE Transactions on Evolutionary Computation*, 4:380–387, 2000.

[119] E. Mandler and J. Schurmann. Combining the classification results of independent classifiers based on the dempster-shafer theory of evidence. In *Pattern Recognition and Artificial Intelligence*, pages 381–393, 1988.

[120] S. Markovitch and D. Rosenstein. Feature generation using general constructor functions. *Mach. Learn.*, 49(1):59–98, 2002.

[121] N. F. McPhee and J. D. Miller. Accurate replication in genetic programming. In L. Eshelman, editor, *Genetic Algorithms: Proceedings of the Sixth International Conference (ICGA95)*, pages 303–309, Pittsburgh, PA, USA, 15-19 July 1995. Morgan Kaufmann.

[122] P. Melville and R. J. Mooney. Creating diversity in ensembles using artificial data. *Information Fusion*, 6:99–111, 2004.

[123] Z. Michalewicz. Evolutionary algorithms - an overview. In D. Dasgupta, editor, *Evolutionary Algorithms in Engineering Applications*. Springer-Verlag New York, Inc., Secaucus, NJ, USA, 1997.

[124] T. Mitchell. *Machine Learning*. McGraw-Hill Education (ISE Editions), October 1997.

[125] T. M. Mitchell. Generalization as search. *Artif. Intell.*, 18(2):203–226, 1982.

[126] L. Nanni. Cluster-based pattern discrimination: A novel technique for feature selection. *Pattern Recogn. Lett.*, 27(6):682–687, 2006.

[127] T. H. Nobuhiko Kondo and K. Uosaki. Multiobjective evolutionary rbf networks and its application to ensemble learning. In *Frontiers of Computational Science*, pages 321–325. Springer, 2007.

[128] Y. Nojima and H. Ishibuchi. Genetic rule selection with a multi-classifier coding scheme for ensemble classifier design. *Int. J. Hybrid Intell. Syst.*, 4(3):157–169, 2007.

[129] Y. Nojima and H. Ishibuchi. Effects of diversity measures on the design of ensemble classifiers by multiobjective genetic fuzzy rule selection with a multi-classifier coding scheme. In E. Corchado, A. Abraham, and W. Pedrycz, editors, *Hybrid Artificial Intelligence Systems, Third International Workshop, HAIS 2008, Burgos, Spain, September 24-26, 2008. Proceedings*, volume 5271 of *Lecture Notes in Computer Science*, pages 755–763. Springer, 2008.

[130] P. Nordin and W. Banzhaf. Complexity compression and evolution. In *Genetic Algorithms: Proceedings of the Sixth International Conference (ICGA95)*, pages 310–317. Morgan Kaufmann, 1995.

[131] P. Nordin, F. Francone, and W. Banzhaf. Explicitly defined introns and destructive crossover in genetic programming. In J. P. Rosca, editor, *Proceedings of the Workshop on Genetic Programming: From Theory to Real-World Applications*, pages 6–22, 9 July 1995.

[132] P. Nordin, F. Francone, and W. Banzhaf. Explicitly defined introns and destructive crossover in genetic programming. In *Advances in genetic programming: volume 2*, pages 111–134. MIT Press, Cambridge, MA, USA, 1996.

[133] L. S. Oliveira, R. Sabourin, F. Bortolozzi, and C. Y. Suen. Feature selection for ensembles: A hierarchical multi-objective genetic algorithm approach. *Document Analysis and Recognition, International Conference on*, 2:676, 2003.

[134] M. Oltean and D. Dumitrescu. Multi expression programming. Technical report, May 2002.

[135] D. W. Opitz. Feature selection for ensembles. In *Proceedings of 16th National Conference on Artificial Intelligence (AAAI-99), Orlando, Florida*, pages 379–384. MIT Press, 1999.

[136] U.-M. O'Reilly and F. Oppacher. The troubling aspects of a building block hypothesis for genetic programming. In L. D. Whitley and M. D. Vose, editors, *Foundations of Genetic Algorithms 3*, pages 73–88, Estes Park, Colorado, USA, 31 July–2 Aug. 1994. Morgan Kaufmann. Published 1995.

[137] F. E. B. Otero, M. M. S. Silva, A. A. Freitas, and J. C. Nievola. Genetic programming for attribute construction in data mining. In C. Ryan, T. Soule, M. Keijzer, E. Tsang, R. Poli, and E. Costa, editors, *Genetic Programming, Proceedings of EuroGP'2003*, volume 2610 of *LNCS*, pages 384–393, Essex, 14-16 Apr. 2003. Springer-Verlag.

[138] J. D. Owens, D. Luebke, N. Govindaraju, M. Harris, J. Krüger, A. E. Lefohn, and T. J. Purcell. A survey of general-purpose computation on graphics hardware. *Computer Graphics Forum*, 26(1):80–113, March 2007.

[139] N. C. Oza. Aveboost2: Boosting for noisy data. In F. Roli, J. Kittler, and T. Windeatt, editors, *Fifth International Workshop on Multiple Classifier Systems*, pages 31–40, Cagliari, Italy, June 2004. Springer-Verlag.

[140] G. L. Pappa and A. A. Freitas. Towards a genetic programming algorithm for automatically evolving rule induction algorithms.

In J. Furnkranz, editor, *ECML/PKDD 2004 Proceedings of the Workshop W8 on Advances in Inductive Learning*, pages 93–108, Pisa, Italy, 20-24 Sept. 2004.

[141] C. Park and S.-B. Cho. Evolutionary computation for optimal ensemble classifier in lymphoma cancer classification. In N. Zhong, Z. W. Ras, S. Tsumoto, and E. Suzuki, editors, *ISMIS*, volume 2871 of *Lecture Notes in Computer Science*, pages 521–530. Springer, 2003.

[142] D. Parrott, X. Li, and V. Ciesielski. Multi-objective techniques in genetic programming for evolving classifiers. In D. Corne, Z. Michalewicz, M. Dorigo, G. Eiben, D. Fogel, C. Fonseca, G. Greenwood, T. K. Chen, G. Raidl, A. Zalzala, S. Lucas, B. Paechter, J. Willies, J. J. M. Guervos, E. Eberbach, B. McKay, A. Channon, A. Tiwari, L. G. Volkert, D. Ashlock, and M. Schoenauer, editors, *Proceedings of the 2005 IEEE Congress on Evolutionary Computation*, volume 2, pages 1141–1148, Edinburgh, UK, 2-5 Sept. 2005. IEEE Press.

[143] H. Peng, F. Long, and C. Ding. Feature selection based on mutual information: criteria of max-dependency, max-relevance, and min-redundancy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 27:1226–1238, 2005.

[144] S. Piramuthu and R. T. Sikora. Genetic algorithm based learning using feature construction. In *INFORMS AI/DM Workshop*, Pittsburgh, 2006.

[145] R. Poli. Discovery of symbolic, neuro-symbolic and neural networks with parallel distributed genetic programming. Technical Report CSRP-96-14, University of Birmingham, School of Computer Science, Aug. 1996. Presented at 3rd International Conference on Artificial Neural Networks and Genetic Algorithms, ICANNGA'97.

[146] R. Poli. Exact schema theorem and effective fitness for GP with one-point crossover. In D. Whitley, D. Goldberg, E. Cantu-Paz, L. Spector, I. Parmee, and H.-G. Beyer, editors, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-*

2000), pages 469–476, Las Vegas, Nevada, USA, 10-12 July 2000. Morgan Kaufmann.

[147] R. Poli. Hyperschema theory for gp with one-point crossover, building blocks, and some new results in ga theory. In *Proceedings of the European Conference on Genetic Programming*, pages 163–180, London, UK, 2000. Springer-Verlag.

[148] R. Poli. Exact schema theory for genetic programming and variable-length genetic algorithms with one-point crossover. *Genetic Programming and Evolvable Machines*, 2(2):123–163, 2001.

[149] R. Poli and W. Langdon. A new schema theory for genetic programming with one-point crossover and point mutation. In *Genetic Programming 1997: Proceedings of the Second Annual Conference*, pages 278–285. Morgan Kaufmann, 1997.

[150] R. Poli, W. B. Langdon, and S. Dignum. On the limiting distribution of program sizes in tree-based genetic programming. In M. Ebner, M. O'Neill, A. Ekárt, L. Vanneschi, and A. I. Esparcia-Alcázar, editors, *Proceedings of the 10th European Conference on Genetic Programming*, volume 4445 of *Lecture Notes in Computer Science*, pages 193–204, Valencia, Spain, 11 - 13 Apr. 2007. Springer.

[151] R. Poli, W. B. Langdon, and N. F. McPhee. *A field guide to genetic programming*. Published via `http://lulu.com` and freely available at `http://www.gp-field-guide.org.uk`, 2008. (With contributions by J. R. Koza).

[152] R. Poli and N. F. McPhee. General schema theory for genetic programming with subtree-swapping crossover: part i. *Evol. Comput.*, 11(1):53–66, 2003.

[153] R. Poli and N. F. McPhee. General schema theory for genetic programming with subtree-swapping crossover: Part ii. *Evol. Comput.*, 11(2):169–206, 2003.

[154] R. Poli, N. F. McPhee, and J. E. Rowe. Exact schema theory and markov chain models for genetic programming and variable-length genetic algorithms with homologous crossover. *Genetic Programming and Evolvable Machines*, 5(1):31–70, 2004.

[155] W. F. Punch, III, E. D. Goodman, M. Pei, L. Chia-Shun, P. D. Hovland, and R. J. Enbody. Further research on feature selection and classification using genetic algorithms. In *Proceedings of the 5th International Conference on Genetic Algorithms*, pages 557–564, San Francisco, CA, USA, 1993. Morgan Kaufmann Publishers Inc.

[156] J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, March 1986.

[157] J. R. Quinlan. Boosting first-order learning. In *Lecture Notes in Computer Science*, pages 143–155. Springer, 1996.

[158] R. J. Quinlan. *C4.5: Programs for Machine Learning (Morgan Kaufmann Series in Machine Learning)*. Morgan Kaufmann, January 1993.

[159] M. L. Raymer, W. F. Punch, E. D. Goodman, and L. A. Kuhn. Genetic programming for improved data mining: An application to the biochemistry of protein interactions. In J. R. Koza, D. E. Goldberg, D. B. Fogel, and R. L. Riolo, editors, *Genetic Programming 1996: Proceedings of the First Annual Conference*, pages 375–380, Stanford University, CA, USA, 28–31 July 1996. MIT Press.

[160] I. Rechenberg. *Evolutionsstrategie Optimierung technischer systeme nach prinzipien der biologischen evolution*. Friedrich Frommann Verlag, Stuttgart-Bad Cannstatt, 1973.

[161] D. B. Redpath and K. Lebart. Observations on boosting feature selection. In *6th International Workshop on Multiple Classifier Systems, MCS 2005*, pages 32–41, 2005.

[162] R. G. Reynolds, B. Peng, and R. S. Alomari. Cultural evolution of ensemble learning for problem solving. In G. G. Yen, L. Wang, P. Bonissone, and S. M. Lucas, editors, *Proceedings of the 2006 IEEE Congress on Evolutionary Computation*, pages 1119–1126, Vancouver, 6-21 July 2006. IEEE Press.

[163] M. Richeldi and P. Lanzi. Performing effective feature selection by investigating the deep structure of the data. In *Proceedings*

of the Second International Conference on Knowledge Discovery and Data Mining, pages 379–383, 1996.

[164] P. P. S. Kotsiantis. Combining bagging and boosting. *International Journal of Computational Intelligence*, 1(4):324–333, 2004.

[165] M. Saerens and F. Fouss. Yet another method for combining classifiers outputs: A maximum entropy approach. In F. Roli, J. Kittler, and T. Windeatt, editors, *Multiple Classifier Systems*, volume 3077 of *Lecture Notes in Computer Science*, pages 82–91. Springer, 2004.

[166] J. D. Schaffer. Multiple objective optimization with vector evaluated genetic algorithms. In *Proceedings of the 1st International Conference on Genetic Algorithms*, pages 93–100, Mahwah, NJ, USA, 1985. Lawrence Erlbaum Associates, Inc.

[167] G. Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, Princeton, 1976.

[168] W. W. Siedlecki and J. Sklansky. A note on genetic algorithms for large-scale feature selection. *Pattern Recognition Letters*, 10(5):335–347, 1989.

[169] S. Silva and J. Almeida. Dynamic maximum tree depth. In E. Cantú-Paz, J. A. Foster, K. Deb, D. Davis, R. Roy, U.-M. O'Reilly, H.-G. Beyer, R. Standish, G. Kendall, S. Wilson, M. Harman, J. Wegener, D. Dasgupta, M. A. Potter, A. C. Schultz, K. Dowsland, N. Jonoska, and J. Miller, editors, *Genetic and Evolutionary Computation – GECCO-2003*, volume 2724 of *LNCS*, pages 1776–1787, Chicago, 12-16 July 2003. Springer-Verlag.

[170] S. Silva and E. Costa. Dynamic limits for bloat control: Variations on size and depth. In K. Deb, R. Poli, W. Banzhaf, H.-G. Beyer, E. Burke, P. Darwen, D. Dasgupta, D. Floreano, J. Foster, M. Harman, O. Holland, P. L. Lanzi, L. Spector, A. Tettamanzi, D. Thierens, and A. Tyrrell, editors, *Genetic and Evolutionary Computation – GECCO-2004, Part II*, volume 3103 of *Lecture Notes in Computer Science*, pages 666–677, Seattle, WA, USA, 26-30 June 2004. Springer-Verlag.

[171] M. Smith and L. Bull. Improving the human readability of features constructed by genetic programming. In D. Thierens, H.-G. Beyer, J. Bongard, J. Branke, J. A. Clark, D. Cliff, C. B. Congdon, K. Deb, B. Doerr, T. Kovacs, S. Kumar, J. F. Miller, J. Moore, F. Neumann, M. Pelikan, R. Poli, K. Sastry, K. O. Stanley, T. Stutzle, R. A. Watson, and I. Wegener, editors, *GECCO '07: Proceedings of the 9th annual conference on Genetic and evolutionary computation*, volume 2, pages 1694–1701, London, 7-11 July 2007. ACM Press.

[172] M. G. Smith and L. Bull. Feature construction and selection using genetic programming and a genetic algorithm. In C. Ryan, T. Soule, M. Keijzer, E. Tsang, R. Poli, and E. Costa, editors, *Genetic Programming, Proceedings of EuroGP'2003*, volume 2610 of *LNCS*, pages 229–237, Essex, 14-16 Apr. 2003. Springer-Verlag.

[173] M. G. Smith and L. Bull. GAP: Constructing and selecting features with evolutionary computation. In A. Ghosh and L. C. Jain, editors, *Evolutionary Computing in Data Mining*, volume 163 of *Studies in Fuzziness and Soft Computing*, chapter 3, pages 41–56. Springer, 2004.

[174] M. G. Smith and L. Bull. Using genetic programming for feature creation with a genetic algorithm feature selector. In X. Yao, E. Burke, J. A. Lozano, J. Smith, J. J. Merelo-Guervós, J. A. Bullinaria, J. Rowe, P. T. A. Kabán, and H.-P. Schwefel, editors, *Parallel Problem Solving from Nature - PPSN VIII*, volume 3242 of *LNCS*, pages 1163–1171, Birmingham, UK, 18-22 Sept. 2004. Springer-Verlag.

[175] M. G. Smith and L. Bull. Genetic programming with a genetic algorithm for feature construction and selection. *Genetic Programming and Evolvable Machines*, 6(3):265–281, Sept. 2005. Published online: 17 August 2005.

[176] T. Soule and J. A. Foster. Removal bias: a new cause of code growth in tree based evolutionary programming. In *1998 IEEE International Conference on Evolutionary Computation*, pages 781–186, Anchorage, Alaska, USA, 5-9 May 1998. IEEE Press.

[177] C. R. Stephens and H. Waelbroeck. Effective degrees of freedom in genetic algorithms and the block hypothesis. In T. Bäck, editor, *Proceedings of the 7th International Conference on Genetic Algorithms*, pages 34–40, San Francisco, 1997. Morgan Kaufmann.

[178] J. Su and H. Zhang. Full bayesian network classifiers. In *ICML '06: Proceedings of the 23rd international conference on Machine learning*, pages 897–904, New York, NY, USA, 2006. ACM.

[179] B.-Y. Sun, X.-M. Zhang, and R.-J. Wang. On constructing and pruning svm ensembles. In *Signal-Image Technologies and Internet-Based System, 2007. SITIS '07. Third International IEEE Conference on*, pages 855–859, 2007.

[180] R. Swinburne. *Simplicity As Evidence of Truth*. Marquette Univ Press, January 1997.

[181] W. A. Tackett. *Recombination, Selection, and the Genetic Construction of Computer Programs*. PhD thesis, University of Southern California, Department of Electrical Engineering Systems, USA, 1994.

[182] B. tak Zhang and H. Mhlenbein. Balancing accuracy and parsimony in genetic programming. *Evolutionary Computation*, 3:17–38, 1995.

[183] E. Tang, P. Suganthan, X. Yao, and A. Qin. Linear dimensionality reduction using relevance weighted lda. *Pattern Recognition*, 38(4):485–493, April 2005.

[184] A. Teller. Evolving programmers: the co-evolution of intelligent recombination operators. pages 45–68, 1996.

[185] J. Torres, A. Saad, and E. Moore. Application of a ga/bayesian filter-wrapper feature selection method to classification of clinical depression from speech data. In *Soft Computing in Industrial Applications*, pages 115–121. Springer, 2007.

[186] J. Torres-Sospedra, C. Hernández-Espinosa, and M. Fernández-Redondo. A comparison of combination methods for ensembles of rbf networks. In *Neural Networks, 2005. IJCNN '05. Proceedings. 2005 IEEE International Joint Conference on*, volume 2, pages 1137– 1141, 2005.

[187] J. Torres-Sospedra, C. Hernández-Espinosa, and M. Fernández-Redondo. New results on combination methods for boosting ensembles. In *ICANN '08: Proceedings of the 18th international conference on Artificial Neural Networks, Part I*, pages 285–294, Berlin, Heidelberg, 2008. Springer-Verlag.

[188] A. Tsymbal, S. Puuronen, and D. Patterson. Feature selection for ensembles of simple bayesian classifiers. In *ISMIS '02: Proceedings of the 13th International Symposium on Foundations of Intelligent Systems*, pages 592–600, London, UK, 2002. Springer-Verlag.

[189] H. Vafaie and K. De Jong. Feature space transformation using genetic algorithms. *IEEE Intelligent Systems*, 13(2):57–65, 1998.

[190] H. Vafaie and K. DeJong. Robust feature selection algorithms. *Proc. 5th Intl. Conf. on Tools with Artifical Intelligence*, pages 356–363, 1993.

[191] H. Vafaie and K. D. Jong. Genetic algorithms as a tool for restructuring feature space representations. In *In Proc. of the International Conference on Tools with Artificial Intelligence. IEEE Computer Soc*, pages 8–11. Press, 1995.

[192] G. Valentini. Random aggregated and bagged ensembles of svms: An empirical bias-variance analysis. In *Multiple Classifier Systems*, pages 263–272, 2004.

[193] R. Vilalta and L. Rendell. Integrating feature construction with multiple classifiers in decision tree induction. In *In 14th International Conference on Machine Learning*, pages 394–402. Morgan Kaufman, 1997.

[194] L. Wang, N. Zhou, and F. Chu. A general wrapper approach to selection of class-dependent features. *IEEE Trans. Neural Networks*, 19(7):1267–1278, 2008.

[195] G. I. Webb. Multiboosting: A technique for combining boosting and wagging. In *MACHINE LEARNING*, pages 159–196, 2000.

[196] J. D. Wichard and C. Merkwirth. Building ensembles with heterogeneous models, 7th course of the international school on neural nets iiass, 2002.

[197] J. Wickramaratna, S. B. Holden, and B. F. Buxton. Performance degradation in boosting. In *MCS '01: Proceedings of the Second International Workshop on Multiple Classifier Systems*, pages 11–21, London, UK, 2001. Springer-Verlag.

[198] I. H. Witten and E. Frank. *Data Mining: Practical Machine Learning Tools and Techniques with Java Implementations*. Morgan Kaufmann, October 1999.

[199] D. H. Wolpert. Stacked generalization. *Neural Networks*, 5:241–259, 1992.

[200] D. H. Wolpert and W. G. Macready. No free lunch theorems for search. Working Papers 95-02-010, Santa Fe Institute, Feb. 1995.

[201] J. Yang and V. Honavar. Feature subset selection using a genetic algorithm. *IEEE Intelligent Systems*, 13(2):44–49, 1998.

[202] X. Yao, S. M. Ieee, and Y. Liu. A new evolutionary system for evolving artificial neural networks. *IEEE Transactions on Neural Networks*, 8:694–713, 1996.

[203] X. Yao and M. Islam. "evolving artificial neural network ensembles". *Computational Intelligence Magazine, IEEE*, 3:31–42, February 2008.

[204] X. Yao, S. Member, Y. Liu, and S. Member. Making use of population information in evolutionary artificial neural networks. *IEEE Transactions on Systems, Man and Cybernetics, Part B: Cybernetics*, 28:417–425, 1998.

[205] C. Yu and D. B. Skillicorn. Parallelizing boosting and bagging, 2001.

[206] Y. Zhang and S. Bhattacharyya. Genetic programming in classifying large-scale data: an ensemble method. *Information Sciences*, 163(1-3):85–101, June 2004.

[207] Z. Zheng. *Constructing New Attributes for Decision Tree Learning*. PhD thesis, 1996.

[208] E. Zitzler, K. Deb, and L. Thiele. Comparison of multiobjective evolutionary algorithms: Empirical results. *Evolutionary Computation*, 8(2):173–195, 2000.

[209] E. Zitzler and L. Thiele. Multiobjective optimization using evolutionary algorithms — A comparative case study. *Lecture Notes in Computer Science*, 1498:292–??, 1998.